



Ingeniería del Software

Código de Curso: CY440

Versión: 4.3

Guía del Estudiante

Libro 1: Ingeniería del Software

IBM Learning Services
Worldwide Certified Material

Información Sobre la Publicación

Esta publicación ha sido producida usando Microsoft Word 2000 y Microsoft PowerPoint 2000 para Windows.

Marcas Registradas

IBM ® es una marca registrada por International Business Machines Corporation.

Otras compañías, productos, y nombre de servicios pueden ser marcas registradas o marcas de servicios de otros.

Trademarks of International Business Machines Corporation

DB2	Informix
Lotus Script	Net.data

Marcas Registradas de otras Compañías

Windows, Microsoft Visual Studio	Microsoft Corporation
Sybase	Sybase Inc.

Edición Noviembre 2007

La información contenida en este documento no ha sido sometida a ninguna prueba formal de IBM y es distribuida básicamente "como es" sin ninguna garantía ya sea expresa o implícita. El uso de esta información o la implementación de cualquiera de estas técnicas es responsabilidad del comprador y dependerá de la habilidad de éste para su evaluación e integración en el ambiente operacional del comprador. A pesar de que cada tema ha sido revisado por IBM para su exactitud en una situación específica, no hay garantía de obtener el mismo resultado o uno similar a éste en otra situación. Los compradores que intenten adaptar estas técnicas a sus propios ambientes lo hacen bajo su propio riesgo.

Copyright International Business Machines Corporation, 2007. All rights reserved. Este documento no puede ser reproducido en su totalidad o en parte sin el previo permiso escrito de IBM.

Instrucciones Especiales para la impresión de este Curso:

No elimine páginas en blanco que puedan aparecer al final de cada unidad o entre unidades. Estas páginas fueron insertadas intencionalmente.

Contenido

Descripción del Curso.....	1
Descripción de Unidades	3
Volumen 1: Fundamentos de Análisis y Diseño	5
Unidad 1: Fundamentos de Ingeniería de Software.....	7
1. Introducción	8
2. Características del Software	8
3. Mitos que Prevalecen en la Industria de Software	9
4. Tipos de Aplicaciones de Software	10
5. Procesos de Software	11
6. Categorías de Modelos de Procesos de Software	13
7. Modelos Secuencial Lineal	15
8. El Modelo de Creación de un Prototipo	19
9. Modelos Evolutivos	21
10. El Modelo de Ensamblado de Componentes	26
11. Técnicas de Cuarta Generación (4GTs)	27
Unidad 1: Examen de Autoevaluación	30
Unidad 2: Especificación de los Requerimientos	35
1. Introducción.	36
2. ¿Qué son los Requerimientos?	37
3. Análisis del Problema y Descripción del Producto	40
4. Importancia de la Fase de Requerimientos	41
5. Ingeniería de Requerimientos	41
6. Conducir el Estudio de los Requerimientos	46
7. Usos de la SRS	48
8. Contenido de la SRS	49
9. Atributos de una SRS de Alta Calidad	50
10. Estándares en SRS	51
Unidad 2: Examen de Autoevaluación	54
Unidad 3: Lab. de Especificación de Requerimientos.....	57
1. Ejercicio de Laboratorio	58
Unidad 4: Diseño de Software	61
1. Introducción al Diseño de Software	62
2. Factores que Afectan los Enfoques de Diseño de Software	63

3. Comprender el Proceso de Diseño	64
4. Enfoques de Diseño Top-Down y Bottom-Up	67
5. Conceptos Básicos de Diseño	68
6. Arquitectura del Software	69
7. Diseño Modular	70
Resumen	73
Unidad 4: Examen de Autoevaluación	74
Respuestas de la Unidad 4: Examen de Autoevaluación	77
Unidad 5: Planeamiento del Software	79
1. Introducción	80
2. Estimación	81
3. Administración del Riesgo	81
4. Administración y Monitoreo de Riesgos	84
5. Cronograma del Proyecto de Software	86
6. Decisiones de Adquisición	89
7. Re-ingeniería	89
8. Planeamiento Organizacional	90
9. Administración de Configuración	91
Unidad 5: Examen de Autoevaluación	97
Respuestas de la Unidad 5: Examen de Autoevaluación	100
Volumen 2: Pruebas y Calidad.....	101
Unidad 1: Fundamentos de Pruebas del Software.....	103
1. Introducción	104
2. Beneficios de la Prueba	105
3. Niveles de Prueba	105
4. Técnicas de Prueba de Software	107
5. Prueba de Caja Blanca	108
6. Prueba de Caja Negra	126
Resumen	135
Unidad 1: Examen de Autoevaluación	136
Respuestas de la Unidad 1: Examen de Autoevaluación	138
Unidad 2: Metodologías de Prueba	139
1. Introducción	140
2. Verificación y Validación del software	141

3. El Proceso de Prueba	141
4. Prueba de Unidad	143
5. Prueba de Integración	149
6. Prueba del Sistema	155
7. Prueba de Aceptación	156
Resumen	160
Unidad 2: Examen de Autoevaluación	161
Respuestas de la Unidad 2: Examen de Autoevaluación	164
Unidad 3: Control de Calidad del Software.....	165
1. ¿Qué es Calidad?	166
2. Algunos Conceptos sobre Calidad	167
3. ¿Cómo se Controla la Calidad del Software?	167
4. Aseguramiento de la Calidad del Software	168
5. Algunas Operaciones SQA	168
6. Confiabilidad del Software	169
7. Seguridad del Software	170
8. El Plan de SQA	170
9. El Modelo de Capacidad de Madurez (CMM)	171
Resumen	178
Unidad 3: Examen de Autoevaluación	179
Respuestas de la Unidad 3: Examen de Autoevaluación	181
Unidad 4: Introducción a RUP	183
1. Introducción	184
2. Definiendo Rational Unified Process (RUP)	184
3. Características de RUP	186
4. Elementos Básicos de RUP	187
5. Estructura de RUP	193
6. Fases del Ciclo de Vida de RUP	195
7. ¿Quiénes deben usar RUP?	211
8. ¿Cuándo usar RUP?	211
Resumen	213
Unidad 4: Examen de Autoevaluación	214
Respuestas de la Unidad 4: Examen de Autoevaluación	216

Descripción del Curso

Nombre del Curso

Ingeniería de Software.

Duración

La duración de este curso es de 28 horas académicas.

Propósito

El propósito de este curso es proporcionar al estudiante una visión general sobre los diferentes conceptos relacionados al diseño y desarrollo de software como disciplina. El curso de Ingeniería del Software relaciona al estudiante con conceptos y procesos involucrados en el ciclo de desarrollo de software. Las metodologías usadas en el desarrollo de software y las técnicas de prueba y verificación del producto de software antes de ser entregado al cliente.

Audiencia

Estudiantes, profesionales y desarrolladores que desean conocer acerca de Ingeniería del Software.

Prerrequisitos

- Introducción a las computadoras personales y a Windows XP.
- Fundamentos de programación.

Objetivos del Curso

Al final de este curso Ud. será capaz de:

- Aplicar los conceptos básicos relacionados a la Ingeniería del software, procesos y producto de software.
- Determinar cuál es la mejor metodología para el desarrollo un producto de software.
- Realizar el correcto levantamiento de los requerimientos para un producto de software.

- Evaluar la calidad de un diseño de software.
- Determinar el nivel de calidad de los procesos de software aplicados dentro de una empresa de desarrollo.
- Diseñar pruebas para medir los diferentes aspectos de la calidad de un producto de software.

Agenda

Cada unidad del curso tiene una duración de 2 a 3 horas académicas.

Descripción de Unidades

Volumen 1: Fundamentos de Análisis y Diseño

Unidad 1: Fundamentos de Ingeniería de Software

Esta Unidad proporciona los fundamentos de la ingeniería de software. Define la ingeniería de software y explica su importancia. Analiza la necesidad de modelos de proceso de software y discute varios modelos del proceso de software. Se discute la necesidad de diversos Modelos Evolutivos. También se discuten las Técnicas de Cuarta Generación.

Unidad 2: Especificaciones de Requerimientos de Software

Las especificaciones de requerimientos de software forman la base de la ingeniería de software, es una de las fases más tempranas del ciclo de vida. Esta Unidad describe la naturaleza de los requerimientos, define los Requerimientos de Software y describe la Especificación de Requerimientos de Software (Software Requirements Specification - SRS). Por otra parte, describe las actividades del análisis de requerimientos, lo que deberá hacerse en el análisis de requerimientos y el impacto de creación de prototipos en los requerimientos, además del ciclo de vida de software. Esta Unidad describe las funciones, los componentes y los atributos de una SRS, adicionalmente, una lista de algunos de los estándares para escribir SRS.

Unidad 3: Laboratorio de Especificaciones de Requerimientos de Software

Esta Unidad proporciona la práctica para afianzar los conocimientos adquiridos en la Unidad 2, por medio del planteamiento de un ejercicio que le brindará una visión más clara de los conceptos esenciales sobre especificación de requerimientos de software.

Unidad 4: Diseño de Software

Esta Unidad describe en resumen el proceso del diseño y proporciona una introducción para diseñar las metodologías. Explica los enfoques de Diseño Descendente (Top-Down) y Ascendente (Bottom-Up), los principios del aprendizaje del diseño y de los conceptos críticos de la cohesión, aparte del acoplamiento. En síntesis, proporciona una vista general y concisa del proceso de diseño.

Unidad 5: Planeamiento de Software

La planificación del software, el análisis de riesgo y algunos elementos de la planificación de proyectos, son los temas centrales en esta Unidad. Esta Unidad lista los diversos pasos que implica la planificación de proyectos y discute las diversas actividades que abarcan el análisis de riesgo. Trata acerca de la administración de riesgo, la planificación de proyectos de software, planeamiento, la planificación de la organización y la administración de configuración de software desde múltiples

perspectivas. Discute también la administración de versiones como un subconjunto de la administración de configuración.

Volumen 2: Pruebas y Calidad

Unidad 1: Fundamentos de Prueba de Software

En esta Unidad se discuten los fundamentos de pruebas de software. Trata acerca de la necesidad de metodologías de pruebas de software, los diversos niveles de prueba y describe el diseño de casos de prueba. Explica también algunos de los métodos de prueba de 'caja blanca', el proceso de derivar los casos de prueba y algunos de los métodos de prueba de 'caja negra'.

Unidad 2: Metodologías de Prueba

Esta Unidad proporciona una visión general de las metodologías de prueba. Discute la prueba de unidades, verificación y validación de software, pruebas de sistema y pruebas de integración. Se discuten también las estrategias de prueba. Se proporciona una breve visión de algunos de los sistemas de métodos de prueba que se usan comúnmente.

Unidad 3: Control de Calidad del Software

En esta Unidad, se discuten parte de los conceptos importantes de calidad y control de la calidad en el entorno de software. Explica el control de calidad y la confiabilidad y seguridad del software. Esta Unidad proporciona una visión general del Plan para el aseguramiento de la calidad del software y también del modelo de madurez de capacidad de SEI.

Unidad 4: Introducción a RUP

Esta Unidad brinda una introducción a la metodología RUP. Describe las características de este enfoque. Explica las ventajas de la adopción de RUP como metodología de desarrollo de software. Discute los principios esenciales que guían la metodología. Proporciona una descripción de los elementos básicos de RUP. Describe cada una de las fases del ciclo de vida de un proyecto usando RUP.

Volumen 1: Fundamentos de Análisis y Diseño

Unidad 1: Fundamentos de Ingeniería de Software

Objetivos de Aprendizaje

Al finalizar esta unidad, usted será capaz de:

- Explicar el software como producto y proceso.
- Definir la ingeniería de software y explicar su importancia.
- Explicar los diversos modelos de proceso de software.
- Explicar las características de los diversos modelos secuencial lineal y evolutivos.

1. Introducción

Asuma que un ingeniero civil se encuentra supervisando la construcción de un edificio de departamentos de varios pisos. Existen dificultades en la utilización de los materiales, así como también de otros recursos, para asegurar que el edificio, en su forma final, sea estable y esté de acuerdo a los requerimientos de los clientes.

Ahora, considere un escenario paralelo con un ingeniero o varios ingenieros trabajando juntos, escribiendo procedimientos finales de diversa complejidad para desarrollar sistemas de software. Un sistema de software es como un edificio moderno. En muchos casos, los sistemas de software son grandes y complicados. Por lo tanto, diseñar y programar estos sistemas requiere de métodos, procedimientos y herramientas. Si un sistema es rentable en su forma final, definitivamente el ingeniero optimizó el uso de los recursos.

La importancia de la ingeniería de software está en fomentar un enfoque sistemático para el desarrollo, la implementación y el mantenimiento del software a través del ciclo de vida del sistema de software. Es muy diferente escribir programas pequeños y eficientes que desarrollar sistemas de software. Las técnicas utilizadas para desarrollar programas son insuficientes para desarrollar sistemas de software. De igual manera, las técnicas utilizadas tradicionalmente para desarrollar sistemas de software de pequeña escala no parecen adecuadas cuando se aplican al desarrollo de sistemas grandes. Cuando se utilizan metodologías de desarrollo inapropiadas los proyectos tienden a presentar un aumento significativo de los costos y un consumo extra de tiempo.

La ingeniería de software apunta a proveer metodologías y técnicas que ayuden a desarrollar sistemas de software a tiempo, y a su vez, que aseguren que el desarrollador cumpla con las expectativas de calidad y permanezca dentro del presupuesto. Este tipo de ingeniería puede ser apreciado sólo al entender las características significativas del software, las cuales son discutidas a continuación.

2. Características del Software

Un producto de software, al igual que el proceso de desarrollo de software no se puede comparar, ni evaluar utilizando los mismos criterios que se usan para otros productos y procesos de manufactura. Esto subraya aún más la importancia del estudio de la ingeniería de software.

En general, cualquier producto de software debe tener las siguientes deseables características:

- El producto debe ser *confiable* y realizar sólo las tareas especificadas en los requerimientos.
- El producto debe ser *robusto*. Esto significa que el software se comporta de forma razonable, incluso en circunstancias que no fueron anticipadas cuando el producto fue hecho inicialmente.

- El producto de software debe ser lo más *reutilizable* posible, de manera tal que pueda ser incorporado en otro producto de software si se requiere.
- El producto de software debe ser *eficiente* en el uso de los recursos del sistema.
- El producto de software debe ser *mantenible*, es decir, que se puedan realizar cambios fácilmente en el software para corregir o mejorar su funcionamiento.
- Se requiere desarrollar el software en una manera que lo haga *evolutivo*, de forma tal que se pueda agregar funcionalidad adicional sin efectos perjudiciales.
- El producto de software debe cumplir con los requerimientos de *rendimiento* especificados, es decir, debe cumplir algunas de las restricciones relacionadas al rendimiento.
- El producto de software tendrá un valor mayor si es *portable*, es decir que puede trabajar bajo diferentes plataformas y ambientes (hardware, sistemas operativos, etc.).
- El producto de software debe ser *utilizable*, es decir, el aprendizaje de su uso debe ser lo suficientemente sencillo por parte de personas no especialistas.

En esta etapa, es importante entender algunos de los mitos que prevalecen en la industria de software.

3. Mitos que Prevalecen en la Industria de Software

Hasta hace poco en la industria de software, cada pieza de software tenía que ser construida desde cero. Como resultado, hasta hace muy poco, nunca se tuvo ningún componente de software reutilizable que hiciera más fácil el desarrollo de software. Esta es una de las razones principales por las que el crecimiento de la industria de software ha sido lento y pesado comparado con la industria de hardware. Sin embargo, ahora esta situación está cambiando.

Hay un gran número de problemas que preocupan al desarrollador, a la gerencia y a los clientes. Generalmente, los proyectos de software se exceden en los costos. Muchos proyectos de software tienden a retrasarse en los cronogramas. El software es un producto que usualmente no otorga ningún tipo de garantía. La calidad es también una materia de preocupación.

En vista de que el desarrollo de software es una actividad dependiente del diseñador y el programador, el factor humano también debe ser tomado en cuenta. Existen algunas creencias incorrectas, comunes en la mayoría de programadores, que retrasan el crecimiento de la industria. Estas creencias inciden en la calidad y en el incremento de los costos. A continuación se mencionan algunas de ellas:

- Tener un buen programador (o un conjunto de buenos programadores) es suficiente para construir software de calidad.
- Seguir un conjunto de procesos y metodologías tiende a disminuir el 'instinto creativo' de los desarrolladores.

- La disciplina de desarrollar software es un 'arte' y no una disciplina de 'ingeniería'.
- Cuando se enfrenta a presiones de entrega, todo lo que necesita es añadir más programadores.
- Ni el cliente puede proveer todos los requerimientos, ni son necesarios todos los requerimientos. Siempre se puede acomodar requerimientos adicionales o modificar los presentes en una fecha posterior, ya que el software es muy maleable.
- Para el cliente, el único entregable importante es un conjunto de programas que funcione de manera razonable.
- La calidad del software no puede ser evaluada hasta que esté completamente construido.

Desde luego, ahora se sabe que estas opiniones y creencias están lejos de la realidad. El software debe ser desarrollado con un enfoque de ingeniería, así como la calidad del software tiene sus orígenes en etapas muy anteriores a la programación. La gerencia de proyectos de software es diferente de las prácticas seguidas generalmente en la industria manufacturera.

Antes de proseguir, se presentan los diferentes tipos de aplicaciones de software.

4. Tipos de Aplicaciones de Software

Las aplicaciones de software varían desde aplicaciones de negocio, aplicaciones de ingeniería hasta aplicaciones de entretenimiento y otros. La clasificación de las aplicaciones de software es una tarea extremadamente complicada. Pero algún tipo de clasificación ayuda a decidir la metodología o proceso a ser adoptado, especialmente cuando una aplicación particular requiere ser construida. Claramente, los procesos, métodos y técnicas que se usan para desarrollar un compilador difieren de aquellos que se usan para desarrollar un juego multimedia para niños o incluso aplicaciones que controlan la función de navegación de una nave espacial. Algunas de las extensas áreas de aplicaciones de software son:

4.1 Software de Aplicaciones de Sistema

Las aplicaciones de sistema de software son usualmente programas que interactúan principalmente con el hardware. Ejemplos comunes de este tipo de software son los sistemas operativos, compiladores y depuradores simbólicos.

4.2 Software de Aplicaciones en Tiempo Real

Este software requiere de salidas que deben proporcionarse en tiempo real. En otras palabras, debe responder a entradas dentro de restricciones estrictas de tiempo. Las aplicaciones que controlan la navegación de aeronaves y control de velocidad en automóviles son ejemplos de software de aplicación en tiempo real.

4.3 Software para Aplicaciones de Negocio

Estas aplicaciones involucran el manejo de entradas muy grandes, reglas específicas de proceso del negocio y requerimientos de salidas específicos. Algunos ejemplos comunes son: la nómina, aplicaciones bancarias, manejo de inventarios, entre otros.

4.4 Software para Ingeniería y Aplicaciones Científicas

Estas aplicaciones involucran cálculos complejos. Algunos ejemplos comunes son las aplicaciones que efectúan cálculos matemáticos muy grandes en el área de astronomía, reactores de ingeniería, etc.

4.5 Software para Aplicaciones Incorporadas

Tales aplicaciones residen en la Memoria de Sólo Lectura (Read Only Memory -ROM), usualmente son de tamaño compacto y específicas a los dispositivos con los que trabaja. Los ejemplos de estas aplicaciones son los que controlan la operación de lavadoras, hornos, etc.

4.6 Software para Computadoras Personales y Aplicaciones para Individuos

Estos varían desde lo simple hasta lo complejo, pero se pueden ejecutar en computadoras personales y pueden usarlas personas con diversas habilidades para manejar la computadora. Algunos ejemplos de estas aplicaciones son procesadores de palabras, hojas de cálculo, multimedia, etc.

Las siguientes secciones tratan acerca de los diversos tipos de modelos de proceso de software.

5. Procesos de Software

Un *proceso*, se define como una serie de operaciones usadas en la creación de un producto. Un proceso de software se puede definir de las siguientes formas:

- Un proceso de software define el conjunto de tareas, que tienen que ser realizadas para producir un producto de software de alta calidad. En otras palabras, este es el enfoque que se toma para el desarrollo del software.
- Es el proceso que se sigue para construir el producto de software desde la concepción de una idea, hasta la entrega y el retiro final del sistema.

Las características de un proceso de software se resumen a continuación:

- **Comprensión:** Este requiere claridad y declaración de la naturaleza explícita de la definición del proceso.
- **Visibilidad:** Se refiere a la capacidad de observar la salida de varias actividades del proceso, de manera que se mida el progreso del proceso.

- **Confiabilidad:** Se refiere a la capacidad del proceso para evadir errores o detectar errores y manejarlos antes de que estos avancen en el producto.
- **Robustez:** Se refiere a la capacidad del proceso de no detenerse a pesar de problemas inesperados.
- **Facilidad de Mantenimiento:** Se refiere a la cantidad de modificaciones que pueden hacerse al sistema de software sin introducir errores.
- **Facilidad de Verificación:** Un proceso es verificable si sus propiedades pueden ser fácilmente verificadas.
- **Rapidez:** Se refiere a la agilidad y rapidez del proceso para ser capaz de entregar un producto final a partir de las especificaciones.
- **Facilidad de Soporte:** Se refiere a la posibilidad de que las actividades del proceso sean soportadas por un conjunto de herramientas automatizadas.
- **Facilidad de Aceptación:** Se refiere a la capacidad del proceso a ser aceptado y usado por el equipo de ingenieros.
- **Facilidad de Adaptación:** Se refiere a la capacidad del proceso a ser modificado para satisfacer las necesidades de cambio en el ambiente de desarrollo.

Después de haber discutido las características del proceso de desarrollo de software, se presenta a continuación las diferentes fases del proceso de desarrollo de software.

5.1 Fases del Proceso de Desarrollo de Software

El proceso de desarrollo de software ha sido descrito en términos de las siguientes fases:

- **Fase de Definición:** Esta fase se concentra principalmente en '*qué*' tiene que ser completado por el proceso de software. Se identifican los requerimientos claves del sistema y el software que describen lo que debe completarse. Durante esta fase, los métodos varían dependiendo del paradigma de ingeniería de software que se aplica. Las tres tareas principales que se toman en cuenta durante esta fase son:
 - Ingeniería de Información.
 - Planeamiento del Proyecto de Software.
 - Análisis de Requerimientos.
- **Fase de Desarrollo:** Esta fase se enfoca en el '*cómo*' los requerimientos de un sistema y el software serán completados. Durante esta fase, se lleva a cabo la implementación de los detalles de procedimientos, caracterización de interfaces y la traducción del diseño a un lenguaje de programación; es decir, *cómo* las tareas se completarán. Finalmente, se lleva a cabo la prueba del diseño. Las siguientes son las tres principales tareas de esta fase:
 - Diseño de Software.
 - Codificación.
 - Prueba.

- **Fase de Mantenimiento:** Esta fase se enfoca en 'cambio'. El mantenimiento incluye la corrección de errores y la adaptación, conforme evoluciona el entorno del software. Esta fase reaplica los pasos del diseño y desarrollo, basado en el cambio de los requerimientos del cliente y las consecuentes mejoras.

5.2 La Necesidad de Modelos de Procesos de Software

En los primeros tiempos de la computación, el desarrollo de software era principalmente un esfuerzo individual. No había diferencia entre el programador y el usuario final de la aplicación. El usuario final desarrollaba la aplicación como una función de soporte para su propia actividad. Consistía sólo en codificar en un lenguaje requerido y representaba sólo la fase de desarrollo discutida anteriormente.

Algunos problemas pueden ser tan pequeños que cada una de las actividades mencionadas anteriormente no podían ser distinguidas y reconocidas explícitamente. Por ejemplo, en ciertos proyectos pequeños, las actividades de diseño y codificación pueden estar tan entrelazadas que pueden ser consideradas como una sola actividad. Sin embargo, para sistemas grandes donde la resolución del problema puede durar unos cuantos años y el desarrollo puede involucrar a mucha gente, realizar estas actividades sin descomponer el problema y documentar los diversos aspectos de la solución del problema, no funcionará.

Esto conduce al concepto de modelos de procesos de software. Los modelos se utilizan para precisar descripciones con la finalidad de hacer el proceso predecible y controlable.

Los modelos son abstracciones que asisten en la representación del proceso de software. Se construyen con la idea del desarrollador acerca de qué es lo que se necesita en el producto final. Al definir el modelo, se pueden obtener algunos de los beneficios de un proceso estandarizado.

La organización de un proceso de software permite producir un producto de software de alta calidad, confiable, predecible y eficiente. Los diversos modelos capturan este proceso. Como la naturaleza de los productos varía, se necesitan diferentes modelos de procesos. La siguiente sección discute las diversas categorías de modelos de procesos de software.

6. Categorías de Modelos de Procesos de Software

Los modelos de procesos de software se pueden clasificar de la siguiente manera:

- Modelo secuencial lineal.
- Modelos de creación de un prototipo.
- Modelo evolutivo.
- Modelo de métodos formales.
- Sistema ensamblado a partir de componentes reutilizables.

A continuación se discuten cada uno brevemente.

6.1 Modelo Secuencial Lineal

Este modelo procede de una manera lineal ordenada, con transiciones de entregables bien definidos en cada etapa. Consiste de dos tipos:

- Modelo de Cascada.
- Modelos de Desarrollo Rápido de Aplicación (Rapid Application Development - RAD).

Cada uno de estos será discutido en detalle más adelante en esta unidad.

6.2 Modelo de Creación de un Prototipo

El proceso procede en la forma de iteraciones. En cada iteración, se crea un prototipo basado en los requerimientos iniciales del cliente. Para cada iteración del prototipo hacia adelante, se obtiene por parte del cliente retroalimentación sobre el modelo. Esto continúa hasta que el cliente esté satisfecho con el modelo desarrollado.

6.3 Modelo Evolutivo

Este modelo se usa cuando los objetivos generales y las entradas y salidas se conocen. Inicialmente, un producto central se desarrolla y al usuario se le permite usarlo. A medida que surgen nuevos requerimientos, se añaden características adicionales al sistema existente.

6.4 Modelo de Métodos Formales

El modelo de métodos formales usa algunas técnicas matemáticas para derivar un programa de computadora de un conjunto de especificaciones del sistema. Normalmente, las especificaciones se escriben en lenguaje natural como el inglés o usando algún método de diagramas.

6.5 Ensamblar un Sistema Usando Componentes Reutilizables

Un sistema consiste en un conjunto de partes que interactúan. En vez de construir el sistema completo desde el inicio se puede ensamblar reutilizando algunos de los componentes existentes e integrándolos con el resto del sistema.

Las primeras tres categorías se usan ampliamente para desarrollo de sistemas prácticos. Por lo tanto, se discutirán estos tres enfoques.

7. Modelos Secuencial Lineal

7.1 El Modelo de Cascada

Winston Royce propuso el modelo de cascada en 1970. En el modelo original, las fases eran iterativas. En la práctica, sin embargo, se volvió rígidamente secuencial y llegó a ser conocido en consecuencia como el *modelo secuencial lineal*. La Figura 1.1 representa el modelo de cascada con fases iterativas.

Las etapas principales del modelo de cascada son:

7.1.2 Ingeniería de Sistemas

El software es siempre una parte de un gran sistema. Por lo tanto, los requerimientos se establecen para todos los elementos del sistema y una parte de estos requerimientos están asignados para el software. Esta vista del sistema es necesaria cuando el software tiene una interfaz con otros elementos como hardware, personas y bases de datos.

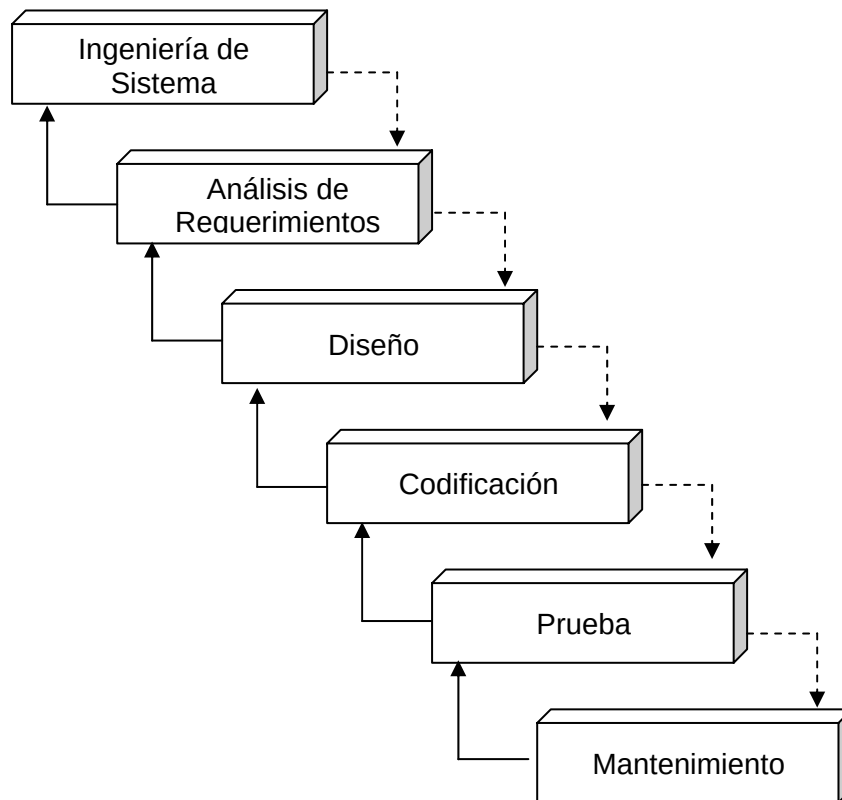


Figura 1.1: Modelo de Cascada

7.1.2 Análisis de Requerimientos

Los requerimientos son analizados y entendidos antes de proceder a otros procesos. La representación gráfica del análisis de requerimientos motiva su uso para evitar la

ambigüedad de los requerimientos. La simulación extensiva y la creación de prototipo, se usan a veces, para capturar y analizar los requerimientos del sistema concernientes a la interacción humana.

7.1.3 Fase de Diseño

El propósito de la fase de diseño es planificar una solución para el problema especificado en el documento de requerimientos. Este es el primer paso para moverse del dominio del problema al dominio de la solución. La entrada para esta fase es la especificación de requerimientos y la salida es el documento de diseño. La fase de diseño incluye la propuesta de una solución, el modelo de la solución, evaluación contra los requerimientos originales y después de alguna iteración de estas actividades, se obtiene como resultado un documento de diseño o especificación.

7.1.4 Fase de Codificación

El objetivo de la fase de codificación es traducir el diseño del sistema a un código de programa en un lenguaje de programación apropiado. Para un diseño dado, la meta es implementarlo de la mejor manera posible. Durante la codificación, el foco debe ser desarrollar programas simples y entendibles. Todo código bien escrito puede reducir la fase de prueba y el esfuerzo de mantenimiento.

7.1.5 Fase de Prueba

La prueba es la principal medida de control de calidad empleada durante el desarrollo de software. Es el paso final de la verificación y validación del proceso de desarrollo. Se denomina *validación*, cuando se realiza contrastando los requerimientos originales. Cuando se realiza contra las especificaciones, se denomina *verificación*. La prueba no sólo debe descubrir errores introducidos durante la fase de codificación, sino también los errores introducidos en fases previas.

7.1.6 Fase de Mantenimiento

Esta fase incluye todas las actividades realizadas para mantener el sistema operacional después de la instalación de software. Los cuatro tipos principales de actividades de mantenimiento son:

- Mantenimiento correctivo: Se ocupa del diagnóstico y corrección de errores.
- Mantenimiento adaptativo: Modifica el software en un entorno cambiante.
- Mantenimiento mejorativo: Incluye la adición de nuevas capacidades, modificaciones a funciones existentes y perfeccionamiento en general.
- Mantenimiento preventivo: Ocurre cuando el software se modifica para mejorar la confiabilidad.

Las ventajas de este modelo son:

- Permite una máxima organización en el proceso de implementación.
- Proporciona una plantilla estructurada para ingeniería de software.

Las desventajas de este modelo son:

- Es difícil para los clientes proporcionar todos sus requerimientos de una sola vez, siendo esto necesario en este modelo.
- Es difícil para el usuario anticipar si el sistema final construido de acuerdo a las especificaciones, eventualmente cumplirá con sus requerimientos.
- Cualquier cambio realizado durante la implementación puede causar confusión, ya que el modelo es inherentemente secuencial.
- La versión de trabajo puede verse sólo en una etapa posterior y por esta razón, en caso de un error serio, el error debe ser rastreado hacia atrás, hasta la fase de requerimientos.
- Los clientes deben tener paciencia cuando trabajan con este modelo.

Considere la situación a continuación:

- 1) Peter y David consultan a un arquitecto para diseñar una casa. En vez de darles un plano, el arquitecto presenta un documento técnico de 100 páginas, referente a la construcción de la casa, el cual Peter y David no entienden. Sin embargo, para evitar leer el documento de 100 páginas que es demasiado técnico como para ser entendido por Peter y David, prefieren decir: '¡Si, esto es! ¡Construya la casa!'.

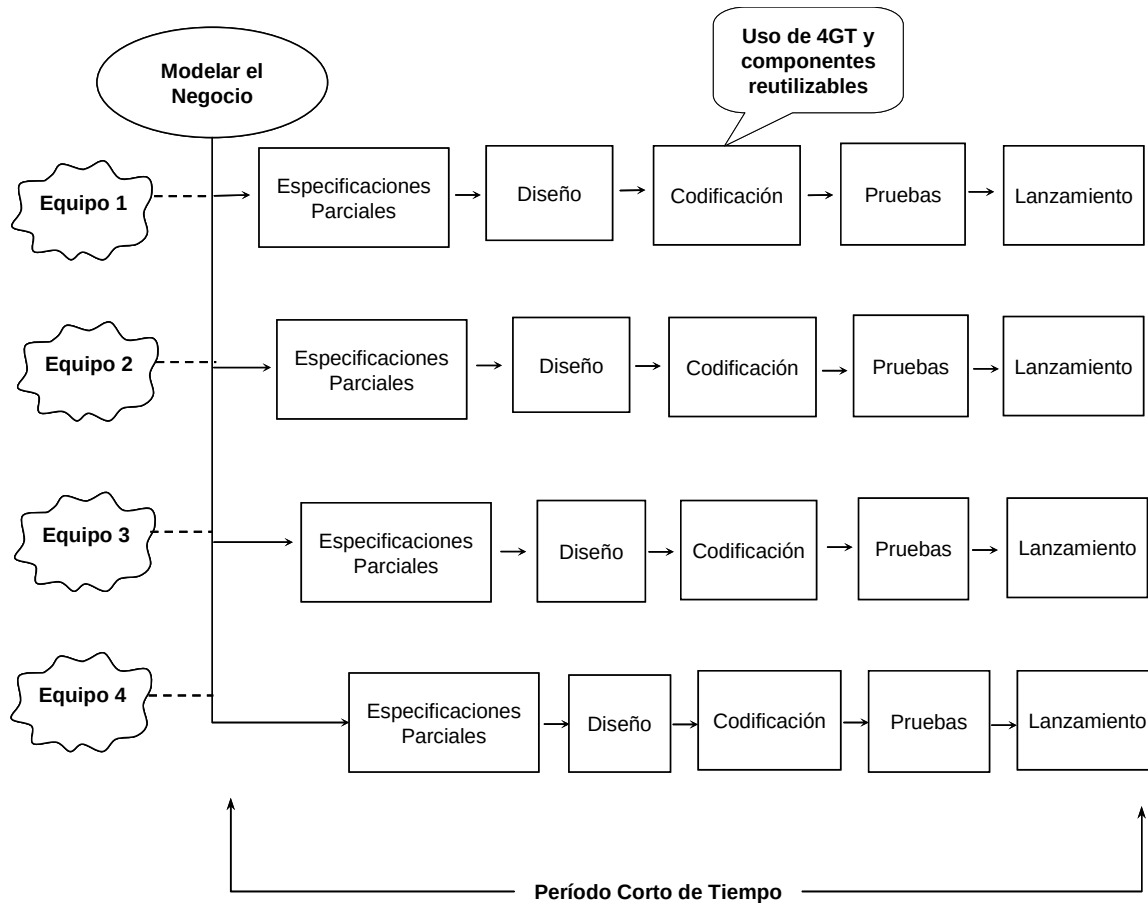
Esta situación es improbable. Pero tipifica la manera en que funciona el modelo de cascada. El proceso inicia con las especificaciones. Esto es realmente una recolección de información detallada, con la que el cliente no está familiarizado. En efecto, el documento es firmado aunque no se haya entendido apropiadamente.

El cliente ve el producto funcionando sólo después de que se ha completado totalmente. No es una sorpresa que los diseñadores de software vivan ante el temor de escuchar que su cliente no pidió esto o aquello.

Dado que las especificaciones existen sólo en el papel, el cliente puede no entender cómo se verá el producto final. El modelo de cascada, que depende crucialmente de 'especificaciones detalladas', puede llevar a la construcción de productos que realmente no corresponden a las necesidades del cliente.

7.2 El Modelo de Desarrollo Rápido de Aplicación - RAD

El desarrollo de software usando los modelos discutidos recientemente, toma mucho tiempo. Frecuentemente, existe un período de tiempo más corto para desarrollar software. Esta falta de tiempo condujo al desarrollo del modelo RAD (por las siglas en inglés, Rapid Application Development). El modelo RAD es un modelo secuencial lineal que trabaja con tiempos cortos de desarrollo. Este modelo, se ilustra en la Figura 1.2.

**Figura 1.2: Modelo RAD**

En este modelo, varios equipos participan en el proceso de desarrollo. Cada equipo maneja el desarrollo de una parte del sistema. El desarrollo a través de múltiples equipos ocurre concurrentemente.

Una organización típicamente tiene varias reglas de negocio y funciones que requieren sean usadas en el desarrollo del software. Usando el proceso de modelado del negocio, se puede generar un conjunto de especificaciones parciales que cubran una parte de estas funciones del negocio. Basados en estas especificaciones parciales, un equipo puede empezar el trabajo de desarrollo y llevarlo a través de las etapas de diseño, codificación, prueba y edición. Un aspecto significativo de este desarrollo son las partes de 'diseño' y 'codificación'.

La función de diseño se hace específicamente para promover el reciclaje de componentes existentes. La función de codificación usa técnicas de Cuarta Generación (Fourth Generation - 4G) para generar aplicaciones usando componentes de software reutilizables.

De la misma manera, otras especificaciones parciales son generadas concurrentemente y entregadas a otros equipos de desarrollo. Cada equipo trabaja con tiempos de desarrollo cortos, liberando productos parcialmente probados. En cada etapa de

liberación, los productos parciales de cada equipo de desarrollo son integrados, probados y liberados.

Las desventajas de este modelo se listan a continuación:

- El modelo RAD se ajusta mejor a un ciclo de desarrollo extremadamente corto, en los que los requerimientos son bien entendidos y el alcance del proyecto es restringido.
- Se requiere más desarrolladores, ya que múltiples equipos trabajan concurrentemente.
- Requiere del compromiso por parte de los desarrolladores, así como también de los clientes, para completar el sistema en un período de tiempo reducido.
- No es adecuado para sistemas que no pueden ser mantenidos apropiadamente.
- En caso de riesgo técnico alto, no es recomendable usar el modelo RAD, porque no se enfoca en detalles minuciosos.

Habiendo ya discutido unos cuantos modelos, se discute la necesidad de usar los modelos evolutivos en un proceso de desarrollo de software.

8. El Modelo de Creación de un Prototipo

El modelo de creación de un prototipo está basado en el modelo iterativo. Un cliente define un conjunto de objetivos generales para el software, pero no identifica entradas detalladas, procesamiento o requerimientos de salida. La necesidad del cliente de un 'diseño rápido' y la retroalimentación llevaron al surgimiento de este modelo.

En este modelo, el prototipo se desarrolla basado en requerimientos ya existentes. El desarrollo de este prototipo también experimenta diseño, codificación y prueba, pero cada una de estas fases no se hace de manera muy formal o exhaustiva. Al usar este prototipo el cliente puede obtener una percepción del sistema actual. El prototipo es aplicable a sistemas grandes y complicados donde los requerimientos no se conocen claramente. Este modelo se ilustra en la Figura 1.3.

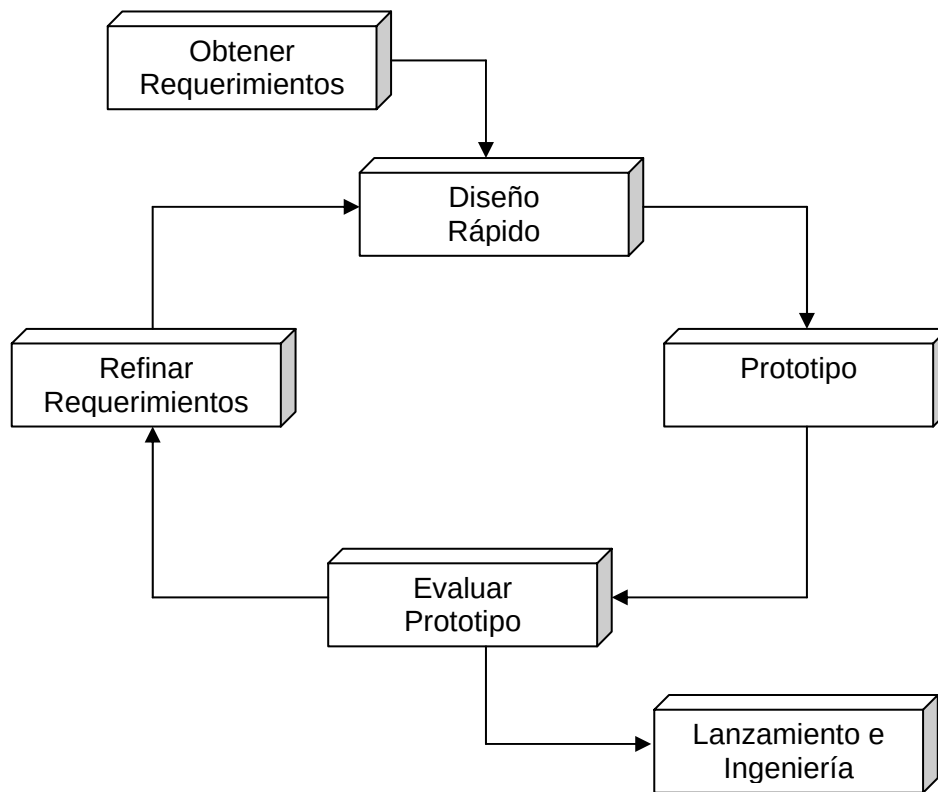


Figura 1.3: Modelo de Creación de un Prototipo

La ventaja de este modelo es que, como ocurre un diseño rápido sin las entradas y salidas detalladas, el cliente puede de alguna forma ver la salida. De esta manera, el cliente tiene la oportunidad de modificar los requerimientos temprano durante el proceso de desarrollo.

Las desventajas de este modelo son:

- El cliente puede confundirse y pensar que el prototipo es el producto final y mostrarse renuente a proceder con el desarrollo del producto. Esto resultará en un producto pobremente formado e incompleto.
- Debido a que el cliente tiene la oportunidad de ver el prototipo desarrollado, espera que el producto final esté listo en un período corto. En realidad, el proceso de desarrollo del producto puede tomar un tiempo mayor.
- Los desarrolladores no están seguros de qué hacer con el prototipo - ¿Deben deshacerse de él o usarlo como un componente reutilizable? La buena práctica sugiere que el prototipo debe ser descartado después de que los objetivos de creación de un prototipo se logren.

9. Modelos Evolutivos

A continuación se revisan algunas de las desventajas asociadas a los modelos presentados anteriormente.

- El modelo de cascada sólo puede ser aplicado para desarrollo mediante una línea directa, esto es, hasta que la secuencia lineal se complete, el sistema no está listo para su uso.
- El modelo de creación de prototipo no puede entregar un sistema completo en un modo operacional, porque ha sido diseñado ante todo para presentar los requerimientos del cliente de una manera más fácil y eficiente.

Otras restricciones también deben ser consideradas junto con estas limitaciones:

- Calendario ajustado para mercadeo.
- Dificultad para entender y desarrollar los detalles del producto claramente.
- Cambio en los requerimientos en el tiempo.

El modelo evolutivo sigue un enfoque no monótono y flexible. Por ello, no sólo está libre de las limitaciones anteriores, sino que también responde a los problemas mencionados anteriormente. Los pasos básicos del modelo evolutivo son los siguientes:

- Entregar algo al usuario.
- Recibir retroalimentación del usuario.
- Ajustar el diseño y objetivos basado a las realidades observadas.

En el modelo evolutivo, se encuentran principalmente los siguientes enfoques:

- Modelo Incremental.
- Modelo Espiral.

9.1 El Modelo Incremental

El enfoque incremental consiste en un desarrollo paso a paso, dónde las actividades de algunas etapas se posponen. Esto ayuda en la producción de algunos conjuntos útiles de funciones tempranamente en el desarrollo del proyecto. Aquí, cada etapa consiste de expandir incrementos de un producto de software operacional. Los incrementos pueden ser entregados al cliente a medida que se desarrollan. Los modelos se ilustran en la Figura 1.4.

Esto le agrega valor al modelo al obtener una retroalimentación anticipada del usuario. Cuando un incremento es entregado, no sólo se prueba el incremento, también es integrado con todos los incrementos previos. Entonces, se realiza la prueba de integración y se entrega el sistema parcial. Este enfoque incremental puede ser extendido a todas las etapas del ciclo de vida.

En este modelo, cada incremento es diseñado, codificado, probado, integrado y entregado por separado. En otras palabras, se puede seguir todavía el modelo de

cascada, pero sólo para cada incremento por separado. Los incrementos se desarrollan uno después de otro, basados en la retroalimentación recibida del cliente. A medida que los usuarios usan las partes entregadas, empiezan a entender mejor lo que necesitan realmente. Ya que cada incremento es más simple que el sistema completo, es más fácil predecir, administrar y controlar la utilización de recursos en el proyecto.

Las ventajas de este modelo son:

- En la mayoría de otros modelos, el trabajo obviamente no puede empezar hasta que los recursos de desarrollo estén disponibles. Sin embargo, no existe tal requerimiento para modelos incrementales. El trabajo puede iniciarse con incrementos específicos aún si todos los recursos de desarrollo no están disponibles. Así, este modelo es conveniente para ser usado cuando existe una disponibilidad limitada de recursos de desarrollo.
- El modelo incremental es apropiado para esos sistemas en los que es difícil establecer todos los requerimientos por anticipado. Existen casos en los que los requerimientos se conocen o perfilan sólo a través del proceso de aprendizaje, después de usar parte de un sistema.
- Un ejemplo típico es el diseño de interfaz de usuario para aplicaciones. En estas situaciones el modelo incremental es una excelente opción.

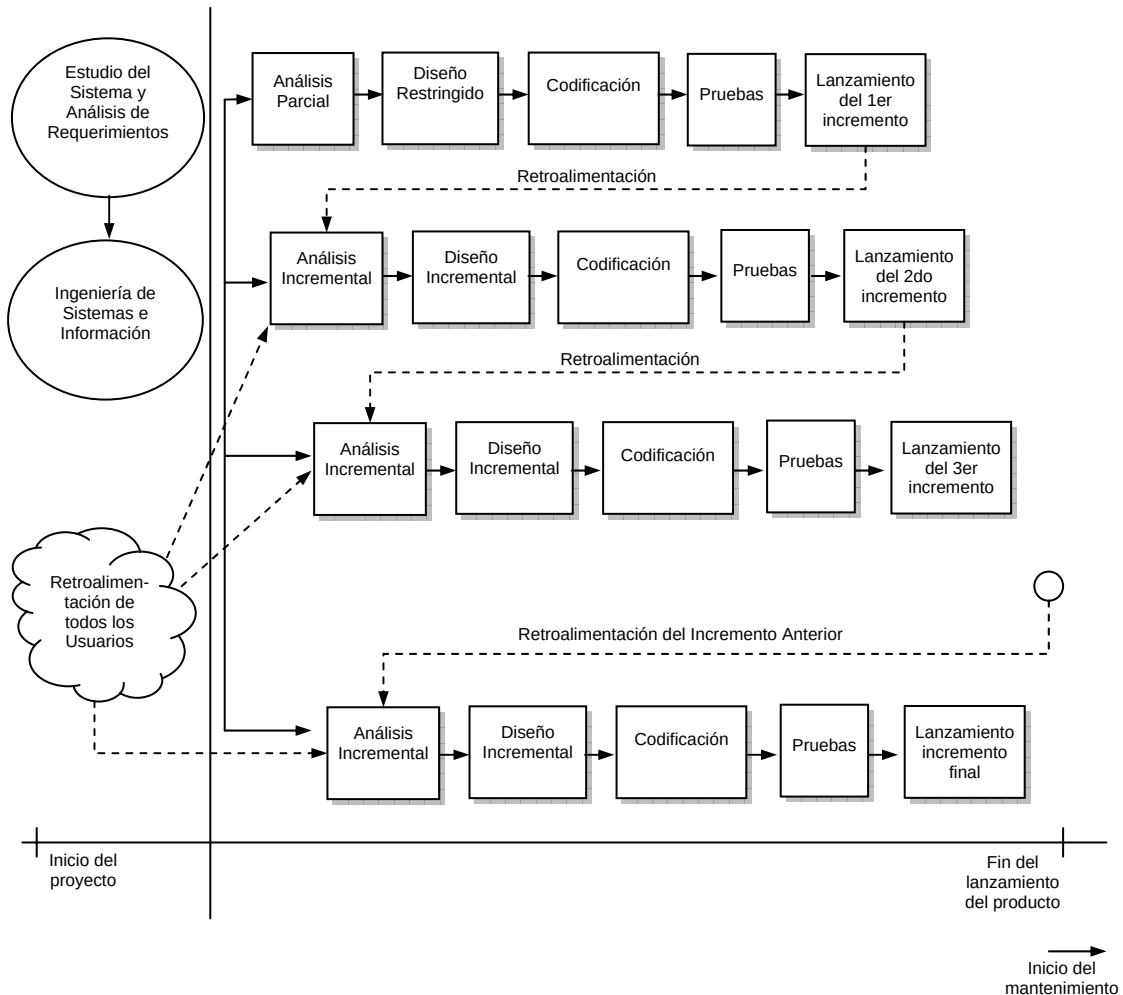


Figura 1.4: Modelo Incremental

Las desventajas de este modelo son las siguientes:

- Debido a la naturaleza evolutiva de este modelo, a medida que los requerimientos crecen, la arquitectura y el diseño original adoptados pueden cambiar dramáticamente. Esto puede resultar en el debilitamiento progresivo de la arquitectura y el diseño original. Por consiguiente, con la finalidad de causar una mínima alteración a la arquitectura y diseño en cualquier etapa, puede tomarse una vista restringida de los requerimientos. Es posible que esto no contribuya con el manejo de todos los requerimientos, necesidades y aspiraciones del cliente.
- Algunas organizaciones pueden percibir el modelo incremental como un trabajo poco sistemático en todas las etapas. En tales organizaciones, un cambio de actitud (para considerar que un enfoque incremental no tiene por qué, necesariamente acarrear las connotaciones negativas de un enfoque poco sistemático) es necesario antes de que el modelo pueda ser empleado sin problemas.

9.2 El Modelo Espiral

Los modelos secuenciales lineales, especialmente el modelo de cascada es tal vez el modelo más adoptado en la industria. Mientras que el modelo de cascada tiene ventajas concretas, también ha sido muy criticado. La mayor parte de esta crítica está enfocada en su naturaleza lineal y en su incapacidad de cambiar de curso frente al riesgo.

Barry Boehm originalmente propuso el modelo espiral en 1988. Este modelo incluye explícitamente el riesgo. El modelo espiral es un modelo de proceso evolutivo. Combina las fortalezas de la creación de prototipo con los beneficios del modelo secuencial lineal.

El modelo espiral se divide en un número de actividades de marco de trabajo, denominadas regiones de tarea. El número de regiones de tarea usualmente varía entre tres y seis. El modelo original Boehm era mucho más complejo que el simplificado con seis regiones de tarea, ilustrado en la Figura 1.5.



Figura 1.5: Modelo Espiral de Boehm con Seis Regiones de Tarea

Las seis regiones de tareas involucran:

Comunicación con el cliente – En esta fase, el desarrollador y el cliente conversan respecto a los requerimientos del sistema en varios niveles de detalle. Se consideran las tareas para establecer la comunicación.

Planificación del Proyecto – Se definen y documentan en esta fase, los recursos necesarios para el proyecto, el calendario o la agenda, entre otros. Además, se establecen las tareas requeridas para detallar el plan del proyecto.

Análisis de Riesgos – En esta fase las diversas facetas de riesgos (técnicos y gerenciales) se analizan desde un punto de vista de continuar el desarrollo del proyecto. Se establecen las tareas que identifican y ayudan a determinar los riesgos.

Ingeniería del Software – Las diversas tareas que conducen hacia la ingeniería del producto o la aplicación se ponen en marcha en esta fase. Se establecen las tareas de análisis, especificación y diseño.

Codificación, Prueba y Lanzamiento – Las tareas relacionadas a la codificación, prueba e instalación en el lado del cliente son todas definidas en esta fase.

Evaluación y retroalimentación del cliente – En esta fase el cliente evalúa el producto o aplicación y proporciona una retroalimentación. Se consideran las tareas que permiten al cliente evaluar el producto y proporcionar una retroalimentación estructurada.

Siguiendo este modelo, los equipos de desarrollo se mueven a través de la espiral en la dirección en la que giran las agujas del reloj, comenzando con el núcleo. Movimientos subsecuentes de una etapa a otra alrededor del espiral, se usan para desarrollar primero un prototipo, y después las versiones más avanzadas del software. El modelo espiral puede ser adaptado para aplicarse a lo largo de toda la vida del software. Este permanece operativo hasta que el software se completa, se entrega y finalmente se discontinúa su funcionamiento. Este modelo asocia la naturaleza iterativa de la creación de prototipo con los aspectos controlados de desarrollo del modelo secuencial lineal.

El modelo espiral es en realidad un enfoque realista al desarrollo de software. Siguiendo este modelo, el software evoluciona mientras el proceso progresa, por lo tanto, el desarrollador y el cliente, entienden y reaccionan mejor ante los riesgos en cada etapa del proceso evolutivo.

Las ventajas de este modelo son:

- Prueba su utilidad cuando es necesario un desarrollo rápido de versiones incrementales del software. Esto puede conducir a la entrega de sólo un prototipo en la primera iteración, pero en las iteraciones posteriores es posible entregar una versión completa de manera rápida.
- Este modelo puede ser usado incluso después que el software haya sido entregado. Otros modelos clásicos dejan de ser operativos una vez que el software ya está operativo, pero el modelo espiral puede ser aplicado a través del ciclo de vida del software.
- Utiliza la creación de prototipo como un mecanismo de reducción de los riesgos. Más aún, permite al desarrollador aplicar el enfoque de prototipo en cualquier etapa durante la evolución del producto.

Las desventajas de este modelo son:

- Demanda una experiencia considerable en determinación de riesgos por parte del cliente, así como del equipo de desarrollo en todas las etapas del proyecto.

- Los clientes generalmente son aprensivos respecto a sí la propuesta evolutiva puede ser controlada o no (por ejemplo si el proceso literalmente se saliera de control).

10. El Modelo de Ensamblado de Componentes

El modelo de ensamblado de componentes funciona de manera similar al modelo espiral, debido a que éste es iterativo. Sin embargo, utiliza un método específico en las regiones de tarea de 'ingeniería de software' y 'programación, prueba y lanzamiento'. Hace uso del enfoque orientado a objetos para el componente ensamblador, en vez de diseñar subsistemas y desarrollar programas para implementar estos desde el inicio. Los pasos involucrados en estas dos regiones de tareas, en el espiral, se listan a continuación:

- Considerar el diseño e identificar varios componentes que pueden ser reutilizados.
- Buscar los componentes en una librería de componentes o un repositorio similar que promueva la reutilización.
- Seleccionar los componentes requeridos de la librería o el repositorio.
- Retocar los componentes seleccionados si es necesario, para satisfacer las necesidades actuales.
- Construir componentes adicionales para ayudar a realizar el diseño en la etapa de la iteración.
- Ensamblar los componentes para ayudar a la entrega del producto o aplicación en la etapa de iteración.
- Adicionar los componentes retocados en la librería de reutilizables o en el repositorio.
- Adicionar los nuevos componentes creados en la librería de reutilizables o repositorio.
- Completar la iteración actual del componente ensamblador.

El modelo trabaja tal como lo hace el modelo espiral hasta que se alcanza la fase de ensamblado de componentes. Esto ocurre durante las partes de ingeniería y programación de las regiones de tareas. Una vez que los componentes son ensamblados, y el producto o aplicación en esa iteración es entregado, se reasume la travesía en el espiral. Específicamente, moverse a la siguiente región de tareas, por ejemplo, análisis de riesgos. Siempre que el movimiento a través de la espiral conduzca hacia la ingeniería de software y a las regiones de tareas de programación, son seguidos por el ensamblaje de componentes. Debe ser claro que el éxito del uso de este modelo depende de la habilidad para reutilizar los componentes de manera efectiva.

Las ventajas del modelo son:

- Proporciona un medio para reutilizar los componentes de manera efectiva. Normalmente, la reutilización de componentes durante el desarrollo puede

reducir el tiempo de desarrollo significativamente, quizás tanto como un 70%, como algunos estudios han demostrado.

- Los componentes reutilizables son generalmente componentes bien probados. Reutilizar dichos componentes contribuye a aumentar la confiabilidad de los sistemas ensamblados.

Las desventajas del modelo son:

- Contrario a la creencia popular, el reutilizar no es gratis, pues hay un costo significativo asociado a esto. Los componentes, por ejemplo, tienen que ser colocados en una librería o repositorio. Ellos tienen que ser entendidos y a veces reelaborados para que de esa forma sean convenientes para el nuevo entorno.
- Algunos ingenieros de software prefieren reescribir los componentes, además creen que pueden mejorar componentes anteriores, de tal forma que obvian las ventajas de la reutilización del componente.

11. Técnicas de Cuarta Generación (4GTs)

Estas técnicas involucran especificar las características del software en un alto nivel de abstracción. Diversas herramientas se utilizan para generar el código fuente. Las 4GTs se denominan también 'generadores de aplicaciones'. Hay unos cuantos entornos de desarrollo de software disponibles hoy en día que soporten 4GT. Muchos de los entornos que soportan 4GTs tienen las siguientes capacidades:

- Al menos un lenguaje no-procedural es normalmente utilizado para las consultas de base de datos.
- Diversas herramientas de generación de reportes que no requieren programación.
- Múltiples formas para el manejo de los datos, para los programadores y no programadores.
- Los mecanismos de definición e interacción de pantallas son fáciles de utilizar.
- Facilidad para la generación de código.

Como otros paradigmas las 4GTs, también se inician con las especificaciones y el análisis de los requerimientos. Pocas veces es posible utilizar las 4GTs para producir prototipos que requieren funcionalidad. Desde que se reconoce que los requerimientos pueden cambiar, las interacciones entre el desarrollador y el cliente continúan y permanecen activas en este método. Una vez que los requerimientos están claros, el equipo puede proceder con una estrategia de diseño. La implementación final empleará un Lenguaje de Cuarta Generación (4GL). Muchos 4GLs, como Focus, SQL, RAMIS y RPG-II están comercialmente disponibles hoy en día.

Las ventajas de esta técnica son:

- La generación automática de código para generar las salidas.
- El tiempo requerido para producir software es dramáticamente reducido para aplicaciones pequeñas e intermedias.

- Esta técnica, acoplada con las herramientas CASE (Ingeniería de Software asistida por computadora) y los generadores de código, pueden ayudar a resolver muchos problemas de software.
- Las desventajas de esta técnica son:
 - El código resultante generado de dichas herramientas es a veces ineficiente.
 - A pesar que la codificación se reduce en cierto grado por el uso de 4GTs, el método requiere más esfuerzo y tiempo en términos de análisis, diseño y pruebas.

Resumen

Ahora que ha completado esta unidad, usted debe ser capaz de:

- Explicar el software como producto y proceso.
- Definir la ingeniería de software y explicar su importancia.
- Discutir las ventajas y desventajas de los diversos modelos de procesamiento de software.
- Discutir los Modelos Evolutivos y Secuencial Lineal.

Unidad 1: Examen de Autoevaluación

- 1) ¿Cuáles de las siguientes son algunas características de cualquier producto de software?
 - a) El aprendizaje de su uso deber ser lo suficientemente sencillo por parte de personas no especialistas.
 - b) Deber ser eficiente en el uso de los recursos del sistema.
 - c) Debe cumplir algunas de las restricciones relacionadas al rendimiento.
 - d) Deber permitir agregar funcionalidad adicional sin efectos secundarios perjudiciales.

- 2) ¿Cuáles de las siguientes son algunas creencias incorrectas que prevalecen en la industria del software?
 - a) La calidad del software no puede ser determinada hasta que esté completamente construido.
 - b) El desarrollo del software puede llevarse a cabo como si fuese un proceso de ingeniería.
 - c) Añadir más programadores puede aliviar las presiones respecto a la entrega y a las fechas límite.
 - d) Los procesos y metodologías tienden a ayudar al mejor manejo del ciclo de vida del desarrollo de software.

- 3) ¿Cómo se puede describir el proceso del software?
 - a) Un proceso de software define un conjunto de tareas, las cuales tienen que ser realizadas para producir un software de alta calidad.
 - b) Un proceso de software es el enfoque tomado para el desarrollo del software.
 - c) Es el proceso que se sigue para construir, entregar y evolucionar el software desde la concepción de una idea hasta la entrega del sistema.
 - d) Es el tiempo transcurrido desde el requerimiento del cliente hasta la entrega del producto.

- 4) ¿Qué características presenta un proceso del software?
 - a) Comprensión.
 - b) Visibilidad.
 - c) Rapidez.
 - d) Entrega.

- 5) ¿Cuáles de los siguientes son modelos secuenciales lineales?
- a) El modelo de cascada.
 - b) El modelo RAD.
 - c) El modelo de la entrega incremental.
 - d) El modelo de creación de prototipos.
- 6) El modelo de creación de prototipo es un _____.
- a) Modelo secuencial lineal.
 - b) Un modelo iterativo.
 - c) Un modelo evolutivo.
 - d) Un modelo RAD.
- 7) ¿Cuáles de las siguientes son desventajas del modelo de cascada?
- a) Es difícil para el cliente tener todos los requerimientos de una vez, pero es una necesidad para este modelo.
 - b) Es difícil para el usuario anticipar si el sistema final construido de acuerdo a las especificaciones eventualmente cumple con sus requerimientos.
 - c) Cualquier cambio en la implementación puede causar confusión, ya que el modelo es inherentemente secuencial.
 - d) Cualquier versión trabajada puede ser vista demasiado tarde, y por lo tanto en el caso de un error serio, éste tiene que ser rastreado hasta la fase de requerimientos.
- 8) ¿Cuáles de los siguientes son modelos evolutivos?
- a) Modelo incremental.
 - b) Modelo espiral.
 - c) Modelo de ensamblado de componentes.
 - d) Modelo RAD.

- 9) ¿Cuáles de las siguientes son ventajas del modelo espiral?
- a) Resulta bueno cuando se requiere un rápido desarrollo de versiones incrementales del software. Esto puede significar la entrega de sólo un prototipo en las primeras iteraciones, pero en las siguientes iteraciones será más probable entregar una versión completa rápidamente.
 - b) Este modelo se puede usar aún después de que el software es entregado. Otros modelos clásicos dejan de ser operativos una vez que el software está operativo, pero el modelo espiral puede ser aplicado a través del ciclo de vida del software.
 - c) Utiliza los prototipos como un mecanismo de reducción de riesgos. Además, permite al desarrollador aplicar el enfoque de prototipo en cualquier etapa en la evolución del producto.
 - d) Provee la capacidad de verificar formalmente los requerimientos del cliente antes de proceder con las otras fases.
- 10) ¿Cuáles son los pasos básicos de los modelos evolutivos?
- a) Ajustar el diseño y objetivos basado en las realidades observadas.
 - b) Entregar algo al usuario.
 - c) Ajustar el diseño y objetivos según los requerimientos iniciales.
 - d) Recibir retroalimentación del usuario.

Respuestas de la Unidad 1: Examen de Autoevaluación

- 1) a, b, c y d
- 2) a y c
- 3) a, b y c
- 4) a, b y c
- 5) a y b
- 6) b
- 7) a, b, c y d
- 8) a, b, c y d
- 9) a, b y c
- 10) a, b y d

Unidad 2: Especificación de los Requerimientos

Objetivos de Aprendizaje

Al finalizar esta unidad, usted será capaz de:

- Discutir la naturaleza y la importancia de los requerimientos.
- Definir los requerimientos de software y el término SRS.
- Explicar en que consisten las Especificaciones de Requerimientos de Software (ERS).
- Describir las actividades del análisis de requerimientos.
- Describir las funciones y componentes de una SRS.

1. Introducción.

En cualquier proyecto de desarrollo de software, especificar los requerimientos es un paso extremadamente crucial y contribuye significativamente al éxito del proyecto. En esta unidad, se discute la especificación de los requerimientos de software. Para comprender la importancia y dificultad de este proceso, se va a considerar un escenario imaginario como el que se presenta a continuación.

Imagine una situación de una reunión que está teniendo lugar en una sala de conferencias de una gran corporación, que quiere transformarse a sí misma en una oficina con la menor cantidad de papeles posible. Ud está invitado a asistir a la reunión como consultor o ingeniero especialista en requerimientos. Debido al congestionamiento del tráfico en la autopista, llega unos minutos tarde. La reunión ya ha comenzado, y está algo caótica, con varias personas tratando que sus opiniones sean escuchadas al mismo tiempo.

Confundido, pero impaciente por marcar la diferencia, se sienta y trata de empaparse por varios minutos de la reunión. El propósito principal de esta reunión es obtener una comprensión preliminar de los requerimientos del sistema, hardware y software, que ayudarán a implementar una oficina con menos papel en la corporación. Rápidamente nota que muchos de ellos están hablando de diferentes cosas a diferentes niveles y en diferentes ámbitos.

A continuación una síntesis de la conversación:

Susan Kessler (VP, Administración): Mi departamento está virtualmente debajo de una cantidad de papeles. ¡Verdaderamente necesitamos una oficina con menos papeleo!

Adam Kahn (VP, Operaciones): Susan, Esa es una frase muy general. Necesitamos ser realmente precisos en este punto. Lo que sugiero es que entrevistemos a una gran parte de nuestros empleados, circulemos un memorando entre todos los empleados pidiendo una lista de lo que desean y analizar todos los productos que ofrecen una oficina con menos papel.

Layla Abu (VP, Procuraduría): Yo estoy en desacuerdo con Adam. Lo que él sugiere será una tremenda pérdida de tiempo. Yo he trabajado para FlexiCorp International, New York, por siete años antes de este empleo. Estas personas implementaron una oficina con menos papel como se hacía en el inicio de los noventa. Yo, personalmente, he estudiado varios productos buenos de este tipo en el mercado y mi sugerencia es que hagamos una corta lista de los cinco mejores vendedores de soluciones, los invitemos para que nos hagan una presentación y seleccionemos al que contribuya mejor a las metas de nuestra empresa.

Shimon Netanyahu (VP, Negocios Internacionales): No es sólo un producto o una solución lo que nosotros estamos buscando. Lo que nosotros buscamos debe ser la pieza de trabajo de más clase, que no sólo mejore la productividad y nuestras metas, sino también que contribuya a la imagen de la corporación. Así que no sólo hagamos

una lista de los servicios que debe proporcionar el producto y solución, sino también indicar claramente cómo deben interactuar los usuarios entre sí con el producto.

Viral Vora (VP, Sistemas): Miren chicos, Yo he trabajado en Análisis y Diseño de Sistemas Estructurados (ADSE) los últimos 14 años. Esta es la mejor técnica y la más usada para el estudio de sistemas. Sugiero que cada uno de los miembros de este grupo asista a un curso corto de un día en ADSE y regresemos luego. Entonces todos podremos usar ADSE, hablar en un lenguaje en el que nos entendamos, que corte el parloteo y la pérdida de tiempo.

Kim Versendaal (VP, Tecnología): Esto es tan irreal. ¡Están hablando de que los vendedores vayan hacia ADSE! ¿Acaso no saben que el Análisis y Diseño Orientado a Objetos (ADOO) y UML son los últimos métodos que se están utilizando? No podemos definir claramente nuestros requerimientos a menos que vayamos por la ruta del ADOO.

David McGrath (VP, Finanzas): Yo sólo quiero recordarles que cualquier cosa que hagamos, debe estar en un límite de \$56 millones. Debemos seleccionar algo que este dentro de nuestro presupuesto o que nos ahorre unos cuantos millones.

Philip Chang (VP, Investigación y Desarrollo): Yo he investigado sobre oficinas con menos papel. Todo es demasiado confuso y es como perseguir un sueño. Lanzaremos por la borda \$56 millones y algo más. Al final tendremos que trabajar con más papeles que antes. Existe una necesidad real de dinero para hacer un adelanto importante en el proyecto del genoma humano en el que estamos trabajando. Sugiero que desechemos este esfuerzo y canalicemos \$12 millones en nuestra investigación del genoma. Esto nos daría un valor de algunos billones de dólares, que ningún proyecto nos brindará para reducir papel.

Como se puede observar, aquí hay un caos. Cada uno tiene diferentes requerimientos y quiere que el sistema le brinde las soluciones. Pero ¿Cuáles son estos requerimientos? ¿Cómo son analizados? ¿Por qué el análisis de los requerimientos debe ser considerado importante? ¿Cuál es el resultado del estudio de los requerimientos? ¿Quién debería hacerlo? ¿Cómo debería hacerse?

Esta unidad trata de dar respuestas a algunas de estas preguntas.

2. ¿Qué son los Requerimientos?

El estándar 729 de la IEEE define *requerimiento* de las dos siguientes formas:

- Una condición o capacidad requerida por un usuario para resolver un problema o para alcanzar un objetivo.
- Una condición o capacidad que debe ser satisfecha o debe poseer un sistema para satisfacer un contrato, estándar, especificación u otro documento formalmente impuesto.

Sin embargo, ninguna de las dos definiciones es suficientemente específica para sugerir claramente qué debe ser incluido en los requerimientos y qué debe ser excluido de

ellos. En el caso del software, la fase de requerimientos se inicia cuando uno o ambos de los siguientes hechos ocurren:

- Un problema existe y quizás requiera una solución basada en un software.
- Hay un alcance para crear un software basado en una idea.

El problema por el cual se requiere una solución a través de un software puede ser orientado a aplicaciones, orientado al negocio, orientado a la mejora del producto o incluso una genérica multipropósito. Esta naturaleza polifacética del problema reconocido se representa en la Figura 2.1.

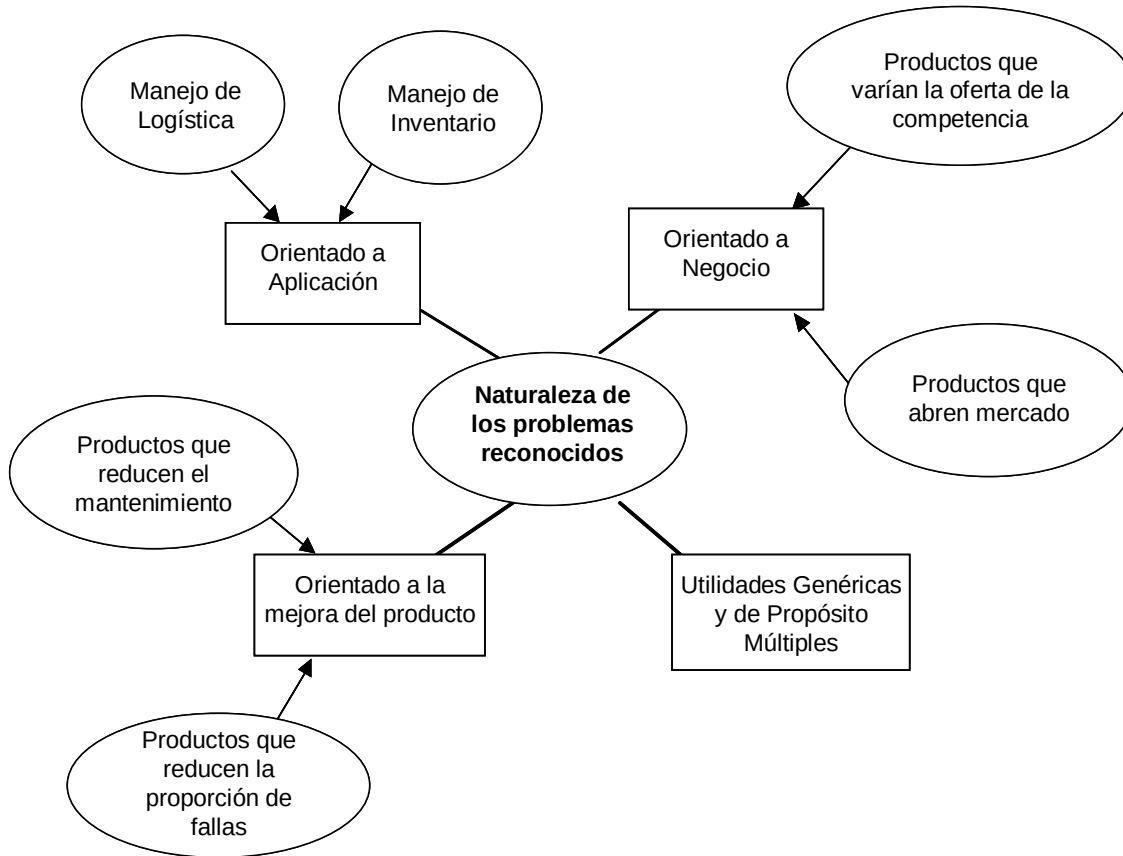


Figura 2.1: Naturaleza Polifacética de los Problemas

Un sistema de software exhibe un comportamiento que puede ser visto por un observador externo, tal como un usuario. Este comportamiento externo es usualmente lo que caracteriza una funcionalidad específica del software. Es diferente del comportamiento interno que se observa, a través del uso de memoria durante la ejecución o el estado de variables usadas. Una vez que se es capaz de generar el comportamiento externo completo del software a ser desarrollado, se puede decir que la fase de requerimientos de software está cerca de su final. La descripción completa del comportamiento externo del software se documenta. Este especifica todas las interfaces entre el software y el entorno del software. El entorno del software consiste de

computadoras, productos de hardware, otros productos de software y los usuarios humanos. Esta descripción completa se registra en una especificación de requerimientos de software (SRS).

Una SRS es un documento que contiene una descripción completa de lo que el software hará, sin describir cómo lo hará.

Nota: Es importante entender que los requerimientos son una declaración de 'qué' será hecho y no 'cómo' deben hacerse las cosas.

Una SRS es un documento que contiene una descripción completa del comportamiento externo de un producto de software y puede ser detallado o general. Esto depende de la naturaleza del software a ser desarrollado y también de la persona que escribe la SRS.

Para comprender los puntos del párrafo anterior de manera más clara, se presentan varios escenarios posibles.

- **Escenario 1:** Una agencia gubernamental, por ejemplo la Organización de Investigación Espacial Xouter (XSRO), desea procurar un encargo de piezas de equipo (con el software incluido) a través de una licitación abierta y debe escribir una SRS para especificar sus necesidades.
- **Escenario 2:** Un contratista gubernamental, por ejemplo Company X, ha ganado una licitación competitiva para un contrato y debe escribir una SRS para especificar el sistema que construirá para la agencia gubernamental.

Se va a considerar cada escenario y ver cómo la SRS puede ser detallada o general, dependiendo de la naturaleza de cada escenario.

Escenario 1: En el primer escenario, es necesario para la SRS ser lo suficientemente general para facilitar a las compañías potenciales interesadas en hacer una oferta para el proyecto y fomentar la competencia entre los postulantes potenciales, pero lo suficientemente específico para asegurar que cualquier sistema que realmente cumpla con las especificaciones satisfaga la necesidad real. Vea el ejemplo siguiente:

'El sistema le proporcionará a los pilotos en los aviones la habilidad de ver en la oscuridad desde altitudes elevadas'.

Los proveedores potenciales de soluciones en una gran variedad de maneras pueden satisfacer este requerimiento.

Por ejemplo, un contratista puede ofrecer un sistema de visión infrarroja con múltiples facilidades para guiar a través de órdenes generadas por la voz, este será a través de un casco visor. Otro puede proponer un proyector que refleja los datos requeridos en una pantalla. Un tercero puede proponer simplemente unos lentes infrarrojos sencillos pero rentables. Claro, un sistema que requiere que el avión dirija una luz halógena de alta potencia que apunte en el blanco a ser visto no es uno aceptable, sobre todo si el avión es de combate y vuela sobre el territorio enemigo.

Escenario 2: En el segundo escenario, el contratista, la CompanyX, tiene que ser mucho más específica de manera de comunicar a la agencia gubernamental lo que será el sistema, debido a que es solamente con estos detalles que la agencia puede planear los componentes específicos del avión y las interfaces entre los sistemas relacionados. El ser específico detallando los requisitos, asegura también a la agencia gubernamental que el contratista entiende el problema realmente, que está construyendo el sistema correcto y que el sistema puede ser probado. Sin embargo, el personal que no está familiarizado con la jerga de la computación debe entender esta especificidad. En este escenario, clientes, escritores técnicos, diseñadores, analistas y verificadores deben proporcionar la entrada en el proceso de escribir la SRS.

Habiendo discutido sobre los requerimientos y su importancia, a continuación se discute sobre el análisis del problema y la descripción del producto.

3. Análisis del Problema y Descripción del Producto

Hay dos tipos diferentes de actividades que ocurren durante la fase de requerimientos - el análisis del problema y la descripción del producto.

Durante el análisis del problema, los analistas dedican una cantidad considerable de tiempo haciendo lo siguiente:

- Tormenta de ideas.
- Dirigir las entrevistas con los involucrados con el sistema.
- Obtener la información de las personas más familiarizadas con ese dominio.

En esta fase se tiene una gran cantidad de información. Un desafío es organizar la gama de información. Otra tarea es identificar todas las posibles restricciones en la solución del problema. Esto puede conducir a que algunos de los requerimientos sean incompatibles con otro conjunto de requerimientos. El problema más grande durante este tiempo es encontrar las maneras de compensar las restricciones y organizar la gama de información. El análisis del problema debe llevarse a cabo hasta que se alcance una comprensión completa del problema.

Cuando se hace la descripción del producto, se describe el comportamiento externo del software en un documento. En esta fase, se organizan las diferentes ideas, se resuelven ciertas visiones contradictorias y se eliminan las inconsistencias. Las ambigüedades, si hubiera alguna, también se eliminan.

Las fases de análisis del problema y descripción del producto no son fases mutuamente exclusivas, ni son secuenciales en el tiempo. Ciertas situaciones pueden requerir un pequeño o ningún análisis del problema. En el caso de problemas bien-entendidos, por ejemplo, hay poca necesidad del análisis del problema. Hay situaciones en que la naturaleza del propio problema no se entiende claramente. En tal caso, no es aconsejable realizar un análisis del problema total antes de escribir la SRS. En vez de eso, el análisis del problema se hace en una serie de pasos. A medida que las partes del problema se entienden bien, las correspondientes secciones de la SRS se escriben. Finalmente, el último aspecto del problema se analiza y la SRS se completa.

Las próximas secciones describen la importancia de la fase de requerimientos en el proceso de desarrollo del software.

4. Importancia de la Fase de Requerimientos

En la Unidad 1 - *Fundamentos de Ingeniería de Software* de este volumen, se definió la fase de los requerimientos del software y se estableció su contexto respecto al resto del ciclo de vida de desarrollo del sistema. El propósito de esta sección, es explicar por qué hacer un buen trabajo al definir y especificar los requerimientos del software, no sólo vale la pena sino que también es rentable. La importancia de especificar los requerimientos del software está determinada por lo siguiente:

- Los errores, si hay alguno, que se esconden sin percatarse durante la fase de especificación de requerimientos, son difíciles de localizar, manejar y arreglar si se descubren durante las fases posteriores del desarrollo. Normalmente, los errores que se descubren más tarde en el ciclo de vida del proyecto de desarrollo de software, tendrán un costo más elevado para repararlos.
- Entender los requerimientos es una fase crucial, ya que cualquier error que se oculta en esta fase tiende a propagarse a otras fases como diseño y codificación. Aquí, estos errores tienden a sufrir transformaciones que hacen que descubrir la fuente original del error sea difícil.
- La fase de requerimientos involucra mucha comunicación entre los involucrados (stakeholders) con el sistema y los desarrolladores. Esta es la fase más propensa a que se introduzcan varios errores.
- Se deben tomar ciertas precauciones en la fase de requerimientos para asegurar que las especificaciones estén libre de hechos (aseveraciones) incorrectas, de omisión de detalles, inconsistencias y ambigüedades en los requerimientos.
- Un aumento significativo de costo y tiempo en los proyectos de desarrollo de software puede atribuirse a los errores e insuficiencias en la fase de análisis de requerimientos.

5. Ingeniería de Requerimientos

Uno de los desafíos principales de cualquier ejercicio de ingeniería de sistemas es asegurar que se ha especificado un sistema que satisface las necesidades del cliente y asegurar que cumple las expectativas. Pero, ¿cómo se asegura esto? Aunque no es una tarea fácil, décadas de investigación, desarrollo, aprendizaje y experiencia han demostrado que seguir buenas metodologías de ingeniería de requerimientos es de gran ayuda. La ingeniería de requerimientos pone a disposición un mecanismo para llevar a cabo un conjunto de actividades, las cuales se muestran en la Figura 2.2.

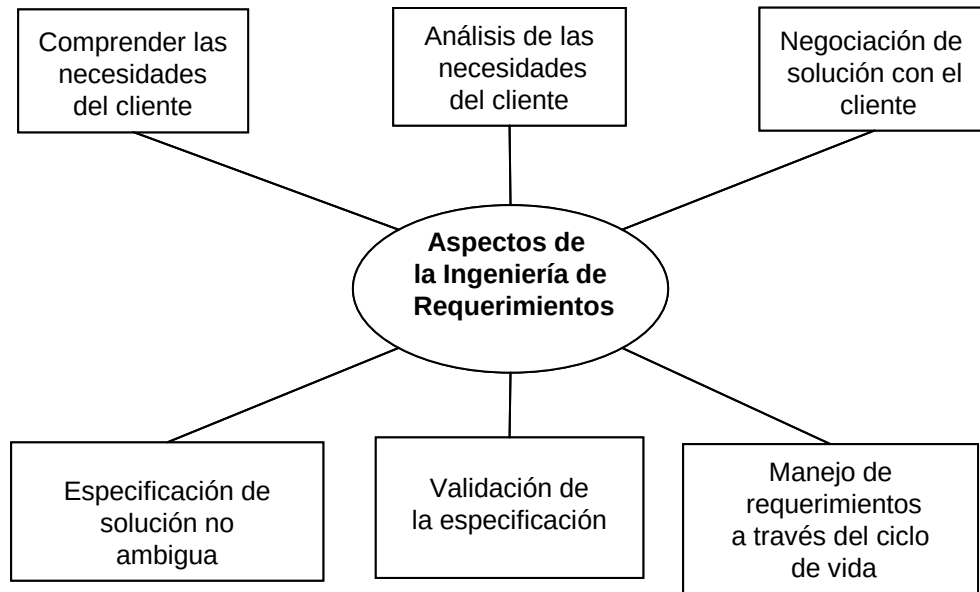


Figura 2.2: Aspectos de la Ingeniería de Requerimientos

De hecho, el proceso de la ingeniería de requerimientos tiene los siguientes pasos, que se muestran en la Figura 2.3.

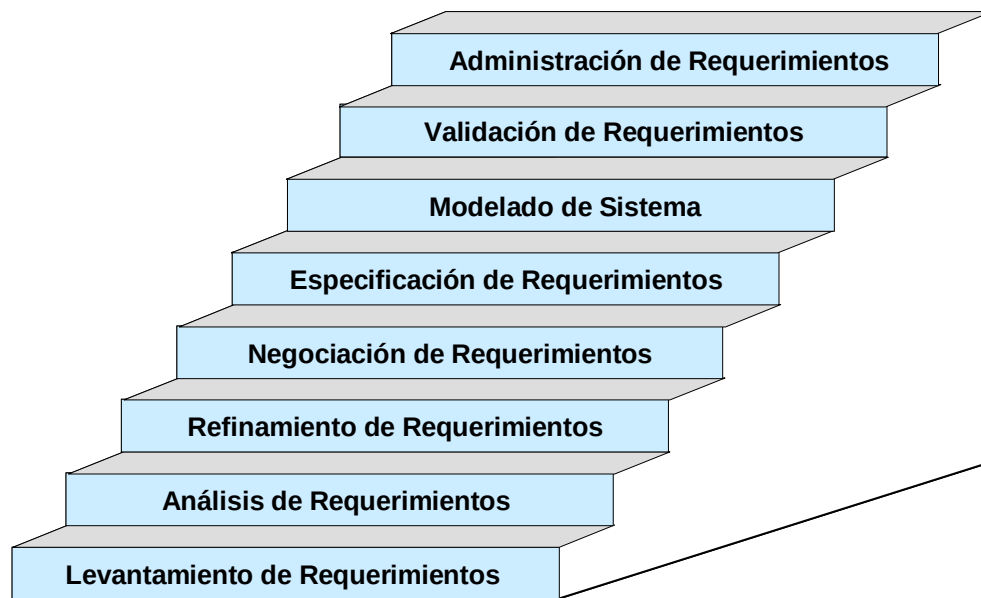


Figura 2.3: Pasos Involucrados en la Ingeniería de Requerimientos

A continuación se explican brevemente algunos de los pasos de la ingeniería de requerimientos.

5.1 Levantamiento de Requerimientos

Levantamiento es el proceso de recibir un conjunto de requerimientos del cliente, el usuario y la gerencia. Estos participantes se hacen las siguientes preguntas. Aunque estas preguntas parecen relativamente simples, realmente es muy difícil obtener la información específica de ellas.

- ¿Cuáles son los objetivos del sistema o producto?
- ¿Qué debe lograr el producto o sistema?
- ¿Cómo ayuda el sistema o producto en las necesidades del negocio?
- ¿Cómo usaría usted el sistema o producto en el día a día?

El proceso de obtención de requerimientos es considerado muy difícil en la práctica debido a los problemas que se muestran en la Figura 2.4.

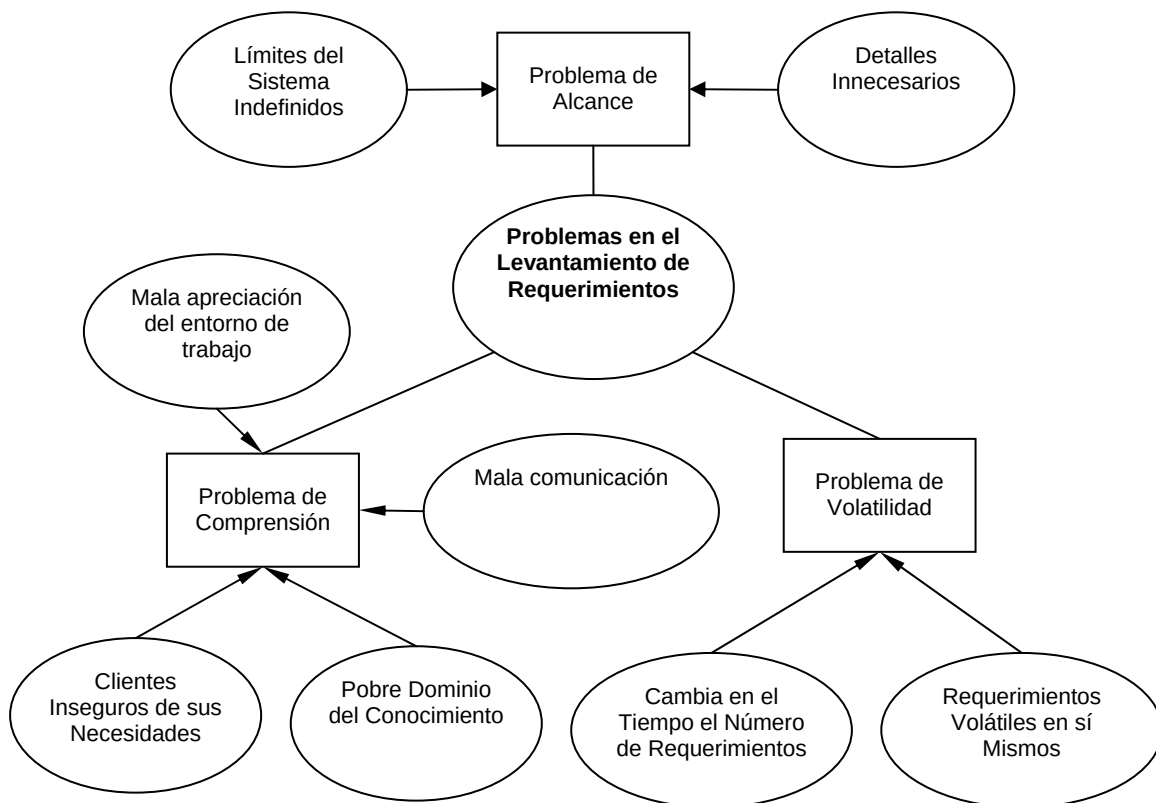


Figura 2.4: Problemas en el Levantamiento de Requerimientos

Después del proceso de levantamiento de requerimientos, se obtiene la siguiente información:

- Un documento especificando la necesidad y factibilidad del sistema a construirse.

- Alcance del sistema o producto a desarrollarse.
- Lista de todos los involucrados que participaron en el levantamiento de requerimientos.
- Un documento describiendo el entorno tecnológico para el sistema en consideración.
- Una lista de todos los requerimientos organizados por función.
- Un conjunto de sentencias que especifican las restricciones de dominio que aplican a cada requerimiento.
- Especificación de los casos de uso o escenarios de uso.

De manera de asegurar la precisión de estos documentos, los involucrados que participaron en el proceso de levantamiento de requerimientos deben revisar cada uno de los ellos.

5.2 Análisis de Requerimientos

Ahora, se tiene una cantidad de requerimientos que deben ser analizados, clasificados y organizados. Estos requerimientos se analizan uno con respecto al otro y se exploran atributos como consistencia, completitud, ambigüedad y prioridad. Por ejemplo, si se declara un requerimiento como "el software proporcionará todas las salidas sólo en impresoras láser" y otro requerimiento establece "el software debe ser capaz de apoyar una amplia variedad de dispositivos de salida", hay algo de ambigüedad así como de inconsistencia. Se pueden hacer las siguientes preguntas como una especie de guía básica para realizar el análisis de requerimientos:

- ¿Es cada requerimiento consistente con los objetivos globales para los cuales el software está desarrollándose?
- ¿Proporcionan cada uno de los requerimientos suficientes detalles para que se pueda pasar a la próxima fase de trabajo?
- ¿Cuáles de los requerimientos especificados son absolutamente necesarios y cuáles de ellos son puramente estéticos?
- ¿Existe un alcance bien definido que proporcione un límite en cada requerimiento?
- ¿Están completos cada uno de los requerimientos en cuanto a descripción, alcance, especificidad, etc.?
- ¿Está el conjunto de todos los requerimientos completo y libre de ambigüedad?
- ¿Es posible identificar la fuente (la persona en la organización) de cada requerimiento?
- ¿Han sido eliminados todos los requerimientos contradictorios?
- ¿Hay un método específico para probar si cada requerimiento es implementado correctamente?

Se debe recordar que los clientes piden a menudo más cosas a ser logradas de las que son posibles. Por consiguiente, es necesario pedirles que le den prioridad a sus

requerimientos. Es necesario negociar con los clientes cuando hay requerimientos contradictorios. En esta fase, se atan las figuras de riesgo con cada requerimiento. Se establece un cálculo estimado del costo de los requerimientos en el costo del proyecto y un cronograma de entrega. Es una especie de proceso iterativo mediante el cual los requerimientos inconsistentes y contradictorios se eliminan, y las partes se ponen de acuerdo en una plataforma común de requerimientos.

5.3 Especificación de Requerimientos

Una especificación de requerimiento puede ser uno de lo siguientes:

- Un documento escrito.
- Un modelo gráfico.
- Un modelo formal (usando base matemática).
- Una colección de casos de uso.
- Un conjunto de prototipos.

Incluso puede ser cualquier combinación de éstos.

5.4 Modelado del Sistema

Modelar el sistema proporciona un mecanismo para entenderlo mejor y obtener un análisis más profundo del mismo. Sobre todo para sistemas que están siendo investigados por primera vez (por el analista o la organización), el modelar sistemas es altamente recomendado. Esto involucra identificar las diferentes entidades en el sistema, los atributos de las diferentes entidades, la naturaleza de las relaciones entre las entidades, los diferentes eventos que ocurren en el sistema y cómo el sistema cambia su estado cuando ocurren los eventos. Hay varias técnicas para modelar sistemas. Algunas de estas técnicas son modelados físicos, modelados por diagramas de flujo, modelado de la relación de cambio evento-estado, etc. Las técnicas pueden emplear los métodos gráficos simples hasta técnicas de modelados basadas en matemáticas complejas. De tal forma que el modelar el sistema es un paso recomendado para tener un mejor entendimiento del mismo. El detalle de varias técnicas de modelado de sistemas está fuera del alcance de este curso.

5.5 Validación de Requerimientos

En la fase de validación de requerimientos, la especificación de requerimientos está sujeta a algunos parámetros de aseguramiento de calidad. La evaluación se hace para asegurar que la especificación de requerimientos tenga algunas o todas las cualidades que se muestran en la Figura 2.5.



Figura 2.5: Cualidades de la Especificación de Requerimientos

Uno de los mecanismos usados para la validación es la revisión técnica formal. Aquí, la especificación intenta erradicar los errores. Se rastrean conflictos, inconsistencias, omisiones y ambigüedades. Se pueden hacer las siguientes preguntas como una serie de pautas básicas:

- ¿Es ambiguo alguno de los requerimientos?
- ¿Se ha comprobado y verificado la fuente de cada requerimiento?
- ¿Cada requerimiento está limitado en su alcance?
- ¿La manera en que los requerimientos se relacionan entre sí está clara?
- ¿Es posible probar cada uno de los requerimientos?
- ¿Los requerimientos son legibles y entendibles?
- ¿Hay suficientes índices y referencias cruzadas para facilitar la ubicación fácil de puntos o ítems en la SRS?
- ¿Los requerimientos relacionados con el desempeño están claramente establecidos?

En la práctica, se puede necesitar trabajar con una lista de control más exhaustiva de preguntas para la validación.

5.6 Administración de Requerimientos

La administración de requerimientos se ocupa de un conjunto de actividades conectadas con el control e identificación de requerimientos, además del rastreo de los requerimientos durante la implementación. También, se ocupa de manejar los cambios en los requerimientos. La administración de requerimientos también se estudia bajo el tópico de administración de la configuración.

6. Conducir el Estudio de los Requerimientos

Un estudio de requerimientos comienza con la comunicación entre el cliente y el ingeniero de requerimientos. La tarea del cliente es comunicar los requerimientos (listas de necesidades, aspiraciones y deseos) al ingeniero de requerimientos. La tarea del

ingeniero de requerimientos es descubrir todos los requerimientos, estructurarlos y escribir la especificación de requerimientos después de entender las necesidades del cliente. Esta comunicación no es tan fácil como aparenta y se llena a menudo de problemas de varias clases.

Los problemas en la comunicación pueden ocurrir si el dominio a ser explorado requiere experiencia para entenderlo, si el lenguaje de la comunicación tiene brechas, si la efectividad de la comunicación es pobre, etc.

Las siguientes son algunas de las actividades principales que deben ser parte del estudio de los requerimientos. Observe que estas actividades no necesitan ser emprendidas en una manera secuencial.

- Examinar el escenario.
- Arreglar la primera reunión con los clientes.
- Procurar entender la misión de la organización.
- Procurar familiarizarse con la estructura de la organización e identificar a los diversos involucrados con el sistema.
- Utilizar la reunión para establecer credibilidad con los clientes y ganar su confianza.
- Elaborar un documento preliminar que resuma los diferentes pasos en la operación del sistema actual describiendo claramente los límites del sistema, las entradas, salidas e interfaces del mismo.
- Describir claramente los límites del sistema, entradas, salidas e interfaces.
- Desarrollar un documento que indique claramente el problema.
- Recopilar todos los datos necesarios y relevantes de la organización.
- Verificar la comprensión del problema revisando los documentos que se han preparado con los clientes.
- Poner a la disponibilidad de la gerencia el informe de evaluación del escenario.
- Estudiar detalladamente el sistema existente.
- Conducir las entrevistas con profundidad con los diferentes involucrados para los aspectos específicos del sistema actual.
- Desarrollar un documento que describa el sistema actual en una manera comprensible, pero sin ningún detalle de implementación.
- Desarrollar una clara comprensión del flujo de datos, así como la lógica operacional del sistema y de un conjunto preliminar de requerimientos basados en el estudio del sistema existente.
- Desarrollar el documento SRS basado en los nuevos requerimientos.
- La definición del problema y el estudio del sistema existente, establecen las bases para la enumeración del nuevo conjunto de requerimientos.
- Especificar la funcionalidad del conjunto de requerimientos.
- Especificar los requerimientos referentes al desempeño y restricciones.

Es posible que incluso con todas estas actividades mencionadas anteriormente, sea difícil descubrir los requerimientos en ciertos casos. Esto sucede cuando los mismos clientes están inseguros de sus necesidades y no son capaces de articularlas. En tales casos, la técnica de creación de prototipo puede ayudar a que el cliente se enfoque mejor en los requerimientos. Este tema se discute a continuación.

6.1 Creación de prototipo y los requerimientos

Se ha visto anteriormente el proceso de creación de prototipo en la Unidad 1 - *Fundamentos de la Ingeniería de Software* en este volumen. Se construye un prototipo basado en un conjunto limitado de requerimientos. En la fase de los requerimientos, un prototipo puede ser construido y dado a un usuario o a un cliente potencial para trabajar con él, para hacer lo siguiente:

- Determinar la viabilidad de un requerimiento.
- Validar que una función en particular es realmente necesaria.
- Descubrir los requerimientos faltantes.
- Determinar la viabilidad de una interfaz de usuario.

Una vez que haya servido a los propósitos mencionados, el prototipo es usualmente desechado, y por lo tanto, se denomina prototipo desechable. Con el uso de un prototipo, los clientes podrán proporcionar requerimientos más enfocados. El equipo que está conduciendo el estudio de los requerimientos puede ahora proceder a terminar la SRS.

7. Usos de la SRS

Uno de los resultados principales de la fase de requerimientos es la SRS. Usualmente, la culminación del desarrollo de la SRS que es validada, señala el final de la fase de requerimientos aunque la parte de gerencia de los requerimientos continúa durante la implementación y quizás más allá de ésta.

Una SRS puede ser escrita por un cliente del sistema o por el desarrollador del sistema. Claramente, las perspectivas y los usos son absolutamente diferentes cuando estas dos diferentes entidades preparan la SRS.

Cuando los clientes preparan una SRS, su propósito principal es articular y definir sus necesidades. Generalmente, estas necesidades según lo perciben los diferentes usuarios del sistema serán diferentes, grandes y más amplias en alcance y cobertura.

Por otra parte, una SRS escrita por la organización de desarrollo como el primer paso del proceso de desarrollo del software debe ser más específica. El tema de esta unidad es este tipo de SRS y que realiza un conjunto de funciones completamente diferentes de la SRS preparada por el cliente. Su propósito incluye lo siguiente:

- Proporcionar los medios de comunicación entre clientes, usuarios, analistas y diseñadores.

- Apoyar las actividades de prueba del sistema.
- Proporcionar una evolución controlada del sistema.

Una SRS bien escrita reduce la probabilidad de que el cliente quede en desacuerdo con el producto final. Los desacuerdos entre el cliente y el desarrollador se resuelven durante la etapa de requerimientos y no durante la etapa de prueba, ya que aumentarán los costos. La SRS debe ser muy específica acerca de cómo el sistema se verá externamente en el ambiente del sistema.

La SRS también sirve como base para las actividades de prueba y verificación del sistema. El propósito de la prueba del sistema es comprobar que el sistema construido cumple los requerimientos. Si la SRS es ambigua o inconsistente, entonces la prueba es imposible.

8. Contenido de la SRS

Una SRS debe incluir una descripción completa, concisa, de la totalidad de la interfaz externa del sistema con su ambiente, incluyendo otro software, puertos de comunicación, hardware y usuarios. Esto incluye dos tipos de requerimientos: de comportamiento y no-comportamiento.

- **Requerimientos de comportamiento:** Define lo que hace el sistema. Estos requerimientos describen todas las entradas al sistema, las salidas del sistema, así como la información referente a cómo estas entradas y salidas se interrelacionan. Tales descripciones de cómo las entradas se corresponden con las salidas se denominan *descripciones de comportamiento* o especificaciones operacionales.
- **Requerimientos de no-comportamiento:** Define los atributos del sistema cuando trabaja. Incluyen una descripción completa de los niveles requeridos del sistema en cuanto a eficiencia, confiabilidad, seguridad, facilidad de mantenimiento, portabilidad, visibilidad, capacidad y la conformidad con los estándares.

Lo siguiente no debe de ser incluido en la SRS:

- **Requerimientos del Proyecto:** Mostrar los requerimientos tales como personal, horario, costos, hitos de control, actividades, fases y los procedimientos de reporte no están incluidos en la SRS.
- **Diseños:** Hay varias razones por las que se debe excluir el diseño de la SRS. Algunas de ellas se presentan a continuación:
 - Para un conjunto de requerimientos, puede haber múltiples soluciones de diseño que se puedan desarrollar. Unir el diseño al SRS crea dificultades para rastrear los cambios en los requerimientos en relación a diferentes diseños, diseños que evolucionan, etc.
 - La SRS es leída por un conjunto de personas, generalmente diferente al grupo de personas que leen el documento de diseño. Con este grupo de personas

mutuamente excluyente, no hay mucho que ganar manteniendo el diseño en la SRS.

- Los ingenieros de requerimientos son generalmente muy competentes en analizar y especificar. Pero no necesariamente son buenos en el diseño. Por lo tanto, el diseño se puede excluir de SRS.
- **Planes de Aseguramiento del Producto:** Los planes de aseguramiento como los planes de manejo de la configuración, planes de verificación y de validación, planes de prueba y planes de aseguramiento de calidad no se incluyen en la SRS.

A continuación se discuten las diversas cualidades de una SRS de alta calidad.

9. Atributos de una SRS de Alta Calidad

Se sabe, por supuesto que no existe tal cosa como una 'SRS perfecta'. Pero si se asume que hay una, esta poseerá todos los atributos que se enumeran a continuación. La idea es que, con estos atributos 'deseables', se pueda intentar escribir SRS de alta calidad.

- **Correcta:** Una SRS es correcta sí y sólo sí cada requerimiento establecido representa alguna característica del sistema que se desarrollará. Asuma que el software propuesto debe proporcionar resultados a un comando particular P en X segundos, es incorrecto especificar el requerimiento como 'el software debe proporcionar resultados al comando P en Y segundos o menos'.
- **No Ambigüedad:** Sí cada requerimiento especificado en una SRS tiene uno y solamente un significado, entonces se considera no-ambigua.
- **Verificable:** Si hay un método para comprobar si el producto desarrollado cumple con el requerimiento especificado para cada requerimiento establecido, entonces se dice que la SRS es verificable.
- **Consistencia:** Una SRS es consistente sí y sólo sí, ninguno de los subconjuntos individuales establecidos tienen conflicto. Es decir los términos usados, características del producto y requerimientos plasmados en la SRS no se contradicen entre sí.
- **Comprensible:** Los requerimientos plasmados en la SRS son entendibles tanto para desarrolladores como para los usuarios.
- **Facilidad de Modificación:** Una SRS es modificable si su estructura y contenido hacen fácil la implementación de cambios en los requerimientos, mientras se preserva su consistencia y la completitud. Las cosas mínimas que deben estar presentes para la facilidad de modificación son una tabla de contenido, un índice y las referencias cruzadas que permiten modificaciones en las porciones relevantes de la SRS en caso de cualquier cambio en los requerimientos.
- **Comentado:** Una SRS se dice que es comentada, cuando proporciona una guía significativa a los desarrolladores, al equipo y a la organización de sí misma.

- **Rastreable:** Una SRS se llama 'rastreable', si el origen de cada uno de los requerimientos en la SRS es claro y fácilmente referenciado.
- **Integridad:** Una SRS está completa si posee las siguientes cuatro características.
 - Todos los requerimientos relacionados con la funcionalidad y el rendimiento están presentes en la SRS. No hay nada que se suponga que hace el software que no esté en la SRS.
 - Para cada entrada del sistema mencionada en la SRS, la SRS especifica cuáles serán la (s) salida(s) apropiadas.
 - Todas las páginas, figuras y tablas en la SRS son enumeradas; por otro lado se proporcionan todos los términos y unidades de medida, además de todo el material y secciones referenciados.
 - Ninguna sección está marcada 'Para ser determinada (TBD -To Be Determined)'.

Sin embargo, alcanzar todas las cualidades en una SRS es imposible. Las siguientes son algunas de las dificultades cuando se intenta alcanzar todas las cualidades de una buena SRS según lo mencionado anteriormente:

- Se procura eliminar inconsistencia y ambigüedad reduciendo el uso del lenguaje natural como el inglés en la SRS y aumentando el uso de métodos matemáticos formales; de esta manera la SRS llega a ser menos comprensible para quien no es especialista en computación.
- A medida que se intenta ser absolutamente completo, el costo de la SRS aumenta y el documento llega a ser extremadamente grande y difícil de leer.
- Si se trata de aumentar las modificaciones eliminando toda la redundancia, la SRS llega a ser difícil de seguir y ambigua.

Por lo tanto, la conclusión es que una SRS perfecta no es posible. De tal modo que hay que esforzarse por alcanzar una SRS de una calidad aceptable.

Hay algunos estándares que fueron desarrollados para alcanzar este nivel de calidad. La siguiente sección los describe brevemente.

10. Estándares en SRS

A través de los años, varios estándares se han desarrollado para servir como pautas para desarrollar SRS. También sirven como indicadores de las mejores prácticas que siguieron. Algunos de los estándares para SRS son:

- IEEE 830 (por la Institución de los Ingenieros Eléctricos o Electrónicos (IEEE)).
- DoD (por el Departamento de la Defensa, de los EE.UU.).
- NASA (por la NASA).

El DoD y NASA fueron estándares definidos principalmente para los proyectos del DoD y de la NASA respectivamente. Estos no serán discutidos. El IEEE 830 es más general y se puede utilizar para una amplia variedad de proyectos de desarrollo de software.

Resumen

Ahora que usted ha completado esta unidad, estará en la capacidad de:

- Discutir la naturaleza e importancia de los requerimientos.
- Definir requerimientos de software y el término SRS.
- Describir las actividades del análisis de los requerimientos.
- Describir las funciones y componentes de una SRS.
- Discutir los diferentes atributos de una SRS bien redactada.

Unidad 2: Examen de Autoevaluación

- 1) En el caso de que los sistemas requieran una solución del software, ¿Cuándo se inicia la fase de los requerimientos?
 - a) Cuando se reconoce que un problema existe y requiere una solución.
 - b) Cuando una nueva idea del software se presenta (el equivalente de una invención en hardware).
 - c) Cuando el cliente proporciona la orden de compra.
 - d) Cuando el consultor está claro sobre la naturaleza del sistema y del problema.

- 2) ¿Cuál de los siguientes son parte del contenido de una SRS?
 - a) Requerimientos del proyecto.
 - b) Requerimientos de comportamiento.
 - c) Planes de aseguramiento del producto.
 - d) Conformidad con los estándares.

- 3) ¿Con cuál de los siguientes trata el estudio de los requerimientos?
 - a) Qué del sistema.
 - b) Porqué del sistema.
 - c) Cuándo del sistema.
 - d) Cómo del sistema.

- 4) ¿Para cuáles de las siguientes funciones es útil una SRS?
 - a) La fuente primaria de los requerimientos de los diseñadores para construir.
 - b) Como base para verificar que el sistema construido resuelve los requerimientos.
 - c) Como entrada a la organización de comercialización para desarrollar el material de la promoción de ventas.
 - d) Para servir como base para la gerencia de proyecto.

- 5) ¿Por qué son importantes los requerimientos?
 - a) Mientras más tardía es la detección de un error de software en el ciclo vital del desarrollo, será más costoso de reparar.
 - b) Muchos errores siguen siendo latentes y no se detectan hasta después de la etapa en la cual se hacen.
 - c) Ayuda a estimar el costo de un proyecto.
 - d) Los errores realizados en las especificaciones de requerimientos son típicamente hechos incorrectos, omisiones, inconsistencias y ambigüedades.

- 6) ¿Cuáles de los siguientes mecanismos hace que la ingeniería de requerimientos sea útil?
- a) Entender lo que desea el cliente.
 - b) Analizar necesidades.
 - c) Validar la especificación.
 - d) Suministrar un diseño preliminar.
- 7) ¿Cuáles de los siguientes no son pasos de la ingeniería de requerimientos?
- a) Levantamiento de los requerimientos.
 - b) Análisis de requerimientos, especificación y administración.
 - c) Planeamiento del proyecto.
 - d) Seleccionar la tecnología para la implementación.
- 8) ¿Por qué el proceso de levantamiento de los requerimientos es difícil?
- a) Debido al problema del alcance.
 - b) Debido al problema de entendimiento.
 - c) Debido al problema de la volatilidad.
 - d) Debido al problema del tiempo y presupuesto.
- 9) ¿Cuáles de los siguientes puede ser una parte de la especificación de requerimientos?
- a) Requerimientos de mano de obra.
 - b) Un modelo gráfico.
 - c) Un documento escrito.
 - d) Un conjunto de prototipos.
- 10) ¿Cuáles de las siguientes es o son las características que posee un buen SRS?
- a) Comprensible.
 - b) Rentable.
 - c) Rastreadable.
 - d) Modificable.

Respuestas de la Unidad 2: Examen de Autoevaluación

- 1) a y b
- 2) b
- 3) a
- 4) a, b y d
- 5) a, b y d
- 6) a, b y c
- 7) c y d
- 8) a, b y c
- 9) b, c y d
- 10) a, c y d

Unidad 3: Lab. de Especificación de Requerimientos

Objetivos de Aprendizaje

Al finalizar esta unidad, estará en la capacidad de:

- Aplicar los conocimientos obtenidos sobre especificación de requerimientos.
- Redactar el alcance de un proyecto.
- Identificar funcionalidades, restricciones de un sistema.
- Determinar los usuarios de un sistema.
- Identificar requerimientos funcionales y de interfaz.

1. Ejercicio de Laboratorio

Una empresa líder en servicios de transporte turísticos y convenciones (taxis, limusinas, autobuses, aviones y helicópteros), debido al crecimiento turístico en el país ha experimentado un alza en la demanda de sus servicios y un crecimiento inesperado de clientes, lo que ha llevado a la empresa a tomar la decisión de optimizar y automatizar la solicitud y reserva de servicios por parte de sus clientes.

La empresa desea tener un sistema que permita a sus clientes solicitar sus servicios de diferentes maneras, sin limitaciones de lugar o medios de solicitud. Se desea que los clientes puedan reservar o solicitar servicios mediante un sitio Web (sitio Web de la empresa) o mediante mensajes de texto (sin importar la compañía telefónica). Con el sistema se espera:

- Tener el control de los clientes habituales de la empresa tanto individuales como corporativos.
- Tener un seguimiento de los nuevos clientes.
- Agilizar la asignación de chóferes o pilotos para ejecutar los servicios.
- Llevar el control del personal disponible como de la flotilla de la empresa.

La empresa presta servicios a escala nacional y hacia algunos parajes turísticos de América latina, Puerto Rico, Curazao, Aruba, etc. Siendo prioridad de la empresa el correcto funcionamiento para Venezuela, Aruba y Curazao.

Actualmente se maneja la solicitud de servicios vía telefónica y con un registro manual de los mismos. Para clientes individuales se manejan datos como nombre, C.I., edad, empresa donde trabaja, dirección de trabajo, teléfono de la oficina, dirección de habitación, teléfono de habitación, celular, email. Para clientes corporativos se maneja: nombre de la empresa, persona de contacto, dirección, teléfono, Rif, Nit. Para ambos tipos de clientes se manejan los datos del último servicio solicitado y el responsable del último servicio.

Del servicio prestado se registra: tipo de servicio (taxi, avión, etc.), fecha del servicio, hora, origen, destino, responsable del servicio, número de personas, presupuesto, fecha de la solicitud, persona de contacto.

Para el problema presentado, se solicita lo siguiente:

- Elabore el documento de SRS.

A efectos del ejercicio, el documento de SRS se limitará a esbozar los siguientes puntos:

1. Introducción

1.1 Propósito de la SRS: Explica el propósito del documento de SRS y no del software.

1.2 Alcance del Producto: Qué aspectos de la aplicación cubre el documento.

2. Descripción General: Resumen de todo el software a desarrollar. Esta incluye las principales metas, tareas y usuarios.

2.1 Funciones del Producto: Sumario de la mayoría de funciones de la aplicación.

2.2 Características del Usuario: Indica qué tipo de persona es el usuario típico del sistema. Ej., Novato, profesional del software, contadores con 5 años de experiencia usando computadores.

2.3 Restricciones Generales: todas las condiciones que pueden limitar las opciones del desarrollador. Estas pueden ser originadas desde diferentes fuentes.

3. Requerimientos Específicos

3.1 Requerimientos Funcionales: Servicios o funciones que proveerá el sistema. Ej. Se deben ingresar cédula y nombre.

3.1.1 Requerimiento Funcional 1

3.1.1.1 Introducción

3.1.1.2 Entradas

3.1.1.3 Proceso

3.1.1.4 Salidas

3.2 Requerimientos externos de Interfaz

3.2.1 Interfaces de Usuario

3.2.2 Interfaces de Hardware

3.2.3 Interfaces de Software

3.3 Otros Requerimientos

3.3.1 Base de datos

Asuma que Ud. es parte del equipo de desarrollo para este sistema, elabore el documento de SRS tomado en cuenta que sólo tiene 10 meses para la entrega del primer prototipo del sistema, utilice esto para definir los límites del sistema.

Unidad 4: Diseño de Software

Objetivos de Aprendizaje

Al finalizar esta unidad, estará en la capacidad de:

- Explicar de forma breve el proceso de diseño.
- Explicar los Enfoques de Diseño Top-Down y Bottom-Up.
- Establecer los principios y conceptos del diseño de software.

1. Introducción al Diseño de Software

El diseño de software es parte fundamental de la ingeniería de software, tiene por objetivo planear una solución para un problema especificado en la SRS. Es el primer paso para avanzar del dominio del problema al dominio de la solución, sin importar cual proceso de software se utilice. Es la primera de tres actividades técnicas en el proceso de ingeniería de software, a saber: diseño, programación y pruebas. El proceso de diseño de software comienza después del análisis y la especificación de requerimientos.

Existen diversos métodos disponibles para realizar la actividad de diseño. Sin importar el método elegido, los siguientes serán algunos de los resultados del proceso de diseño:

- Diseño de datos.
- Diseño arquitectónico.
- Diseño de interfaz.
- Diseño de componentes.

A continuación se discuten brevemente cada uno de ellos.

1.1 Diseño de Datos

El diseño de datos ayuda en la creación de las estructuras de datos requeridas para implementar el software. Los objetos de datos y las relaciones que se definen en el diagrama entidad-relación proveen la base para las actividades de diseño de datos.

1.2 Diseño Arquitectónico

El diseño arquitectónico especifica las diversas entidades estructurales en el sistema y su interacción con otras o la relación entre sí. El diseño arquitectónico considera los elementos estructurales y diversos patrones de diseño.

1.3 Diseño de Interfaz

El diseño de interfaz especifica la manera en la cual se comunican e interactúan los diversos componentes entre sí. El diseño de interfaz también especifica las formas en las que los usuarios se comunican con el software y viceversa.

1.4 Diseño de Componentes

Las entradas principales para el diseño de componentes provienen del diseño arquitectónico, el cual provee los elementos estructurales del sistema. El diseño de componentes especifica los componentes (descripción procedural) que corresponde a la funcionalidad sugerida por los elementos estructurales.

Reiterando, el diseño de software es muy importante para el proceso de la ingeniería de software. El software bien diseñado tendrá menos errores, aparte de ser más eficiente que uno mal diseñado.

2. Factores que Afectan los Enfoques de Diseño de Software

Una metodología puede ser definida simplemente como un conjunto de procedimientos que se siguen, desde el comienzo hasta el término del proceso de desarrollo de software. La naturaleza de la metodología es dependiente de un número de factores. Estos factores se presentan en la Figura 4.1.

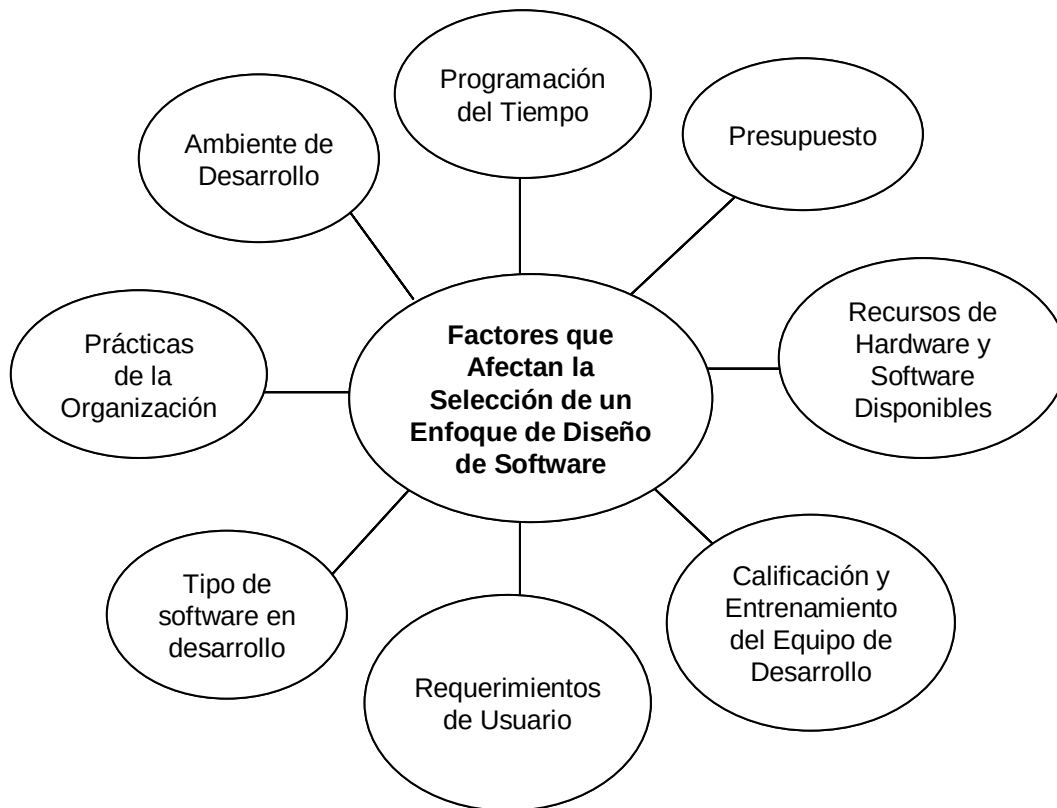


Figura 4.1: Factores que Afectan la Elección de Enfoques de Diseño de Software

Existen dos categorías de metodologías de diseño:

- Sistemática.
- Formal.

La categoría sistemática consiste de componentes procedurales, la cual dicta qué acciones o tareas realizar. Por otra parte, el componente de representación especifica cómo debe representarse la estructura de software.

La categoría formal usa extensivamente notaciones matemáticas para las transformaciones de objetos y para revisar la consistencia.

En general, las técnicas de la metodología de diseño sistemático pueden integrarse y usar esquemas de representación de otras técnicas cuando se requiera. No existe una plataforma común sobre la que se pueda evaluar o comparar metodologías, ya que han sido desarrolladas específicamente para ocuparse de ciertos problemas o grupos de problemas. Pero se pueden analizar y entender sus principios subyacentes para tener una mejor comprensión de los fundamentos de cada metodología. Esto ayudará a aplicar el dominio de cada metodología más efectivamente.

Otro beneficio importante es que la familiaridad con varias metodologías permite crear diseños competitivos, que sean más lógicos, sistemáticos y que se apoyen menos en la inspiración.

3. Comprender el Proceso de Diseño

El diseño de software implica traducir los requerimientos básicos en un plan de acción para construir el software requerido. El diseño se describe inicialmente a un nivel alto de abstracción; por ejemplo qué módulos se necesitan para el sistema, las especificaciones de estos módulos y cómo se deben interconectar los módulos. Se puede entonces remontar los objetivos del sistema, el diseño interno de los módulos o cómo se pueden satisfacer las especificaciones de los módulos junto con la incorporación de requerimientos funcionales y de comportamiento.

3.1 Características de un Buen Diseño de Software

El diseño de un sistema es, quizás, el factor más crítico que afecta la calidad del software. Tiene un gran impacto en las fases posteriores, particularmente en la de pruebas y de mantenimiento. En consecuencia, se realizan revisiones de diseño para asegurar que el diseño satisfaga los requerimientos y sea de buena calidad. Las siguientes tres características pueden usarse como guías para un mejor diseño:

- El diseño debe implementar todos los requerimientos especificados en la SRS.
- El diseño debe ser fácil de entender por parte de los desarrolladores y también para los que probarán el software en una etapa posterior.
- El diseño debe cubrir todos los aspectos de la implementación del software, tales como: el punto de vista funcional, el punto de vista de comportamiento y el punto de vista del flujo de datos.

Las siguientes guías ayudarán a obtener un buen diseño y de alta calidad.

- Un diseño debe tener una estructura arquitectónica que:
 - Esté basada en patrones de diseño.
 - Contenga elementos que muestren características de un buen diseño.
 - Conduzca a una implementación que sea comprobable y robusta.
- El diseño debe especificar claramente los diversos componentes como: componentes de diseño de datos, diseño de elementos estructurales a partir del

diseño arquitectónico y aspectos de diseño de interfaz para la comunicación entre componentes, así como con los usuarios.

- El diseño debe sugerir estructuras de datos que puedan ser usadas.
- El diseño debe tener independencia funcional entre módulos.
- El diseño debe ofrecer interfaces sencillas para la comunicación entre módulos.
- El diseño debe poseer la característica de capacidad de repetición, es decir la posibilidad de especificar componentes de diseño basándose en el análisis de requerimientos.

Algunas guías para buen diseño se ilustran en la Figura 4.2.

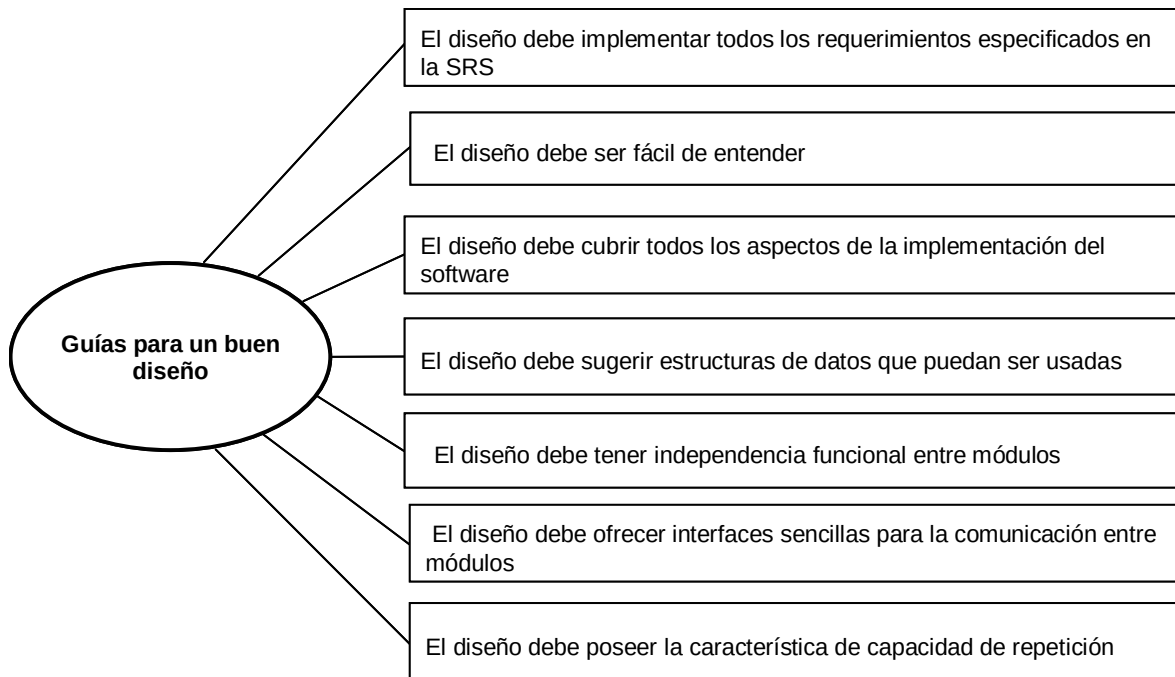


Figura 4.2: Guías para un buen diseño

Los principios básicos para el diseño de software dados aquí son sólo un subconjunto de los principios de diseño definidos por Alan Davis en 1995 en su libro titulado "201 Principles of Software Development", publicado por McGraw Hill.

- El proceso de diseño no debe restringirse a uno o a un conjunto limitado de enfoques. Debe ser sensible a considerar enfoques alternativos basados en la situación y los méritos de ese enfoque.
- El diseño debe ser fácil de seguir hasta la SRS.
- El diseño debe promover la reutilización de componentes y alentar el uso de patrones de diseño.

- El diseño debe mantenerse cercano a la naturaleza del problema que intenta resolver.
- El diseño debe adherirse a los estilos y lineamientos que hayan sido adoptados.
- El diseño debe asegurar que las interfaces reveladas por los componentes sean simples, no-ambiguas y estén bien definidas.
- El diseño debe ser lo suficientemente flexible como para permitir modificaciones con un mínimo esfuerzo o efectos secundarios.
- El diseño debe ser capaz de manejar excepciones en situaciones anormales.
- El diseño nunca debe avanzar hasta el nivel de codificación.
- La calidad del diseño debe ser evaluada durante el proceso de diseño.
- El diseño debe prestarse a revisiones, para asegurar la eliminación de errores de diseño de diversos tipos.

3.2 Evolución del Diseño de Software

Los siguientes son algunas de las etapas principales por las que el diseño de software ha progresado a través de los años:

- Desarrollo de programas mediante diseño modular.
- Refinamiento paso a paso.
- Programación estructurada.
- Diseño orientado a flujo de datos.
- Diseño orientado a estructuras de datos.
- Diseño orientado a objetos.
- Diseño derivado de la arquitectura de software.

Existen numerosos métodos de diseño. Las características comunes que todos estos métodos comparten se ilustran en la Figura 4.3.

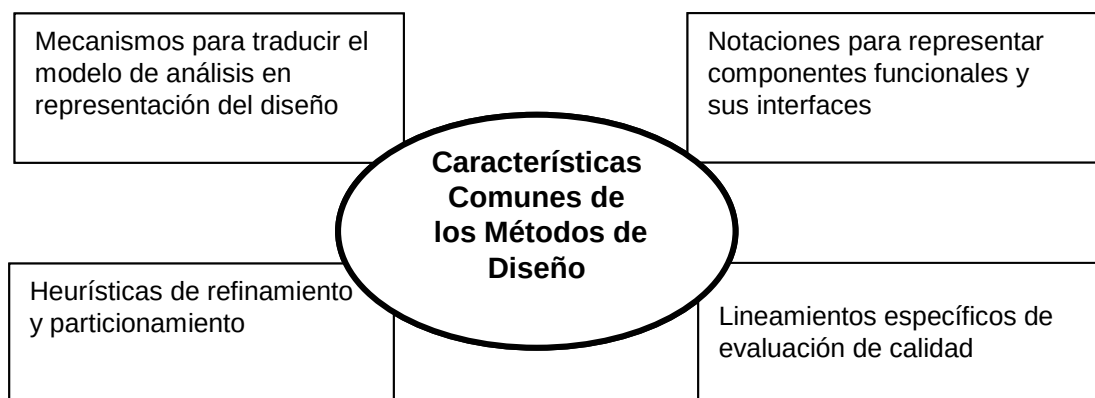


Figura 4.3: Características Comunes de los Métodos de Diseño

Es claro, a partir de la Figura 4.3, que todos los métodos de diseño tienen mecanismos para derivar un diseño a partir de un modelo de análisis. Cada método provee una notación de diseño para mostrar componentes e interfaces.

Los métodos de diseño tienen sus propias técnicas para refinar (elaboración) y particionar el diseño (restringir la complejidad).

Cada método de diseño proveerá formas para evaluar objetivamente la calidad del diseño.

4. Enfoques de Diseño Top-Down y Bottom-Up

El diseño top-down (arriba hacia abajo) es un enfoque de diseño orientado a niveles, donde los diseñadores comienzan con una descripción de alto nivel de un sistema y luego refinan esa visión paso a paso. El sistema se fragmenta en módulos más pequeños y de más bajo nivel con cada refinamiento. Este enfoque requiere que los diseñadores identifiquen los requerimientos y funciones principales del sistema, además las dividan en niveles sucesivos hasta que se puedan diseñar módulos de funciones específicas.

El diseño top-down es iterativo por naturaleza, reduciendo el alcance y el tamaño de cada módulo y concentrándose más en cuestiones específicas. En este método, el enfoque central está en el requerimiento del cliente y en la naturaleza general del problema a resolver. Además, buscar errores es más sencillo aquí que en algunos otros métodos. Una desventaja de este enfoque es que existe la posibilidad de que decisiones hechas en el nivel superior puedan resultar en una descomposición inaceptable o ineficiente en los niveles inferiores. Los diseñadores tienen que superar esta dificultad emprendiendo una actividad considerable de seguimiento, donde las decisiones de nivel superior son reevaluadas y luego reestructuradas de acuerdo a esto.

El enfoque bottom-up (de abajo hacia arriba) involucra identificar un conjunto básico de módulos y sus interrelaciones, que pueden ser usadas como las bases para la solución. En este enfoque, los diseñadores formulan los conceptos de alto nivel basándose en el conjunto básico de módulos. El diseño bottom-up es también un proceso iterativo, y puede resultar en una cantidad considerable de seguimiento si los elementos primitivos no se construyen correctamente. El beneficio principal de este enfoque es que permite la evaluación de sub-módulos durante el proceso de desarrollo del sistema.

Los diseñadores raramente usan un enfoque top-down o bottom-up puro en la realidad. El enfoque top-down es más adecuado cuando el problema y su entorno están bien definidos, por ejemplo, cuando se diseña un compilador. El enfoque debe ser principalmente bottom-up o una mezcla de enfoques, cuando el problema está mal definido. El enfoque top-down ha resultado en la evolución de una metodología de diseño muy popular llamada diseño estructurado, el cual se discutirá en esta unidad.

5. Conceptos Básicos de Diseño

El diseño de software descansa en algunos conceptos básicos como: abstracción, refinamiento, modularidad y arquitectura. Una de las ideas principales detrás del diseño es reducir la complejidad, la cual, se puede reducir a través del proceso de abstracción. Un buen proceso de diseño se desarrolla de lo simple a lo complejo, lo cual se hace usando el proceso de refinamiento. Es importante notar que cada elemento de diseño tiene que desempeñar una función específica. El concepto de modularidad promueve esto. En años recientes, ha habido una creencia firme de que el diseño detallado de componentes de software debe provenir de elementos arquitectónicos. A continuación, se discuten brevemente cada uno de estos conceptos.

5.1 Abstracción

La abstracción es el proceso por el cual el diseñador (o el analista) se concentra sólo en aquellos aspectos que son relevantes e ignora detalles irrelevantes. Puede haber diferentes niveles de abstracción en una solución modular para un problema particular. En el nivel más alto de abstracción, se habla de la solución en términos genéricos, usando el lenguaje del ambiente del problema. En niveles más bajos, el enfoque es más procedural. En los niveles más bajos de abstracción, se da una solución de tal forma que pueda ser directamente implementada.

5.2 Refinamiento

El refinamiento ayuda al desarrollo secuencial de un programa, dentro de una jerarquía específica. Sugiere claramente el enfoque top-down, procediendo de lo simple a lo complejo o de pocos detalles a más detalles. Básicamente, el refinamiento es una especie de elaboración. La funcionalidad se define en un alto nivel de abstracción y es luego dividida gradualmente, proveyendo más detalle con cada paso. De cierta forma, la abstracción y el refinamiento son complementarios. Mientras que la abstracción ayuda a un diseñador a especificar procedimientos y datos, suprimiendo detalles de bajo nivel, el refinamiento ayuda al diseñador a revelar detalles de bajo nivel, a medida que el diseño progresa desde niveles simples hasta complejos.

5.3 Modularidad

El software puede ser dividido en componentes individuales llamados módulos. Dividir un programa en muchos módulos diferentes lo simplifica.

La evaluación de un método de diseño se muestra en la Figura 4.4.

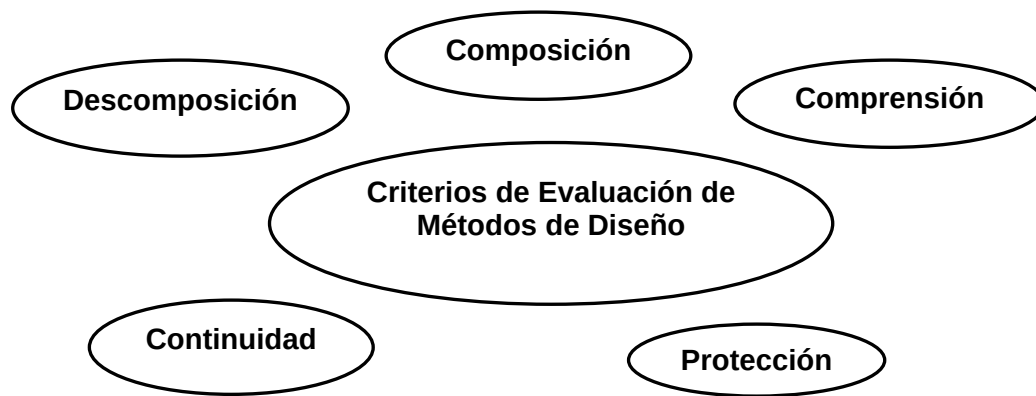


Figura 4.4: Criterio de Evaluación de Métodos de Diseño

Estos ayudan a evaluar un método de diseño en términos de su capacidad o a definir un sistema modular activo.

- **Descomposición:** Se refiere a la posibilidad del diseño de permitir la reducción de la complejidad de un problema al tener grandes problemas divididos en problemas más pequeños que sean manejables.
- **Composición:** Se refiere a la capacidad del diseño para utilizar componentes más pequeños para construir un nuevo sistema.
- **Comprensión:** Se refiere a la posibilidad de un módulo para ser utilizado y entendido mejor, sin la necesidad de referirse a otros módulos y sus usos.
- **Continuidad:** Se refiere a la capacidad de un módulo de contener los efectos de un cambio dentro de sí (o dentro de un ámbito más pequeño), en vez de propagarlos a través del sistema en un efecto expansivo.
- **Protección:** La protección es la capacidad de un módulo para contener los efectos de un error o una condición excepcional dentro de sí mismo, sin permitir que tales efectos se propaguen hacia otros módulos y al resto del sistema.

La modularidad conforma uno de los conceptos principales detrás del diseño moderno. Cualquiera que sea el método de diseño empleado, la modularidad sirve como un vehículo importante.

6. Arquitectura del Software

La arquitectura del software concibe al sistema como una colección de componentes estructurales que interactúan unos con otros. Presenta a los diversos componentes del software organizados como una estructura (frecuentemente jerárquica) en donde los componentes interactúan entre ellos.

Los diferentes modelos para representar la arquitectura del software se listan a continuación:

- **Modelos Estructurales:** Representan la arquitectura como una colección de diversas partes de un programa.
- **Modelos de Marcos de Trabajo:** Ayudan a identificar patrones de diseño que puedan ser reutilizables al crear varias aplicaciones.
- **Modelos Dinámicos:** Se concentran en los aspectos de comportamiento del software.
- **Modelos de Proceso:** Se encargan de los procesos técnicos y de negocio en un sistema.
- **Modelos Funcionales:** Ayudan a acomodar las diversas funciones en un sistema como una jerarquía en forma de árbol.

7. Diseño Modular

El diseño modular es una de las metodologías de diseño más usada. La implementación del diseño modular ayuda en las siguientes formas:

- Reduce la complejidad en el proceso de diseño.
- Ayuda a dar lugar al cambio, el cual es un aspecto vital del mantenimiento del software.
- Permite una implementación más sencilla, al dar cabida al desarrollo paralelo de diversas partes del sistema.

7.1 Tipos de Módulos

Dentro de la arquitectura del software, pueden definirse módulos usando abstracción y atributos que oculten información. El ocultamiento de información es el proceso por el cual un módulo ocultará a otros módulos ciertos detalles de los datos utilizados por él mismo, así como las operaciones realizadas por él. Estos atributos tienen que ser traducidos después a características operacionales del módulo. Se caracterizan por lo siguiente:

- **Histórico de tiempo de incorporación:** Este es el momento en que un módulo es incorporado dentro de una descripción de lenguaje del software.
- **Mecanismos de activación:** Tiene dos mecanismos de activación:
 - Invocación por referencia.
 - Invocación por interrupción.

La última es vista en aplicaciones de tiempo real, donde un evento externo causa una suspensión del proceso actual, resultando en el pase del control a un módulo diferente. Los mecanismos de activación pueden afectar la estructura de un programa y son, por lo tanto, muy importantes para el mismo.

- **Patrón de control:** Es la descripción de la ejecución interna de un módulo. Usualmente, un módulo normal tendrá un único punto de entrada y uno de salida, además será ejecutado en secuencia, como parte de la tarea de sólo un usuario. A veces, puede presentarse la necesidad de patrones más avanzados

de control. En tales situaciones, un único módulo puede ser utilizado por múltiples tareas simultáneamente.

Un módulo puede ser clasificado dentro de una estructura de programa como sigue:

- **Módulo secuencial:** Referenciado y ejecutado sin interrupción por el software de aplicación.
- **Módulo incremental:** También llamado *corutina*. Puede ser interrumpido por el software de aplicación antes de culminar la ejecución, y luego reanudado desde el punto donde fue interrumpido.
- **Módulo paralelo:** También llamado *conrutina*. Se ejecuta junto con otro módulo simultáneamente en ambientes de multiprocesadores.

Los módulos que se ven más frecuentemente son los secuenciales. Contienen macros en tiempo de compilación y subprogramas normales, tales como subrutinas, funciones, procedimientos, etc. Los módulos incrementales, siendo un poco más avanzados que los secuenciales, mantienen un puntero de entrada. Este le permite al módulo reanudar en el punto de interrupción. Los módulos paralelos se observan en momentos en que cálculos de alta velocidad requieren que dos o más CPUs trabajen en paralelo. No existe jerarquía de control fija para estas corutinas y conrutinas, así que estas estructuras requieren rutas especiales de diseño. Otros tipos especiales de módulos, vistos sólo en lenguajes como Modula y Ada, son el *module* de Modula y el *package* de Ada. Una discusión de los mismos está más allá del alcance de esta unidad.

7.2 Independencia Funcional

Desarrollar módulos con funciones singulares es la base para alcanzar independencia funcional. Esto implica desarrollar módulos que no tengan una tendencia hacia una interacción extra con otros módulos. El software tiene que ser diseñado de tal manera que cada módulo se ocupe de una sub-función específica de requerimientos. La interfaz de cada módulo también tiene que verse simple, cuando es vista desde otras partes de la estructura del programa.

La independencia funcional es vital para el software, pues es más simple y más fácil desarrollar software con módulos independientes. Las funciones serán entonces divididas en compartimentos y las interfaces simplificadas. Los módulos independientes funcionalmente son también más fáciles de mantener, ya que habrá una muy pequeña probabilidad de efectos secundarios a causa de modificaciones de código o diseño. La Independencia funcional es la clave para mejorar la calidad del software.

La cohesión y el acoplamiento son los criterios usados para medir la independencia funcional, éstos son discutidos brevemente a continuación.

7.3 Cohesión

La cohesión es la propiedad por la cual las cosas que son similares se mantienen juntas. Un módulo se considera completamente cohesivo sólo si realiza una única función o tarea. Un módulo cohesivo normalmente no requiere mucha interacción con

otro módulo. Siempre se debe aspirar a un alto grado de cohesión, aunque incluso un nivel mediano de cohesión será fácilmente aceptable. Los bajos niveles de cohesión son contraproducentes para propósitos de diseño y deben ser evitados en lo posible. Una cohesión de nivel medio es mejor que una cohesión de niveles mínimos. Si bien puede ser muy difícil lograr una alta cohesión, en muchos casos una cohesión de rango medio es suficiente.

En vez de tratar de determinar el nivel exacto de cohesión, los desarrolladores deben aspirar a lograr una alta cohesión, ya que la baja cohesión es perjudicial para la independencia funcional. Esto les ayudará a modificar el diseño del software para alcanzar una independencia funcional mayor.

7.4 Acoplamiento

El acoplamiento es la medida de la interconexión entre diferentes módulos dentro de una estructura de software. Depende de:

- La complejidad de la interfaz entre dos módulos diferentes.
- El modo como se hace una referencia al módulo.
- La manera en que los datos se pasan a través de la interfaz.

Niveles más simples de conectividad entre diferentes módulos aseguran simplicidad y eficiencia en el software. Por lo tanto, es natural que se esté buscando siempre bajos niveles de acoplamiento. Una conectividad simple asegura un efecto secundario nulo cuando ocurre un error en un punto y se extiende a través del sistema.

Resumen

Ahora que ha completado esta unidad, debe estar en capacidad de:

- Explicar de forma breve el proceso de diseño.
- Discutir las metodologías de diseño.
- Explicar los enfoques top-down y bottom-up.
- Discutir los principios y objetivos del diseño.
- Discutir los conceptos cruciales de cohesión y acoplamiento.

Unidad 4: Examen de Autoevaluación

- 1) Un módulo que puede ser interrumpido por el software de aplicación antes de culminar la ejecución y luego reanudado desde el punto donde fue interrumpido, se denomina:
 - a) Módulo Dinámico.
 - b) Módulo Incremental.
 - c) Módulo Paralelo.
 - d) Módulo Secuencial.

- 2) Cuando el problema y su entorno están bien definidos, ¿Cuál es el enfoque de diseño más adecuado?
 - a) Bottom-Up (de abajo hacia arriba).
 - b) Top-Down (de arriba hacia abajo).
 - c) Un mezcla entre Bottom-Up y Top-Down.
 - d) Diseño arquitectónico.

- 3) Cuando se desarrollan módulos que no tiene una tendencia hacia la interacción extra con otros módulos, se dice que se está aplicando el concepto de:
 - a) Acoplamiento.
 - b) Refinamiento.
 - c) Independencia Funcional.
 - d) Cohesión.

- 4) ¿Cuáles de los siguientes son algunos de los resultados del proceso de diseño?
 - a) El diseño de componentes.
 - b) El diseño arquitectónico.
 - c) Patrón de control.
 - d) El diseño de datos.

- 5) ¿Cuáles son las características comunes de los métodos de diseño?
 - a) Todos tienen un mecanismo para traducir un modelo de análisis en una representación de diseño.
 - b) Todos ellos tienen una notación que les ayuda a representar componentes funcionales y sus respectivas interfaces.
 - c) Todos tienen heurísticas para refinar y particionar.
 - d) Todos tienen guías específicas para evaluar la calidad.

- 6) ¿Cuáles de los siguientes son factores que afectan en la elección de un enfoque de diseño de software?
- a) Presupuesto.
 - b) Ambiente de Desarrollo.
 - c) Programación Estructurada.
 - d) Recursos de Hardware y Software disponibles.
- 7) Una metodología de diseño puede ser definida como un conjunto de procedimientos que se siguen desde el comienzo hasta el término del proceso de desarrollo de software.
- a) Verdadero.
 - b) Falso.
- 8) ¿Cuáles de los siguientes son modelos para representar la arquitectura del software?
- a) Modelos estáticos.
 - b) Modelos de proceso.
 - c) Modelos dinámicos.
 - d) Modelos de factorización.
- 9) En relación con las características para un buen diseño, ¿Cuáles de las siguientes afirmaciones son ciertas?
- a) El diseño debe cubrir todos los aspectos de la implementación del software.
 - b) El diseño debe ser fácil de entender sólo por parte de los desarrolladores.
 - c) El diseño debe implementar todos los requerimientos especificados en la SRS.
 - d) El diseño no debe cubrir aspectos como el punto de vista de comportamiento.

- 10) En relación con los criterios para evaluar un método de diseño en términos de su capacidad, la descomposición se refiere a:
- a) Capacidad del diseño para utilizar componentes más pequeños para construir un nuevo sistema.
 - b) Capacidad de un módulo de contener los efectos de un cambio dentro de sí, en vez de propagarlos a través del sistema en un efecto expansivo.
 - c) Posibilidad de especificar los componentes basándose en el análisis de requerimientos.
 - d) Posibilidad del diseño de permitir la reducción de la complejidad de un problema, al tener grandes problemas divididos en problemas más pequeños.

Respuestas de la Unidad 4: Examen de Autoevaluación

- 1) b
- 2) b
- 3) c
- 4) a, b y d
- 5) a, b, c y d
- 6) a, b y d
- 7) a
- 8) b y c
- 9) a y c
- 10) d

Unidad 5: Planeamiento del Software

Objetivos de Aprendizaje

Al finalizar esta unidad, usted será capaz de:

- Describir la planificación de proyecto.
- Explicar las diferentes actividades de la administración del riesgo.
- Discutir los diferentes pasos que tiene el planeamiento de un proyecto.
- Explicar en qué consiste la administración de la configuración del software.

1. Introducción

En la ejecución de un proyecto de desarrollo de software, hay múltiples involucrados, múltiples etapas de trabajo y múltiples recursos que necesitan ser administrados. La administración del proyecto juega un rol significativo en la ejecución exitosa de un proyecto de desarrollo de software. La administración de un proyecto tiene tres grandes fases:

- Planeamiento.
- Monitoreo y control.
- Análisis de culminación.

El planeamiento del proyecto es la responsabilidad más grande de la administración del proyecto. Al desarrollar un plan para desarrollar software, los objetivos de un proyecto deben ser cumplidos exitosamente y eficientemente.

La administración del tiempo es uno de los aspectos vitales en la administración de los proyectos de software. La falta de una apropiada administración del tiempo conduce frecuentemente a fechas límites imprácticas y también a presiones sobre los desarrolladores de software que pudieran ser evitadas. Un proceso apropiado, es por lo tanto necesario para maximizar la utilización del tiempo.

Los principales pasos para el planeamiento de un proyecto eficientemente se muestran en la Figura 5.1.



Figura 5.1 Planeamiento de un Proyecto

Seguidamente, se describen los diferentes pasos del planeamiento en forma breve, desde la estimación hasta el planeamiento organizacional.

2. Estimación

El primer paso en el planeamiento de un proyecto de software es la estimación. Esto implica estimación de costos y tiempo. La estimación provee al planificador del proyecto la información requerida para las otras actividades del proyecto.

3. Administración del Riesgo

La administración del riesgo tiene varias actividades, algunas de las cuales se listan a continuación:

- Análisis de Riesgo.
- Identificación de Riesgo.
- Proyección del Riesgo.
- Evaluación del Riesgo.

La administración del riesgo consiste principalmente de cuatro actividades, como se muestra en la Figura 5.2.

Se discuten cada una brevemente.

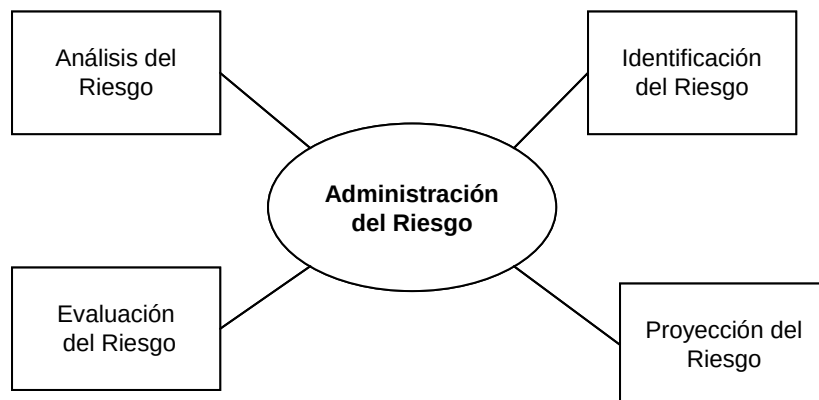


Figura 5.2: Análisis de Riesgo

3.1 Análisis de Riesgo

En el contexto de los proyectos de software, el riesgo se refiere a los problemas que pudieran afectar adversamente el costo, la calidad o el cronograma de un desarrollo de software. Otros factores como el método de implementación y operación de un programa o aplicación, el tipo de herramientas usadas, el número de personal involucrado, etc., son tratados también con análisis de riesgo. El análisis del riesgo

consiste en analizar cada riesgo identificado, para determinar la probabilidad de que pueda ocurrir y el daño que éste puede causar si ocurre.

3.2 Identificación del Riesgo

Los riesgos son usualmente agrupados bajo varias categorías. Son estos:

- Los Riesgos del Proyecto: Están relacionados a problemas de cronograma, de personal, de recursos, de requerimientos, etc.
- Los Riesgos Técnicos: Están relacionados con la escogencia de tecnología, plataforma, ambiente, asuntos de portabilidad, seguridad, confiabilidad, etc.
- Los Riesgos del Negocio: Involucran aspectos relacionados al retorno de la inversión y el tiempo requerido para cubrir gastos.

Agrupar riesgos bajo varias categorías ayuda a determinar los riesgos específicos del proyecto. Otro método para identificar los diferentes tipos de riesgos involucrados, es hacer uso de un conjunto de preguntas para cada uno de estos riesgos. Obtener las respuestas ayuda a un planificador de proyectos a entender el riesgo en términos técnicos.

3.3 Proyección del Riesgo

La proyección del riesgo apunta a medir cada elemento de riesgo realizando las siguientes preguntas:

- ¿Cuál es la probabilidad de que el riesgo sea real y que pueda ocurrir?
- Si este riesgo ocurre, ¿Cuáles son los problemas que se tendrán que manejar?

Cada pregunta en la lista de elementos de riesgo puede ser respondida con un 'Si' o un 'No' pero esto no es práctico. Sería mucho más realista responder tales preguntas con una escala de probabilidad cualitativa que indique la baja probabilidad en un extremo, moderado en el medio y alta probabilidad en el otro extremo.

Un tercer método de proyectar los riesgos es percibiendo el impacto del riesgo en el proyecto y luego darle prioridad. Los tres factores que influyen el impacto del riesgo son:

- La naturaleza del riesgo: Proporciona una idea acerca de los tipos de problemas que pueden darse si el riesgo ocurre. Código incorrecto, por ejemplo, resulta en errores durante la compilación o la ejecución de una aplicación de software.
- Alcance del riesgo: Este tiene que ver con la extensión del daño que el riesgo puede causar en realidad, junto con la distribución global. En otras palabras es un indicativo de cuánto se afectará al proyecto.
- Tiempo del riesgo: Analiza el tiempo y la duración para la cual el impacto se sentirá.

Las cuatro actividades que se dan en la proyección del riesgo son:

- Medir la probabilidad de ocurrencia del riesgo.

- Listar un conjunto de problemas que se necesita manejar si el riesgo ocurre.
- Hacer un estimado del impacto del riesgo en el proyecto en cuestión.
- Evaluar el nivel de confiabilidad con el cual se ha proyectado el riesgo.

Cualquier administración estará más preocupada con el impacto del riesgo y la probabilidad y no excesivamente preocupada con un factor de riesgo con un peso de impacto alto, pero con poca probabilidad de ocurrencia. Ellos se enfocarán en aquellos factores de riesgo que son de alto impacto, con una mediana probabilidad de ocurrencia o riesgos de bajo impacto con alta probabilidad de ocurrencia.

Estos puntos deben ser tratados apropiadamente y los pasos adecuados deben ser tomados para resolverlos eficientemente.

3.4 Evaluación del Riesgo

Los tres factores principales tomados en consideración durante la evaluación del riesgo son: riesgo proyectado, probabilidad del riesgo e impacto del riesgo.

Durante la fase de evaluación del riesgo, la exactitud de los cálculos hechos durante la proyección del riesgo es cuidadosamente examinada, y se les da prioridad a los riesgos. Las formas y medios para controlar estos riesgos también son estudiados.

Un nivel de referencia de riesgo tiene que ser definido por este análisis para que sea efectivo. El costo, cronograma y rendimiento son los tres niveles de referencia de riesgo para proyectos de software o desarrollo de procesos.

Cada uno de estos tres niveles: “excederse en los costos”, “demora en los cronogramas” o “rendimiento pobre”, tienen un cierto nivel alto. El trabajo en un proyecto determinará si un riesgo o una combinación de riesgos causan que se excedan alguno o una combinación de estos niveles de referencia.

En el análisis de riesgo de software, un nivel de referencia de riesgo tiene un solo punto, llamado punto de referencia o punto de quiebre. Es el punto donde la decisión de seguir adelante un proyecto o terminarlo, es aceptable. La cancelación puede ocurrir como resultado de muchos problemas en el proceso de desarrollo de software.

Los siguientes pasos son iniciados en la fase de evaluación de riesgo:

- Definir niveles de referencia de riesgo para un proyecto.
- Construir una relación entre los factores individuales, es decir, riesgo, probabilidad de riesgo e impacto de riesgo y niveles de referencia individuales.
- Llegar a un conjunto de puntos de quiebre que definirá la región de culminación.
- Entender la manera en la que las combinaciones compuestas de riesgos afectarán posiblemente un nivel de referencia.

4. Administración y Monitoreo de Riesgos

Las actividades que ayudan a administrar los riesgos se muestran en la Figura 5.3. En cada una estas actividades se estudian diferentes aspectos del riesgo. En la identificación del riesgo se establece si es un riesgo de proyecto, de producto o de negocio. En el análisis del riesgo se determina la probabilidad del riesgo y sus consecuencias. En la planificación se establece un plan para evitar o minimizar los efectos del riesgo y en el monitoreo del riesgo se hace un seguimiento del mismo a través de todo el proceso desarrollo.

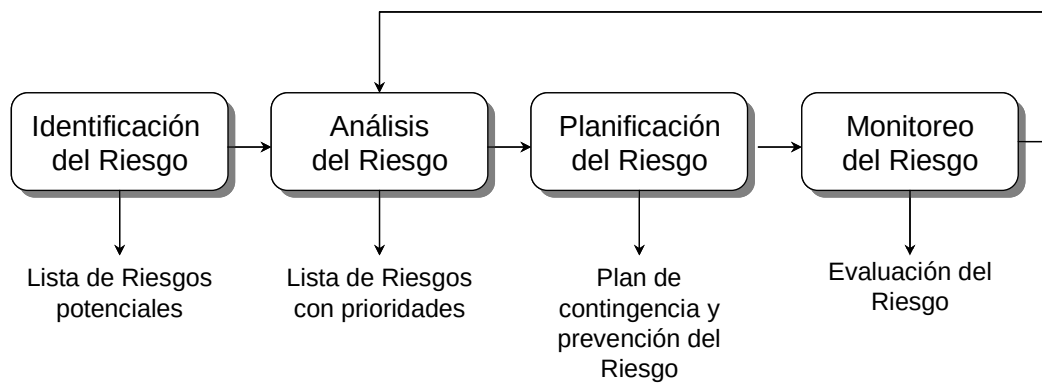


Figura 5.3: Actividades de la Administración de Riesgos

Los factores asociados con los riesgos individuales se toman como la base para generar los *pasos de administración del riesgo*. Se presenta un ejemplo para ilustrar este punto. Considere una compañía de software presenciando una alta pérdida de personal, la cual se toma como un riesgo proyectado. Basado en ocurrencias pasadas, suponga que la probabilidad de que la gente se vaya (probabilidad de riesgo) es cerca de 60%, un número alto y el impacto es probable que incremente el tiempo del proyecto en un 10%, incrementando el costo total en 15%. A continuación se presentan los pasos de administración del riesgo que la compañía debe tomar:

- Tener una reunión con el “staff” existente y determinar la razón por la cual se están retirando.
- Superar esas causas que están dentro del control de la compañía antes del comienzo del proyecto.
- Actuar asumiendo que las personas se retirarán aún después del comienzo del proyecto, además de tomar medidas para asegurar que el trabajo progrese aún cuando el personal se retire.
- Preparar equipos del proyecto y asegurar que la información acerca de cada proyecto circule ampliamente entre los trabajadores.
- Iniciar procesos de documentación estándar y usar mecanismos para asegurar que los documentos sean desarrollados a tiempo.
- Realizar revisiones de grupo de todo el trabajo que se está realizando.

- Implementar un plan de respaldo para cada trabajador que es crítico para un proyecto.

Los pasos para administrar el riesgo incrementan el costo del proyecto. Es por esto, que la administración tiene que asegurar que el gasto extra de dinero esté bien gastado. Esto implica que la administración del riesgo también involucra evaluar si los beneficios del proceso de administración del riesgo sobrepasan los costos incurridos cuando se implementan, por ejemplo, un análisis de costo-beneficio. Es deber del planificador del proyecto, analizar si el proceso de administración del riesgo vale el costo involucrado en implementarlo.

El número de riesgos involucrados varía de proyecto a proyecto, normalmente depende del tamaño del proyecto. Para un proyecto grande, pueden ser identificados cerca de 40 a 55 riesgos, mientras que para un proyecto pequeño, podría haber menos de 7 ó 9. Si para un proyecto grande, 4 a 6 pasos se identifican para cada riesgo, la administración del riesgo podría volverse por sí misma un proyecto independiente. Para evitar tales posibilidades, se hace uso de la regla de Pareto 80/20, que aumenta los riesgos de software. De acuerdo a esta regla, el 80% de todos los riesgos del proyecto, pueden ser ocasionados por el 20% de los riesgos identificados. Los planificadores de proyecto tendrán que identificar los riesgos que caen en ese 20 por ciento, observando el trabajo hecho en pasos previos del análisis de riesgo. De ser este el caso, algunos de los riesgos que han sido identificados, evaluados y proyectados podrían no figurar en el plan de administración del riesgo.

Los pasos de administración del riesgo están clasificados en Mitigación del Riesgo, Monitoreo y Plan de Administración (RMMMP). Este documenta todo el trabajo realizado como parte del proceso de análisis del riesgo. El gerente de proyecto entonces usa este documento como parte de un plan global del proyecto.

El siguiente paso es el monitoreo del riesgo, el cual es principalmente una actividad de rastreo con tres objetivos principales. Estos son:

- Evaluar si un riesgo que ha sido pronosticado realmente ocurre.
- Asegurar que los pasos de administración del riesgo han sido correctamente aplicados.
- Recolectar datos e información que puedan ser útiles en futuros análisis de riesgo.

Otra importante función del proceso de monitoreo de riesgo, es determinar cuál riesgo causa un problema en particular en el proceso de desarrollo.

El análisis de riesgo puede tomar cantidad de tiempo significativa del planeamiento del proyecto, pero vale el esfuerzo ya que resultará en procesos de desarrollo mejores, más rápidos y más eficientes, así como, productos o aplicaciones de una alta calidad.

A continuación se discute el cronograma del proyecto de software.

5. Cronograma del Proyecto de Software

El cronograma de un proyecto de software puede hacerse de dos maneras. En la primera, la fecha límite para la publicación del producto o programa ya ha sido fijada y la compañía tiene que distribuir el esfuerzo dentro de ese espacio de tiempo específico. En el segundo método, los períodos de tiempo aproximado han sido definidos y asignados, pero la compañía dará la fecha final. El esfuerzo es cuidadosamente distribuido a través del proceso y la compañía establecerá la fecha de publicación sólo después de un análisis comprensivo del software. Usualmente, el primer punto de vista prevalece más en las organizaciones de software que la segunda.

La certeza en el cronograma algunas veces juega un rol más importante que la exactitud en los costos. Volver a evaluar el precio del producto puede aumentar el costo, pero un retardo del cronograma probablemente tendrá un efecto negativo en la forma de reducción del impacto en el mercado, insatisfacción del cliente e incremento de los costos internos por problemas adicionales durante la integración del sistema. El cronograma del software debe tomar varios factores en consideración, incluyendo relaciones de personas-trabajo, definición de tarea, distribución de esfuerzos, métodos de planificación a ser implantados, seguimiento y control del proyecto, etc. Se aprenderá acerca de cada una de estas brevemente.

5.1 La Relación Trabajo-Personas

Es posible para una persona tomar un pequeño proyecto de desarrollo de software, analizar los requerimientos del producto, diseñar el producto, generar código y también probarlo todo el mismo. Pero a medida que el tamaño del proyecto aumenta, se requerirá de más personas para lograr el resultado final en un espacio de tiempo dado.

Si el proyecto está retrasado, incrementar el número de programadores seguramente acelerará el proceso. Pero también puede tener un impacto negativo en el proceso de desarrollo, resultando en más demoras. Esto se debe al tiempo y esfuerzo adicional incurrido para enseñar a los nuevos programadores acerca del sistema. Un incremento en el número de personas también conduce a un incremento en el número de canales de comunicación dentro del sistema, por lo tanto, aumenta la complejidad de los procesos de comunicación, lo cual toma tiempo.

Aquí, es posible hacer la siguiente pregunta: ¿Resultará contraproducente tener múltiples equipos en un proyecto? Realmente no. Pero los procesos de comunicaciones deben mejorar la calidad del software y el mantenimiento.

5.2 Paralelismo y Definición de Tareas

Los proyectos de desarrollo de software usualmente se emprenden en paralelo cuando más de una persona está involucrada en el proceso de desarrollo.

La base para tareas paralelas es el análisis, especificación y la revisión resultante de los requerimientos. Estas son las primeras tareas que se llevan a cabo en un proceso de software. Una vez que los requerimientos han sido identificados y revisados, las

actividades de diseño y planeamiento de pruebas se llevan en paralelo. Ya que un buen diseño de software es modular por naturaleza, se presta para el desarrollo en paralelo del diseño, codificación y pruebas. Las pruebas de integración empiezan tan pronto varios componentes de software han sido completados. Al final de todo, se realiza la prueba de validación, lo cual hace que el producto esté listo para la publicación al cliente.

Hitos colocados en intervalos regulares a través del proceso de desarrollo de software da al gerente, actualizaciones regulares del progreso del proyecto. Un hito se logra cuando la documentación que es parte del proceso de desarrollo ha sido revisada exitosamente. Un número de requerimientos de cronograma surge debido a la naturaleza concurrente del proceso de desarrollo de software. Como las tareas paralelas ocurren asincrónicamente, es responsabilidad del planificador determinar las dependencias entre tareas y asegurar el progreso continuo hasta completarlo. Además, el gerente de proyecto debe conocer qué tareas están en el camino crítico, es decir, cuáles deben ser completadas a tiempo para que el proyecto completo se culmine a tiempo.

5.3 Distribución de Esfuerzo

Todas las técnicas de estimación de software generalmente ayudan a determinar el número de mes-persona / año requeridos para terminar un proceso de desarrollo de software. El esfuerzo de distribución de un proyecto está dirigido sólo por las características de ese proyecto en particular. Generalmente, el análisis de requerimiento constituye sólo de un 10 a 25 por ciento del esfuerzo del proyecto. El esfuerzo del planeamiento del proyecto es usualmente de dos a tres por ciento del total de esfuerzo del proyecto. El esfuerzo invertido en prototipo o análisis usualmente se incrementa en proporción directa al tamaño y complejidad del proyecto. Otro 20 a 25 por ciento de esfuerzo se dedica al diseño del software. Junto con estos, se debe tener en cuenta el tiempo que toma la revisión de los diseños y la reiteración subsiguiente. Cuando el esfuerzo aplicado en el diseño de software es insignificante, el desarrollo del código debe ser relativamente fácil. En total, puede lograrse un rango de 15 a 20 por ciento. Probar y depurar toma otro 30 a 40 por ciento del esfuerzo de desarrollo de software. El tiempo de pruebas requerido para un software específico se decide por cuán crítico es el software.

5.4 Diferentes Modelos para Cronogramas

El cronograma de un proyecto de software es similar al cronograma de un esfuerzo de desarrollo multitarea. Esto significa que las herramientas y técnicas que se usan para el cronograma general del proyecto, también se pueden usar para el cronograma del proyecto de software sin modificaciones significativas. Los dos métodos más usados para los cronogramas de desarrollo de software son:

- Evaluación del Programa y Técnica de Revisión (PERT).
- Método de la Ruta Crítica (CPM).

Ambos métodos involucran el desarrollo de una descripción de una red de tareas para el cronograma del proyecto. Esta red, se incorpora usualmente en una tabla o representación gráfica de las diferentes tareas a ser realizadas durante el proceso de desarrollo de software. Se crea una lista de tareas, usualmente llamada la estructura de descomposición de trabajo (Work Breakdown Structure - WBS). Junto con la WBS, se hace una lista ordenada, la cual muestra el orden en el que las tareas deben llevarse a cabo.

Ambos el PERT y CPM proveen al planificador del proyecto de software de herramientas cuantitativas que le permiten hacer lo siguiente:

- Llegar a la ruta crítica, es decir, la secuencia de tareas que calcula el tiempo que tomará en completar el proyecto.
- Fijar los tiempos estimados para tareas individuales.
- Fijar los tiempos límites para calcular el tiempo aproximado en el cual el proyecto debe ser completado.

La determinación del límite de tiempo es muy importante en el cronograma del proyecto de software, ya que la demora o problema en el diseño de una función puede afectar el desarrollo de otras funciones.

El cálculo de los límites de tiempo provee al gerente de un método cuantitativo para evaluar el progreso del proyecto, así como también estimar cuándo se completarán las tareas en el proceso de desarrollo, aparte de ayudar a determinar los caminos críticos. Los planificadores deben recordar que el esfuerzo invertido en el software nunca se sobrepone con la etapa final de desarrollo. Los esfuerzos de mantenimiento figurarán como el factor de costos más grande, pero esto no se tratará en esta unidad.

5.5 Controlar y Hacer Seguimiento de un Proyecto de Software

Es vital hacer seguimiento del proceso de desarrollo de software, ya que se tiende a sobrepasar los tiempos del cronograma. El seguimiento se puede hacer de diferentes maneras:

- Teniendo reuniones del estado del proyecto. Todos los miembros reportan el progreso realizado y los problemas encontrados.
- Haciendo una evaluación de los resultados de las revisiones realizadas a través del proceso de desarrollo de software.
- Determinando si los hitos formales fijados anteriormente se han alcanzado para la fecha del cronograma.
- Comparando la fecha actual de comienzo con la fecha inicial planeada para cada tarea del proyecto listada en la tabla de recursos.
- Interactuando con otros equipos de software, para obtener evaluaciones subjetivas del progreso alcanzado y de los problemas que pueden encontrarse en el futuro.

Todas las técnicas anteriores para el seguimiento las utilizan los gerentes de proyectos en la vida real.

Los gerentes deben tener el control cuando:

- Administran recursos del proyecto.
- Tratan con varios problemas.
- Dirigen al staff del proyecto.
- Superan obstáculos en el proceso de desarrollo.
- Asignan recursos adicionales a las áreas de problemas que han sido identificados.

6. Decisiones de Adquisición

Es usualmente más rentable adquirir software que desarrollarlo. Los ingenieros de software se enfrentan usualmente con un dilema, si desarrollar el software o comprarlo. Este dilema se complica, además por la amplia variedad de opciones disponibles que pueden adquirirse:

- El software se puede comprar o adquirir una licencia.
- El software se puede adquirir y luego adaptarlo para que se ajuste a las necesidades.
- El software también puede ser hecho a la medida dando el desarrollo a un contratista externo.

La adquisición del software se basa usualmente en cuán crítico es y su costo final. Los siguientes lineamientos se pueden seguir para los paquetes de software más costosos:

- Determinar si el software se tendrá en menor tiempo si es desarrollado internamente a diferencia de ser adquirido.
- Revisar si el costo de adquisición y personalización del software es menor que el costo de un desarrollo interno.
- Comparar el costo de soporte interno y externo para determinar cuál será menor.

7. Re-ingeniería

Ciertas aplicaciones de software se pueden haber desarrollado con un diseño en particular, pero las diferentes adiciones (debido a nuevos requerimientos) y actividades de mantenimiento (debido a cambios en los procesos y/o errores) pueden afectar el diseño original. Estas aplicaciones son difíciles y costosas de mantener. Es posible rediseñar este sistema de software y desarrollarlo de nuevo (basado en la versión actual de trabajo y las lecciones aprendidas). Esto se llama re-ingeniería de la aplicación de software.

Las compañías que usan los sistemas de información tienen que lidiar con el problema de que el software se vuelve antiguo y redundante. Aunque mantener programas antiguos de software es caro, la mayoría de las compañías dudan en hacerles una re-ingeniería, ya que piensan que la re-ingeniería será más costosa que mantener los programas y aplicaciones antiguas. Pero la re-ingeniería de software es usualmente una opción de bajo costo en comparación al mantenimiento de software. Las compañías de software deben escoger programas que se van a usar por cinco años o una década.

- Deben hacer una estimación del costo anual de mantenimiento de los programas seleccionados, lo cual debe incluir el costo de corrección de error, de mejoras funcionales y adaptación al medio.
- A los programas seleccionados se les debe dar una prioridad, basándose en cuán críticos son y el costo de mantenimiento.
- El costo de reingeniería o reconstrucción de los programas seleccionados deben ser calculados, junto con su costo anual de mantenimiento.
- El costo de mantenimiento y el costo de reingeniería de cada programa seleccionado debe ser comparado, además de calcular el tiempo requerido para mostrar un retorno de la inversión en el caso de la reingeniería.

8. Planeamiento Organizacional

Una entidad del desarrollo del software tiene varias estructuras de organización de personas dentro de ella. Estas estructuras son bastante estáticas y no es fácil reestructurarlas o modificar su perspectiva, funciones, etc., una vez que las estructuras han tomado forma. No se puede cargar a los planificadores del proyecto con el trabajo de velar por los cambios de la organización, sino que pueden encargarse de la organización del personal que está involucrado directamente en un nuevo proyecto del software. Pueden asistir a la selección de las personas apropiadas para las tareas a mano.

Considere las opciones que un planificador tendrá de manera de asignar recursos humanos a un nuevo proyecto que requiera, por ejemplo, "x" número del personal y "y" número de años para concluirse:

- X individuos son asignados a un número k de diferentes tareas. La magnitud combinada de trabajo es menor; el gerente del software se debe encargar de coordinar el esfuerzo completo. Pero, este gerente puede también tener otros cinco proyectos en que ocuparse.
- X individuos son asignados a k tareas diferentes, donde $x < k$, para establecer equipos. Es posible designar a un líder del equipo. El gerente del software debe coordinar con los diversos equipos, mediante la interacción directa con el equipo o a través del líder del equipo.
- X número del personal se divide en z equipos. Asignan a cada equipo una o más tareas específicas, cada equipo tiene una estructura de organización específica; la coordinación es manejada por ambos, tanto por el equipo así como por el gerente del software.

Se ha encontrado que la tercera opción - organización formal del equipo - es la opción más productiva.

8.1 Una Breve Mirada en el Plan del Proyecto del Software

Cada fase en un proceso de desarrollo del software debe producir normalmente un entregable que pueda ser revisado y después formar la base para las actividades futuras. Esto resulta en el Plan del Proyecto del Software al final del planeamiento de las tareas. Proporciona el costo básico y la información de cronograma, que serán utilizados a través del proceso del software.

Un plan de proyecto del software debe ocuparse de los siguientes puntos:

- Estimación del Costo.
- Cronogramas e hitos.
- Plan del personal.
- Aseguramiento de la calidad de software.
- Plan de la administración de configuración.
- Plan de monitoreo del proyecto.
- Administración del riesgo.

La presentación del costo y del cronograma puede diferir, dependiendo de las audiencias a las cuales se está presentando. Los resultados de técnicas individuales de costeo pueden ser presentados si el plan del proyecto se utiliza como documento interno. Pero un informe detallado de reconciliación de costo será suficiente si el plan se presenta a una audiencia externa. Incluso los detalles ofrecidos en el cronograma variarán dependiendo de las audiencias

Un plan de proyecto del software no necesita ser muy técnico o muy largo, puesto que su objetivo es establecer la factibilidad del proyecto de desarrollo del software. Es generalmente una declaración general de los requerimientos y una declaración específica de los costos y del tiempo involucrado.

9. Administración de Configuración

La Administración de Configuración (CM, por sus siglas en inglés) o la Administración de Configuración del Software (SCM, por sus siglas en inglés) aplican la dirección técnica y administrativa, además de la vigilancia sobre el ciclo de vida de elementos. Sus funciones se dan en la forma de un diagrama de flujo en la Figura 5.4.

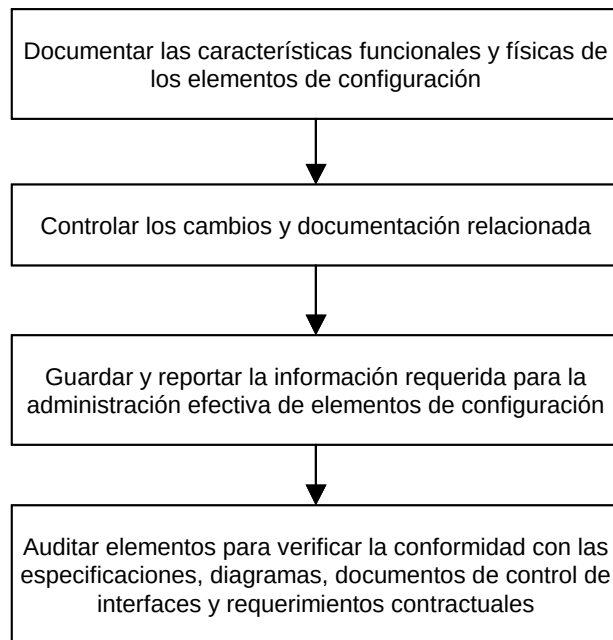


Figura 5.4: Funciones de la Administración de Configuración del Software

Los cuatro elementos claves de la administración de configuración se muestran en la Figura 5.5:

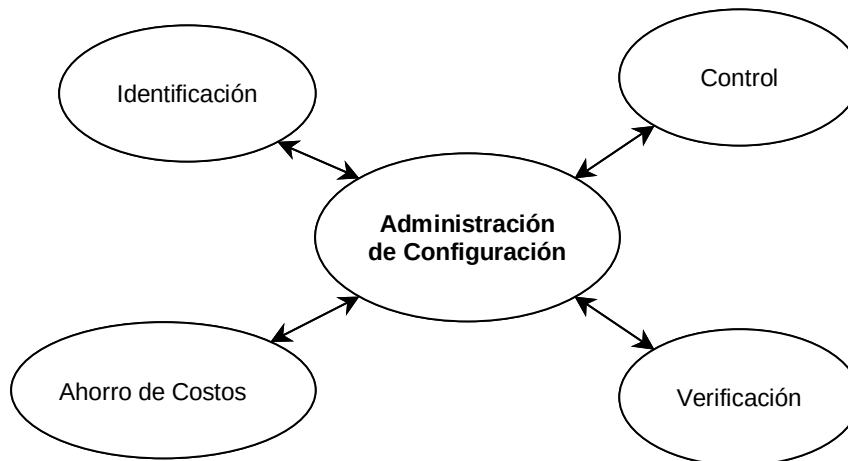


Figura 5.5: Elementos de la Administración de la Configuración

La administración de configuración puede también ser llamada como la práctica de manejar cambios sistemáticamente. Sus tres componentes se muestran en la Figura 5.6.



Figura 5.6: Componentes de la Administración de la Configuración

La administración de la configuración en el sentido tradicional incluye el control de entrada y salida de fuentes (y algunas veces binarios) además de la capacidad de realizar construcciones (o compilación) de entidades. Otras actividades incluyen la administración del proceso el cual es un control de las actividades de desarrollo de software. Por ejemplo el SCM debe verificar para asegurar que una solicitud de cambio existe y está aprobada para modificar; así como también que el diseño asociado, documentación y revisión hallan sido completados antes de permitir que el código sea 'ingresado' nuevamente. Mientras que la administración del proceso y control son necesarias para un proceso de desarrollo repetitivo y optimizado, una sólida base de administración de la configuración para el proceso es esencial.

SCM es responsable de un control ordenado de los productos de software durante sus desarrollos. También provee registro de cambios en el código, tanto durante el desarrollo como en producción. Este tipo de control ordenado ayuda a los desarrolladores a registrar cualquier cambio en el software (o sus proyectos relacionados, como la documentación). Asiste a los desarrolladores a identificar los módulos cambiados en respuesta a un error particular. Ayuda a recrear cualquier versión del producto. Permite a los desarrolladores el control de acceso a las fuentes y al repositorio en el cual las fuentes están almacenadas.

Una estrategia bien diseñada de SCM ayudará a identificar objetivos del proyecto, define los puntos de comienzo y final, además de cómo llegar de uno al otro. Esta estrategia también identificará y reproducirá entregables del proyecto en el tiempo, tomando en cuenta las necesidades particulares de los desarrolladores y de los que realizan las pruebas. Los proyectos del desarrollo pueden tomar caminos paralelos de desarrollo con la ayuda de SCM. Esto permite a los equipos del desarrollo y de mantenimiento trabajar con el mismo conjunto de fuentes, pero con diversas revisiones del archivo fuente.

Las ventajas de la administración de la configuración son:

- Implementar un buen proceso y sistema de SCM, ahorrará dinero si lo usa correctamente un personal de desarrollo bien entrenado. Esto se logra mejorando la calidad, reduciendo los problemas, realizando mantenimiento y haciendo reconstrucciones más confiables de los diferentes productos.
- Muchas compañías adoptan prácticas de SCM porque desean poder controlar y dirigir el proceso complejo lo mejor que puedan. Cuánto o cómo se miden los ahorros de costo pueden depender del seguimiento que la compañía ha estado haciendo a todos los gastos del desarrollo. Los gastos incluyen gastos en depuración, rediseño, correcciones, etc., a través de toda la vida del producto y

no solamente los gastos antes de la primera publicación. Si las compañías no tienen métricas para hacer el seguimiento, es poco probable que se puedan ver ahorros que puedan reconocer (incluso es posible ver la implementación de SCM como si costara más).

Las razones para que las compañías adopten la administración de la configuración son:

- El deseo de proteger su enorme inversión en software y de poder reproducir una estructura con los componentes correctos o continuar el desarrollo de un proyecto, aún si los que trabajaban previamente en él han salido de la compañía.
- El deseo de mejorar la calidad y de reducir los errores causados por construir productos con la versión incorrecta o con algún código viejo, el cual no incluye un último arreglo.
- La simplificación de un proceso complicado de construcción y/o procesos de lanzamiento.
- El deseo de racionalizar procesos y dejar a los desarrolladores preocuparse del desarrollo actual.
- La reducción del trabajo cotidiano, permitiendo así que un equipo falto de personal o sobre-ocupado produzca un trabajo más útil.
- Facilitar mover el personal de un proyecto a otro con poca o nada de pérdida de productividad puesto que todos los proyectos siguen el mismo proceso.
- La eliminación de instancias en las cuales el software necesitó investigar problemas reportados por el cliente no debe ser reproducidos o reconstruidos.
- Emplear nuevos miembros al equipo (particularmente líderes y/o gerentes) que hayan tenido buenas experiencias con SCM en negocios anteriores y apoyen esas mejoras.
- La necesidad de realizar el desarrollo concurrente en múltiples lugares. La ejecución del desarrollo concurrente en múltiples lugares es una actividad sensible que puede salirse fácilmente de control.
- Aumentar la confianza que el grupo de control de calidad (QA) pueda tener en una nueva versión del producto.

Una herramienta bien diseñada de SCM permite que una organización pueda alcanzar lo siguiente:

- Mantener una historia de los cambios sobre los archivos bajo control.
- Recrear las versiones anteriores del software fácilmente.
- Deshacer los cambios cuando hay un problema.
- Aplicar un nombre simbólico a un grupo de revisiones para agruparlas lógicamente.
- Producir reportes en archivos.

Adicionalmente, algunas de las otras características deseables incluyen la capacidad de lograr lo siguiente:

- Restringir el acceso a los archivos fuente, monitorear y restringir algunos privilegios a quienes se les ha dado acceso.
- Combinar dos revisiones de un archivo fuente o dos archivos diferentes, para crear un archivo fuente que incorpore las características y cambios en ambos.
- Almacenar los archivos ejecutables u otros archivos binarios.
- Alcanzar la compatibilidad a través de múltiples plataformas.
- Incorporar SCM fácilmente en el ambiente actual del desarrollo.

Resumen

Ahora que ha finalizado esta unidad, usted será capaz de:

- Describir los diferentes pasos que tiene el planeamiento de un proyecto.
- Explicar la administración del riesgo.
- Discutir el planeamiento y cronograma de un proyecto de software.
- Describir el planeamiento organizacional.
- Explicar la administración de la configuración del software.

Unidad 5: Examen de Autoevaluación

- 1) ¿Cuál de los siguientes no es uno de los pasos en el planeamiento del proyecto?
 - a) Estimación.
 - b) Análisis del riesgo.
 - c) Cronograma.
 - d) Análisis de requerimientos.

- 2) ¿Cuáles de las siguientes son actividades de la administración del riesgo?
 - a) Identificación del riesgo.
 - b) Proyección del riesgo.
 - c) Evaluación del riesgo.
 - d) Análisis del riesgo.

- 3) ¿Cuáles de los siguientes son maneras en las cuales la proyección del riesgo evalúa los riesgos?
 - a) La probabilidad que el riesgo sea verdadero.
 - b) Las consecuencias de los problemas conectados con estos riesgos.
 - c) Exceder el tiempo previsto.
 - d) Exceder el costo previsto.

- 4) ¿Cuáles son las actividades conectadas con la proyección del riesgo?
 - a) Establecer una escala para reflejar la probabilidad percibida de un riesgo.
 - b) Definir las consecuencias del riesgo dicho.
 - c) Hacer una estimación del impacto del riesgo dicho en el proyecto así como el producto.
 - d) Intentar determinar la exactitud total del riesgo dicho para evitar malentendidos.

- 5) ¿Cuál de los siguientes son métodos válidos para cronogramas?
 - a) PERT.
 - b) CPM.
 - c) Programación Lineal.
 - d) El Algoritmo de la Ruta Más Corta.

- 6) ¿Cómo se puede hacer el seguimiento del proyecto?
- a) Conduciendo reuniones periódicas del estado del proyecto en donde cada miembro del equipo tendrá que reportar el progreso alcanzado y los problemas encontrados.
 - b) Determinando los resultados de las revisiones conducidas a través del proceso del desarrollo.
 - c) Descubriendo si los hitos formales fijados anteriormente se han alcanzado en la fecha programada.
 - d) Comparando la fecha real de comienzo con la fecha planeada del comienzo para cada tarea listada del proyecto.
- 7) ¿Cuáles son los elementos de la administración de la configuración?
- a) Identificación de la configuración.
 - b) Control de configuración.
 - c) Verificación del estado de la configuración.
 - d) Revisión.
- 8) ¿Cuál de los siguientes son componentes de la administración de la configuración?
- a) Control de la construcción.
 - b) Control de la versión.
 - c) Control de cambio.
 - d) Control de calidad.
- 9) ¿Qué pueden lograr los desarrolladores con el control ordenado disponible a través de la práctica de la administración de la configuración del software?
- a) Seguir el quién, el qué, el cuándo y porqué de los cambios al software (o a cualesquiera de sus proyectos relacionados, ejemplo la documentación).
 - b) Identificar los módulos cambiados en respuesta a un arreglo particular de error.
 - c) Recrear cualquier versión del producto.
 - d) Controlar el acceso a las fuentes y al repositorio en los cuales se almacenan las fuentes.

- 10) ¿En cuáles de las siguientes situaciones deben los gerentes de un proyecto de software ejercer el control y seguimiento?
- a) Cuando se asignan recursos adicionales a las áreas de problemas identificadas.
 - b) Cuando se tienen reuniones del estado del proyecto.
 - c) Cuando se tratan con varios problemas.
 - d) Cuando se administran recursos del proyecto.

Respuestas de la Unidad 5: Examen de Autoevaluación

- 1) d
- 2) a, b, c y d
- 3) a y b
- 4) a, b, c y d
- 5) a y b
- 6) a, b, c y d
- 7) a, b, c y d
- 8) a, b y c
- 9) a, b, c y d
- 10) a y b

Volumen 2: Pruebas y Calidad

Unidad 1: Fundamentos de Pruebas del Software

Objetivos de Aprendizaje

Al finalizar esta unidad, usted será capaz de:

- Discutir diversos niveles de prueba.
- Describir el diseño del caso de prueba.
- Explicar algunos de los métodos de prueba Caja Blanca.
- Explicar algunos de los métodos de prueba Caja Negra.

1. Introducción

La prueba es una de las fases más importantes del ciclo de vida de desarrollo del software. Un producto de software que se desarrolla, se debe entregar al cliente libre de defectos o de errores. La prueba es el proceso de ejecutar un programa con la intención de descubrir defectos en el programa.

En el ciclo de vida de desarrollo del software, la fase de prueba ocurre en la penúltima etapa, es decir, después de la fase de programación pero antes de la fase de implantación del programa. Cualquier retraso de tiempo que pudiera haber ocurrido durante las fases anteriores, tales como análisis de requerimientos, diseño y programación, tienden a colocar una gran presión en la fase de prueba. Pero no se puede hacer ningún compromiso en la fase de prueba, ya que esto resulta en la entrega de un producto de software defectuoso. Estos defectos pueden dar un sin fin de problemas que pueden ser desde menores hasta catastróficos. Por lo tanto, la fase de prueba se debe realizar de la manera más robusta y eficiente.

El proceso de prueba se debe llevar a cabo bajo condiciones controladas, como en cualquier otro proceso científico. Esto es necesario de manera que se pueda replicar el funcionamiento erróneo de los programas y que la fuente del defecto sea detectada y corregida. Estas condiciones controladas deben implicar las condiciones normales que conducen a los resultados correctos y esperados, así como las condiciones anormales que conducen a los resultados erróneos e inesperados. El proceso de prueba se debe diseñar para revelar todos los defectos no detectados en el software.

¿Cómo se incluyen los defectos en el software? Uno de los mitos frecuentes entre muchos principiantes en la profesión del desarrollo de software es que estos defectos son incluidos en el software por el programador solamente durante la fase de programación. Esta creencia está lejos de la verdad. Los defectos o los errores pueden incluirse durante cualquiera de las fases, a partir de la fase de análisis de requerimientos hasta la fase de mantenimiento después de que el software se haya entregado al cliente. Por supuesto, se admite que cierta clase de defectos es incluida por el programador comúnmente durante la fase de programación.

El proceso de prueba implica la planificación y selección de casos de prueba (el conjunto de entradas de datos usados para probar el software) de tal forma que ayuden a descubrir el máximo número de defectos. Uno de los métodos es seleccionar el tipo de caso de prueba, que asegure que todos los posibles caminos en un programa sean ejecutados por lo menos una vez. Esto es un objetivo altamente deseable, pero no es fácil de lograr.

¿Es la prueba responsabilidad de un sólo individuo, de un grupo de prueba o del equipo de desarrollo? Esto depende de la organización y de la forma como se ha estructurado la función de aseguramiento de calidad y prueba. En algunos casos, es responsabilidad de una sola persona. Comúnmente, la responsabilidad se asigna a un grupo encargado de realizar las pruebas. A menudo, un grupo de prueba y un grupo de desarrolladores trabajan de cerca bajo supervisión de un gerente de proyecto. La estructura de los

equipos de prueba depende del tamaño de la organización y de la manera en la que se estructura el negocio.

2. Beneficios de la Prueba

Los beneficios de adoptar buenas metodologías y procesos de prueba en una organización son:

- Acentuar la calidad del software hará que los programadores y quienes realizan las pruebas del programa se den cuenta de la necesidad de un software sin error.
- Los programas de computadora analizados desde el punto de vista de quien realiza las pruebas, asegura casi automáticamente la detección de las clases de errores más peligrosos, tales como los que tienden a hacer caer el sistema.
- Un proceso de prueba sistemático que actúa como respaldo para otras técnicas, tales como revisiones de diseño y guías estructuradas, incluso si no identifica errores significativos.
- Establecer una actividad de prueba sistemática permitirá que nuevas tecnologías de aseguramiento de la calidad sean aplicadas tan pronto como estén disponibles.

3. Niveles de Prueba

Probar el software tiene cuatro niveles. Estos son:

- Prueba de Unidad.
- Prueba de Integración.
- Prueba del Sistema.
- Prueba de Aceptación.

A continuación se discute brevemente cada uno de estos niveles.

3.1 Prueba de Unidad

La prueba de unidad se realiza durante la fase del desarrollo. El término 'unidad' denota un programa, un módulo, un objeto ActiveX o DLLs. El énfasis de la prueba de unidad está en el funcionamiento apropiado del programa individual.

3.2 Prueba de Integración

La prueba de integración se relaciona con la prueba de sistemas mediante la interacción. La integración, en este contexto es de los subsistemas bajo consideración. En la prueba de integración, el método de prueba se enfoca en el flujo de datos entre los subsistemas.

3.3 Prueba del Sistema

La prueba del sistema denota la prueba de módulos o de programas relacionados, sobre una plataforma, en un ambiente restringido, sin el acceso a otros sistemas, con el fin de probar este sistema. Los elementos como 'stubs' o 'archivos vacíos' se usan cuando otros sistemas necesitan ser integrados con el sistema bajo prueba. La prueba del sistema pone énfasis en el flujo de datos de un programa a otro dentro del sistema bajo prueba. Los sistemas externos al sistema bajo prueba son ignorados.

3.4 Prueba de Aceptación

La prueba de aceptación se relaciona y define la aceptación formal de un producto acabado. Comprueba si el producto satisface los requerimientos originales del negocio. Es realizado por los representantes del negocio, usando los documentos originales de los requerimientos como referencias y no por el personal técnico.

Todos estos cuatro niveles de prueba son importantes y se realizan, generalmente en el mismo orden, durante el desarrollo de la mayoría de los productos de software. Cada nivel de prueba se construye sobre el anterior, asumiendo que el nivel anterior se ha realizado por completo y correctamente.

La prueba se ve comprometida, especialmente en proyectos grandes, puesto que es el penúltimo paso en el ciclo de vida de desarrollo del software y las fechas límites ponen una gran presión en la organización. Muchas compañías también saltan completamente la fase de prueba del sistema, con la esperanza de que la prueba de integración detecte cualquier problema presente. Esto significa que el mayor énfasis está puesto en la prueba de integración. Pero los problemas descubiertos son mucho más difíciles de resolver en esta etapa, puesto que se encuentran varios niveles por encima del punto de generación y diferentes conjuntos de personas están involucradas. Muchas compañías también deciden saltar la prueba de aceptación formal cuando las fechas límites están cercanas. Esto puede ser un error grave, especialmente en proyectos grandes, ya que es la única vez que cualquier persona mira lo que se está entregando y comprueba realmente si el producto final satisface los requerimientos del cliente.

La única manera que un proyecto grande puede ser puesto en ejecución correctamente, es realizar pruebas rigurosas en todos los niveles. Las siguientes son algunas de las estipulaciones referentes a pruebas rigurosas en todos los niveles.

- Es responsabilidad de los programadores entregar un producto, trabajando sin errores.
- La prueba de integración no comienza hasta que todos los módulos se han probado individualmente.
- La prueba del sistema no comienza hasta que se ha terminado la prueba de integración.
- La prueba de aceptación no comienza hasta que se ha terminado la prueba del sistema.

Los líderes de los equipos en cada etapa tendrán documentos para demostrar que la prueba se ha implementado correctamente. La prueba se detiene si se nota que los criterios de entrada no se han satisfecho en otra etapa. El trabajo entonces se transfiere a los niveles anteriores, para rectificar el problema antes que la prueba se reanude.

Hay dos maneras extremas de realizar pruebas a sistemas:

- Prueba improvisada.
- Prueba automatizada.

A continuación se discuten estos métodos.

3.5 Prueba Improvisada

La prueba improvisada implica el hacer casos de prueba en el momento, lo más rápido posible. Esta clase de prueba detecta generalmente la mayoría de los errores en un nuevo software, en el tiempo más corto posible. Pero no es completo, puesto que es difícil pensar todo en una sesión corta. La prueba improvisada es a menudo emocionante y gratificante, dando a aquellos que realizan la prueba la satisfacción de hacer fallar las cosas.

3.6 Prueba Automatizada

La prueba automatizada se realiza con una herramienta de prueba que puede generar acción en el teclado, clic del ratón automáticamente, supervisar el contenido, así como posición de ventanas y de botones. Los ejemplos de las herramientas automatizadas de prueba son SQA, WinRunner, MS Test y Visual Test. La ventaja de una herramienta automatizada de prueba es que los scripts se pueden generar para probar una gama de condiciones y después almacenarlos en una librería. Más adelante, cuando se ha cambiado el software, estas pruebas se pueden usar automáticamente para verificar los cambios.

La desventaja de una herramienta automatizada de prueba, es que la mayoría de estas requieren un cierto conocimiento de programación, puesto que muchos de los scripts generados automáticamente pueden requerir de alguna modificación. Esta modificación no es una tarea fácil. Se puede perder mucho tiempo en la regeneración, si la interfaz de usuario que es probada experimenta muchos cambios. Pero el hecho es que, las herramientas automatizadas pueden emprender una cantidad considerable de pruebas estándar antes de cada lanzamiento de un producto de software. Se asegura por lo menos que las funciones probadas trabajen.

4. Técnicas de Prueba de Software

La prueba del software es el proceso de ejecutar software con el propósito de probar su funcionalidad y exactitud. La prueba del software se realiza generalmente por las siguientes razones:

- Detección de defecto.
- Estimación de la confiabilidad.

El éxito del proceso completo de prueba del software depende de muchas piezas de código. Se compromete el proceso completo si incluso uno de los pedazos de código falla. La clave de la prueba del software consiste en intentar detectar todos los modos de falla, algo que requiere la prueba exhaustiva del código con todas las entradas posibles. Pero la prueba exhaustiva no es una proposición factible. Lo que realmente sucede, es que los intentos están hechos para probar tantas características sintácticas de código como sea posible. Para ello, se emplean dos técnicas importantes, estas son:

- Prueba Caja Blanca.
- Prueba Caja Negra.

Las técnicas de prueba de software de Caja Blanca intentan probar tanto del código como sea posible, dentro de un cierto conjunto de restricciones de recursos; mientras que las técnicas que no se preocupan de la estructura del código cuando se seleccionan los casos de prueba, se llaman técnicas de Caja Negra. A continuación se discuten ambas técnicas detalladamente.

5. Prueba de Caja Blanca

La prueba de caja blanca, también llamada la prueba de Caja de Cristal, se basa en el conocimiento de la estructura y de las sentencias del programa, además requiere un conocimiento exhaustivo del código del programa.

Utiliza las estructuras de control del programa y del diseño procedimental empleados en el código para derivar casos de prueba. La Figura 1.1 describe los objetivos de la prueba Caja Blanca.

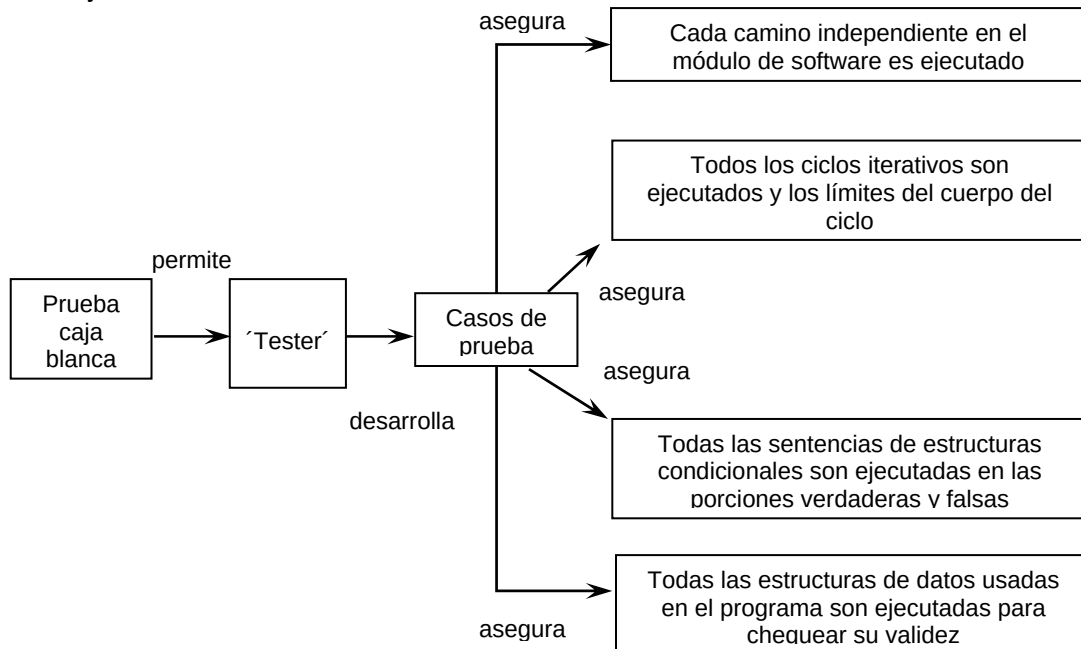


Figura 1.1: Objetivos de la Prueba de Caja Blanca

La prueba Caja Blanca emplea los dos métodos de prueba siguientes:

- Prueba de la Estructura de Control.
- Prueba de Camino Base.

No son realmente dos métodos diferentes, puesto que la prueba de camino base se utiliza como uno de los métodos de prueba de la estructura del control. Sin embargo, se hace una diferenciación puesto que la prueba del camino base logra algo más que solo realizar la prueba de la estructura del control.

5.1 Prueba de Estructura de Control

Los siguientes son algunos de los métodos principales de prueba de estructura de control:

- Prueba de Condición.
- Prueba del Flujo de Datos.
- Prueba de Bucles.

A continuación se explica brevemente cada uno de los métodos para la prueba de estructura de control.

5.1.1 Prueba de Condición

La prueba de condición se utiliza para diseñar los casos de prueba que examinan las condiciones lógicas en un programa. Analiza todas las condiciones en el programa e incluye la prueba de expresiones relacionales y aritméticas. Esto se puede hacer usando los dos siguientes enfoques:

- **Prueba de Bifurcación:** Ejecuta las ramas verdaderas y falsas de una condición al menos una vez.
- **Prueba del Dominio:** Utiliza valores en el lado izquierdo de la relación haciéndolos mayores que, igual o menor que el valor del lado derecho. Este método prueba ambos valores, así como, operadores relacionales en la expresión y se utiliza tres o cuatro pruebas para cada operador relacional.

Los errores en expresiones pueden ser debido a las siguientes razones:

- Error del operador booleano.
- Error de la variable booleana.
- Error de paréntesis booleano.
- Error del operador relacional.
- Error de la expresión aritmética.

A continuación, se definen los siguientes términos para permitir el entendimiento de la prueba de condición:

- **Expresión Relacional:** Una expresión relacional consiste en dos expresiones aritméticas conectadas por un operador relacional. Por ejemplo, $(a+b) < (c/d)$.
- **Condición Simple:** Es solo una variable booleana o una expresión relacional, precedida posiblemente por un operador NOT. Por ejemplo: $x = y$, $\text{NOT}(x = y)$.
- **Condición Compuesta:** Se compone de dos o más condiciones simples, operadores booleanos y paréntesis. Algunos ejemplos de condiciones compuestas son:
 $x = y \ \&\& \ c/d$
 $!(a < b) \ \&\& \ (e > 10)$
- **Expresión Booleana:** Es solo una condición sin expresiones relacionales. Algunos ejemplos de expresiones booleanas son:
 $\text{isGenius} \ \&\& \ \text{highIQ}$
- $\text{isGenius} \ \&\& \ !(highIQ)$

A continuación se presentan algunos ejemplos para ilustrar la prueba de condiciones.

Ejemplo 1.1: Prueba de Condición para Determinar el Máximo de dos Números Enteros

Se da a continuación el programa en C que determina cuál es el máximo de dos enteros:

El código en C comienza aquí...

```
#include <stdio.h>
main(){
    int num1, num2;
    /* pide al usuario que ingrese los valores de num1 y de
    num2*/
    printf("Introduzca 2 enteros: ");
    scanf("%d %d",&num1,&num2);
    printf("Valor de num1 : %d\n", num1);
    printf("Valor de num2 : %d\n", num2);

    /* comprobar que ambos números tienen mismo valor */
    if (num1 == num2)
        printf("Los números son iguales\n");
    /* comprobar si el valor de num1 es mayor que num2*/
    else if (num1 > num2)
```

```
        printf("El máximo es %d\n", num1);  
    /* comprobar si el valor de num1 es menor que num2 */  
    else  
        printf("El máximo es %d\n", num2);  
} /* End de main */
```

El código de C termina aquí...

Según la estrategia de prueba de bifurcación, el caso de prueba debe ser preparado de tal manera que cada rama se ejecute por lo menos una vez. Las dos sentencias de bifurcación que ocurren en este programa son:

```
if (num1 == num2)  
else if (num1 > num2)
```

Para probar la primera sentencia de bifurcación, se puede tener {num1 = 10, num2 = 10}.

Para probar la segunda sentencia de bifurcación, se puede tener {(num1 = 10, num2 = 15), (num1 = 15, num2 = 10)}.

De acuerdo a la estrategia de prueba de dominio, se pueden utilizar tres o cuatro casos de prueba para cada operador relacional. Por lo tanto, los casos de prueba para la primera sentencia de bifurcación dada pueden ser:

num1	num2
10	10
5	15
15	5

Los mismos casos de prueba se pueden utilizar con la segunda declaración de la bifurcación dada.

Fin del Ejemplo 1.1

Ahora, ¿cuáles deben ser los casos de pruebas para probar la siguiente condición?

$(x < y) \ \&\& \ (u > v)$

Se sabe que para la expresión $(x < y)$, se pueden tener los siguientes casos de prueba:

x	y
10	10
5	15
15	5

Para la expresión $(u > v)$, se pueden tener los siguientes casos de prueba:

u	v
10	10
5	15
15	5

Pero cuando se tiene que considerar la expresión $(x < y) \ \& \ (u > v)$, se deben tener los siguientes casos de prueba:

x	y	u	v
10	10	10	10
5	15	10	10
15	5	10	10
10	10	5	15
5	15	5	15
15	5	5	15
10	10	15	5
5	15	15	5
15	5	15	5

La tabla anterior muestra que si el número de variables en la condición es grande, se requiere que se realicen más pruebas. En general, si se tienen n variables se deben realizar 2^n pruebas. La limitación es que este tipo de clase de prueba solamente es factible cuando el valor de n es razonablemente pequeño.

A continuación se procede a discutir la prueba de flujo de datos.

5.1.2 Prueba de Flujo de Datos

El método de prueba de flujo de datos sirve para localizar los caminos de prueba de un programa según las variables usadas en el programa. Hay muchas variaciones del método de prueba de flujo de datos. En esta unidad se explicará un método muy

general y simple. Los pasos involucrados en la prueba de flujo de datos se explican a continuación:

- Enumerar cada sentencia del programa fuente que se probará.
- Asegurar que el programa que es probado no modifique las variables globales. Si lo hace, hacer un intento para rediseñar e implementar el programa tal que no modifique las variables globales. Este paso no es parte de la prueba de flujo de datos, pero es esencial para la aplicabilidad correcta del método.
- Asegurar que los subprogramas que son probados no modifiquen los parámetros dados por los programas que los llaman. Si ocurre esto, intente rediseñar e implementar los subprogramas de tal forma que no modifique sus parámetros. Este paso no es parte de la prueba de flujo de datos, pero es esencial para la aplicabilidad correcta del método.
- Considere una variable en el programa en el cual la prueba de flujo de datos debe ser realizada. Sea esta variable x .
- Localice el número de sentencia en la que se define o inicializa la variable. Sea este número de sentencia m .
- Localice los números de sentencia en los cuales se utiliza la misma variable. Sean estos números de sentencia $n_1, n_2, \dots, n_x$.
- Defina el conjunto de declaraciones entre los números de declaración m y n_1 como una cadena de definición-uso. El conjunto de sentencias entre los números de sentencia m y n_2 forman otra cadena de definición-uso. De este modo, se puede definir la cadena de definición-uso como un conjunto de números de sentencia a partir de un número de sentencia donde una variable es definida hasta un número de sentencia donde la misma variable es utilizada.
- El método de prueba de flujo de datos basado en las cadenas de definición-uso, indica que el conjunto de sentencias en cada cadena de definición-uso debe ser cubierta por los casos de prueba por lo menos una vez.

Los pasos involucrados en la prueba de flujo de datos serán ilustrados con un ejemplo. Se toma el ejemplo de un programa que determina el número de dígitos en un entero positivo.

Ejemplo 1.2: Prueba de Flujo de Datos de un Programa que Cuenta el Número de Dígitos en un Número Entero Positivo.

El programa siguiente cuenta el número de dígitos en un número entero positivo:

```
1. # include <stdio.h>
2. main(){
3.   int num, digit, save;
4.   /* Pide al usuario ingresar un número positivo */
5.   printf("Introduzca un entero positivo : ");
6.   scanf("%d",&num);
7.   /*Chequea que el número ingresado sea positivo */
```

```
8.  if (num <= 0)
9.      printf("%d no es un entero
positivo\n",num);
10.     else {
11.         /*Cuando el número es mayor que cero */
12.         save = num;      /* guarda el número */
13.         digit = 1;
14.
15.         /*Extrae cada dígito del número y cuenta */
16.         while (num/10 != 0) {
17.             digit++;
18.             num = num /10;
19.         }/*Fin del while*/
20.         printf("El número de dígitos en %d es : \
%d\n",save,digit);
21.     }/*Fin del else*/
22. }/*Fin del main*/
```

Nota: Las sentencias en el programa se numeran. Se seguirán los pasos para la prueba descrita anteriormente.

Se selecciona la variable num para localizar las cadenas de definición-uso. La variable num se define en la sentencia número 6, este es el m definido anteriormente. La misma variable se utiliza en los números de sentencia 8, 9, 12 y 16. De acuerdo a la definición de anterior de cadena de definición-uso, se tienen las siguientes cadenas de definición-uso:

6-8
6-9
6-12
6-16

Hay cuatro cadenas de definición-uso. Según este método de prueba, cada una de las cadenas de definición-uso deben ser cubiertas por lo menos una vez por los casos de prueba.

Se puede ver que si se considera la cadena de definición 6-16, se cubren todas las cadenas de definición-uso anteriores. También se observa que num se define otra vez en la sentencia número 18. El siguiente uso de éste, está en la sentencia número 16. Claramente, la cadena de definición-uso abarca la construcción iterativa entre los números de sentencia 16 a 18.

Una vez que las cadenas de definición-uso se identifican, se puede recurrir a identificar los casos de prueba que cubrirán las cadenas de definición-uso por lo menos una vez.

Fin del ejemplo 1.2

Del mismo modo, es posible hacer las pruebas de flujo de datos para todas las variables usadas en un programa. Las pruebas de flujo de datos pueden revelar errores con eficacia cuando se prueban los caminos seleccionados. La desventaja de las pruebas de flujo de datos es que seleccionar los caminos es difícil para los programas complejos.

A continuación se discute otra prueba de estructura de control, la prueba de bucle.

5.1.3 Prueba del Bucle

Muchos algoritmos se basan en las construcciones iterativas, que implican bucles. La prueba del bucle es un método de prueba de estructura del control que trata de detectar errores en bucles.

Considere un ejemplo que utiliza una construcción simple del bucle, para aplicar este método de prueba.

Ejemplo 1.3: Prueba de Bucle para un Programa con Bucle Simple

En este ejemplo, se considera un programa para contar el número de dígitos en un número entero positivo. Para que el programa realice esta tarea, se presenta el ejemplo 1.3. La única construcción de bucle que se tiene se reproduce a continuación.

```
1.      while (num/10 != 0) {  
2.          digit++;  
3.          num = num /10;  
4.      }/*Fin del while*/
```

La prueba de bucle implica generar casos de prueba tales que el bucle se pruebe a fondo. Lo siguiente, puede servir como pautas generales:

- Seleccionar casos de prueba que no ejecutan el bucle.
- Seleccionar casos de prueba que ejecuten el bucle exactamente una vez.
- Seleccionar casos de prueba que ejecuten el bucle más de una vez, hasta un máximo número de veces (sí ese valor es conocido). En este ejemplo, no se puede decir el máximo número de veces que el bucle se ejecutará.
- Seleccionar casos de prueba que verifican si el bucle siempre termina.

En este ejemplo específico, los siguientes casos de prueba se pueden utilizar:

num = 1	El bucle no se ejecuta.
num = 22	El bucle se ejecuta exactamente una vez.
num = 32767	El bucle se ejecuta varias veces.
num = 65537	Otro número grande para chequear si el bucle termina.

Otros valores de num, tal como el 0 ó un número negativo no son apropiados aquí, debido a que el programa verifica por ellos y no los considera para que sean ejecutados en el bucle.

Fin del Ejemplo 1.3

A continuación se presenta otro ejemplo para ilustrar la prueba de bucle en programas que tienen bucles anidados.

Ejemplo 1.4: Prueba de Bucle para un Programa con Bucles Anidados

Considere el problema de encontrar ordenar un arreglo usando el método de burbuja. El siguiente programa realiza esta función:

El código de C comienza aquí...

```
#include <stdio.h>
main(){
    int arreglo[6]={260, -15, 20, 30, 11, -19};
    int i, temp, interchange, j;

    interchange = 1;
    j = 1;
    while(interchange) {
        interchange = 0;
        for(i=0; i < n-j; i++) {
            if(arreglo[i] > arreglo[i+1]) {
                // Intercambia los elementos
                temp = arreglo[i];
                arreglo[i] = arreglo[i+1];
                arreglo[i+1] = temp;
                interchange = 1;
            }
        }
        j++;
    }/*Fin del while*/
}/*Fin del main*/
```

El código en C termina aquí

El programa anterior utiliza un bucle anidado. La prueba de bucles anidados es un ejercicio bastante simple y directo. Sin embargo, el único problema es que el número de

pruebas requeridas tiende a ser inmanejable. Uno de los métodos prácticos de hacer la prueba con las estructuras anidadas es como sigue:

- Mantener los bucles externos fijos para valores mínimos de las variables de control del bucle.
- Probar el bucle más interno completamente como si fuera un bucle simple.
- Luego, de la misma manera, considere el bucle externo y pruébelo manteniendo todas las variables de bucles más externos en valores mínimos.

En este ejemplo, se mantiene el valor de `interchange` en el valor que satisface el `while`. Entonces se prueba el bucle interno, es decir, el bucle `for`.

Por un momento, considere que el bucle externo es también un `for` que implica una variable del control del bucle `x1`, como se presenta a continuación:

```
for(x1 = 1;; x1 <=1000; x1++) {
```

El método de prueba del bucle indica que se debe mantener el valor `x1` en el mínimo, es decir en 1 y proceder a probar el bucle interno. Una vez que se termine eso, el bucle externo puede ser probado. En este caso, no hay otro bucle así que puede ser probado como bucle simple.

Fin del Ejemplo 1.4

Muchos programas pueden ser escritos tales que los bucles no estén estructurados, estos se denominan bucles no estructurados. Los programas que involucran bucles no estructurados no son favorables para la prueba y es recomendable que tales programas sean rediseñados con bucles estructurados, ya sean simples o anidados.

A continuación se discute uno de los métodos más poderosos de pruebas de Caja Blanca, la prueba del Camino Base.

5.2 La Prueba del Camino Base

La prueba del camino base, también llamada prueba estructural, es la principal estrategia de prueba basada en el código. La premisa fundamental detrás de este tipo de prueba es que los resultados de la decisión dentro de una función de software se deben probar independientemente. Verifica las interacciones entre las construcciones en vez de simplemente ejecutar la construcción. Es decir, es insuficiente solo ejecutar cada una de las construcciones (una construcción condicional o una construcción iterativa). Es necesario verificar la interacción entre cualquiera de estas construcciones.

Cuando se tiene que probar un programa que implique sentencias condicionales, la creencia común es que es adecuado probar cada una de las sentencias y cada una de las condiciones. De hecho, esto puede ser inadecuado en términos de descubrir errores. Antes de presentar un ejemplo que ilustre el uso de la prueba de camino base se explicará cómo calcular el valor de la complejidad ciclomática de un código, valor necesario para desarrollar la prueba de camino base.

El Número de Complejidad Ciclomática de McCabe

Numerosos esfuerzos se han hecho para medir la complejidad de los algoritmos. Una de las cosas más deseables es ser capaz de asignar un valor numérico que signifique la complejidad del algoritmo. Thomas McCabe desarrolló una métrica llamada número de complejidad ciclomática en 1971. Útil como un indicador de la dificultad de prueba y mantenimiento de un programa o módulo.

Esta medida calcula el número de caminos independientes en un programa, lo cual conduce a un valor numérico de la complejidad. Para calcular este valor de complejidad, se debe representar el programa o módulo, que se está considerando como un grafo. En general, cualquier algoritmo o programa puede ser representado como un grafo. Sea G al grafo del programa que se está considerando. La complejidad ciclomática del grafo G se denota como V y se puede calcular por cualquiera de las siguientes tres (3) maneras:

- $V(G) = \text{Número (aristas)} - \text{Número (nodos)} + 2$.
- $V(G) = \text{Número de regiones de } G$, donde una región de un grafo es un área delimitada por aristas y nodos. Al contar las regiones de un grafo también se cuenta el área exterior del grafo como una región más.
- $V(G) = \text{Número de nodos predicados en } G + 1$ (se denomina nodo predicado al que contiene una condición).

Aplicar cualquiera de los anteriores métodos debe coincidir con el número de caminos independientes del programa. Donde, se conoce como camino independiente a cualquier camino que contemple al menos un conjunto de sentencias, no considerado en caminos anteriores.

Se presentan a continuación un ejemplo para ilustrar el proceso para calcular esta métrica.

Ejemplo 1.5: Complejidad Ciclomática de una Función para Insertar un Nodo en un Árbol.

En este ejemplo, se considera una función escrita en C que toma un grafo como entrada y le inserta un nodo. Esta función también fue estudiada en el curso de Algoritmos y Estructuras de Datos. La función en cuestión se da a continuación.

```
void insertarNodo(Tipo_Arbol *nodo, int ítem) {
    Tipo_Arbol *q, *r;
    if(nodo == NULL) {
        printf("No se puede Insertar\n");
        return;
    }
    r = NULL;
    q = nodo->hijo;
```

```

while (q != NULL) {
    r = q;
    q = q->next;
}
q = getNode(item);
q->padre = nodo;
if (r == NULL)
    nodo->hijo = q;
else
    r->next = q;
}

```

El grafo de flujo para la función, se muestra en la Figura 1.5.

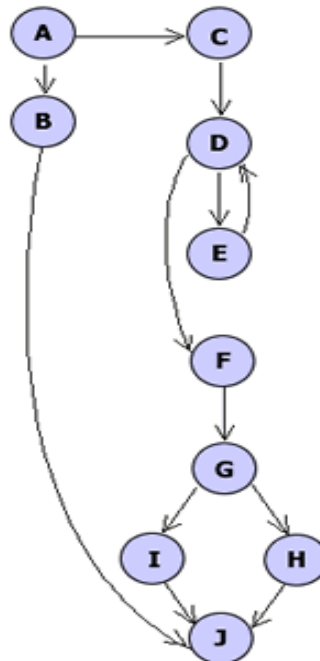


Figura 1.5: Grafo de Flujo de la Función para Insertar un Nodo en un Grafo

La figura muestra que hay 12 aristas y 10 nodos. Por lo tanto, la complejidad ciclomática de este algoritmo es:

$$\begin{aligned}
 V(G) &= \text{número de aristas} - \text{número de nodos} + 2 \\
 &= 12 - 10 + 2 = 4
 \end{aligned}$$

Fin del Ejemplo 1.5

Seguidamente, se ilustra la prueba de camino base mediante un ejemplo.

Ejemplo 1.6: Necesidad de la Prueba del Camino Base

Considere la siguiente función de software con dos decisiones secuenciales.

```
int myFunction(int x) {  
    if (x != 0)  
        myValue++;  
    if (x > 1)  
        myValue--;  
    return (myValue);  
}
```

La intención es que la función myFunction deje el valor de myValue sin cambiar, no importa el valor de x. La función es realmente muy pequeña y una inspección minuciosa revela que la función no deja el valor de myValue sin cambio para todos los valores de x.

Comience confiando en la prueba de la condición. Esto es, considere probar el programa anterior con valores de x tales que hagan que ambas sean condiciones TRUE para comenzar. Esto es posible cuando, por ejemplo, se elige un valor de x = 5. Porque:

```
if (x != 0)
```

es TRUE. También, la declaración

```
if (x > 1)
```

será TRUE. En tal caso, inicialmente, myValue será incrementado y después decrementado. El efecto neto de esto es que myValue no tendrá cambios. Esto apoya el argumento de que no hay error en el programa.

Ahora considere un valor de x que hará las dos condiciones FALSE. Esto puede suceder si por ejemplo x = 0. En este caso, ni unas ni otras de las declaraciones de incremento o decremento serán ejecutadas. Esto también deja a myValue sin cambio.

El problema con la prueba de la condición es que sugiere que solamente se requieren dos pruebas. Estas dos pruebas no revelan el error. Si se utiliza la prueba de camino base, sugiere tres pruebas, en lugar de dos. También, esta prueba tendrá que hacer que una de las dos declaraciones de condición de un valor TRUE mientras que el otro tenga el valor FALSE. Elija x = 1. En este caso, la primera condición dará TRUE, incrementando myValue. Pero la segunda condición dará FALSO y myValue no se decrementará. El efecto neto es que myValue se incrementa y cambia. Esto apunta a un error en el programa.

La prueba del camino base requiere que todos los caminos independientes sean probados. Esto permite que la prueba del camino base detecte errores que se puedan

descubrir solamente con la interacción de las condiciones, y no solamente ejecutando las condiciones.

Fin del Ejemplo 1.6

La prueba del camino base implica los pasos siguientes:

- Derivar el grafo de flujo basado en el programa fuente.
- Calcular la métrica de la complejidad ciclomática. Esto sugiere el número de los caminos independientes que se probarán en el programa.
- Enumerar los caminos independientes en el programa que forman el conjunto base.
- Derivar los casos de prueba que aseguran que cada una de los caminos independientes en el conjunto base sean ejecutadas.

Para recordar, la complejidad ciclomática del grafo G se denota como V y se calcula como sigue:

$$V(G) = \text{número (aristas)} - \text{número (nodos)} + 2$$

Considere el siguiente ejemplo que ilustra la prueba del camino. Se reproducen por completo los pasos de la complejidad ciclomática, pues también se requieren para la prueba del camino base.

Ejemplo 1.7: Prueba del Camino Base del Programa en el Ejemplo 1.6

Reconsidere el programa en el Ejemplo 1.5. Se estableció que se necesitan tres caminos independientes para ser probados. Ahora se demostrará cómo hacer la prueba del camino base del programa, según los pasos indicados anteriormente.

Se comienza reproduciendo el programa con números de línea por conveniencia.

```
1. int myFunction(int x) {  
2.   if (x != 0)  
3.     myValue++;  
4.   if (x > 1)-  
5.     myValue--;  
6.   return (myValue);  
7. }
```

Para derivar el grafo del flujo, se puede primero dibujar el flujograma. Este se muestra en la Figura 1.2.

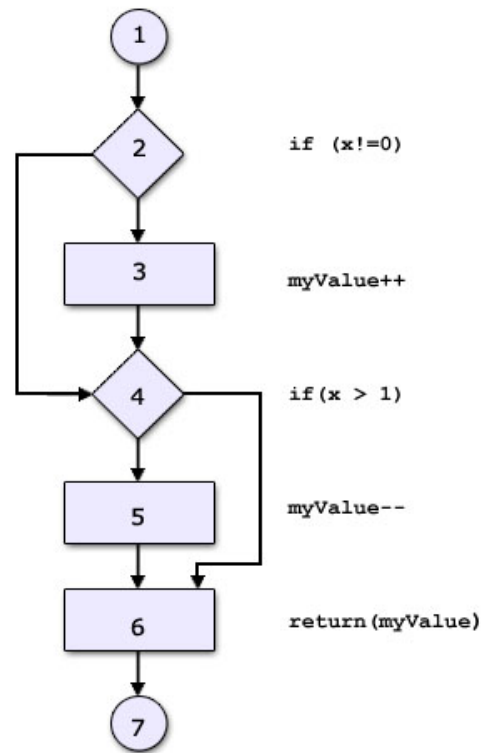


Figura 1.2: Flujograma del Programa en el Ejemplo 1.6

El grafo del flujo correspondiente para el flujograma en la Figura 1.2 se muestra en la Figura 1.3.

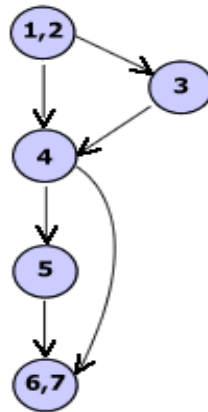


Figura 1.3: Grafo de Flujo para el Ejemplo 1.5

Se puede ver que en el grafo del flujo, el número de aristas es seis y el número de nodos es cinco. La complejidad ciclomática del programa es:

$$V(G) = \text{número (aristas)} - \text{número (nodos)} + 2$$

$$= 6 - 5 + 2$$

$$= 3$$

Según la prueba del camino base, la complejidad ciclomática indica el número de caminos independientes en el conjunto base. Aquí se deben tener tres caminos independientes en el conjunto base. En la Figura 1.3 se pueden enumerar los caminos independientes como sigue:

Camino 1: 2-3-4-5-6,7

Camino 2: 2-4-6,7

Camino 3: 2-3-4-6,7

El camino 2, 4, 5, 6, 7, no es considerado un camino independiente, pues este no recorre ninguna arista nueva, no contemplada ya en los demás caminos.

Lo que queda es derivar los casos de prueba que aseguren que cada una de los tres caminos independientes anteriores en el conjunto base sean ejecutados. Esto se presenta como sigue:

Camino 1: 2-3-4-5-6,7	$x = \{5\}$
Camino 2: 2-4-6,7	$x = \{0\}$
Camino 3: 2-3-4-6,7	$x = \{-1\}$

Es al ejecutar el caso de prueba para Camino 3 se revela la presencia de un error

Fin del Ejemplo 1.7

A continuación se ilustra con un programa que utiliza la iteración, cómo se hace la prueba del camino base.

Ejemplo 1.8: Prueba del Camino Base de un Programa para Contar el Número de Dígitos en un Número Entero Positivo.

En este ejemplo, considere un programa que encuentre el número de dígitos en un número entero positivo de entrada. El programa que hace esta tarea se observa a continuación:

```

1. # include <stdio.h>
2. main(){
3.   int num, digit, save;
4.   /*Pide al usuario un número positivo*/
5.   printf("Introduzca un entero positivo: ");
6.   scanf("%d",&num);
7.   /*Chequea que el número ingresado sea positivo */
8.   if (num <= 0)
9.       printf("%d no es un entero positivo\n",num);

```

```

10. else {
11.     /*Cuando el número es mayor que cero */
12.     save = num;
13.     digit = 1;
14.     /*Extrae cada dígito del número y cuenta
    cuantos dígitos ingreso */
15.     while (num/10 != 0) {
16.         digit++;
17.         num = num /10;
18.     }/*Fin del while*/
19.     printf("El número de digitos en %d es : \
    %d\n",save,digit);
20. }/*Fin del else*/
21. }/*Fin del main*/

```

De acuerdo con este programa, se puede dibujar el flujograma equivalente que describe los números de las líneas relevantes. Esto se muestra en la Figura 1.4.

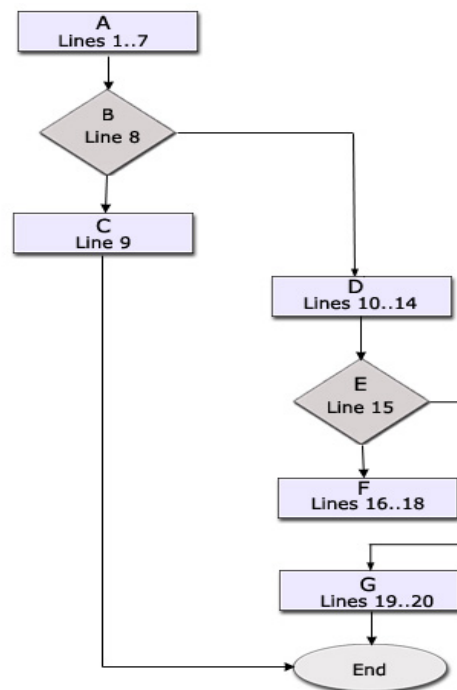


Figura 1.4: Flujograma del Programa en el Ejemplo 1.7

El paso siguiente es dibujar el grafo equivalente del flujo y se muestra en la Figura 1.5.

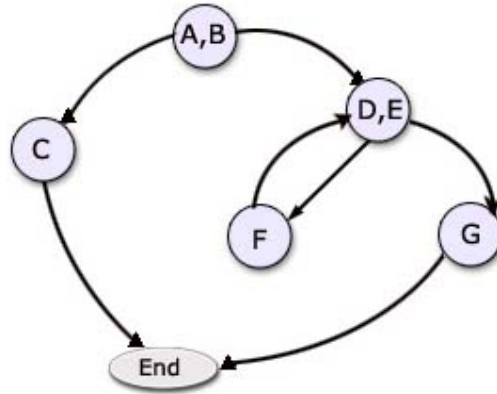


Figura 1.5: Grafo de Flujo que corresponde a la Figura 1.4

Se puede calcular la métrica de la complejidad ciclomática de la Figura 1.5 como sigue:

$$\begin{aligned}
 V(G) &= \text{número (aristas)} - \text{número (nodos)} + 2 \\
 &= 7 - 6 + 2 \\
 &= 3
 \end{aligned}$$

Por lo tanto, el conjunto base debe consistir de tres caminos independientes. Éstos se enumeran a continuación:

Camino 1: A,B-C-End
 Camino 2: A,B-D,E-G-End
 Camino 3: A,B-D,E-F-D,E-G-End

El caso de la prueba del camino base se elige de tal forma que cada uno de estos caminos independientes sea ejecutado. Algunos ejemplos de prueba se enumeran a continuación:

Camino 1: A,B-C-End	num = {0, -1}
Camino 2: A,B-D,E-G-End	num = {1,2,...,9} solo dígito, positivo
Camino 3: A,B-D,E-F-D,E-G-End	num = {21,10} Positivo, 2 dígitos o más

Fin del Ejemplo 1.8

Algunos de los beneficios principales de la prueba del camino base son:

- El número de pruebas es igual a la métrica de complejidad ciclomática de un módulo.
- El esfuerzo de prueba se centra en un software propenso a errores, puesto que la complejidad se correlaciona con los errores.

- El proceso de prueba se puede planear y supervisar en mayor detalle que con la mayoría de las otras estrategias de prueba, puesto que el número mínimo de pruebas requeridas se sabe por adelantado.

Así termina la discusión de la prueba de Caja Blanca. A continuación se discute otra estrategia de prueba, el enfoque de la Caja Negra.

6. Prueba de Caja Negra

La prueba de Caja Negra considera la selección de los datos de prueba y la interpretación de los resultados de la prueba realizada basándose en las características funcionales del software. El desarrollador original de un programa no debe realizar la prueba de Caja Negra, pues conoce mucho acerca de la parte interna del programa. Esto puede conducir a la selección de los casos de prueba que son sesgados e insatisfactorios. Los sistemas de software se entregan generalmente a un equipo independiente de prueba que realiza las pruebas de Caja Negra, después de terminar con éxito la prueba de Caja Blanca.

La prueba de Caja Negra, también conocida como prueba funcional, se concentra en la funcionalidad global del software y permite la prueba funcional para descubrir fallas tales como las siguientes:

- Funciones erróneas o faltantes.
- Errores de interfaz.
- Errores de estructura de datos y base de datos.
- Errores de inicialización y finalización.
- Errores en el desempeño.

Las pruebas de la Caja Negra se ocupan de aspectos tales como los que se ilustran en la Figura 1.6.

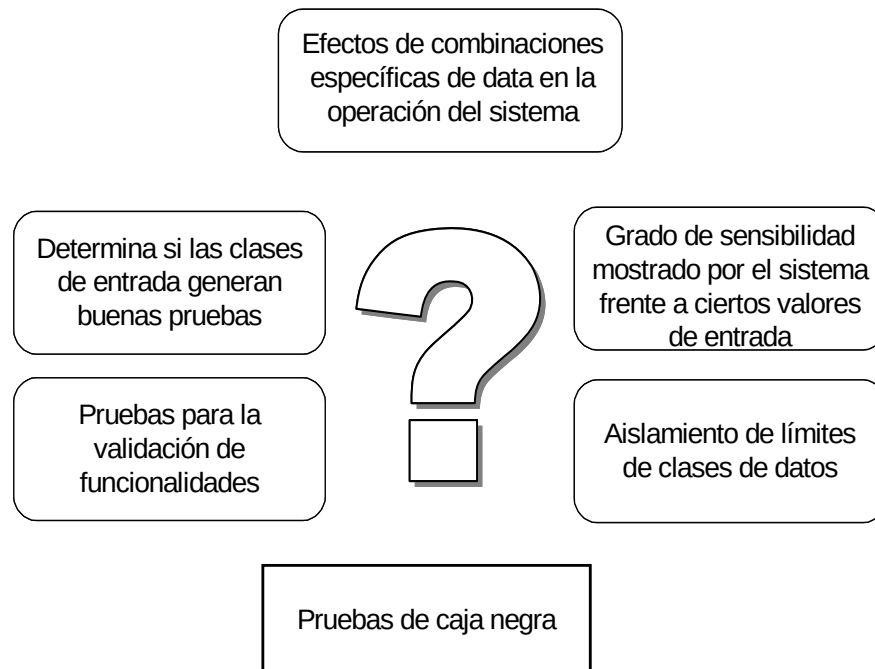


Figura 1.6: Pruebas de Caja Negra

Las cuatro técnicas principales que se usan en el enfoque de la Caja Negra se muestran en la Figura 1.7.

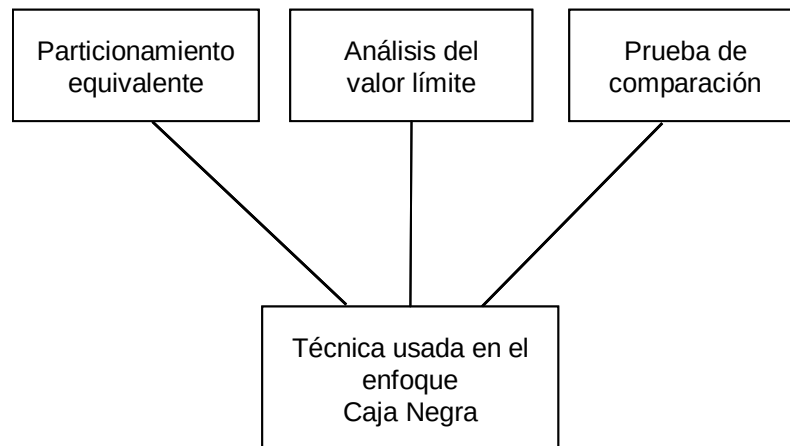


Figura 1.7: Técnicas de la Prueba de Caja Negra

En esta unidad se discutirán brevemente cada una de estas técnicas de prueba de Caja Negra.

6.1 Partición Equivalente

Primero se establecerá el fundamento de este procedimiento para la prueba de Caja Negra. Para hacer esto, se presenta un ejemplo simple.

Ejemplo 1.9: Necesidad de Partición Equivalente

Considere un programa simple que tome un número entero positivo como entrada, cuente el número de dígitos en el número e imprima el número de dígitos. Se ha utilizado este programa anteriormente, pero se reproduce a continuación para facilitar la referencia.

El Código en C empieza aquí...

```
# include <stdio.h>
main(){
    int num, digit, save;

    /* Pide al usuario un número positivo */
    printf("Enter a positive Integer : ");
    scanf("%d",&num);

    /* Chequea si el número ingresado es positivo */
    if (num <= 0)
        printf("%d is not a positive
integer\n",num);
    else {
        /* Cuando el número es mayor que cero */
        save = num;      /* keep num safely */
        digit = 1;

        /* Extrae cada dígito del número y cuenta
cuentos dígitos ingreso */
        while (num/10 != 0) {
            digit++;
            num = num /10;
        }/*Fin del while*/

        printf("The number of digits in %d is : \
%d\n", save, digit);
```

```
}/*Fin del else*/
```

```
}/*Fin del main*/
```

El Código en C termina aquí

Considere la primera sentencia condicional.

```
if (num <= 0)
```

Para ejecutar esta parte de la declaración, se deben especificar los siguientes casos de prueba:

```
num = 0
num = -1
num = -2
num = -3
```

Esto puede continuar. La pregunta es: ¿Se continúa teniendo casos de prueba cuando num es negativo? ¿Cuántas de esas pruebas se requieren? Claramente, no es útil proporcionar todos los valores negativos como casos de prueba.

Repartir los valores de la entrada en particiones equivalentes, permite restringir el número de los casos de prueba requeridos.

Fin del Ejemplo 1.9

La partición equivalente es un método de dividir entradas en clases de equivalencia. Es una práctica común, por ejemplo dividir las entradas en tres clases de equivalencia: números enteros negativos, cero y números enteros positivos. El objetivo principal es definir tan solo un caso de prueba en cada clase de equivalencia, en lugar de múltiples casos de prueba que pertenecen a la misma clase de equivalencia.

En el Ejemplo 1.9, las clases de equivalencia para la variable de entrada num son:

- Números enteros negativos.
- El número entero cero.
- Números enteros positivos.

Por consiguiente, se necesita tener solo tres casos de prueba, uno en cada clase de equivalencia. Por ejemplo, un conjunto válido de casos de prueba es:

- | | |
|-----------------------------|-----|
| • Números enteros negativos | -1 |
| • El número entero cero | 0 |
| • Números enteros positivos | 555 |

Nota: La generación de particiones equivalentes y casos de prueba depende solamente de las condiciones de entrada. Una clase de equivalencia representa un conjunto de estados válidos o de estados inválidos para las condiciones de entrada.

Un procedimiento que se puede utilizar para obtener las particiones equivalentes se ilustra en la Figura 1.8:

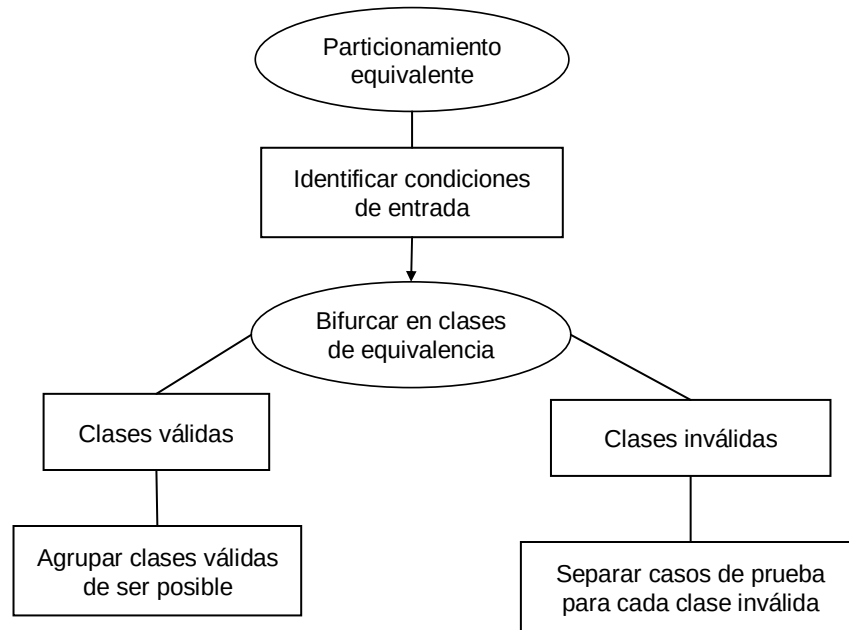


Figura 1.8: Procedimiento de Partición Equivalente

Las clases de equivalencia se pueden definir según las pautas ilustradas en la Figura 1.9.

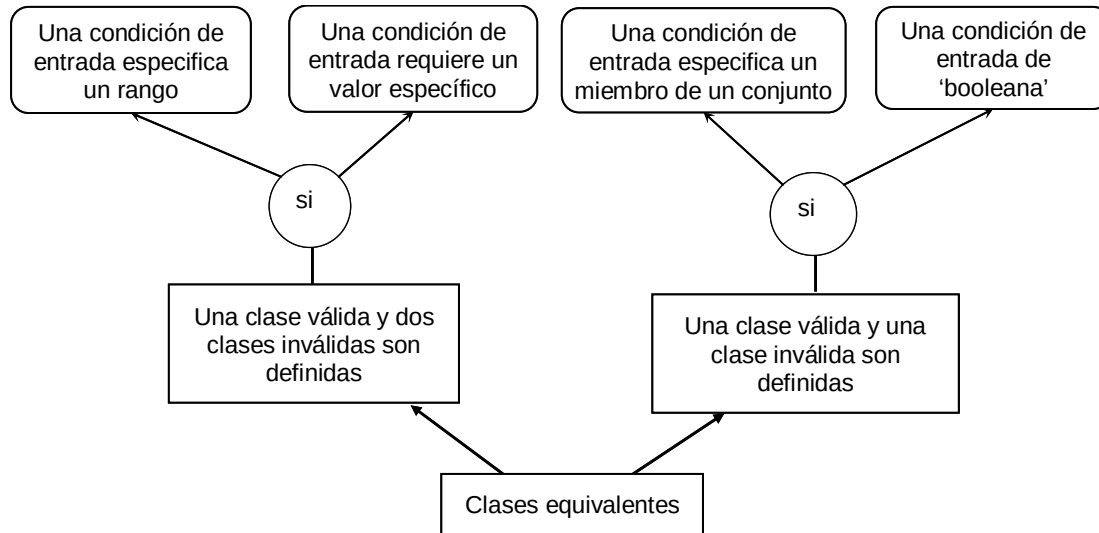


Figura 1.9: Clases de Equivalencia

Considere el Ejemplo 1.9, donde se discutió el programa que cuenta el número de dígitos en un número entero positivo. ¿Cuándo se prueba el programa con los casos de prueba? ¿Se puede estar realmente seguro de que no hay defectos? Realmente, la respuesta depende de la validez de la partición de equivalencia, así como del valor particular del caso de prueba elegido.

Para la clase del número entero positivo, se seleccionó el valor 555. Esto puede ser insuficiente para detectar defectos, puesto que el programa puede no funcionar correctamente para una entrada de un número de un solo dígito. La directriz normal es que se debe seleccionar un valor del punto medio de una clase, puesto que es representativa. En este caso, la partición equivalente debería ser:

- Números enteros negativos.
- El número entero cero.
- Números enteros positivos de un dígito.
- Números enteros positivos con más de un dígito.

Se deben dividir correctamente las clases y también seleccionar solamente el valor representativo apropiado para un caso de prueba.

Así como con las particiones de la entrada, se puede también tener particiones para la salida. En este caso, se deben generar casos de prueba tales que se pueda obtener por lo menos un valor de cada clase de equivalencia.

Considere otro ejemplo para consolidar la comprensión de partición equivalente.

Ejemplo 1.10: Partición Equivalente para una Función que Calcula la Raíz Cuadrada

Asuma que se tiene que probar una función llamada $\text{sqrt}(x)$ usando la partición equivalente. Primero, se hace la partición equivalente para los valores de entrada:

- Números reales negativos.
- El número real cero, es decir, 0.0.
- Números reales positivos.

Por consiguiente, los casos de prueba pueden ser los siguientes:

$x < 0$
 $x = 0.0$
 $x > 0$

Las particiones de la salida pueden ser como sigue:

$\text{sqrt}(x) > 0$ (para los casos $x > 0$)
 $\text{sqrt}(x) = 0.0$ (para los casos $x = 0.0$)
el resultado es error (para los casos $x < 0$)

Fin del Ejemplo 1.10

Observe que esto es un mecanismo de prueba de Caja Negra, así que ninguna información de las sentencias del programa se puede utilizar para derivar casos de prueba. Se hace la partición equivalente usando el documento o los documentos de la especificación que explican cómo utilizar los programas.

¿Otra pregunta que se presenta es si las particiones siempre deben ser discretas? La respuesta es negativa. Las particiones pueden también solaparse. Esto depende de la naturaleza de la especificación que conduce a tal partición.

La partición equivalente es una técnica bastante simple. A medida que el software se torna complejo, determinar las particiones equivalentes mismas puede ser muy complejo.

A continuación se explica otra técnica de prueba de Caja Negra, la técnica del análisis del valor límite.

6.2 Análisis del Valor Límite

Esta técnica, se concentra en la generación de los casos de prueba que trabajan con valores límite. El análisis del valor límite trabaja con valores de salida, así como valores de entrada. Para entender esta técnica, considere el ejemplo de una función que calcula la raíz cuadrada de un número real que es entrada, según se presentó en el ejemplo 3.9.

Ejemplo 1.11: Análisis del Valor Límite de una Función para Calcular la Raíz Cuadrada de un Número de Entrada

Usando el método de partición equivalente, la entrada para la función $\text{sqrt}(x)$ fue enumerada como sigue:

- Números reales negativos.
- El número real cero, es decir, 0.0.
- Números reales positivos.

La salida de la función se puede repartir en las clases de equivalencia siguientes:

$\text{sqrt}(x) > 0$ (para los casos $x > 0$)
 $\text{sqrt}(x) = 0.0$ (para los casos $x = 0.0$)
El resultado es error (para los casos $x < 0$)

El análisis del valor límite se realiza basado en las pautas siguientes:

- Incluya los puntos finales del rango de entrada: Si el rango de los valores de la entrada es $[u, v]$, entonces se debe incluir el valor de u y v en el caso de prueba.
- Incluya los valores justo debajo de los puntos finales del rango de entrada: Los casos de prueba deben incluir valores apenas debajo de los puntos finales, es decir. $u' < u$ y $v' < v$.
- Incluya los valores apenas sobre los puntos finales del rango de entrada: Los casos de prueba deben incluir valores apenas sobre los puntos finales $u'' > u$, y $v'' > v$.
- Las mismas tres pautas se pueden aplicar a la salida: Las tres pautas no pueden ser directamente aplicables a los valores de la salida, pero donde quiera que sea posible directamente, los casos de prueba deben ser generados convenientemente y utilizados.
- Especifique los casos de prueba para las estructuras de datos usadas por separado: Los casos de prueba se deben crear y utilizar por separado para cada una de las estructuras de datos usadas en el programa.

De acuerdo con estas pautas, los siguientes casos de prueba pueden ser enumerados:

- **Caso de Prueba 1**

Entrada { el número real más negativo }
Salida esperada – la condición de error
Ejecuta el límite más bajo de la partición (i).

- **Caso de Prueba 2**

Entrada { apenas menos de 0 }
Salida esperada – la condición de error

- **Caso de Prueba 3**

Entrada 0

Salida esperada - 0

- **Caso de Prueba 4**

Entrada { apenas mayor que 0 }

Salida esperada - un número real positivo que representa la raíz cuadrada de la entrada

- **Caso de Prueba 5**

Entrada {el número real más positivo}

Salida esperada - un número real positivo que representa la raíz cuadrada de la entrada

Fin del Ejemplo 1.11

El análisis del valor límite es una técnica útil puesto que muchos errores se centran solo en los valores límite. Es especialmente útil en software pequeño, no-complejo. Para el software extremadamente complejo, el análisis del valor límite puede llegar a ser tedioso y difícil de llevar a cabo.

6.3 Prueba de la Comparación

La prueba de comparación normalmente se emprende cuando hay que ocuparse de los sistemas que se consideran críticos. Pueden ser 'críticos' porque tratan con aspectos humanos de seguridad, tales como sistemas de computadoras en aviones o en la administración de la dosificación de la radiación para los pacientes cáncer. Pueden también ser 'críticos' porque una interrupción o un malfuncionamiento del sistema, puede dar lugar a enormes pérdidas.

En la prueba de comparación, se desarrollan dos versiones independientes pero idénticas del mismo software. Equipos independientes desarrollan las dos versiones. Durante la prueba de comparación, los mismos casos de prueba se utilizan para evaluar la adhesión a las especificaciones funcionales del software. Generalmente, en la prueba de comparación, ambas versiones del software funcionan en paralelo para comparar su rendimiento en tiempo real.

¿Qué sucede cuando los mismos casos de prueba en ambas versiones no proporcionan los resultados idénticos esperados? En este caso, ambas versiones del software se revisan para detectar la presencia de defectos. Al final, con los mismos casos de prueba, se identifican y se rectifican todos los defectos.

De las dos versiones, solamente una estará funcionando normalmente, mientras que la otra sirve como respaldo.

La prueba de la comparación también se llama prueba de apoyo.

Resumen

Ahora que ha finalizado esta unidad, usted será capaz de:

- Discutir diversos niveles de prueba.
- Describir el diseño de caso de prueba.
- Explicar algunos de los métodos de prueba de Caja Blanca.
- Discutir el proceso de derivar casos de prueba.
- Explicar algunos de los métodos de prueba de Caja Negra.

Unidad 1: Examen de Autoevaluación

- 1) ¿Cuáles de las siguientes sentencias son correctas?
 - a) La prueba es una fase obligatoria en la ingeniería de software.
 - b) La prueba puede demostrar la ausencia de errores.
 - c) La prueba puede demostrar la presencia de errores, pero no su ausencia.
 - d) La prueba es un método para obtener todos los errores.

- 2) ¿Por qué se hace la prueba del software?
 - a) Para descubrir defectos en software.
 - b) Para probar que el software esta sin defectos.
 - c) Para reducir el costo de mantenimiento.
 - d) Para estimar la confiabilidad del software.

- 3) ¿Cuáles son los diferentes niveles de prueba?
 - a) Prueba de Unidad.
 - b) Prueba de Integración.
 - c) Prueba del Sistema.
 - d) Prueba de Aceptación.

- 4) La prueba improvisada _____.
 - a) Es inútil.
 - b) Es útil ya que detecta grandes defectos en corto tiempo, pero no es recomendado.
 - c) Es útil ya que detecta grandes defectos en corto tiempo y es recomendado.
 - d) No es un método de prueba.

- 5) SQA, WinRunner, MS TEST y Visual Test son todos ejemplos de _____.
 - a) Métodos de prueba de unidad.
 - b) Productos comerciales para la prueba de unidad.
 - c) Productos comerciales para la prueba de integración.
 - d) Herramientas de prueba automatizadas.

- 6) ¿Cómo es conocida también la prueba de caja blanca?
- a) Prueba opaca.
 - b) Prueba transparente.
 - c) Prueba de caja de cristal.
 - d) Prueba de unidad.
- 7) ¿Cuál es el significado de la métrica de la complejidad ciclomática en la prueba del camino base?
- a) Indica el número máximo de los casos de prueba que requerimos para probar el programa.
 - b) Indica el número de trayectorias independientes en el grafo del flujo de programa que necesita ser probado.
 - c) Indica el número de trayectorias dependientes en el grafo del flujo de programa que necesita ser probado.
 - d) Indica el número mínimo de los casos de la prueba que requerimos para probar el programa.
- 8) ¿Cuál de los siguientes da la complejidad ciclomática?
- a) $e - n + 2$
 - b) $e + n - 2$
 - c) $n - e + 2$
 - d) $n + e - 2$
- 9) Cuando la misión crítica del software necesita ser probada, se utiliza una técnica que requiere dos versiones del mismo software para ser desarrollada y probada usando el mismo conjunto de casos de prueba. ¿Cómo se llama esta técnica?
- a) Partición equivalente.
 - b) Prueba de la comparación.
 - c) Prueba de apoyo.
 - d) Análisis del valor límite.
- 10) La prueba basada en grafo y la partición equivalente son ejemplos de _____.
- a) Prueba de caja blanca.
 - b) Prueba de caja negra.
 - c) Prueba de unidad.
 - d) Prueba de integración.

Respuestas de la Unidad 1: Examen de Autoevaluación

- 1) a y c
- 2) a y d
- 3) a, b, c y d
- 4) b
- 5) d
- 6) c
- 7) b
- 8) a
- 9) b y c
- 10) b

Unidad 2: Metodologías de Prueba

Objetivos del Aprendizaje

Al finalizar esta unidad, usted será capaz de:

- Distinguir entre la verificación y la validación.
- Describir el proceso de prueba.
- Explicar los métodos involucrados en el proceso de prueba de unidad.
- Discutir los enfoques involucrados en la prueba de integración.
- Describir los diversos componentes de la prueba del sistema.

1. Introducción

Se han discutido diversas técnicas de prueba de software en la Unidad 3 - Fundamentos de Pruebas del Software. El proceso de prueba requiere una planificación considerable. La planificación de la prueba es vital para la acertada concepción y culminación de un proceso de prueba, la cual debería hacerse antes de que la prueba realmente comience y simultáneamente con las fases de codificación y diseño. La Figura 2.1 ilustra los pasos involucrados en la planificación de prueba del software.



Figura 2.1: Planificación de la Prueba del Software

En general, la prueba comienza con un plan de prueba y termina con la prueba de aceptación. Un plan de prueba es un documento general elaborado por el equipo del desarrollo para el proyecto completo. El plan de prueba define el alcance, el enfoque que se tomará y el cronograma de prueba. También identifica los tipos de pruebas que se llevarán a cabo y los casos de prueba para el proceso completo de prueba, además del personal responsable de las diversas actividades de la misma.

Cada modelo puede también tener su propio plan de prueba que comprende un conjunto de casos y de procedimientos para realizar pruebas de módulo. Los procedimientos de prueba pueden ofrecer descripciones de los diferentes manejadores (drivers) de software requeridos para probar, mientras que el portafolio de casos de prueba proporciona respuesta a cada prueba. El proceso de prueba implica el desarrollo de los manejadores, que no son más que los módulos de software que llaman, controlan y supervisan la ejecución de varios módulos. El proceso de prueba también requiere el desarrollo de los subprogramas simulados (stubs), que permiten el proceso de prueba incremental de los módulos integrados.

El criterio de aceptación para cada módulo es aquel que debe ser aceptado para la integración con otros módulos. Debido a que los ciclos de desarrollo en la mayoría de los proyectos de software son largos, los desarrolladores encontrarán conveniente revisar los criterios de aceptación de vez en cuando, además de chequear si siguen siendo aplicables al proyecto dado. Esto es algo muy útil, porque a menudo existen cambios en los requerimientos del cliente en el ambiente operacional, durante el curso del proceso de desarrollo.

La planificación para probar comienza tempranamente en el ciclo de desarrollo. Sin embargo, las actividades de prueba del software comienzan solamente después de que las actividades principales del desarrollo del software y la etapa de programación concluyen. Generalmente, en esta etapa la organización estará haciendo frente a una fuerte presión para entregar el producto a tiempo. Esto puede imponer severas

restricciones y presiones a la prueba. La fase de prueba implica no sólo detectar los defectos, sino también implica el rastrear la fuente de los errores y corregirlos.

Cuando la prueba se hace bajo presiones de fechas límites, puede ocurrir que el proceso de prueba se salga de control. Hay una necesidad de ejercer disciplina en el proceso de prueba. Un requisito simple es instalar una herramienta apropiada de administración de la configuración del software.

En esta unidad se estudiarán algunas de las principales metodologías para pruebas que incluyen prueba de unidad, prueba de integración y prueba del sistema.

2. Verificación y Validación del software

Los dos términos usados con frecuencia en el desarrollo y prueba del software son la verificación y validación. A continuación se definen estos dos términos y las actividades que representen.

2.1 Verificación del Software

La verificación define todas las actividades que ocurren al final de un ciclo de desarrollo particular. La verificación confirma que el producto se está desarrollando correctamente y satisface las condiciones impuestas en el principio de la etapa del desarrollo. La verificación, por ejemplo se puede hacer al final de la fase de ingeniería de requerimientos o de la fase del diseño, inclusive al final de la fase de implantación del software según las premisas del cliente. Se asegura que en el software en particular esté implementado correctamente una función específica requerida en la SRS. La verificación responde a la pregunta ¿Se está construyendo el producto correctamente?.

2.2 Validación Del Software

La validación confirma que el producto se está desarrollando correctamente y refleja la SRS. Se refiere a un conjunto de actividades (diferentes de aquellas para la verificación), las cuales aseguran que el software desarrollado coincida con los requerimientos del cliente. La validación intenta asegurar que el software se comporta de una manera que está en conformidad con cada uno de los requerimientos funcionales, característicos del comportamiento y requerimientos de desempeño establecidos explícitamente en la SRS. La validación contesta a la pregunta ¿Se está desarrollando el producto requerido?.

3. El Proceso de Prueba

El proceso de prueba del software implica lo siguiente:

- Definición de los Criterios Formales de Aceptación para el Sistema: El equipo de prueba tiene que preparar los criterios formales de aceptación para el sistema. Este paso comienza tan pronto se concluya la fase de análisis de

- requerimientos. Se realiza en paralelo con las tareas de ingeniería de sistema y desarrollo de la SRS.
- **Plan para la Integración y Prueba del Sistema:** El equipo de prueba es también responsable de desarrollar el plan para la integración y prueba del sistema. Éste también se hace junto con las actividades de ingeniería de sistema y desarrollo de la SRS.
 - **Desarrollar una Estrategia de Prueba:** La estrategia de prueba se desarrolla para indicar la naturaleza de la prueba de unidad que se realizará. También, indicará el proceso por el cual los módulos serán integrados en 'estructuras' para la prueba.
 - **Sincronizar las Actividades de Prueba con el Cronograma del Desarrollo:** Es importante sincronizar la actividad de prueba con el cronograma del desarrollo del módulo, ya que cualquier aspecto fuera de la sincronización resultará en retrasos y problemas.

Una parte del proceso de prueba se extiende durante la implantación del sistema. La Figura 2.2 ilustra el proceso de prueba para un programa de desarrollo grande de software.

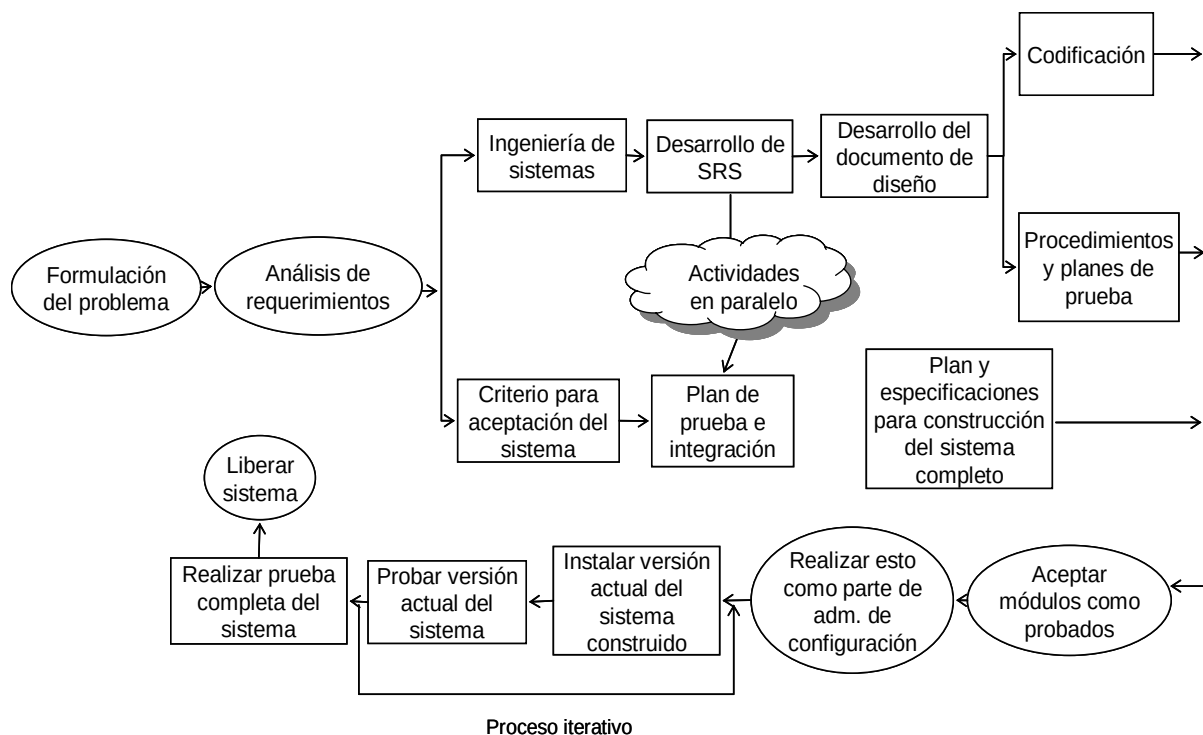


Figura 2.2: Proceso de Prueba para el Programa de Desarrollo del Software

El proceso de prueba e integración, también conocido como estructura incremental, se basa en la construcción del sistema en incrementos de módulos integrados. Considere un sistema que requiera 3 funcionalidades diferentes en total. Es posible integrar solamente los módulos que proporcionan sólo una de las tres funcionalidades como una

estructura. Se prueba esta estructura. Luego, el siguiente conjunto de módulos que proporcionan la segunda funcionalidad (entre las tres funcionalidades) se integra con la primera funcionalidad. Esto da lugar a una estructura incremental que proporciona dos de las funcionalidades. Finalmente, los módulos que proporcionan la tercera funcionalidad serán integrados con la estructura anterior para obtener la estructura incremental siguiente. Cada estructura incremental tiene una función específica. La ejecución de las pruebas de aceptación al nivel de sistema también ocurre progresivamente en este proceso.

4. Prueba de Unidad

La prueba de unidad se ocupa de la prueba de módulos en un sistema de software. El término 'unidad' en el contexto de prueba de unidad, se puede referir a:

- Una función individual.
- Una clase en un software, desarrollada con el enfoque orientado a objeto.
- Un DLL (Biblioteca de Enlaces Dinámica), que es una biblioteca de funciones ejecutables utilizada por las aplicaciones de Microsoft Windows.
- Una colección de las funciones cohesivas que realizan algunas tareas específicas.

Así, la prueba de unidad se concentra en algunos de los conjuntos de componentes más pequeños de un sistema de software, que se llaman 'módulos' en un sentido genérico. Aquí, se utilizarán los términos unidad y módulo alternativamente. La prueba de unidad se hace generalmente usando los métodos de prueba de caja blanca. La prueba de unidad puede hacerla un individuo o un grupo de personas y las pruebas de varias unidades pueden proceder en paralelo. El objetivo principal de la prueba de unidad es detectar los defectos que pueden existir en el componente probado. Las unidades del software se crean basándose en el documento detallado del diseño. Por lo tanto, el documento detallado del diseño servirá como base para la selección del método de prueba apropiado de caja blanca.

La prueba de unidad es un proceso que implica un conjunto de actividades bien definidas que se ilustran en la Figura 2.3.

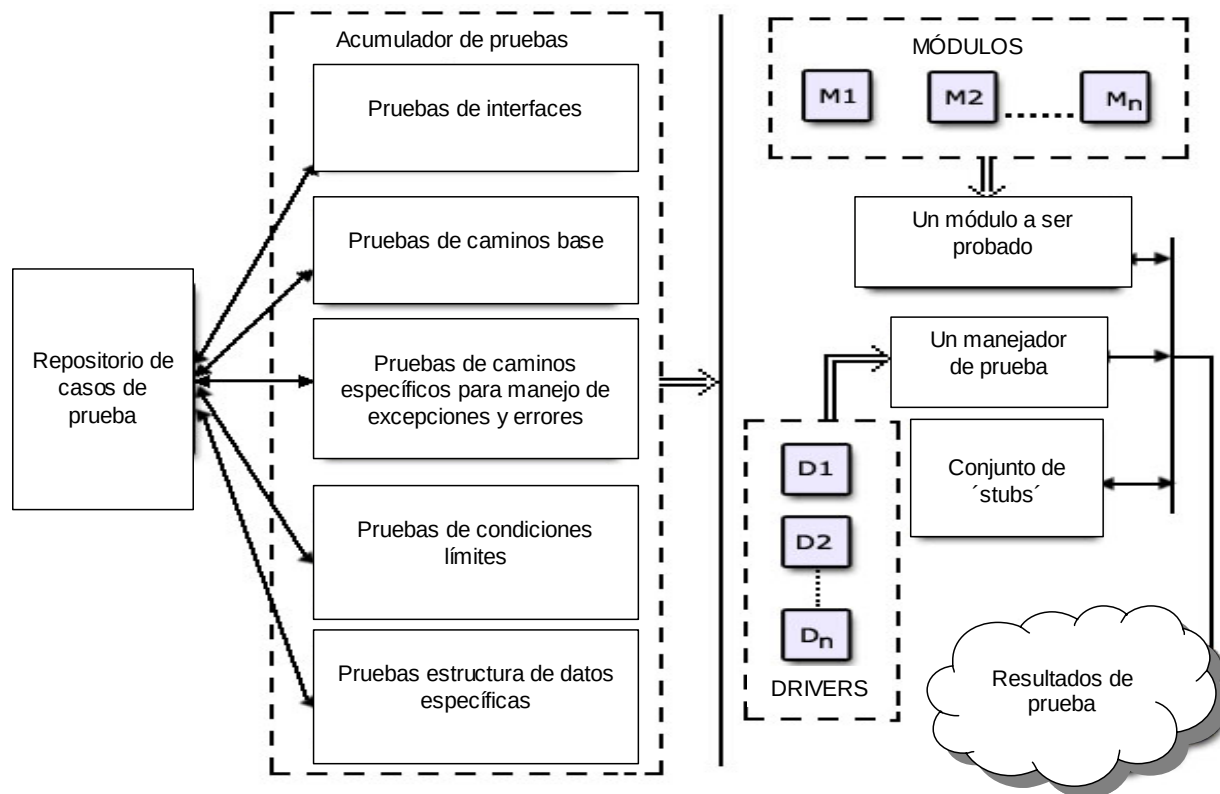


Figura 2.3: El Proceso de Prueba de Unidad

La Figura 2.3 muestra que el proceso de la prueba de unidad implica llevar a cabo una serie de pruebas, generar un conjunto de resultados de la prueba, que serán analizados y descubrirán defectos. La serie de pruebas que se llevarán a cabo en las unidades del software incluyen:

- **Prueba de la Interfaz del Módulo:** La prueba de la interfaz del módulo es una de las primeras pruebas que se realizan en las unidades. Esto se hace principalmente para asegurarse de que hay un pase apropiado de información hacia y desde el módulo.
- **Prueba del Camino Base:** La prueba del camino base asegura que todas los caminos independientes dentro de la unidad se ejecutan con una opción apropiada de los casos de prueba.
- **Prueba de Caminos Específicos para Errores y Manejo de Excepción:** Algunas de las unidades que son probadas tendrán caminos específicos que están conectados con el manejo de errores o de excepción. Éstos se prueban para asegurar que el proceso de manejo de error y excepción se ejecuta para verificar si hay presencia de defectos.

- Prueba de las Condiciones Límites: Las unidades que son probadas involucran parámetros de entrada y de salida.
- Prueba de las Estructuras de Datos Locales: Generalmente, las unidades que son probadas hacen uso de ciertas estructuras de datos localmente dentro de la unidad. Las estructuras de datos deben ser probadas. Esto se hace para asegurar la integridad de los datos usados por la unidad.

Está claro que el proceso de la prueba de unidad implica llevar a cabo algunas o todas las series de pruebas. El proceso por sí mismo no especifica una secuencia rígida en la cual estos métodos individuales de pruebas unitarias tengan que ser utilizados. Una secuencia sugerida para realizar estas pruebas puede ser:

- Prueba de la interfaz modular.
- Prueba de las estructuras de datos locales al módulo.
- Prueba del módulo en condiciones límite.
- Prueba de caminos independientes a través de la prueba del camino base.
- La prueba de caminos conectados con el manejo de error y excepción.

Casi todos los métodos de prueba de caja blanca se pueden utilizar aquí como parte del proceso de prueba de unidad. Por ejemplo, la prueba de flujo de datos. Algunos practicantes prefieren realizar la prueba de flujo de datos antes de que se realice cualquiera de las otras pruebas. Mientras se realiza la prueba de flujo de datos de unidades se hacen los esfuerzos necesarios para asegurar que el número y los atributos de los parámetros de entrada coincidan con el número y los atributos de los argumentos. La prueba puede también ver si los tipos de datos usados en los parámetros y los argumentos son iguales y consistentes. En resumen, la prueba cubre todos los aspectos de la definición, uso de parámetros y argumentos. Algunos practicantes también recomiendan la inclusión de la variable global usada (y/o modificada) por la unidad.

Algunas unidades que son probadas pueden realizar una tarea de E/S haciendo uso de archivos. Las pruebas deben incluir la verificación de que los archivos se abren y cierran apropiadamente, el manejo de error y el uso de los buffers intermediarios de E/S. Estas pruebas se pueden realizar como parte de las pruebas de la interfaz del módulo.

La Figura 2.3 también muestra los módulos que se prueban usando manejadores y 'stubs' apropiados de prueba. Un manejador de prueba es un programa que ayuda a invocar el módulo que es probado con las entradas apropiadas. Es decir 'conduce' la prueba del módulo. Los módulos que son probados pueden llamar otros subprogramas. Los 'stubs' son programas simulados que se usan para que se ocupen de los requerimientos del módulo probándose con respecto a llamadas a subprogramas específicos. El proceso de la prueba de unidad hace uso de un repositorio de casos de prueba que se construyen basándose en las unidades que se probarán y los métodos de prueba a utilizarse.

Los errores observados en el proceso de prueba de unidad se registran en alguna plantilla estándar establecida por la compañía para el análisis adicional. A continuación

se discute brevemente la naturaleza de los errores que serán detectados en el proceso de prueba de unidad.

4.1 Naturaleza de los Errores Detectados en la Prueba de Unidad

Durante el proceso de prueba de unidad, pueden ser descubiertos muchos tipos de errores. Las estructuras de datos locales usadas en un módulo pueden ser una fuente de errores común.

Los errores se pueden clasificar como: errores de cómputo, lógicos, E/S, interfaz del manejo de datos, la definición de los datos y base de datos.

Las categorías en las cuales los errores pueden ser encontrados se ilustran en la Figura 2.4. Los nombres de variables tipográficas y nombres de variables truncadas son detectados como errores en la fase de la compilación, algunos errores de este tipo pueden pasar desapercibidos. El nombre variable `temp1`, por ejemplo puede ser escrito incorrectamente como `temp l`, pero el error tipográfico puede ser consistente de tal forma que el proceso de compilación puede no señalarlo como error.

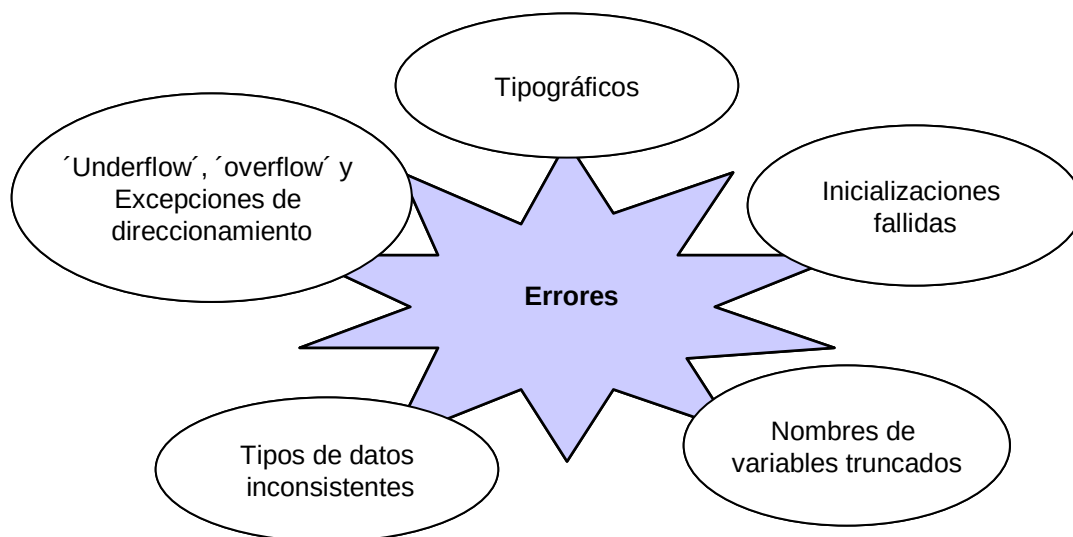


Figura 2.4: Errores en el Desarrollo del Software

Algunas de las categorías principales en las cuales los errores tienden a encontrarse son fallas en la inicialización de variables, uso de tipos de datos inconsistentes y una variedad de excepciones. También, pueden ocurrir varios errores durante el cálculo de expresiones debido a una variedad de factores, tales como: control del flujo incorrecto y comparaciones incorrectas en sentencias condicionales. La prueba de unidad debe ser llevada a cabo de tal manera que estos tipos de errores sean también descubiertos. Los tipos principales de errores que ocurren durante los cálculos se ilustran en la Figura 2.5.

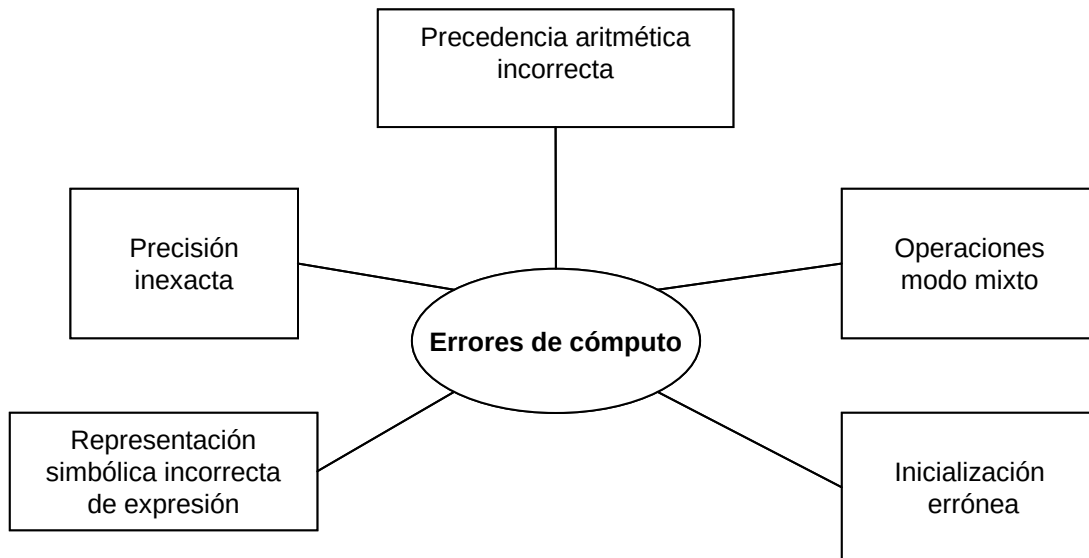


Figura 2.5: Errores de Cómputo en el Desarrollo del Software

La Figura 2.5 muestra que los errores de cómputo que ocurren principalmente debido a: precedencia aritmética incorrecta en las expresiones usadas, efectos indeseables al usar operaciones mezcladas en expresiones y asignaciones aritméticas, problemas relacionados con la precisión cuando se trabaja con números de punto flotante e inicialización incorrecta de variables.

En declaraciones condicionales, así como en declaraciones iterativas, generalmente existe una comparación que se hace usando una expresión relacional seguida por un cambio del flujo. En este sentido, el flujo de la comparación y del control van juntos en las sentencias condicionales e iteraciones. Puede haber un número de errores que estén conectados con las comparaciones y el flujo del control según se muestra en la Figura 2.6.

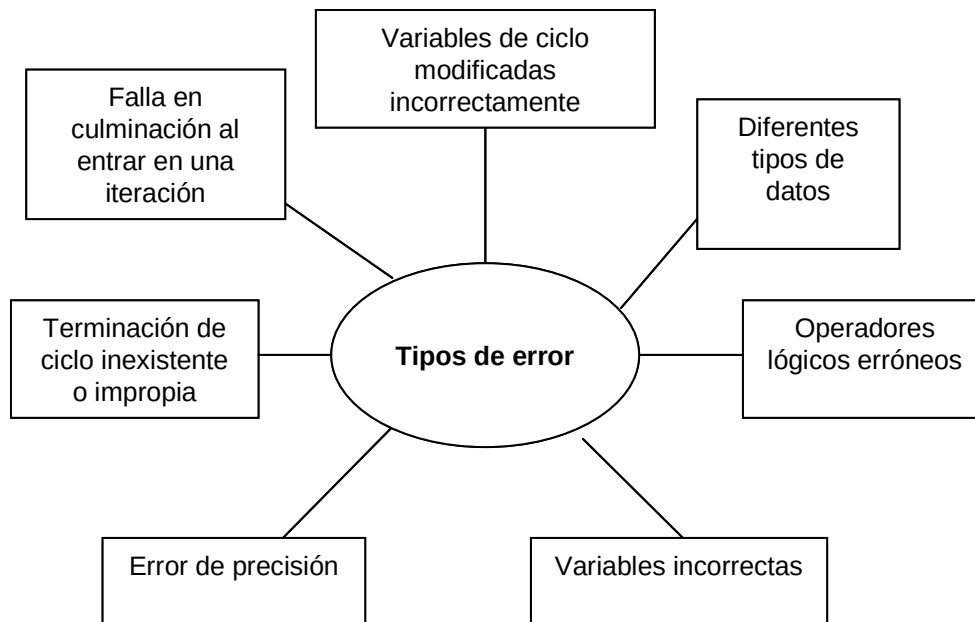


Figura 2.6: Errores en Comparaciones y Flujo del Control

El proceso de prueba de unidad debe implicar la selección de los casos apropiados de prueba que ayudan a descubrir las clases de errores que se describen en las Figuras 2.4, 2.5 y 2.6.

¿Cuándo comienza el procedimiento de prueba de unidad? Los métodos de prueba de unidad comienzan una vez que los programas se hayan desarrollado en el lenguaje fuente, revisado, verificado y eliminados los errores de sintaxis. Sin embargo, la preparación para la prueba de unidad comienza mucho antes. Puede comenzar tan pronto como el documento detallado del diseño para el software esté listo. Una revisión del documento detallado del diseño del software, proporciona pautas excelentes para generar casos de prueba.

Se ha mencionado que para realizar la prueba de unidad, se necesita preparar programas manejadores y 'stubs'. Los programas manejadores tienen que ser desarrollados para cada unidad que se probará. Los 'stubs' son subprogramas simulados, así que los 'stubs' tendrán que ser preparados para cada uno de los subprogramas que las unidades invocarán. Los 'stubs' sirven como reemplazo para las interfaces del subprograma invocado. Esencialmente, un 'stub' solamente realiza manipulaciones de datos muy elementales y quizás imprime un mensaje de diagnóstico de que el 'stub' fue ejecutado.

Se ha discutido el proceso de prueba de unidad. A continuación se explicará la prueba de integración.

5. Prueba de Integración

Una vez que las diversas unidades del software fueron sometidas al proceso de prueba de unidad, los defectos que aparecieron durante el proceso habrán sido eliminados y corregidos los errores. Idealmente, en esta etapa, se tendrán cada una de las unidades trabajando correctamente. Ahora se deben ensamblar las diferentes unidades en el sistema total. Este proceso de ensamblar las diferentes unidades en el sistema de software completo se llama integración. Probar el sistema como la integración de varias unidades se llama prueba de integración.

Un plan de prueba de integración contesta generalmente a preguntas como:

- ¿Qué se está probando?
- ¿Qué constituye éxito o falta?
- ¿Qué asignación de recursos hacer?, incluye tiempo, mano de obra y casos de prueba, entre otros para cada prueba.
- ¿Qué características debe tener el ambiente de prueba?
- ¿Que características deben ser probadas?
- ¿Qué criterios se deben tomar en cuenta para la documentación?
- ¿Qué responsabilidades asignar a los diversos individuos y a las organizaciones?

La Figura 2.7 ilustra dos posibilidades de integrar un sistema conformado por numerosos módulos de software, desarrollados independientemente y que se probaron con éxito. Observe que el sistema puede también consistir de módulos de hardware. Un enfoque es integrar los diferentes módulos probados de forma incremental en un sistema. Este sistema realizará más pruebas en cada paso incremental para comprobar si se cumplen con los criterios de aceptación del sistema o no. El segundo enfoque es integrar el sistema completo y después realizar pruebas a nivel sistema. Esto se llama explosión grande (big bang) o enfoque todo- arriba (all-up).

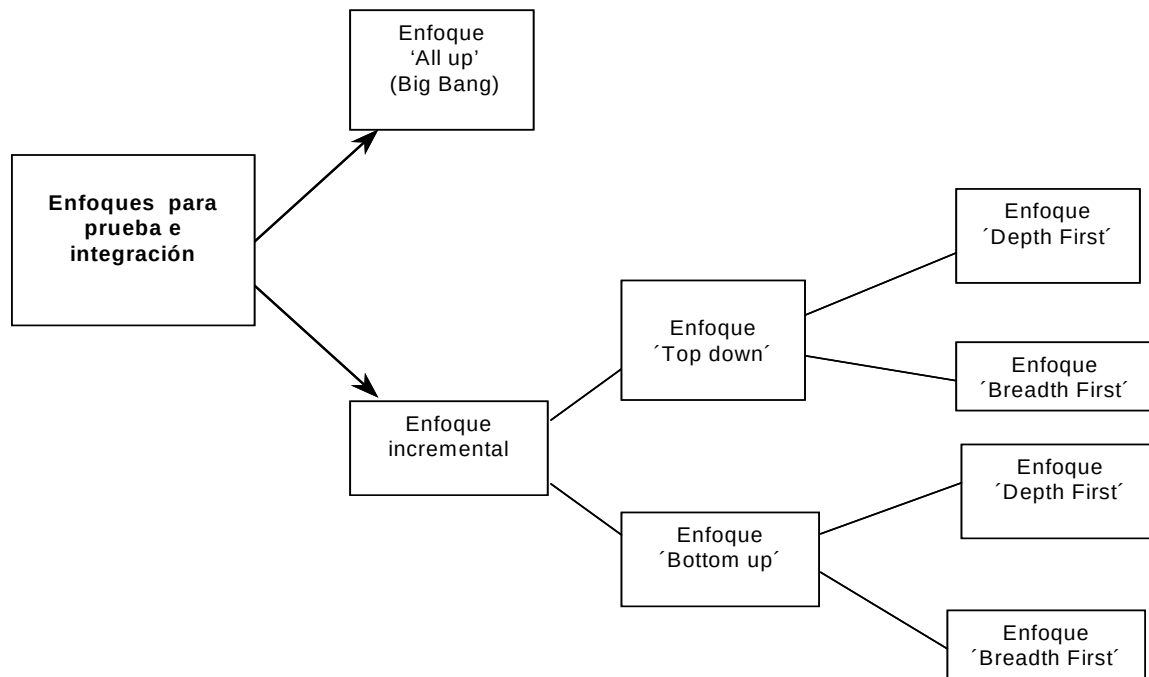


Figura 2.7: Enfoques Posibles para Prueba de Integración

A continuación se describen brevemente algunos de los enfoques principales para prueba e integración. Se comenzará con el enfoque 'hacia arriba' o enfoque de la 'explosión grande'.

5.1 El Enfoque 'Hacia Arriba' o Enfoque de la 'Explosión Grande'

En este enfoque, el conjunto completo de unidades primero se integra en el sistema final. Luego, el sistema final se prueba. En esta etapa, los métodos de prueba caja negra serán utilizados. Este enfoque puede sonar sistemático, pero no es un enfoque recomendado. Esto es porque, la prueba de un sistema grande que implica muchos módulos llega a ser demasiado complicado con muchas interacciones entre los módulos.

Cuando los defectos aparecen durante la prueba, localizar y corregir la fuente del error se convierte en una tarea difícil. A causa del gran número de módulos que interactúan, el proceso de prueba tiende a ser algo caótico. Las desventajas de usar este enfoque pesan mucho más que las ventajas que pueda ofrecer el enfoque. En vista de esto, el enfoque hacia arriba o enfoque de la explosión grande se emplea raramente a excepción de sistemas extremadamente pequeños que implican apenas algunos módulos.

Esto deja el enfoque incremental, que será discutido a continuación.

5.2 El Enfoque Incremental

De la Figura 2.7. se puede ver que el enfoque incremental para integrar y probar módulos se puede hacer de dos maneras – de arriba hacia abajo (top-down) o de abajo hacia arriba (bottom-up). En cada uno de los enfoques de arriba hacia abajo (top-down) o de abajo hacia arriba (bottom-up), los módulos pueden ser integrados y probados usando una primera profundidad o un enfoque de primer alcance. A continuación, se discuten brevemente los enfoques de arriba hacia abajo o de abajo hacia arriba para la integración y prueba incremental.

5.2.1 El Enfoque de Abajo hacia Arriba (Bottom-Up)

Se debe primero recordar que la estructura de los diferentes módulos en un sistema de software se puede considerar como un árbol de jerarquía. Esto se muestra en la Figura 2.8.

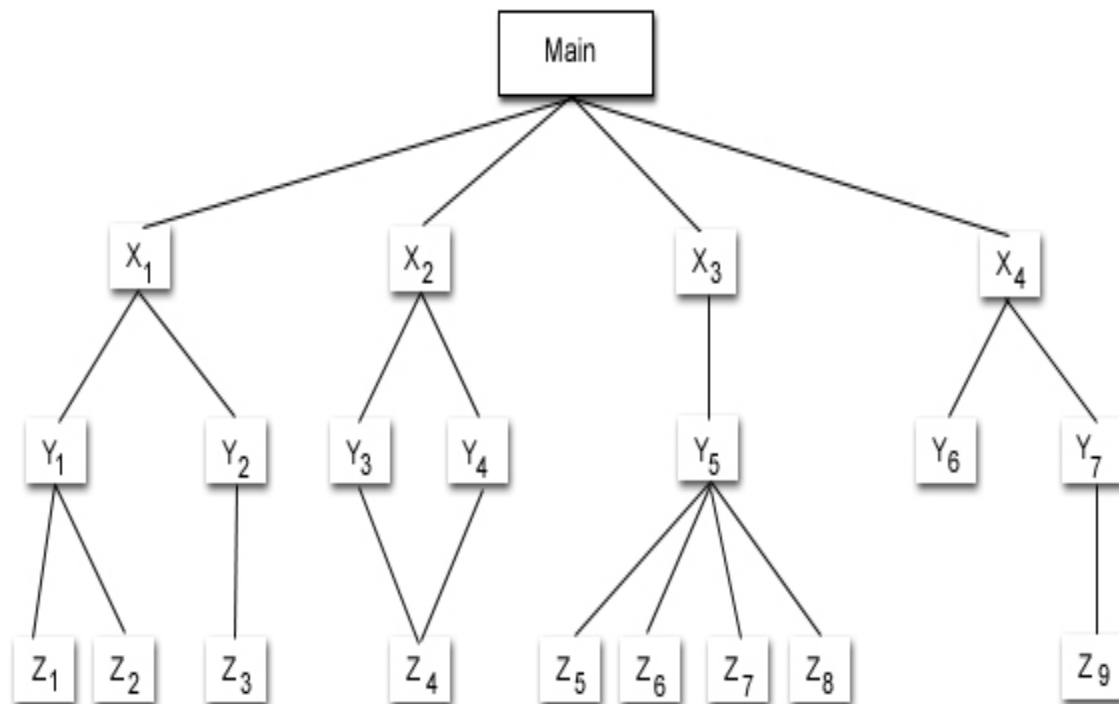


Figura 2.8: Estructura de Módulos en un Sistema

De esta figura, se puede ver que los módulos Z_i para $i = 1, 2, \dots, 9$ son todos módulos atómicos. Son módulos atómicos puesto que son módulos del nivel más bajo y realizan una tarea computacional específica. En el enfoque de abajo hacia arriba, se procede de los módulos atómicos y se integran en una parte funcional significativa. En la Figura 2.8 se pueden agrupar los módulos Z_1 y Z_2 con Y_1 para obtener una tarea de computacional significativa. Similarmente, todos los módulos en el nivel más bajo se pueden agrupar

con el módulo del siguiente nivel al cual se enlazan, para formar una parte computacional significativa. Una agrupación significativa se llama una estructura. Se puede, por ejemplo, tener las siguientes estructuras de la Figura 2.8:

Estructura 1: Z_1 , Z_2 , y Y_1

Estructura 2: Z_3 y Y_2

Estructura 3: Z_4 , Y_3 , Y_4 y X_2

Estructura 4: Z_5 , Z_6 , Z_7 , Z_8 y Y_5

Estructura 5: Z_9 y Y_7

Ahora, cada una de estas estructuras se prueba por separado. Hay varios pasos que se seguirán para probar estas estructuras. Estos son:

- Se debe escribir un programa manejador para cada una de estas estructuras para ayudar a realizar la prueba.
- No se necesita escribir 'stubs' porque todas las funciones llamadas por los módulos en cada una de las estructuras están disponibles. Es posible que el módulo haga una llamada a una función que no esté disponible en una estructura. Esto puede suceder solamente a causa de una estructuración de módulos incorrecta. Sólo se verifica que no existan funciones llamadas por los módulos que no estén presentes en la estructura, debido a una estructuración incorrecta de funciones en los programas. Si un módulo llama una función que no está presente en la estructura, debe ser referido al programador para la corrección y asegurar la estructuración apropiada.
- Los casos de prueba son generados para probar cada estructura.
- Las pruebas se realizan en cada estructura.

Al probar cada estructura, el reporte de error será generado. La idea es hacer uso del reporte de error de cada estructura que se probó, localizar los errores y corregirlos. Cada vez que cada una de las estructuras en un nivel particular haya sido probada, se puede mover hacia arriba para integrar con más módulos en un nivel más alto en otro conjunto de estructuras. De la Figura 2.8 por ejemplo, se pueden tener las estructuras siguientes:

Estructura 6: Z_1 , Z_2 , Y_1 , Z_3 , Y_2 , y X_1

Estructura 7: Z_5 , Z_6 , Z_7 , Z_8 , Y_5 y X_3

Estructura 8: Z_9 , Y_7 , Y_6 y X_4

Nota: De la Figura 4.8 se aprecia que Z_4 , Y_3 , Y_4 y X_2 pueden ser pensados como una estructura a este nivel. Pero se había considerado anteriormente como "estructura 3" en un nivel diferente. Considerar Z_4 , Y_3 , Y_4 y X_2 como otra estructura a este nivel no es

incorrecto, puesto que el conjunto de módulos representa una función significativa en el software.

Ahora, para las estructuras anteriores, el proceso normal para la prueba de integración se puede seguir y realizar. Una vez que se prueben estas estructuras, los módulos se pueden integrar con algunos en un nivel más alto. En la Figura 2.8, esto significa que los módulos se integran con el programa 'principal' para formar una estructura completa, que es el sistema. Este puede ahora ser probado.

Nota: Lo que se describió anteriormente es apenas uno de los enfoques para construir estructuras. Se pueden integrar los módulos para construir estructuras usando una primera profundidad o un enfoque de primer alcance. Un enfoque de primer alcance es lo que se siguió principalmente en la descripción. Si se siguiera el enfoque de profundidad primero, se integrarían los módulos como se presenta a continuación:

X_1, Y_1, Z_1 y Z_2

X_1, Y_2 y Z_3

Z_4, Y_3, Y_4 y X_2

Z_5, Z_6, Z_7, Z_8, Y_5 y X_3

X_4, Y_6, Y_7 y

El método de abajo hacia arriba trabaja con más eficacia para aquellos sistemas donde los módulos del nivel inferior son cruciales para el desempeño del sistema y también los más propensos a errores. Los manejadores se utilizan inicialmente para proporcionar interfaces, mostrar y guardar resultados, invocar llamadas y también dar interfaces de operador.

La desventaja con este enfoque es el número relativamente grande de manejadores requeridos a ser desarrollados mientras se integra de abajo hacia arriba y se prueba.

5.2.2 El Enfoque de Arriba Hacia Abajo (Top-Down)

En el enfoque de arriba hacia abajo se puede hacer uso también del enfoque de una primera profundidad o un enfoque de primer alcance para integrar los módulos y construir estructuras. Considere el enfoque de primer alcance, usando la Figura 2.8.

En la integración de primer alcance para hacer estructuras, la idea principal es considerar todos los módulos presentes en el mismo nivel, subordinarlos a un módulo de alto nivel e integrarlos. En la Figura 2.8, por ejemplo, se pueden integrar X_1, X_2, X_3 y X_4 . Todos estos módulos están en el mismo nivel horizontal y son subordinados al programa principal. El programa principal mismo actúa como manejador de la prueba, de allí que no es ningún requisito escribir un manejador especial de prueba. Pero cada uno de los módulos X_1, X_2, X_3 y X_4 llaman a las funciones en los niveles más bajos. Se

deben sustituir estas funciones en los módulos de nivel inferior por los 'stubs'. Una vez que se hace la estructura, se prueba.

Los defectos que se encuentran deben conducir al equipo de depuración y prueba al proceso de localizar la fuente del error y corregirlos. Pero este proceso tiene algunas limitaciones. Hace uso de un número bastante grande de 'stubs' en las primeras etapas de la integración. Esto significa que muy poco flujo significativo de datos verdaderos sucede de los módulos de nivel inferior a los módulos de un nivel más alto (que es parte de la estructura que es probada). Esto implica que una vez que un defecto se revele en el informe del error, se tiene poca información para rastrear la fuente del error. No se puede, por lo tanto, ignorar el requerimiento de tener un flujo significativo de datos verdaderos de los módulos de nivel inferior. Esto significa que los 'stubs' no pueden ser inútiles. Los 'stubs' deben proporcionar una cierta funcionalidad limitada del módulo y entregar un flujo real de datos. Es decir, los 'stubs' tienden a ser más complicados y a crear enormes gastos indirectos para el proceso de prueba.

En el enfoque de arriba hacia abajo, una vez que se prueba una estructura, el módulo substituye uno de los 'stubs' y se prueba otra estructura. Así, alternadamente, los módulos reales substituyen cada uno de los 'stubs' y se prueba cada estructura.

En el enfoque de primera profundidad, hay una tentativa de integrar todos los módulos que están en un camino principal de control. Así, la integración de primera profundidad depende de la identificación de caminos de control principales. En la Figura 2.8, por ejemplo, se pueden identificar los siguientes caminos principales de control.

Principal y X_1

Principal y X_2

Principal y X_3

Principal y X_4

Un método es tratarlos como estructuras diferentes y probarlas. Una vez que la estructura Principal y X_1 se haya probado, se puede reemplazar el 'stub' de Y_1 por el módulo real Y. Después de que esta estructura integrada sea probada, se puede reemplazar el 'stub' de Y_2 con el real Y_2 . Después de que esta estructura también sea probada, se procede a integrar el resto reemplazando los 'stub' de Z_1 y Z_2 por sus respectivos módulos reales.

En la integración de primera profundidad, se recomienda que se seleccionen los caminos del control en un orden que permita demostrar el funcionamiento de los aspectos específicos de la aplicación. Se deben seleccionar los aspectos más importantes de la funcionalidad para realizar las pruebas.

El método de arriba hacia abajo trabaja mejor para aquellos sistemas donde los módulos de control son cruciales para el desempeño del sistema, y por tanto, más propensos a errores. En este método, los módulos que implementan la estructura de control de un nivel superior se desarrollan primero y luego se agregan a la configuración

del sistema. Las interfaces son probadas y se ejecutan las funciones de control básicas. Los 'stubs', módulos de interfaz y retardadores representan otros módulos que no están disponibles inicialmente, pero que pueden ser llamados por los módulos de control.

También es posible el uso de una combinación de los enfoques de primera profundidad y de primer alcance para integrar los módulos.

El enfoque incremental trabaja mejor, ya que provee para las pruebas solo un número limitado de módulos desconocidos en cualquier momento. Las fallas son más fáciles de rectificar, ya que cuando aparecen se está probando, porciones pequeñas de un programa. El enfoque incremental ayuda en sistemas implementados sobre una base de tiempo real. Este enfoque también ayuda a producir un producto robusto de mejor calidad, debido a las posibilidades de detectar fácilmente los errores.

Un plan de prueba correctamente organizado facilitará que los módulos se agreguen de tal manera que permita la prueba de las capacidades funcionales del sistema en los subconjuntos de los diferentes módulos que comprenden el sistema completo. La arquitectura de software, separar los requerimientos en módulos y los cronogramas efectivos para el desarrollo de los módulos ayudan al proceso de prueba e integración.

A continuación se discute brevemente acerca de la prueba del sistema.

6. Prueba del Sistema

La prueba del sistema se emprende después de que el proceso de prueba de integración es completado y los errores analizados, localizados y corregidos. La prueba del sistema tiene algunos elementos de validación y de aquí que algunas veces se le refiera como prueba de validación. La prueba del sistema se hace para asegurarse de que las funciones del software cumplan con las expectativas del cliente según lo establecido en la SRS. La SRS usualmente contiene una sección llamada 'criterios de validación', los cuales sirven como base para la prueba del sistema.

Se hace hincapié de nuevo que el propósito de la prueba de sistema o prueba de validación es demostrar que el software realiza cada uno de los requerimientos indicados en la SRS. Las pruebas de validación se llevan a cabo para establecer que el software cumple con lo siguiente:

- **Conformidad a los Requerimientos Funcionales:** Para cada uno de los requerimientos funcionales en la SRS, se generan los casos de prueba específicos y las pruebas se realizan para revelar si el software cumple con el requerimiento.
- **Conformidad a Todas las Características de Comportamiento:** Se conducen las pruebas usando casos específicos de prueba para comprobar si cada una de las características del comportamiento indicadas en la SRS se cumplen.
- **Conformidad a los Requerimientos de Funcionamiento:** Se conducen las pruebas usando casos de prueba apropiados para asegurar que el software se ejecuta según requerimientos de desempeño establecidos.

- **Conformidad a las Necesidades de Documentación:** Las pruebas de validación se realizan para asegurar que toda la documentación requerida está presente, correcta y aceptable para el cliente.
- **Conformidad a los Requerimientos Misceláneos:** Las pruebas de validación deben también probar la conformidad con algunos requerimientos específicos tales como compatibilidad, capacidad de mantenimiento y extensión que se indiquen en la SRS.

Todas las pruebas de validación se conducen a través de un proceso de prueba caja negra. Cada prueba revela si el software se ejecuta en conformidad con un requerimiento o no. Si el software no se ejecuta según lo indicado en los requerimientos, se denomina deficiencia. Para detectar tempranamente esto, se llevan a cabo las pruebas alfa y beta. Se debe observar que las pruebas alfa y beta son aplicables para los productos de software y no para todos los tipos de proyectos del software.

6.1 Pruebas Alfa y Beta

En estos métodos, un producto de software se prueba en un ambiente real bajo condiciones reales, con operadores verdaderos.

La prueba alfa considera un equipo de usuarios y operadores del cliente que viene al ambiente del desarrollador y participa en el proceso de prueba. La prueba alfa se hace a veces en una instalación separada del sistema, creada solamente para el propósito de la prueba. El equipo del usuario desplegado para la prueba alfa hace uso a menudo de sus propios tiempos y aplicaciones. Utilizan casos de prueba adicionales y nuevos operadores, dando por resultado el requerimiento y el uso de más tiempo en el sistema. Las ocurrencias de fallas se documentan cuidadosamente y se entregan al equipo de prueba de la organización que desarrolla.

La prueba beta implica entregar una o más copias del software o sistema a la instalación del cliente. En esta instalación del cliente se le asigna el estado beta. Cualquier falla en el software o sistema se debe reportar inmediatamente al equipo de prueba de la organización que desarrolla. Este método implica a más usuarios, diferentes operadores así como diversos tiempos. Más tiempo de CPU para el sistema asegura que más fallas están expuestas, aumentando la validez del producto en la etapa siguiente.

Finalmente se discute la prueba de aceptación.

7. Prueba de Aceptación

El primer paso en el proceso de prueba es ultimar los criterios de aceptación del sistema con el cliente. Esto puede preceder u ocurrir simultáneamente con la evolución del plan de prueba. Los criterios de la aceptación son cruciales para validar el sistema en etapas posteriores para comprobar si resuelve los requerimientos del cliente o no. Los criterios de aceptación se incorporan normalmente en la declaración de la misión o la especificación de sistema. Forma a menudo parte del acuerdo contractual que se

hace parte de la estructura de honorarios. Los criterios de aceptación del nivel de sistema se dan generalmente a la computadora, al operador y al software. La culminación exitosa de todos los criterios de aceptación convenidos por el contratista y el cliente en el comienzo del ciclo de desarrollo del sistema, constituye los requerimientos de la validación. La falla en cumplir con los criterios de aceptación puede reducir las ganancias para la firma del software o resultar en penalizaciones financieras.

La Figura 2.9 ilustra los diversos tipos de criterios de la aceptación:

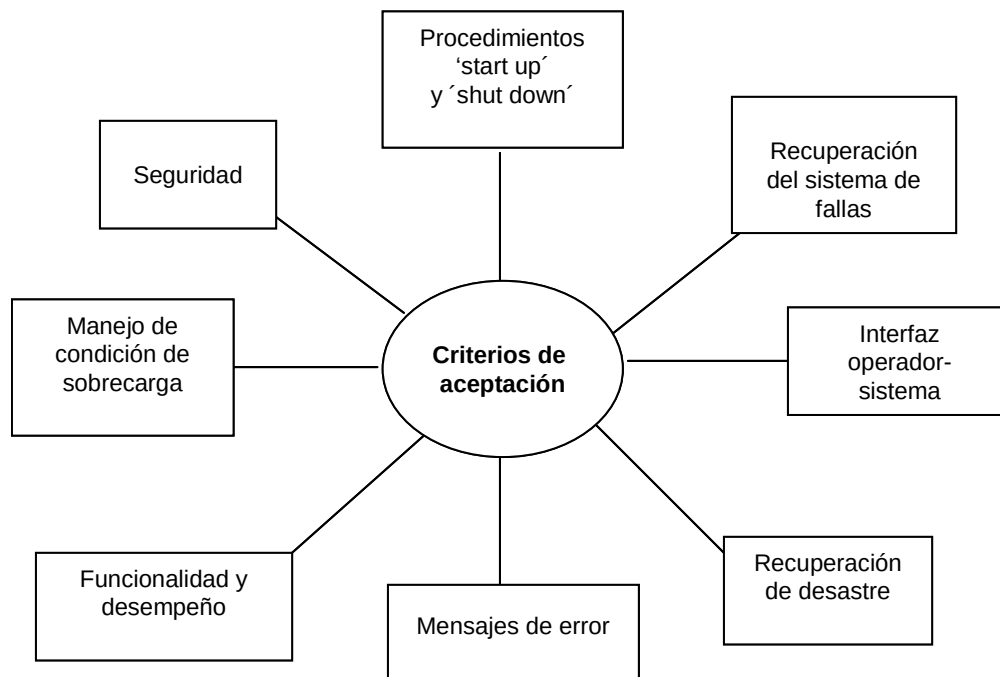


Figura 2.9: Criterios de Aceptación para el Desarrollo del Software

¿Qué debe alcanzar la prueba del sistema? La prueba del sistema debe poder identificar las diferencias entre el desempeño medido y requerido de un sistema, que se hace siguiendo un conjunto de escenarios de prueba. La medida del desempeño de un sistema se debe hacer en condiciones que se asemejen muy de cerca al ambiente operacional real.

Como parte de la prueba del sistema, el software debe ser probado para recuperación, seguridad, estrés y funcionamiento. Se discuten las pruebas para cada uno de estos puntos.

7.1 Prueba de Recuperación

La falla del software al funcionar en un negocio puede ser costosa. Por lo tanto, el software debe ser capaz de recuperarse de fallas. Es decir, el software debe ser capaz de comenzar a funcionar otra vez en el tiempo más corto posible (según lo indicado en la SRS). Por otra parte, los errores individuales que ocurren en un subsistema no deben tumbar el sistema completo.

La prueba de recuperación se diseña para asegurar las funciones de recuperación indicadas en la SRS y que funcionen correctamente. Se generan y utilizan los casos de prueba para hacer que el software falle, y así observar si el proceso de recuperación funciona según lo deseado. El proceso de recuperación puede ser manual o automático.

- **Proceso Manual de Recuperación:** Cuando se utiliza un proceso manual de recuperación, la prueba de recuperación se concentra en asegurar que el tiempo para la recuperación esté dentro de los límites establecidos para este proceso en la SRS.
- **Proceso Automático de Recuperación:** En este caso, la prueba de recuperación hace uso de casos de prueba apropiados para comprobar si los mecanismos de recuperación de los datos trabajan. Los casos de prueba también se utilizan para comprobar los mecanismos de puntos de chequeo (usados en el proceso automático de la recuperación) si hay alguno, y si el reinicio de estos mecanismos, trabajan según lo indicado en la SRS.

7.2 Prueba De Seguridad

La prueba de seguridad se realiza para asegurar que el software mantiene las características de seguridad requeridas y establecidas en la SRS. El proceso de prueba de seguridad incluye un conjunto de actividades diseñadas para intentar burlar la seguridad del sistema. Algunos de los intentos pueden ser los siguientes:

- Explorar el rechazo de ataques a servicios sobre el sistema a través de un número abrumador de peticiones de servicio al mismo.
- Explorar caminos para romper la contraseña de usuarios, archivos y recursos, además de obtener acceso no autorizado.
- Explorar las posibilidades de acceso a los recursos protegidos a través de los agujeros de seguridad en el sistema.

7.3 Prueba de Estrés

El objetivo principal de la prueba de estrés es someter al software a condiciones extremas y observar en qué etapa colapsa el sistema. Es decir, el objetivo del proceso de prueba es determinar el grado al cual el software se puede someter a condiciones extremas. Algunas de estas condiciones extremas pueden ser cualquiera de las siguientes:

- Necesidad de un gran espacio de almacenamiento secundario.
- Necesidad de grandes cantidades de ancho de banda para las transferencias en la red.
- Necesidad de una gran cantidad de tiempo de la CPU.
- Necesidad de ejecutar un programa muy grande que requiera espacio extremadamente grande de datos.

Aparte de esto, una prueba que genere un número anormalmente grande de interrupciones por segundo será una condición extrema también. En la prueba de

estrés, se introducen este tipo de condiciones extremas y se observa el funcionamiento del sistema.

Una variante de la prueba de estrés es la prueba de sensibilidad. A veces, una cantidad pequeña de datos dentro de los límites de los datos válidos para un proceso puede interrumpir el horario del proceso y disminuir el desempeño del sistema. Esto sucede comúnmente en algoritmos matemáticos. Se realiza la prueba de sensibilidad como una tentativa de conseguir librarse de este tipo de datos.

7.4 Prueba de Desempeño

La prueba de desempeño se lleva a cabo para asegurar que los requerimientos de funcionamiento establecidos en la SRS y el funcionamiento del software están en conformidad. Ésta es la prueba de desempeño en tiempo de ejecución. Claramente, la prueba de desempeño se puede llevar a cabo significativamente solamente cuando se integra y se prueba el sistema completo. Sin embargo, algunas pruebas de funcionamiento también se hacen en el ámbito de la prueba de unidad. El propósito principal es revelar deficiencias, si las hay, en el desempeño del software.

Resumen

Ahora que ha finalizado esta unidad, usted será capaz de:

- Distinguir entre la verificación y la validación.
- Describir el proceso de prueba.
- Explicar los métodos involucrados en el proceso de prueba de unidad.
- Discutir los enfoques involucrados en la prueba de integración.
- Describir los diversos componentes de la prueba del sistema.

Unidad 2: Examen de Autoevaluación

- 1) ¿Qué es verificación del software?
 - a) Todas esas actividades que suceden al final de un ciclo de desarrollo particular, que confirman que el producto se está desarrollando correctamente y que satisface las condiciones establecidas al principio de la etapa del desarrollo.
 - b) Todas esas actividades que suceden al final de un ciclo de desarrollo particular, que confirman que el producto se está desarrollando correctamente y reflejan la SRS o su equivalente.
 - c) El proceso que determina que el software cumple con los requerimientos del cliente.
 - d) El proceso de probar un software y determinar que no tiene ningún error.

- 2) ¿Qué es validación?
 - a) La validación es el proceso de asegurar que el software es 100% libre de error.
 - b) La validación es verificación más la certificación.
 - c) La validación es el proceso de asegurar que el software funciona en conformidad con cada uno de los requerimientos indicados en la SRS.
 - d) La validación es el proceso de usar la prueba de caja blanca para descubrir defectos en interfaces.

- 3) ¿Cuáles son las fuentes comunes de errores encontrados en la prueba de unidad?
 - a) Errores del diseño.
 - b) Falla de inicialización o de los valores por defecto.
 - c) Nombres de variables escritos incorrectamente o truncados, que son errores tipográficos.
 - d) Tipos de datos inconsistentes.

- 4) Los errores en comparaciones y flujo del control ocurren en programas principalmente debido a:
 - a) Uso de operadores lógicos erróneos.
 - b) Uso de una condición incorrecta o no existencias de la terminación del bucle.
 - c) Uso de precedencias incorrectas de operadores en expresiones aritméticas.
 - d) Uso de instrucciones complicadas de E/S que generan muchas interrupciones.

- 5) Los 'stubs' son:
- a) Subprogramas simulados usados en la prueba.
 - b) Subprogramas ya probados.
 - c) Una cierta información que se pasa del programa que llama al llamado durante la prueba.
 - d) Direcciones al programa que llama durante la prueba.
- 6) ¿Qué puede hacer un stub?
- a) Reemplazar la interfaz del módulo llamado.
 - b) Realizar ejecuciones simples de manipulación de datos.
 - c) Verificación impresa de la entrada.
 - d) No puede hacer nada, simplemente retorna al programa que lo llama ya que es simulado.
- 7) ¿Cuáles de los siguientes son verdaderos cuando se usa el enfoque de arriba hacia abajo al realizar la prueba de integración?
- a) No se tiene que desarrollar manejadores especiales, puesto que los módulos del nivel superior pueden ser utilizados mientras se conducen las pruebas.
 - b) Se tienen que desarrollar manejadores especiales a pesar de que el módulo del nivel superior es integrado primero.
 - c) Se tiene que hacer uso de stub durante la integración en varias etapas, mucho más cuando se integran los módulos en niveles superiores y menos cuando se integran módulos en niveles inferiores.
 - d) Este enfoque no requiere el uso de stub.
- 8) ¿Cuáles son las desventajas de usar el enfoque de todo arriba o el enfoque de explosión grande para la prueba de integración?
- a) Tiende llegar a ser demasiado complejo ya que todas las interacciones del módulo deben ser consideradas.
 - b) Tiende hacer caótico el proceso de prueba.
 - c) No se puede garantizar que detecta todos los defectos que están en el software.
 - d) El proceso es incompatible con las técnicas de la caja blanca y de la caja negra.

- 9) ¿Cuáles de los siguientes están conectados con la prueba alfa?
- a) La prueba se hace en el sitio del desarrollador, solamente por los representantes del cliente.
 - b) La prueba se hace en el sitio del cliente o cualquier otro sitio señalado como sitio de prueba alfa.
 - c) Los reportes de error son manejados por el mismo equipo de prueba alfa.
 - d) Los reportes de error se envían de nuevo al equipo de prueba en la organización del desarrollador que los maneja.
- 10) ¿Cuáles de los siguientes están conectados con la prueba beta?
- a) La prueba se hace en el sitio del desarrollador solamente.
 - b) La prueba se hace en el sitio del cliente o cualquier otro sitio señalado como sitio de prueba beta.
 - c) Los reportes de error son manejados por el mismo equipo de prueba beta.
 - d) Los reportes de error se envían de nuevo al equipo de prueba en la organización del desarrollador para manejarlos.

Respuestas de la Unidad 2: Examen de Autoevaluación

- 1) a
- 2) c
- 3) b, c y d
- 4) a y b
- 5) a
- 6) a, b y c
- 7) a y c
- 8) a y b
- 9) a y d
- 10) b y d

Unidad 3: Control de Calidad del Software

Objetivos de Aprendizaje

Al final de esta unidad, usted será capaz de:

- Explicar el concepto de la calidad.
- Describir el control de calidad y el aseguramiento de la calidad en el contexto del software.
- Discutir la seguridad y confiabilidad del software.
- Explicar el Modelo de Capacidad de Madurez – SEI.

1. ¿Qué es Calidad?

Se ha estado hablando continuamente acerca de calidad y de la necesidad de ésta en el desarrollo del software. Pero, ¿qué significa el término realmente?. La calidad, en el contexto del software, trata de los métodos que aseguran ciertos estándares preestablecidos en varios aspectos de un desarrollo de programa de software y aplicación. Existen dos aspectos:

- **Calidad de Diseño:** Se ocupa generalmente de las características especificadas por un diseñador para un elemento particular, tal como especificaciones del funcionamiento, tolerancias, etc. En el caso del software, se ocupa de requerimientos, diseño del sistema y las especificaciones.
- **Calidad de Conformidad:** Se ocupa del grado con el cual las especificaciones se cumplen durante el proceso de fabricación. En el caso del software, la calidad de conformidad se relaciona generalmente con la implementación. Se dice que es alto si la implementación sigue el diseño y el sistema resultante cumple los requerimientos especificados.

El concepto de la calidad se ilustra en la Figura 3.1.

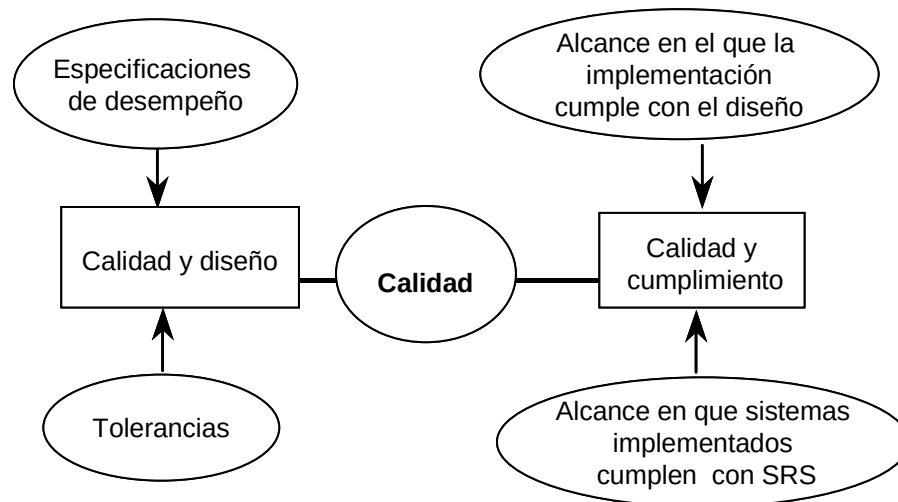


Figura 3.1: Concepto de la Calidad

Las tres dimensiones de la calidad del software son:

- **Operación del Producto:** Implica apropiado, confiabilidad, eficacia, integridad y utilidad.
- **Transición del Producto:** Implica portabilidad, reutilización e interoperabilidad.
- **Revisión del producto:** Implica capacidad de mantenimiento, flexibilidad y prueba.

2. Algunos Conceptos sobre Calidad

Muchos desarrolladores creen que la calidad del software es un concepto que puede ser implementado después de que se haya generado el código. Sin embargo, esto no es verdad. El Aseguramiento de la Calidad del Software (SQA, siglas en inglés) abarca una serie de actividades que tienen que ser implementadas a través del proceso de desarrollo del software y el proceso de la aplicación. Los desarrolladores tienen que generar un conjunto de actividades para asegurar una alta calidad del producto, realizar pruebas del aseguramiento de la calidad y usar métricas para desarrollar las estrategias que mejorarán el proceso del software. También tendrá que asegurar la calidad total del producto, si se proponen desarrollar un software confiable, es decir, un software que las personas no dudarán en usar incluso en situaciones que implican riesgo o lesión a las personas. El proceso de SQA se muestra en la Figura 3.2.

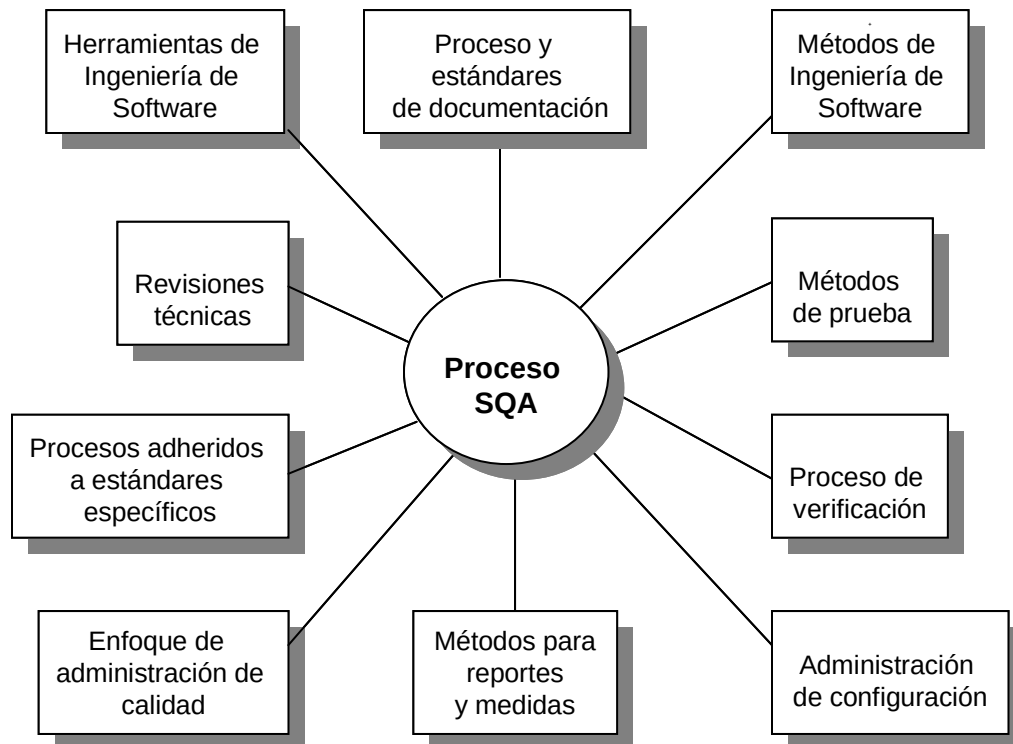


Figura 3.2: Proceso SQA

3. ¿Cómo se Controla la Calidad del Software?

El control de calidad del software se relaciona principalmente con las inspecciones, revisiones y pruebas realizadas durante el proceso del software, de forma de asegurar que los productos desarrollados cumplen los estándares establecidos. El control de calidad también necesita un lazo de retroalimentación del proceso por el cual el producto fue desarrollado. Esta combinación de pruebas y de retroalimentación será útil si el producto no puede cumplir los estándares especificados. En tal caso, el

departamento del control de calidad podrá mejorar el proceso de fabricación para asegurar alta calidad en las versiones futuras del producto.

4. Aseguramiento de la Calidad del Software

El aseguramiento de la calidad se refiere al aspecto de revisión y de reporte de la administración. Su función básica es proporcionar datos sobre la calidad del producto a la gerencia y asegurar que los estándares de calidad especificados se estén cumpliendo. Los datos de aseguramiento de la calidad pueden también discutir cualquier defecto en la calidad del producto. La gerencia puede entonces tomar la acción necesaria para rectificarla. La adherencia a la alta calidad es imprescindible para cada programa o aplicación de software.

El SQA se puede definir como la conformidad a las necesidades funcionales y de rendimiento, a los estándares de desarrollo y a las características implícitas requeridas de todo el software que se ha desarrollado profesionalmente.

Partiendo de esta definición, se tiene que el SQA se relaciona con tres cosas:

- Los requerimientos para un programa / aplicación de software formarán la base para evaluar su calidad. La no-conformidad a estos requerimientos dará lugar a la carencia de la calidad.
- Los estándares especificados influyen en el desarrollo del software. La no-conformidad a estos criterios especificados dará lugar a la mala calidad del programa o aplicación.
- Cada aplicación o programa de software tiene ciertos requerimientos que son implícitos, por ejemplo buena velocidad de procesamiento, facilidad de uso, etc. El conformarse solamente con los requerimientos explícitos, sin atender a los requerimientos implícitos, también dará lugar a la pobre calidad del software.

Hasta ahora, se deben haber dado cuenta que el propósito del SQA es evaluar críticamente el software desarrollado, comprobar si los estándares de calidad se han cumplido, si el producto satisface o no las necesidades del cliente y también identifica las debilidades que el cliente puede detectar.

5. Algunas Operaciones SQA

El SQA se puede identificar como un patrón de acciones planeado y sistemático, que ayudan a asegurar alta calidad en el software de programas y aplicaciones. Hay diversas operaciones que vienen bajo SQA. Éstas se asocian generalmente a los siguientes dos conjuntos de personas:

- Ingenieros de Software.
- Grupo SQA.

Es la responsabilidad de los Ingenieros de Software ocuparse de todo el trabajo técnico involucrado en actividades de aseguramiento y control de calidad. Esto lo hacen

aplicando métodos técnicos, realizando revisiones técnicas y realizando pruebas en el software desarrollado.

Hoy en día, cada organización trabajando con desarrollo de software tiene un equipo de SQA, que actúa como el representante interno del cliente. Es la responsabilidad del grupo SQA ayudar a los Ingenieros de Software a lograr una alta calidad en el programa o aplicación de software terminado. De hecho, un conjunto específico de actividades del SQA ha sido recomendado por el Software Engineering Institute - Carnegie-Mellon University. Estas actividades se ilustran en la Figura 3.3.

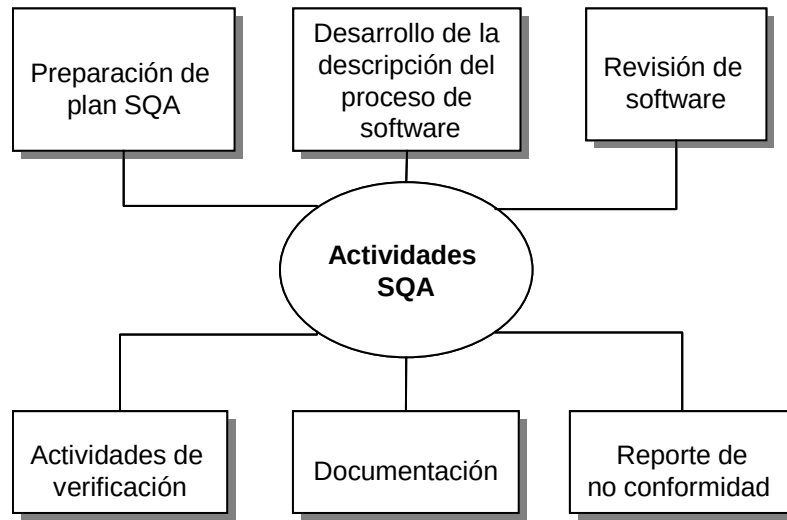


Figura 3.3: Actividades del SQA

Es cierto que algunos errores serán detectados mientras que está ocurriendo el desarrollo del software, mientras que otros serán detectados una vez que se ha desarrollado y lanzado el software. El "Principio de Pareto" se conoce como la regla 80-20, que en este contexto, identifica el 20% de las causas dominantes que contribuyen hasta el 80% de todos los defectos en el software.

6. Confiabilidad del Software

El factor de confiabilidad influye la mayor parte de las veces en la calidad total de un programa o aplicación de software. Si funciona sin fallar siempre que se esté utilizando, se considera de alta calidad. Se indica la mala calidad cuando el programa o aplicación falla repetidamente o con frecuencia, por cualquier razón aún si otros factores parecen aceptables. La confiabilidad del software se puede medir y calcular usando los datos obtenidos durante las pruebas.

Considere un ejemplo donde un programa dado tiene una confiabilidad de 0.90 sobre ocho horas de procesamiento. Esto indica que si el programa en particular debe ejecutarse 100 veces y necesita 8 horas de tiempo de ejecución, funcionará probablemente de manera correcta 90 veces de las 100 posibles.

El término falla, en términos de la confiabilidad y de la calidad del software significa una inconformidad a los requerimientos especificados. Una falla puede ser un inconveniente simple o importante con consecuencias desastrosas. Algunas fallas pueden ser rectificadas fácilmente mientras que otras pueden tomar meses. Existe también la posibilidad de que la corrección de una falla pueda conducir a la aparición de otros errores, dando por resultado finalmente una calidad muy mala.

7. Seguridad del Software

La seguridad del software es una operación de SQA que ayuda a identificar y a determinar las zonas peligrosas potenciales que pueden causar fallas del sistema. Un sistema de control puede llegar a ser muy complejo después de que el software se haya instalado como parte del sistema. Llega a ser muy difícil detectar fallas de diseño sutiles cuando el software está involucrado en el sistema de control.

La seguridad del software es principalmente un proceso de modelación y análisis donde las amenazas potenciales se identifican y se agrupan basándose en lo crítico y el riesgo. Una vez que se hayan identificado las amenazas, las técnicas de análisis determinan su severidad y posibilidad de ocurrencia. Se debe tener presente que el software tiene que ser analizado en el contexto del sistema completo, no independientemente. Los eventos indeseables se pueden detectar usando varios métodos como el análisis del árbol de fallas, lógica en tiempo real o modelos de red de petri. Estos métodos pueden también determinar la probabilidad de que ocurran tales eventos individuales.

El siguiente paso, después de que las amenazas se han identificado y se han analizado, es determinar los requerimientos de seguridad. Los eventos indeseables y las respuestas necesarias del sistema, se enumeran en este paso. Aquí, se enfoca el rol del software en el manejo de los eventos indeseables.

La confiabilidad y la seguridad del software se relacionan estrechamente, sin embargo existen pequeñas diferencias entre los dos. La confiabilidad del software hace uso del análisis estadístico de la calidad para localizar la posibilidad de falla del software. Aquí, una falla no tiene que terminar en un contratiempo. La seguridad del software, por otra parte, trata diversas maneras en las cuales las fallas terminan en contratiempos. En este caso, las fallas no se tratan de una manera aislada. En este caso, se determinan en el contexto del sistema completo.

8. El Plan de SQA

El plan de SQA establece parámetros amplios para la implementación del aseguramiento de la calidad del software. Puede ser descrito como plantilla para implementar las diferentes actividades relacionadas con el SQA para cada proyecto del software, y es desarrollado generalmente por la firma del grupo de SQA. El IEEE ha fijado cierto estándar para los planes de SQA. Algunos se listan a continuación:

- La primera parte de estas especificaciones está relacionada con el alcance y el propósito del documento e indicaciones de esas actividades de software que son cubiertas por el aseguramiento de la calidad. Todos los documentos especificados en el plan de SQA se enumeran junto con estándares aplicables.
- La sección de administración especifica la posición del SQA en la estructura de la organización, las tareas y las actividades del grupo de SQA y su posición a través del proceso completo del desarrollo. También habla de roles y responsabilidades en la organización en lo referente a calidad del producto.
- La tercera sección, es la sección de la documentación, se refiere a cada producto creado durante el proceso del software. Habla del conjunto mínimo de productos de trabajo necesarios para alcanzar una alta calidad.
- La cuarta sección, los estándares, prácticas y la sección de convenciones, listan todos los estándares aplicables y las prácticas aplicadas durante el proceso del desarrollo. También enumera, procesos y métricas del producto que se deben recoger como parte del trabajo de desarrollo.
- La quinta sección es la sección de la revisión y de la auditoria. Especifica todas las revisiones y auditorias que llevarán a cabo el equipo de desarrollo de software, el grupo de SQA y el cliente. La sección ofrece una vista resumida de cómo es considerada cada revisión y auditoria.
- La sexta sección, la sección de prueba, se relaciona con el Plan y el Procedimiento de Pruebas del Software y establece parámetros del mantenimiento de registros de las pruebas.
- La séptima sección, es la sección de reporte del problema y de la acción correctiva, dicta los procedimientos que se seguirán para reportar, realizar seguimiento y eliminar errores y defectos en el producto. También identifica quién es responsable de estas actividades en la organización.

La porción restante del plan del SQA identifica las diferentes herramientas y métodos que soportan las actividades y tareas del SQA. También hace una referencia a los procedimientos para la administración de la configuración del software, define un enfoque apropiado para contratar gerentes, fija maneras de ensamblar, salvaguardar y mantener registros, identifica requisitos de entrenamiento y también establece los métodos para identificar, evaluar, supervisar y controlar el riesgo.

9. El Modelo de Capacidad de Madurez (CMM)

En el año 1982, el Departamento de Defensa de los EE.UU. (DoD, siglas en inglés), uno de los contratistas más grandes de software, formó una fuerza de trabajo conjunta de servicio para revisar los problemas de software que estaba enfrentando. Desde entonces, para ayudar al DoD y a otras organizaciones a abordar sus problemas de software, se estableció en 1984 el Instituto de Ingeniería de Software (SEI) en la Universidad de Carnegie-Mellon. Liderizado por Watts Humphrey, el SEI comenzó a desarrollar un marco de trabajo del proceso de madurez.

El CMM es un marco de trabajo que describe qué elementos son claves para tener un proceso de desarrollo eficiente. Este marco de trabajo describe las prácticas claves para

el planeamiento, ingeniería, desarrollo y mantenimiento del software. Cuando se siguen estas prácticas, se mejora la capacidad de organización para alcanzar las metas en cuanto a costo, cronograma, funcionalidad y calidad del producto. El CMM puede ser usado por una organización para un planear mejoras de su proceso de software.

9.1 Estructura del Modelo de Capacidad de Madurez

El CMM esta compuesto de 5 niveles de madurez. Con excepción del primer nivel, cada nivel de madurez está compuesto de diferentes áreas de práctica clave (KPAs). Cada KPA identifica un conjunto de actividades relacionadas (llamadas características comunes) que, cuando son ejecutadas conjuntamente, se alcanza un conjunto de metas consideradas importantes para establecer la capacidad del proceso en ese nivel de madurez (se cumple con las metas de esa área clave del proceso).

Cada KPA consta de 5 secciones de características comunes: acuerdo para la realización, capacidad de realización, actividades de desempeño, análisis, medición y verificación de la implementación. Las características comunes son las cualidades que indican, si la implementación y la estandarización de la KPA del proceso, es eficaz, repetible y duradera.

Capacidad del proceso, describe el rango de resultados esperados que pueden ser alcanzados siguiendo un proceso de software.

La madurez de proceso de software es el grado al cual un proceso específico es definible, manejable, medible, controlable y eficaz. La madurez implica un potencial para el crecimiento en capacidad e indica la riqueza del proceso del software de una organización y la consistencia con las cuales se aplica en proyectos a través de la organización. La madurez de un proceso de software, implica que tanto la productividad como la calidad que resulta del proceso en una organización, puedan mejorar en un cierto plazo con aumentos constantes en la disciplina alcanzada.

Cada nivel de la madurez proporciona una capa en la fundación para la continua mejora del proceso. Cada área del proceso clave abarca un conjunto de las metas que, cuando son satisfechas, estabilizan un componente importante del proceso de software. Alcanzando cada nivel del modelo de la madurez, se estandariza un componente del proceso de software, dando por resultado un aumento total en la capacidad de proceso de la organización. Los cinco niveles del proceso de madurez se muestran en la Figura 3.4.

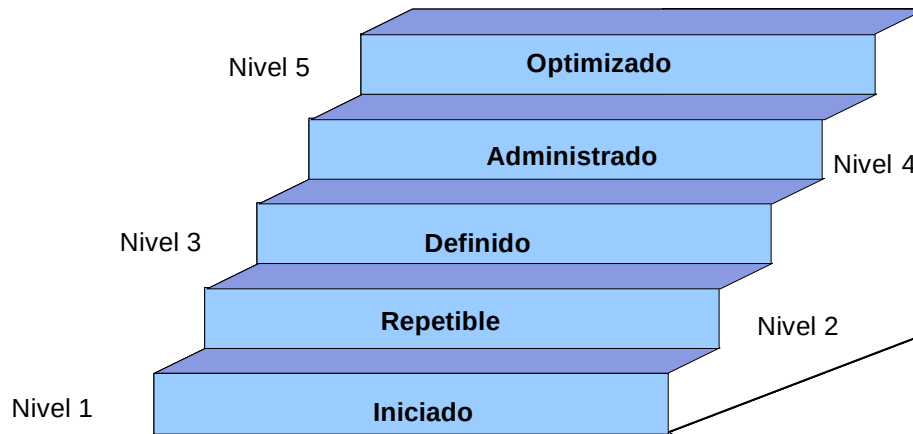


Figura 3.4: Cinco Niveles del Modelo de Capacidad de Madurez (CMM)

Hasta que punto para el cual los KPAs se implementan en cada nivel, determina el nivel del grado del proceso de madurez de una organización. Los KPAs son grupos relacionados de actividades que mejoran la capacidad de proceso. El alcance de la implementación para un KPA específico se evalúa determinando los parámetros mostrados en la Figura 3.5.

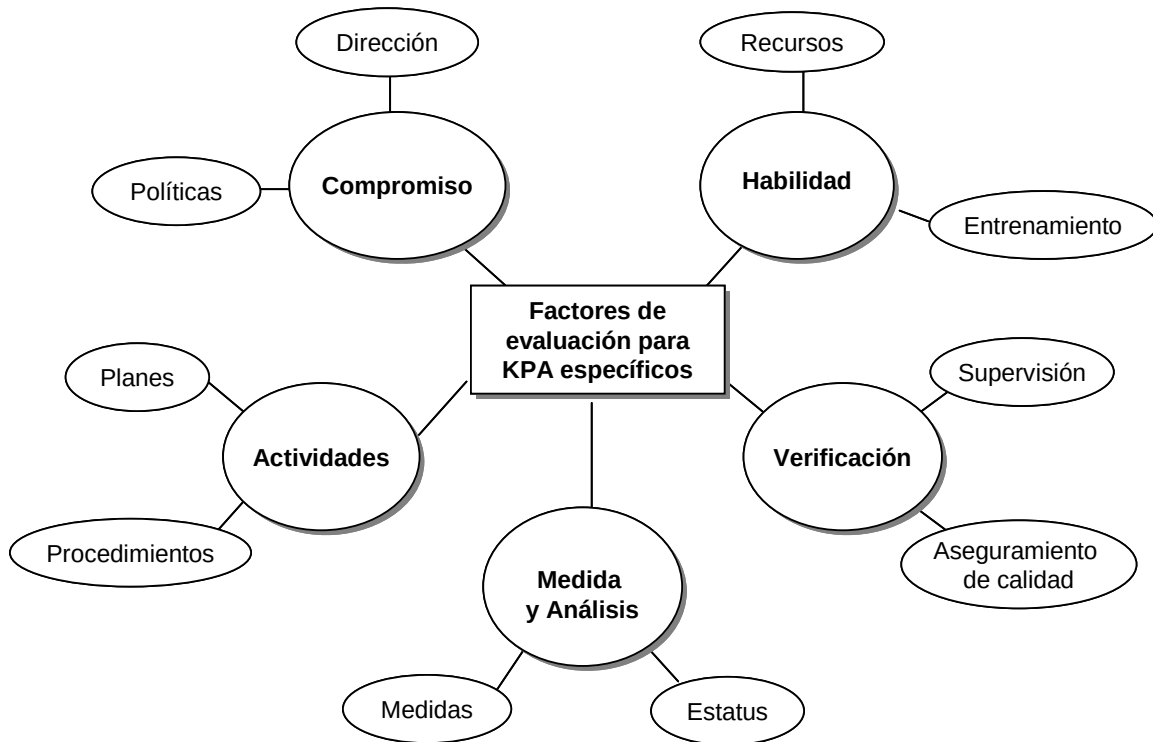


Figura 3.5: Parámetros Determinados para Evaluar el Grado de Implementación para un KPA Específico

Los KPAs para los niveles del 1 al 5 del CMM se presentan a continuación.

Nivel 1: Iniciado

En un nivel iniciado, típicamente una organización no cuenta con un ambiente estable para desarrollar y mantener software. La capacidad del proceso de software de las organizaciones del nivel 1, es impredecible porque el proceso de software cambia constantemente o es modificado según el trabajo progresa. Los cronogramas, presupuestos, funcionalidad y calidad del producto son generalmente impredecibles. El desempeño depende de los individuos, sus habilidades y conocimientos innatos.

Nivel 2: Repetible

Éste es el nivel repetible de CMM. A este nivel, una organización establecerá los procesos principales de la administración de proyecto requeridos para mantener el control sobre costo, cronograma y funcionalidad.

El objetivo a alcanzar en este nivel, es la estandarización de una efectiva administración de procesos para proyectos de software, es decir, un planeamiento y seguimiento de proyectos estables, que le permite a la organización repetir prácticas exitosas desarrolladas en proyectos anteriores. Los KPAs para este nivel se ilustran en la Figura 3.6.

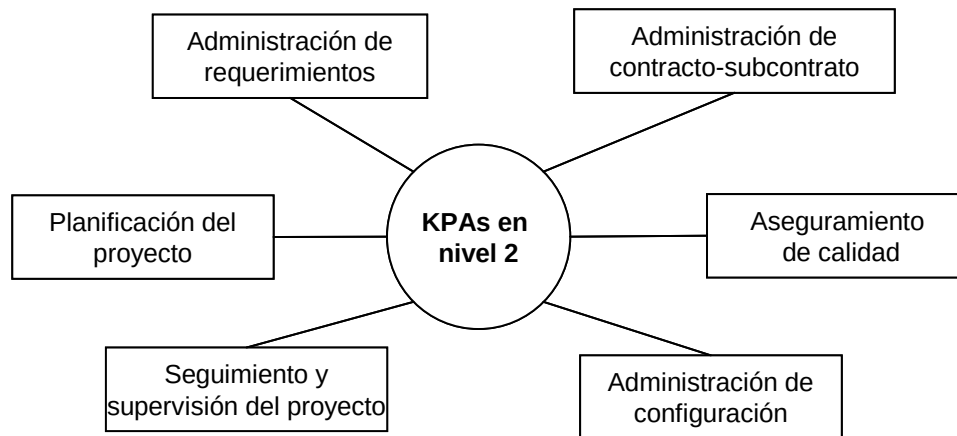


Figura 3.6: KPAs para el Nivel 2 del CMM

Nivel 3: Definido

Este nivel se llama el nivel definido del CMM. En este nivel, una organización documentará, estandarizará e integrará el proceso del software para las operaciones de administración e ingeniería en todos los departamentos. Esto significa simplemente que todos los proyectos del software que se estén desarrollando se referirán a una versión documentada y aprobada del proceso en la organización como un estándar para proceso de software. A este nivel, se puede definir un estándar porque tanto las actividades de administración como las de ingeniería de software, son estables y repetibles. Del mismo modo, están basadas en un amplio y común entendimiento organizacional de las actividades, roles y responsabilidades definidas en un proceso de software.

Los procesos establecidos al nivel 3 son usados por el equipo de proyecto como ayuda para un desempeño más efectivo de sus actividades. Este nivel también incluirá todos los procesos especificados para el Nivel 2. Los KPAs para este nivel se ilustran en la Figura 3.7.



Figura 3.7: KPAs para el Nivel 3 del CMM

Nivel 4: Administrado

Este nivel se refiere a cómo el nivel del CMM es administrado. A este nivel, una organización requiere comparar los datos detallados del proceso de software y de calidad del producto, para esto se fijan metas cuantitativas de la calidad de ambos, producto y proceso. Estos elementos se entienden cuantitativamente y después se controlan.

La capacidad de proceso del software de las organizaciones del nivel 4 se puede resumir como fiable, porque el proceso se mide y funciona dentro de límites medibles. Este nivel de capacidad de proceso permite que una organización prediga tendencias del proceso y la calidad del producto dentro de los límites. Cuando se exceden estos límites, la acción se toma para corregir la situación. Los productos de software están de calidad fiable alta. Este nivel también incluirá todas las especificaciones del Nivel 3.

Los KPAs para este nivel se ilustran en la Figura 3.8.

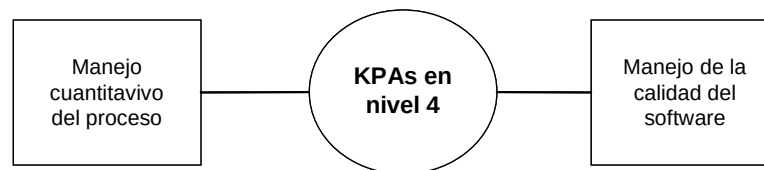


Figura 3.8: KPAs para el Nivel 4 del CMM

Nivel 5: Optimizado

Este nivel, llamado el nivel optimizado, es el nivel más alto que puede alcanzar una organización de software. Los equipos de proyecto del software en organizaciones del nivel 5 analizan defectos para determinar sus causas. Los procesos del software se evalúan para evitar que los tipos de defectos conocidos se repitan y las lecciones aprendidas se reparten entre otros proyectos. Este nivel está caracterizado por la mejora continua del proceso, la cual efectúa retroalimentación cuantitativa del proceso y a través de la prueba de la implementación de ideas y de tecnologías innovadoras. Este nivel también incluirá todos los procesos especificados para el Nivel 4. Los KPAs para este nivel se ilustran en la Figura 3.9.

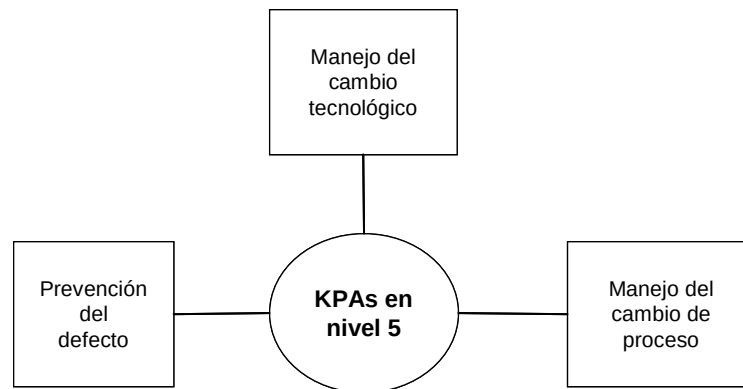


Figura 3.9: KPAs para el Nivel 5 del CMM

Resumen

Ahora que ha completado esta unidad, usted será capaz de:

- Explicar el concepto de la calidad.
- Describir el control de calidad y el aseguramiento de la calidad en el contexto del software.
- Discutir la seguridad y confiabilidad del software.
- Explicar el Modelo de Madurez - SEI.

Unidad 3: Examen de Autoevaluación

- 1) Las dimensiones de la calidad del software son _____.
 - a) Operación del producto, el cual involucra exactitud, confiabilidad, eficiencia, integridad y la utilidad.
 - b) Transición del producto, el cual involucra portabilidad, reutilización e interoperabilidad.
 - c) Revisión del producto, el cual involucra capacidad de mantenimiento, flexibilidad y prueba.
 - d) Integración del producto, el cual involucra la capacidad de integrar.

- 2) La calidad del software _____.
 - a) No puede medirse.
 - b) Puede medirse.
 - c) No definido claramente.
 - d) Definido, pero intangible.

- 3) ¿Cuáles de las siguientes son actividades del SQA?
 - a) Revisión del software.
 - b) Actividades de verificación.
 - c) Reporte de no conformidad.
 - d) Desarrollo de la descripción del proceso de software.

- 4) La seguridad del software ayuda a identificar
 - a) Los eventos indeseables.
 - b) Zonas peligrosas que pueden causar fallas del sistema.
 - c) Posibilidad de falla del sistema.
 - d) Las fallas lógicas del sistema.

- 5) ¿Cuál de los siguientes es una definición apropiada para el aseguramiento de calidad del software?
 - a) Inspecciones, revisiones y pruebas, conducidas durante el proceso del software para cerciorarse de que los productos cumplen los estándares establecidos.
 - b) Una conformidad a las necesidades funcionales y de rendimiento que se han indicado explícitamente, estándares de desarrollo explícitamente documentados y también características implícitas esperadas de todo el software que se ha desarrollado profesionalmente.

- c) Un proceso que utiliza control de calidad estadístico para asegurar calidad del software.
 - d) Un proceso estandarizado que es el SEI-CMM.
- 6) ¿Cuáles de los siguientes se asocian directamente a SQA?
- a) ISO
 - b) SEI
 - c) Grupo de la SQA
 - d) Ingenieros de software
- 7) ¿Cuáles de las siguientes afirmaciones son ciertas en relación con el plan de SQA?
- a) Puede ser descrito como plantilla para implementar las diferentes actividades relacionadas con el SQA.
 - b) El plan desarrollado aplica a todos los proyectos de la organización.
 - c) Es desarrollado por la firma del grupo de SQA.
 - d) Es desarrollado por los analistas de requerimientos.
- 8) El Modelo de Capacidad de Madurez:
- a) Está compuesto por 5 niveles de madurez.
 - b) Es un modelo para desarrollar planes de SQA.
 - c) Es marco de trabajo que describe las prácticas claves para el planeamiento, ingeniería, desarrollo y mantenimiento del software.
 - d) Todos los 5 niveles de madurez están compuestos de diferentes KPAs.
- 9) La madurez de un proceso de software, es el grado al cual el proceso es explícitamente:
- a) Controlable.
 - b) Medible.
 - c) Eficaz.
 - d) Definible.
- 10) ¿Cuáles de las siguientes son KPAs para el Nivel 3?
- a) Prevención del defecto.
 - b) Definición del proceso.
 - c) Revisiones detalladas.
 - d) Planificación del proyecto.

Respuestas de la Unidad 3: Examen de Autoevaluación

- 1) a, b y c
- 2) b
- 3) a, b, y c
- 4) b
- 5) b
- 6) c y d
- 7) a y c
- 8) a y c
- 9) a, b, y c
- 10) b y c

Unidad 4: Introducción a RUP

Objetivos de Aprendizaje

Al finalizar esta unidad, usted será capaz de:

- Definir Rational Unified Process (Proceso Racional Unificado).
- Describir las ventajas del enfoque RUP.
- Explicar los elementos básicos de RUP.
- Explicar las características de la metodología RUP.
- Describir las fases del ciclo de vida de RUP.

1. Introducción

Al principio de este curso se describió un conjunto de modelos para procesos de software, y se observó que no existe dentro de los modelos mencionados, uno que se adapte a la perfección a todos los tipos de procesos de desarrollo, ya que cada desarrollo tiene características diferentes que dependen de factores como tiempo, área de aplicabilidad, complejidad, recursos, entre otros. Un modelo que es aplicado para un desarrollo en particular y funciona efectivamente, no necesariamente es aplicable a otros procesos de software con la misma efectividad. En este sentido, el enfoque que RUP proporciona, una metodología o modelo para el desarrollo de software el cual es configurable y se basa en los estándares de la organización. Esto facilita en primer plano, la flexibilidad necesaria dentro del mundo de desarrollo de software.

En un proceso de desarrollo de software existen muchos factores que cuidar al mismo tiempo, cada uno de los cuales afecta de manera diferente a cada una de las partes involucradas en el mismo. El problema de un ingeniero de sistemas en medio de este panorama, es diseñar e implementar un sistema que cumpla con las necesidades de las personas involucradas (stakeholders) en el proyecto las cuales incluyen:

- Los usuarios, quienes se ven afectados por el funcionamiento y el desempeño.
- Los dueños, quienes son los afectados por los costos de publicación del producto y propiedad intelectual.
- Los inversionistas, quienes son los afectados por la competencia.

2. Definiendo Rational Unified Process (RUP)

La interrogante ¿Qué es RUP?, puede tener diferentes respuestas dependiendo de quién la formule y el contexto de la misma. Lo que genera confusiones sobre RUP, es el hecho de que este se relaciona con los siguientes puntos:

- RUP es una metodología de desarrollo de software que es iterativa e incremental, centrada en la arquitectura y manejada por casos de usos.
- RUP es un proceso de ingeniería del software bien definido y estructurado. Define claramente quién es responsable de qué, cómo deben hacerse las cosas y cuándo deben hacerse las mismas. También provee una estructura bien definida para el ciclo de vida de un proyecto, marcando claramente los hitos (puntos de decisión esenciales).
- RUP también es un marco de trabajo genérico que puede configurarse para una gran variedad de desarrollos.

Resumiendo, podría decirse que RUP es una metodología para procesos de desarrollo de software dentro del contexto de la ingeniería de software, donde existen tres elementos centrales que la definen:

- Un conjunto base de prácticas y conceptos para el éxito del proceso de desarrollo de software. Este conjunto es la base sobre la cual RUP ha sido desarrollado.
- Un modelo de procesos de desarrollo de software y una librería de contenido asociada, por medio de los cuales se define la base del proceso de software del marco de trabajo RUP. Sobre esta base, la empresa creará sus propios procesos configurables.
- Un lenguaje de definición de procesos elementales. Esto es una descripción del modelo subyacente, muchas veces llamado meta-modelo, el cual ofrece un lenguaje de definición de procesos para describir el proceso de software.

No existe un proceso de software universal. Las características de cada proyecto (equipo de desarrollo, recursos, tiempo, etc.) exigen que el proceso de desarrollo de software sea configurable. A continuación se explicará qué ofrece RUP como metodología para lidiar con la variedad de proyectos que se pueden desarrollar.

2.1 Ventajas de RUP

RUP proporciona al desarrollador de software, un ambiente para el proceso de desarrollo configurable basado en estándares, que permite:

- Un proceso de software hecho a la medida para ser publicado y hacerlo accesible para todo el equipo del proyecto.
- Un proceso de software configurable, para satisfacer necesidades específicas de un proyecto.
- Una definición común del proceso que puede ser compartida por todo el equipo de desarrollo, ayudando a asegurar una comunicación clara y sin ambigüedades entre los miembros del equipo.

Los desarrolladores no son los únicos que se ven beneficiados de este enfoque, éste brinda a los demás participantes del proyecto ventajas como:

- Ofrece a cada usuario, un filtrado personalizado de la definición del proceso publicado, acorde con su rol dentro del proyecto.
- Facilitar al inversionista, el entendimiento de qué esperar respecto al esfuerzo de desarrollo, proporcionándole un glosario de términos y una enciclopedia de información para ayudarlo a comunicar sus necesidades de manera efectiva dentro del equipo de desarrollo.
- Suministrarle al gerente o líder de proyecto, un proceso por medio del cual puede comunicarse efectivamente con su personal, administrar la planificación y controlar el trabajo de estos respectivamente.
- Entregarle al ingeniero de procesos, una buena base de la arquitectura y abundante material a partir del cual puede construir sus propias definiciones del proyecto, permitiéndole configurar y extender esas bases como lo desee.

Cuando se comienza a trabajar con RUP, se tiene a veces la concepción de que las diferentes fases de éste son simplemente el cambio de nombre de las fases clásicas de

un proceso de la metodología en cascada. Si se examinan las características de la metodología RUP se podrá ver que esta idea es errada.

3. Características de RUP

La mayoría de los equipos de proyecto dentro de las empresas aún utilizan el modelo en cascada para desarrollar los proyectos, completando cada fase en una estricta secuencia; por el contrario RUP usa un enfoque iterativo (mini-proyectos) que es una secuencia de pasos incrementales (versiones).

Las características esenciales de la metodología RUP son tres: dirigida por casos de uso, iterativa e incremental y centrada en la arquitectura.

3.1 Dirigido por Casos de Uso

Los casos de uso describen cómo los usuarios interactúan con el sistema a desarrollar. Donde un usuario, puede ser una persona u otro sistema que utilice las funcionalidades del sistema a desarrollar. Un caso de uso representa una funcionalidad puntual del sistema. Por ejemplo, una funcionalidad puntual, en un sistema para cajeros automáticos, es la de “retiro”.

Los casos de uso definidos para un sistema, son la base para el desarrollo del sistema entero, ya que son una forma de capturar los requerimientos funcionales del sistema. Los desarrolladores, generan una serie de modelos de diseño e implementación, que llevan a cabo las funcionalidades plasmadas en los casos de uso. Para esto, ellos hacen uso de un modelo llamado modelo de casos de uso, que agrupa un conjunto de casos de uso relacionados.

Dirigido por casos de uso, quiere decir que el proceso de desarrollo sigue una secuencia de actividades que parten de los casos de uso (funcionalidades) identificados.

3.2 Iterativo e Incremental

Un desarrollo iterativo tiene un ciclo de vida que consiste de varias iteraciones. Según el enfoque de RUP, en cada una de las fases del ciclo de vida del proceso, se puede tener un número variable de iteraciones, el cual depende de los objetivos que se desee alcanzar en la fase y de las iteraciones previas. A su vez, cada iteración reproduce un ciclo de vida en cascada a menor escala. La Figura 4.1 ilustra lo que implica el carácter iterativo de RUP.

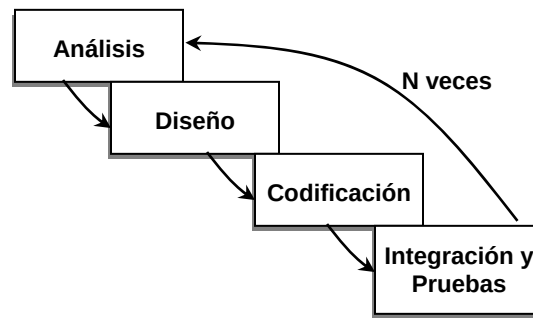


Figura 4.1: Carácter Iterativo de RUP

RUP se basa en la evolución de prototipos ejecutables o versiones del producto final que se muestran a los usuarios e inversionistas del proyecto. Cada paso por el ciclo de vida produce una versión del producto que incrementalmente se va refinando en las iteraciones de las diferentes fases. Si llegado el final del ciclo de vida de RUP, el producto no cumple con los objetivos planteados, se puede realizar un ciclo más para refinar, corregir y agregar funcionalidades que lleven al software a cumplir con las expectativas o cancelar el proyecto en base a los resultados obtenidos. Lo que indica que con un enfoque iterativo e incremental, se tiene un mejor manejo de los riesgos y un refinamiento más efectivo del producto final.

3.3 Centrado en la Arquitectura

En RUP el proceso se basa en diseñar tempranamente una arquitectura base ejecutable. La arquitectura de un sistema, es la organización o estructura de sus partes (componentes) más relevantes dejando de lado los detalles, incluye los aspectos estáticos y dinámicos del sistema. Mientras que una arquitectura ejecutable es una implementación parcial del sistema, construida para demostrar cuáles serán las funcionalidades claves soportadas por el sistema, y lo más importante, esta arquitectura exhibirá las propiedades correctas en términos de desempeño, rendimiento, capacidad, confiabilidad, escalabilidad entre otras.

RUP establece refinamientos sucesivos de la arquitectura ejecutable, construida como un prototipo evolutivo. La arquitectura debe ser: flexible, fácil de modificar, fácil de comprender y debe promover la reutilización de componentes. RUP apoya el desarrollo basado en componentes, tanto nuevos como preexistentes.

A lo largo de cada ciclo e iteración de RUP, interactúan un conjunto de elementos, cada uno con un significado especial dentro del proceso; a continuación se describirán esos elementos.

4. Elementos Básicos de RUP

Con RUP, un proceso de desarrollo es representado usando un conjunto de elementos de modelado, tales como: roles, actividades, artefactos y flujos de trabajo (workflows),

entre otros. Un rol expresa quién (individuo o grupo) hace un trabajo, una actividad describe cómo es hecho el trabajo y un artefacto captura el trabajo realizado. En RUP se encuentran 4 elementos básicos: los roles (el quién), las actividades (el cómo), los artefactos (el qué) y los flujos de trabajo (el cuándo).

La estructura estática de RUP maneja cómo los elementos del proceso (actividades, disciplinas, artefactos y roles) están lógicamente agrupados dentro del corazón de las disciplinas del proceso. En la Figura 4.2, se muestran los elementos básicos de la metodología RUP y cómo éstos se relacionan entre sí.

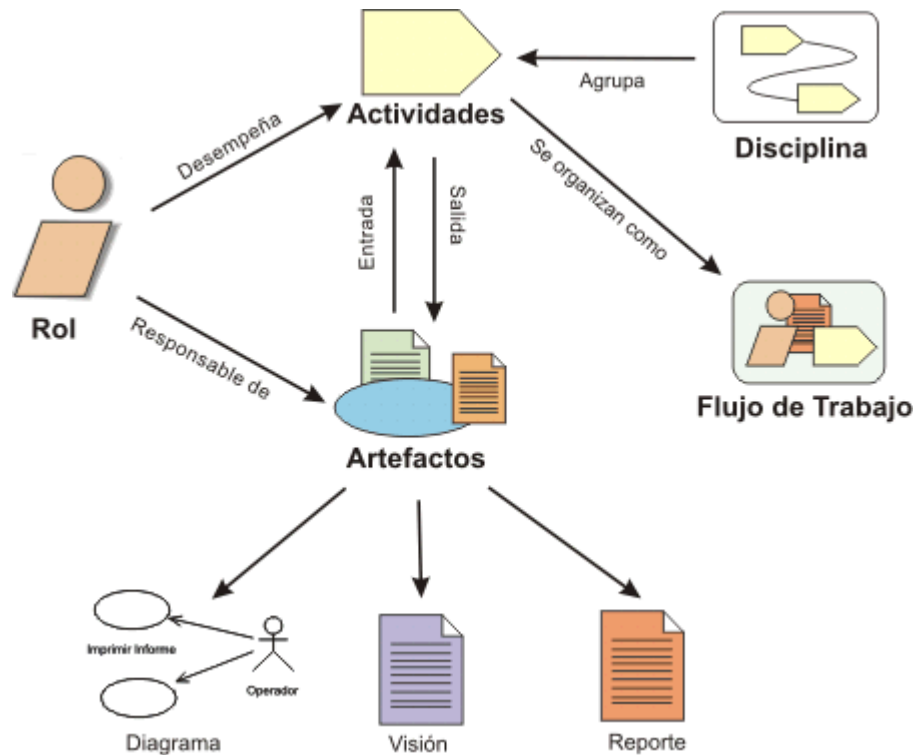


Figura 4.2: Elementos Básicos de RUP.

4.1 Roles

Un rol es una definición abstracta del conjunto de responsabilidades, para las actividades a ser desempeñadas y artefactos a ser producidos dentro del proyecto por un individuo o grupo.

Es natural pensar en un rol como un individuo dentro del proyecto o como un cargo o designación fija, pero en RUP, los roles simplemente definen cómo los individuos deberían trabajar, especificando las competencias y responsabilidades que éstos desempeñan. Una persona usualmente desempeña uno o más roles y diferentes personas pueden desempeñar el mismo rol.

Un proyecto de RUP, es una colaboración entre todo el grupo de roles. Todos los roles están involucrados a través de la mayoría del ciclo (excepto quizás en el inicio).

La asignación de un individuo a un rol, es realizada por el gerente del proyecto cuando planifica y asigna el personal del proyecto, lo que permite a diferentes individuos actuar en diferentes roles y que el rol sea desempeñado por diferentes individuos. RUP provee un conjunto de roles predefinidos, agrupados en subconjuntos según las habilidades y responsabilidades comunes. Estos subconjuntos son:

Analistas: Agrupa los roles que están principalmente involucrados en la captura de requerimientos (investigan). Por ejemplo analista de procesos de negocios.

Desarrolladores: Agrupa a los roles principalmente involucrados en el diseño e implementación del software. Algunos de los roles de esta categoría son el arquitecto de software, el diseñador, el integrador, etc.

Gerentes: Agrupa los roles principalmente involucrados en la dirección y configuración de los procesos de ingeniería. Por ejemplo, gerente de proyecto, gerente de publicación (deployment).

Producción y Soporte: Agrupa los roles para dar soporte al proceso de desarrollo del software. Ejemplos: escritor técnico, especialista de herramientas, administrador de sistema, etc.

Probadores: Agrupa los roles que dirigen las pruebas para habilidades específicas a medir. Ejemplos: probador, analista de pruebas, etc.

Roles Adicionales: Agrupa los roles que no se ajustan en ninguno de los grupos anteriores.

La explicación detallada de cada uno de los roles dentro de las categorías anteriores está fuera del alcance de este curso.

4.2 Actividades

Una actividad es una unidad de trabajo que se asigna a un rol, la cual se requiere sea ejecutada por el individuo asociado a ese rol. Cada actividad es asignada a un rol específico. Una actividad generalmente toma unas pocas horas o días en ser completada, usualmente involucra una persona y afecta a uno o a un número pequeño de artefactos. Una actividad debería ser utilizable como un elemento de planificación y progreso; si ésta es muy pequeña será abandonada y si es muy grande el progreso tendría que ser expresado como parte de una actividad. Las actividades pueden ser repetidas varias veces sobre un mismo artefacto, especialmente cuando se va desde una iteración a otra, refinando y expandiendo el sistema por el mismo rol, pero no necesariamente por el mismo individuo. La actividad tiene un claro propósito, usualmente expresado en términos de crear o actualizar algún artefacto, tal como un modelo, un componente o un plan.

4.2.1 Pasos

Las actividades están fraccionadas en pasos y estos agrupados en tres categorías:

- Pasos de Análisis: Son aquellos que se refieren a cuando el individuo que desempeña el rol comprende la naturaleza de la tarea, recolectando y examinando los artefactos de entrada y formulando resultados o solución.
- Pasos de Ejecución: El rol crea o actualiza algún artefacto.
- Pasos de Revisión: Donde el rol verifica los resultados contra algún criterio.

No todos los pasos son necesariamente ejecutados cada vez que una actividad es invocada, así los pasos pueden ser expresados en forma de flujos alternativos.

En la Figura 4.3 se muestra el rol especificador de requerimientos junto a las actividades asociadas a éste. Se puede observar los pasos en los que está fraccionada la actividad de detallar los requerimientos del software.

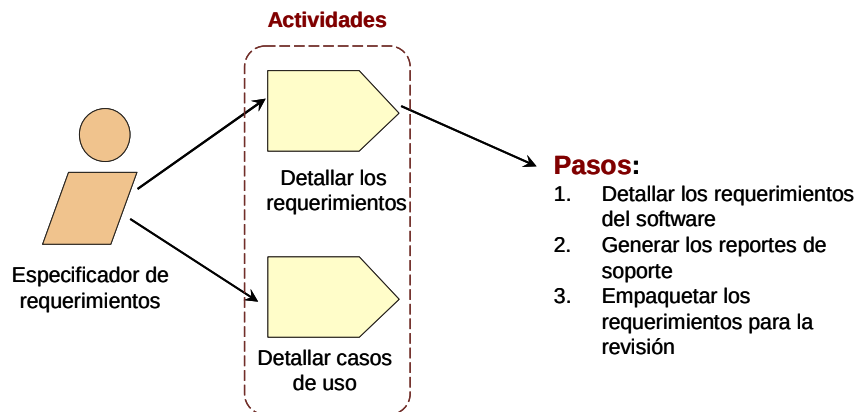


Figura 4.3: Relación entre actividades y pasos.

4.3 Artefactos

Un artefacto es una pieza de información que es producida o utilizada por procesos. Los artefactos son los elementos tangibles de un proyecto, elementos que el proyecto produce o usa mientras se trabaja en busca del producto final. Éstos, pueden tomar varias formas y formatos, como por ejemplo:

- Un documento, tal como la visión o la lista de riesgos.
- Un modelo, por ejemplo un diagrama de casos de uso o el modelo de diseño.
- Un elemento dentro de un modelo, tal como una clase, un caso de uso o un subsistema.
- Ejecutables, por ejemplo el ejecutable del prototipo.

- Código fuente.

Las actividades tienen artefactos como entrada y salida. Los roles usan artefactos para ejecutar actividades y producen artefactos durante la ejecución de sus actividades. Los artefactos son la responsabilidad sencilla de un rol, creando responsabilidades fáciles de identificar y entender, promoviendo la idea de que cada pieza de información producida en un proceso de desarrollo requiere un conjunto apropiado de habilidades. Aunque un rol puede ser el propietario de un artefacto, otros roles pueden hacer uso de éste, incluso podrían actualizar el artefacto si el rol que va a hacerlo, tiene permiso para hacerlo.

En RUP se encuentran conjuntos de artefactos que agrupan artefactos relacionados con el modelo de negocio, los requerimientos, el análisis y diseño, la implementación, las pruebas, la configuración y administración de cambios, el ambiente de desarrollo, entre otros. La lista detallada de cada uno de los posibles artefactos dentro de un proyecto RUP es extensa y su discusión detallada está fuera del alcance de este curso.

4.4 Disciplinas

Una disciplina es una colección de actividades relacionadas a un área de interés principal, dentro de todo el proyecto; por ejemplo, la administración de los requerimientos. La agrupación de actividades en disciplinas se hace principalmente para añadir un entendimiento del proyecto desde la perspectiva del enfoque tradicional en cascada. Por ejemplo, es más común realizar ciertas actividades de la administración de requerimientos en coordinación con las actividades de análisis y diseño. Separando estas actividades en disciplinas, se hace más fácil comprender las actividades pero más difícil fijarlas en el cronograma del proyecto.

El flujo de trabajo de una disciplina es una secuencia semi-ordenada de actividades, la cual es ejecutada para alcanzar un resultado particular, por ejemplo, para la disciplina asociada a los requerimientos que lleva como objetivo establecer un acuerdo con los clientes, sobre qué debe hacer el sistema. La colección de actividades llevadas a cabo debería ser:

- Capturar un vocabulario común, un glosario.
- Desarrollar un plan para la administración de los requerimientos.
- Desarrollar la visión.
- Determinar los requerimientos de los usuarios e inversionistas.
- Encontrar qué o quiénes interactúan con el sistema (actores) y describir esas interacciones (casos de uso).
- Administrar las dependencias.
- Estructurar el modelo de casos de uso.

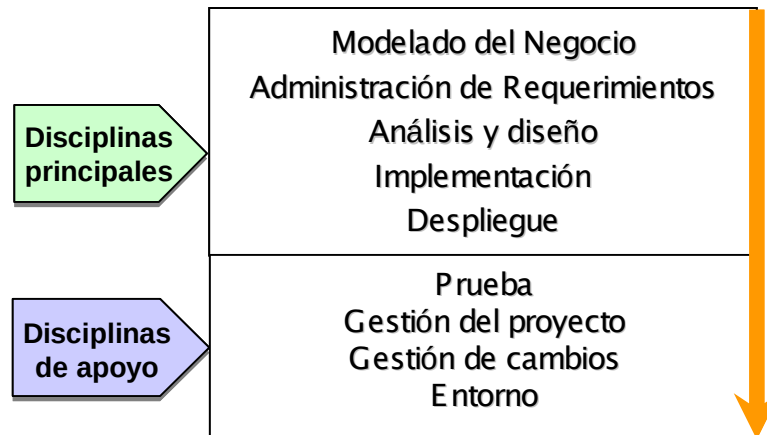


Figura 4.4: Disciplinas de RUP

Por cada disciplina se presenta un conjunto de actividades. Ese conjunto muestra todas las actividades a lo largo de la disciplina, junto a los roles que ejecutan dichas actividades. La discusión detallada de cada una de las actividades dentro cada una de las disciplinas está fuera del alcance de este curso.

El listado de roles, actividades y artefactos no constituyen un proceso de software. Se necesita una manera de describir significativamente, la secuencia de actividades dentro de un proyecto que producen algún resultado de valor, así como también es necesario que la descripción de esas actividades, muestre cómo interactúan entre sí los roles, actividades y artefactos.

4.5 Flujos de Trabajo (Workflows)

Un flujo de trabajo es una secuencia de actividades que producen un resultado de valor observable. Una de las grandes dificultades de describir un proceso de software, es que hay muchas formas de organizar el conjunto de actividades dentro de un flujo de trabajo. No siempre es posible representar flujos de trabajo. En RUP se utilizan flujos de trabajo detallados y disciplinas para organizar el proceso de software.

4.5.1 Flujos de Trabajo Detallados

Son flujos de trabajo dentro de una disciplina. Muestran el grupo de actividades que frecuentemente se ejecutan “juntas”. Estos diagramas muestran los roles involucrados, artefactos de entrada y salida, así como las actividades ejecutadas. Los diagramas de flujo de trabajo detallado están allí por las siguientes razones:

Típicamente, en las reuniones de un equipo de proyecto se trabaja en paralelo algunas actividades, mientras se hace esto, se observa más de un artefacto. Un flujo de trabajo

detallado, da una visión más clara de ese trabajo en paralelo. Esto significa que hay varios diagramas de flujo de trabajo detallado para una disciplina.

Es complicado el mostrar los artefactos de entrada y salida para todas las actividades de una disciplina en un diagrama. El diagrama de flujo de trabajo detallado, permite mostrar las actividades y los artefactos juntos, para una parte del flujo de trabajo.

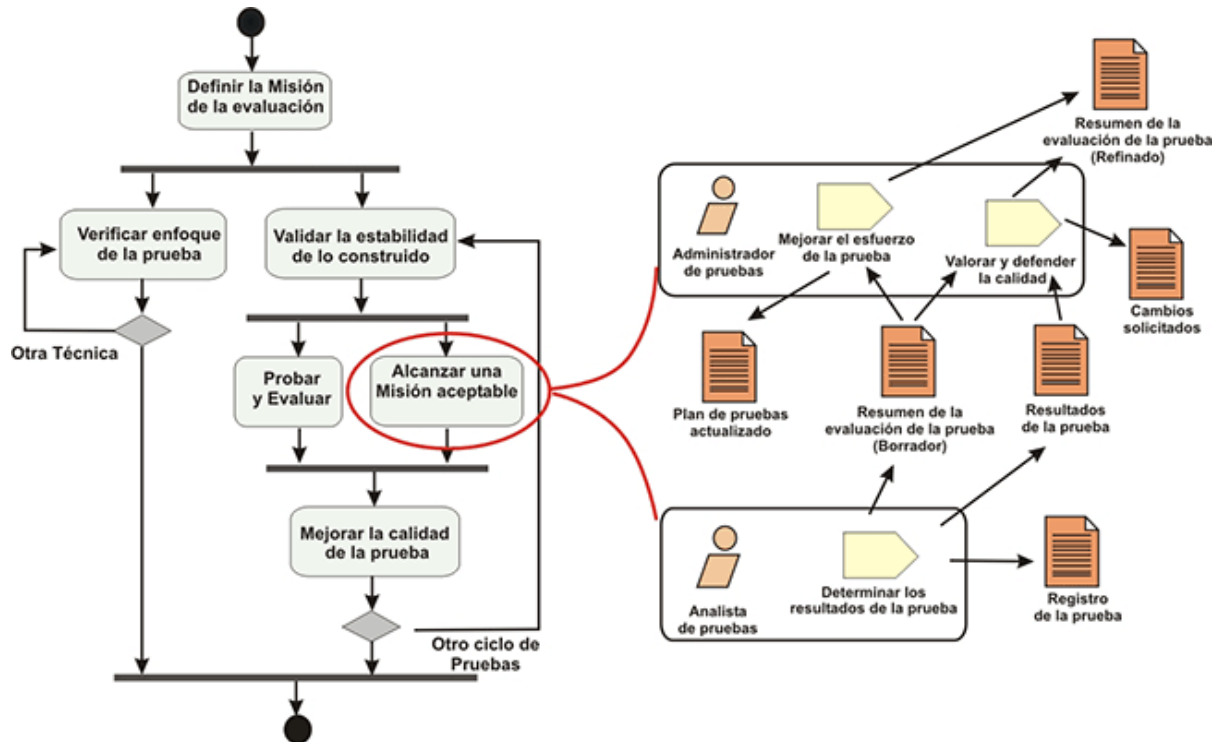


Figura 4.5: Flujo de Trabajo y Flujo de Trabajo Detallado

La Figura 4.5 muestra a la izquierda el flujo de trabajo para la prueba del producto, y a la derecha, el flujo de trabajo detallado de las actividades necesarias para alcanzar una misión aceptable en la prueba del producto, junto a los roles y artefactos relacionados. El flujo de trabajo muestra, la secuencia de actividades para realizar una disciplina pero agrupando las actividades en actividades “más genéricas”, que no son más que un grupo de actividades puntuales que se ejecutan juntas, lo cual simplifica la vista del flujo de trabajo. Mientras que el flujo de trabajo detallado muestra esas actividades “más genéricas” en forma de actividades puntuales, que a su vez podrían desglosar en pasos.

A continuación se discutirá cómo esta estructurado RUP.

5. Estructura de RUP

Como se recordará, un proceso de ingeniería de software es un conjunto parcialmente ordenado de tareas, pasos y operaciones que se llevan a cabo para alcanzar un objetivo: “Producir un producto de software de alta calidad”. Para alcanzar dicho objetivo

en RUP, un proceso de software está organizado o estructurado en dos dimensiones, una dinámica y una estática.

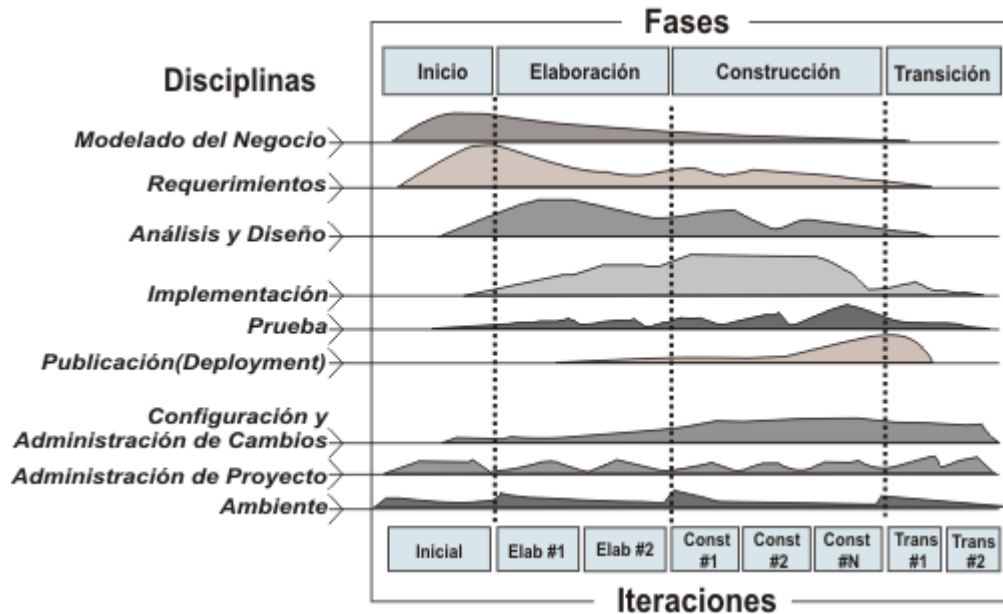


Figura 4.6: Estructura de RUP.

Estructura Dinámica: La dimensión horizontal representa la estructura dinámica o el tiempo del proceso. Muestra cómo el proceso es expresado en términos de ciclos, fases, iteraciones e hitos que evolucionan sobre el ciclo de vida de un proyecto.

Estructura Estática: La dimensión vertical expresa la estructura estática del proyecto. Describe cómo los elementos del proceso, actividades, disciplinas, artefactos roles son lógicamente agrupados dentro de las disciplinas del proceso (o flujos de trabajo básicos).

Cada fase contiene una o más iteraciones, las cuales se enfocan en producir los entregables técnicos necesarios para conseguir los objetivos de la fase. Habrá tantas iteraciones como sean necesarias para cumplir a cabalidad los objetivos de la fase. El número de iteraciones a realizar por cada fase, depende de ciertos factores asociados al tipo desarrollo, a la fase y la iteración en sí. Por ejemplo, como se muestra en la Figura 4.6, se tienen dos iteraciones en la fase de elaboración Elab #1 y Elab #2, esto puede variar a más iteraciones o menos, según algunos criterios que se explicarán más adelante.

Si los objetivos de la fase no pueden ser alcanzados con las iteraciones planeadas, otra iteración deberá ser agregada a la fase, lo cual retrasará el proyecto. Para evitar esto, se debe asegurar que cada iteración se enfoque en los objetivos que deben lograrse en esa fase. Cada fase a su vez, tiene un hito (punto de decisión) que proporciona un punto de transición bien definido de una fase a la siguiente fase. Cada hito está

enfocado en un entregable específico, por ejemplo los objetivos del software a desarrollar.

Cada fase e iteración involucra la ejecución, una o más de las disciplinas esenciales del desarrollo (requerimientos, análisis, diseño e implementación). Cada iteración sucesiva se construye sobre el resultado del trabajo de interacciones previas para propiciar la evolución y refinamiento del sistema hasta que el producto final este terminado. Es así como por ejemplo en la Figura 4.6 se observa que en la fase de inicio hay una cantidad de trabajo asociado a las disciplinas, de modelado del negocio y requerimientos, pero en la fase de elaboración esa carga de trabajo disminuye, ya que se refinará el resultado obtenido de aplicar esas disciplinas en la fase de inicio.

En las primeras iteraciones se hará énfasis en los requerimientos, análisis y el diseño, mientras que en las últimas iteraciones se hará énfasis en la implementación y prueba. Cada fase tiene un conjunto de objetivos y productos bien definidos, como implementaciones parciales del trabajo final.

6. Fases del Ciclo de Vida de RUP

Un desarrollo RUP se da a través de cuatro fases: Inicio, Elaboración, Construcción y Transición. Este ciclo de fases finaliza con una versión completa del producto de software. ¿Qué se hace durante cada una de las diferentes fases de RUP?, ¿Qué se trata de alcanzar en cada fase?, ¿Qué artefactos son producidos?, ¿Qué actividades son ejecutadas? En la siguiente sección se explicará el sentido dinámico de un proyecto RUP. Antes, es necesario aclarar algunas cosas sobre los hitos, flujos de trabajo y artefactos que ayudan a entender el dinamismo de este enfoque.

6.1 Importancia de los Hitos

El propósito de las fases de RUP no es clasificar las actividades por tipo: análisis, codificación, etc. El propósito de una fase es ejecutar cualquier actividad requerida, lo suficiente para resolver los objetivos de esa fase. Es decir, hasta que los hitos que la delimitan se hayan alcanzado.

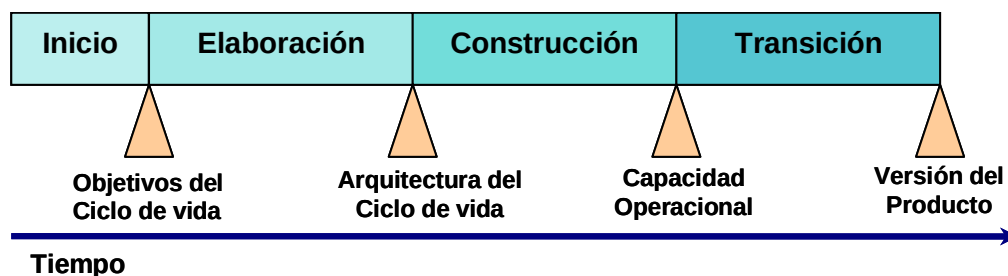


Figura 4.7: Hitos de las Diferentes Fases de RUP.

6.2 Flujos de Trabajo y Artefactos Dinámicos.

En RUP no hay un flujo de trabajo fijo o un sistema predefinido de las actividades que se deben ejecutar en cada fase. Éste ofrece una gama de posibilidades, pero el contexto del proyecto determinará lo que se utilizará realmente durante una fase específica para alcanzar sus objetivos. De un ciclo de desarrollo a otro, como de un proyecto a otro, el contexto será diferente, los riesgos serán diferentes como también los artefactos de inicio, el tamaño del proyecto, su duración y el número de partes implicadas. Todos jugarán un papel dictaminando qué debe ser ejecutado realmente y qué artefactos deben ser producidos o ser revisados.

El peor escenario posible se presenta cuando el equipo de trabajo trata de ejecutar todo RUP, desarrollando ciegamente todos los posibles artefactos y ejecutando todas las actividades planteadas por el enfoque RUP, olvidándose de adaptar RUP para satisfacer el contexto específico. De esta forma, el equipo ejecuta un proyecto con una alta carga de trabajo, se debe racionar RUP para que esté acorde con el proyecto, evitando así ejecutar trabajo innecesario.

Por otro lado, mientras que el proyecto es desarrollado y se entiende más sobre los objetivos, mientras se descubren dificultades y se introducen cambios externos, los artefactos tienen que ser revisados, actualizados y corregidos. Por lo tanto, las actividades que se han ejecutado, tienen que ser ejecutadas nuevamente y los artefactos tienden a cambiar y no permanecer constantes de una fase a otra, de una iteración a otra.

Por ejemplo, los requisitos se construyen gradualmente en el inicio y la elaboración, además deben estar completos para el final de la fase de la elaboración. La arquitectura será diseñada o seleccionada gradualmente y debería ser bastante estable para el final de la fase de la elaboración. Pero estos artefactos no son intocables. Se puede tener que alterar la visión, modificar los requerimientos o cambiar el diseño arquitectónico en las últimas fases.

Éste enfoque, es absolutamente simple: asegurar que se tienen claros los objetivos para cada una de las cuatro fases e imaginarse lo que significan estos objetivos concretamente para su situación. Esto es lo que se busca alcanzar en la primera fase del ciclo de vida de RUP.

6.3 Fase de Inicio

Entender lo que se desea alcanzar en una fase, ayudará a aplicar el enfoque RUP a sus proyectos con más eficacia. Se seleccionará y realizarán solamente las actividades que contribuyan a alcanzar los objetivos de un proyecto particular. La meta más importante de la fase de inicio, es el alcanzar consenso entre todos los inversionistas y afectados por el desarrollo del proyecto, de los objetivos del ciclo de vida del proyecto.

6.3.1 Objetivos de la Fase

La fase de inicio es la primera de las cuatro fases del ciclo de vida de RUP. La idea de esta fase es realmente entender el alcance del proyecto y los objetivos, obtener suficiente información para confirmar qué se debería hacer para proceder con el proyecto o tal vez concluir qué no se debería hacer. La fase de inicio tiene cinco objetivos básicos, que son:

1. Entender qué se va construir

Determinar la visión, el alcance del sistema y los límites de éste, es decir, qué está dentro del sistema y qué está afuera. Establecer el alcance del proyecto de software y las condiciones que lo limitan, incluyendo una visión operacional, criterio de aceptación, además de qué está previsto hacer en el producto y qué no.

La visión debería estar completa y estable al final de la fase de inicio, pero se puede continuar refinando a través del proyecto. Es importante que la visión sea pública, compartida y constantemente revisada de modo que nadie pueda decir que no la entendía o no la conocía. Para asegurar un entendimiento común de lo que se quiere construir se necesita:

- Producir un documento de la visión.
- Proporcionar una buena descripción del alcance del sistema sin entrar mucho en detalles. Esta descripción requiere dos actividades básicas:
 - Identificar los usuarios típicos del sistema (actores).
 - Identificar y describir cómo cada actor interactuará con el sistema. Las descripciones de las interacciones típicas con el sistema son llamadas casos de uso.
- Detallar los actores y casos de uso claves.

2. Identificar la Funcionalidad Clave del Sistema

Este es el segundo objetivo de la fase de inicio y se debería trabajar en él mientras se identifican los casos de uso. Esto es importante para decidir cuáles casos de uso son los más esenciales o significativos a nivel de la arquitectura, para asegurar que se esté dedicando más tiempo a los casos de uso críticos.

3. Identificar por lo menos una arquitectura candidata

Puesto que la meta de la fase de inicio es determinar si tiene sentido continuar con el proyecto, es necesario cerciorarse de que haya por lo menos una arquitectura candidata que permitirá construir el sistema con una mínima cantidad de riesgos (o una cantidad manejable) y un costo razonable.

4. Comprender los costos, cronogramas y riesgos asociados con el proyecto

Entender lo que se va a construir es clave, pero determinar cómo construirlo y cuánto cuesta, es crucial. Para determinar si se debería continuar con un proyecto se necesita

comprender ¿Cuánto costará el proyecto? La mayoría de costos están relacionados a ¿Qué recursos se necesitarán? ¿Cuánto tiempo tomará terminar el proyecto? Combinando todo este conocimiento con el entendimiento de la funcionalidad requerida y el valor de la misma para los usuarios, se puede construir un caso de negocios para el proyecto. Durante la fase de elaboración se enfrentará la gran mayoría de los riesgos relacionados con la tecnología y la arquitectura que se identificaron durante la fase de inicio.

5. Decidir qué proceso seguir y qué herramientas usar

Es importante que el equipo de trabajo comparta un punto de vista común de cómo será construido el software; es decir, cuál será el proceso de software a seguir. Se debería racionalizar el proceso, para minimizar la carga de trabajo innecesario y asegurar que el proceso de software trate las necesidades específicas del proyecto. En proyectos pequeños, se pueden tomar decisiones precisas sobre cuál proceso de desarrollo de software seguir, pero proyectos grandes pueden necesitar más tiempo para considerar qué opción de proceso seleccionar.

La idea es plantear un proceso de software y una herramienta de desarrollo y trabajar en ella en la primera iteración. Desplegar el proceso y la herramienta en la segunda iteración, además de obtener una retroalimentación inmediata sobre qué trabaja y qué no lo hace. Basándose en la retroalimentación, se actualiza el proceso y el ambiente de desarrollo, se extiende a la siguiente iteración y se mantiene iterando hasta que se está satisfecho con el ambiente. Una vez que se ha seleccionado un proceso, se puede elegir qué herramientas utilizar.

6.3.2 Las Iteraciones y la Fase de Inicio

La mayoría de proyectos tienen una iteración en la fase de inicio, no obstante, algunos proyectos pueden necesitar más de una iteración para cumplir los objetivos. Entre las razones para tener varias iteraciones, se encuentran:

- El proyecto es muy grande y es difícil para el equipo de desarrollo captar el alcance del proyecto.
- El sistema no tiene precedentes y es difícil establecer claramente lo que debe hacer.
- Hay muchos inversionistas y las relaciones con éstos son complejas, por ejemplo, las negociaciones del contrato.

Para determinar cuánto dura y qué se quiere de una iteración, se establece un plan de iteración, el cual es el detalle de las tareas a realizar durante una iteración específica. Un plan de iteración se desarrolla en cuatro pasos:

1. Determinar qué se intenta o se quiere lograr en la iteración.
2. Definir un criterio de evaluación para determinar cuándo la iteración ha concluido.
3. Definir qué se necesita hacer y sobre qué artefactos (Actividades de la iteración).

4. Asignar los recursos (personas, herramientas, etc.) para ejecutar las actividades.

La Figura 4.8 muestra las actividades que deberían llevarse a cabo durante la fase de inicio para alcanzar los objetivos antes mencionados, además de algunos de los artefactos producto de esta fase.

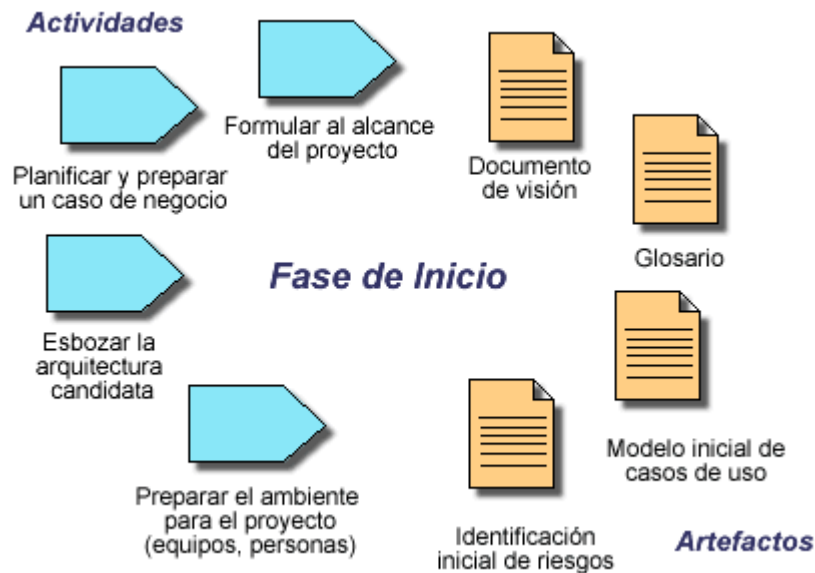


Figura 4.8: Actividades y Artefactos de la Fase de Inicio.

6.3.3 Hito: Objetivos del Ciclo de Vida

Al final de la fase de inicio está el primer y principal hito del proyecto, llamado hito de los objetivos del ciclo de vida. En este punto, se examinan los objetivos del ciclo de vida del proyecto. El proyecto debería abortarse o reconsiderarse si no logra alcanzar estos objetivos. Si el proyecto está condenado a fracasar, es mejor realizar esto al principio del ciclo de vida y no luego. El hito de los objetivos del ciclo de vida evalúa básicamente la viabilidad del proyecto.

Observe que el orden de los objetivos no indica prioridad, tampoco que se traten sin un orden específico. Por el contrario, se tratan todos los objetivos en paralelo. Una revisión del hito del objetivo del ciclo de vida incluye los siguientes criterios de evaluación:

- El consenso tanto de los inversionistas como del equipo de proyecto, en la definición del alcance, estimación inicial del costo y cronograma del proyecto.
- Un acuerdo en que ha sido capturado el conjunto correcto de requerimientos y que existe un entendimiento común de estos.
- Un acuerdo en que los riesgos iniciales, han sido identificados y existe una estrategia de mitigación de riesgos, para cada uno.

Ejemplo 4.1

Para comprender mejor los objetivos, actividades y artefactos de la fase de inicio, se plantea el siguiente escenario:

Se desea diseñar un sistema computarizado para automatizar el funcionamiento de la cadena de clubes de video "Global Videos", en conversaciones con los gerentes de la cadena, se ha obtenido la siguiente información preliminar sobre lo que se desea que haga el sistema:

- El sistema debe llevar el control de afiliación y desafiliación de clientes. Por políticas de empresa, no se permite la afiliación de personas menores de edad. Además, para alquilar y/o reservar una película es necesario estar afiliado.
- El sistema debe soportar funciones básicas como alquilar, reservar, devolver y comprar películas. Además, debe soportar la afiliación y reservación por medio del sitio Web de la empresa, así como también gestionar los pagos con tarjeta de crédito.
- Se desea que un cliente pueda devolver la película por medio de un buzón de devolución, el cual automáticamente realizará el registro (se descontará del depósito del cliente los días de mora, si es el caso). También debe permitir a los clientes consultar las películas disponibles por medio de terminales remotos instalados en la tienda.
- La duración de la reservación es de 24 horas, al cumplirse el lapso, la reservación debe ser cancelada automáticamente y la película estará disponible en la existencia. El alquiler de las películas tiene una duración de 2 días hábiles, los días adicionales son cobrados como días de mora.
- El sistema debe llevar el inventario de las películas de video. La empresa ya cuenta con un sistema contable automatizado y se requiere que el sistema a desarrollar se comunique con el mismo.

Asuma que Ud. forma parte del equipo de desarrollo de la empresa, encargada de este desarrollo. En principio si se asume que es un sistema sin precedentes, que adicionalmente, necesita integrarse con otro sistema de la empresa, con la banca en línea. Esto implica que necesitará más de una iteración para alcanzar los objetivos de la fase de inicio.

Según lo antes discutido, uno de los principales artefactos a obtener de fase de inicio es la visión, la cual ayuda a establecer los objetivos del ciclo de vida. Para el escenario propuesto, un borrador inicial de la visión después de una iteración podría verse como sigue:

Definición del Problema.

La empresa Global Videos, actualmente maneja de forma manual el registro de sus servicios de venta, reserva y alquiler de películas de video, así como también la transferencia de información a otros departamentos de la empresa. Lo que afecta a:

- Los clientes, quienes exigen un servicio más rápido, eficiente y flexible.

- La empresa, que se ve limitada en su proceso de expansión y capacidad de respuesta a las demandas de los clientes.
- El sistema de inventario, que no cuenta con información actualizada de la cantidad de películas disponibles para la venta o alquiler.
- Los empleados, a quienes se les dificulta monitorizar el estado de los servicios, como reserva y alquiler.

El impacto generado por el estado actual, se traduce en un lento y costoso proceso para el manejo de la información, junto a la insatisfacción de los clientes y empleados. Una solución sería mejorar el flujo de información de los servicios prestados, de manera de lograr un mayor grado de satisfacción, tanto en clientes como en empleados, posibilitando una mayor capacidad para captar más clientes.

Enfoque del Producto.

Se requiere un sistema que automatice los servicios de venta, alquiler y reserva de películas. Además de permitir la transferencia eficiente de información veraz al sistema de inventario. El sistema en cuestión debe proporcionar flexibilidad en los servicios de compra, reservación y alquiler de películas al poder ejecutar dichos servicios vía Web, así como también de una forma más automática dentro del local.

Otro producto de la fase de inicio es la identificación de las funcionalidades del sistema y los usuarios que hacen uso de las funcionalidades y cómo hacen uso de las mismas. Debido al alcance del curso, el ejemplo se limitará a identificar las funcionalidades y los usuarios. El diagrama utilizado para representar las funcionalidades del sistema, los usuarios y cómo estos interactúan con el sistema (diagrama de casos de uso), será diseñado en un curso posterior por medio de un lenguaje de modelado especial.

Principales usuarios involucrados (actores):

- Clientes y personal de Global Videos.
- Banco de la empresa.
- Sistema de inventario.

Descripción de los usuarios:

Nombre	Representa	Actividades que realiza
Empleado	Persona encargada de prestar los servicios de la tienda.	Registrar las ventas, alquileres, reservas y devolución de películas, así como también la afiliación y desafiliación de clientes.
Cliente	Persona que hace uso de los servicios de la tienda. Incluye a clientes afiliados como no afiliados.	Consumidor principal de los servicios prestados por Global Videos.
Banco	La entidad bancaria como un todo.	Verificar números de tarjeta, disponibilidad de fondos, validez. Transferir fondos de la cuenta del cliente a la cuenta de la empresa.
Sistema Contable	El módulo de inventario del sistema contable.	Registrar las transacciones contables relacionadas al alquiler, reservación y venta de películas. Tiene conexión directa con el sistema de inventario.

Funcionalidades del sistema (Casos de Uso).

- Afiliar y desafiliar clientes.
- Reserva, alquiler, venta de películas.
- Gestionar el pago con tarjeta de crédito.
- Devolver de forma automática las películas, mediante un buzón mecánico.
- Actualizar de manera automática del inventario.

En posteriores iteraciones, es posible identificar más actores, así como también otras funcionalidades, esto sólo representa un punto de partida. Sin embargo, con este boceto inicial, se tienen respuestas a algunas de las interrogantes planteadas, para obtener el primer objetivo de esta fase: entender qué se va construir.

Otra cosa a hacer en este punto, es definir una arquitectura candidata. Para el escenario planteado se podría proponer una arquitectura que divida el trabajo del sistema en cuatro capas, como:

Capa de Interfaz de Usuario: En esta capa estarán todos los componentes relacionados con el diseño, captura de datos, validación de datos e inicio de sesión.

Capa de Controlador de Interfaz: En esta capa se encuentran todos los componentes que toman decisiones sobre el flujo de eventos, dependiendo de las solicitudes de los usuarios del sistema (llámese solicitud a cualquier tarea que se le pide al sistema ejecutar de manera automática o no, desde consultas hasta actualizar el inventario), aquí están los componentes que determinan qué otros componentes atenderán una solicitud específica. Por ejemplo, pagar con tarjeta de crédito o actualizar inventario.

Capa de Lógica de Negocios: Se encuentran todas las entidades (clases) que fueron definidas para implementar las funcionalidades (comportamiento) del sistema, según: requerimientos, alcance del sistema y políticas de la empresa.

Capa de Acceso a Datos: Aquí se encuentran los componentes dedicados a establecer la conexión con las bases de datos asociadas al sistema. Estos componentes son responsables de establecer los privilegios de los usuarios que intentan acceder a la información de la BD. Se encuentran también en esta capa, las BD definidas para el sistema.

Una representación gráfica para lo anterior se muestra en la Figura 4.9.

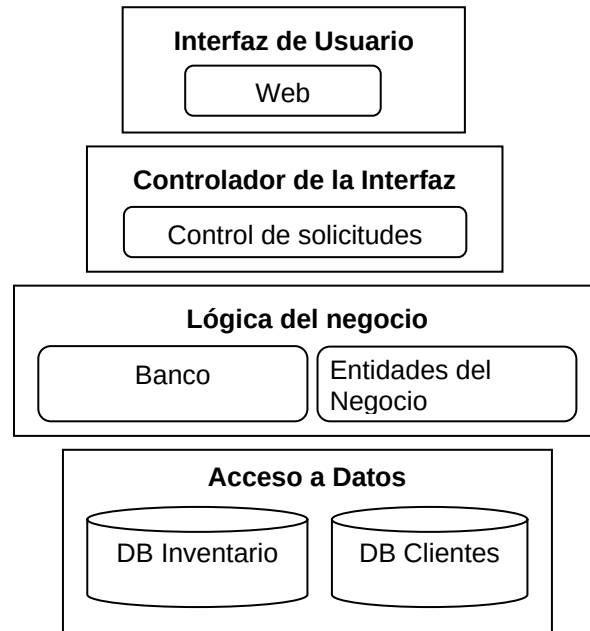


Figura 4.9: Arquitectura Candidata del Ejemplo 4.1

Algunos de los riesgos que podrían identificarse a primera vista en este desarrollo son:

- El software interno de los terminales remotos de la tienda y el software del buzón, no son compatibles con el sistema a desarrollar.
- La red para intercambio de información entre el banco y la tienda es inestable.
- El costo asociado a la compra e instalación del buzón y los terminales es muy alto.
- La seguridad y estabilidad en el intercambio de información entre sucursales.

Esta es una lista inicial de riesgos, conforme se avance hacia el producto final pueden identificarse más riesgos.

Esto representa una pequeña parte de toda la información a obtener de esta fase, luego de sus respectivas iteraciones. Toda esta información es agrupada en el documento de visión, también se deben registrar en este las fechas de cada una de las modificaciones realizadas en las diferentes iteraciones. En las subsiguientes fases se refinará esta información y se generará más.

Fin del ejemplo 4.1

En la fase de inicio se obtienen muchos detalles esenciales como lo son: la visión, arquitectura candidata, la identificación de los usuarios típicos, etc. Pero éstos, sólo son bocetos, los cuales serán refinados y algunos de ellos validados en la siguiente fase: La fase de elaboración.

6.4 Fase de Elaboración

La meta de la fase de elaboración es definir y establecer la base de la arquitectura del sistema, brindando así una base estable para la mayor parte del esfuerzo de diseño e implementación en la fase de construcción.

Esta meta general se traduce en cuatro objetivos importantes, cada uno trata un área importante del riesgo. Se manejan riesgos asociados con los requerimientos y con la arquitectura. También se manejan los riesgos asociados con los costos, cronograma y finalmente se necesita manejar los relacionados al proceso y al ambiente de desarrollo. Manejar estos riesgos asegura un mínimo de riesgos y problemas cuando se pase a la fase de construcción.

6.4.1 Objetivos de la Fase

Los cuatro objetivos de esta fase son:

1. Obtener un entendimiento más detallado de los requerimientos.

Durante la fase de elaboración, se desea tener un buen entendimiento de la gran mayoría de los requerimientos, ya que estos fueron descritos brevemente durante la fase de inicio. Esto permite crear un plan más detallado. Finalmente, ganar un entendimiento amplio de los requerimientos más críticos, para validar que la arquitectura cubra todas las bases, algo que solo puede alcanzarse construyendo una implementación parcial de estos requerimientos.

2. Diseñar, implementar, validar la arquitectura base.

La funcionalidad del sistema no estará completa, pero la mayoría de las interfaces entre los bloques de construcción son implementadas durante la fase de elaboración, para poder compilar y probar la arquitectura. A esto se le denomina "arquitectura ejecutable" hasta el punto que se puede (y debe) conducir alguna carga inicial de datos y ejecutar pruebas sobre la arquitectura.

3. Mitigar los riesgos significativos, producir un cronograma más exacto y estimar el costo.

Durante esta fase, se manejan los riesgos significativos. La mayoría serán manejados como el resultado detallado de los requerimientos y el diseño, implementación y prueba de la arquitectura. Al final de esta fase, se tendrá información más exacta, que permitirá actualizar el cronograma, plan de proyecto y la estimación de costo.

4. Refinar el caso de desarrollo y configurar el ambiente de desarrollo.

Se refina el proceso definido en la fase de inicio, para reflejar lo aprendido en las iteraciones previas. En esta fase, se establece un ambiente de soporte. Se define qué herramientas de desarrollo serán necesarias y qué otras tendrán que ser actualizadas o descartadas. Se instala y configura el ambiente establecido, el conjunto de todas herramientas seleccionadas, repositorios, entorno de desarrollo, servidores, herramientas gráficas, etc. Algunas de las actividades ejecutadas en esta fase se muestran en la Figura 4.10, junto a algunos de los artefactos producto de las actividades de la fase.



Figura 4.10: Actividades y Artefactos de la Fase de Elaboración.

6.4.2 Iteraciones de la Fase de Elaboración.

Si se tiene que construir un sistema con la misma tecnología que se está usando en el proyecto actual, entonces se puede alcanzar este objetivo con una simple iteración porque hay una cantidad limitada de riesgos que dirigir. Se pueden usar soluciones del pasado y así progresar rápidamente.

Pero si hay inexperiencia en el dominio de la aplicación, si el sistema es muy complejo o si se está usando nueva tecnología, entonces se necesitarán dos o tres iteraciones, para obtener la arquitectura correcta y mitigar los riesgos. Otros factores que determinarán si se requiere múltiples iteraciones son:

- Desarrollar un sistema distribuido.
- Muchas personas involucradas en el proyecto (stakeholders).

- La necesidad de cumplir con regulaciones de seguridad y otros estándares externos.

6.4.3 Hito: Arquitectura del Ciclo de Vida

Al final de la fase de elaboración está el hito de arquitectura del ciclo de vida. En este punto se examina en detalle el alcance y los objetivos del sistema, la selección de la arquitectura y la resolución de la mayoría de los riesgos. Si el proyecto falla en encontrar este hito, debe ser abortado o por lo menos seriamente reconsiderado.

La revisión del hito del ciclo de vida de la arquitectura incluye evaluar criterios como:

- ¿Son estables la visión y los requerimientos del producto?
- ¿Es estable la arquitectura?
- ¿Se tiene una prueba de los prototipos ejecutables, que demuestre que la mayoría de elementos de riesgos han sido manejados y resueltos?
- ¿Son los planes de iteración para la construcción lo suficientemente detallados y fiables para permitir iniciar el trabajo?
- ¿Están de acuerdo con la visión actual todos los involucrados en el proyecto, tal como se definió en el documento de visión?
- ¿Son aceptables los gastos actuales de recursos vs. los gastos planeados?

Hasta este punto se han implementado partes del sistema para verificar que se está en el camino correcto. Ahora, se procede a completar las funcionalidades del sistema para obtener una versión completa del mismo.

6.5 Fase de Construcción

La fase de construcción se enfoca de forma detallada en el diseño, implementación y prueba hasta lograr un sistema completo, con una alta calidad a un costo efectivo. La meta de esta fase, es resolver los requerimientos restantes y completar el desarrollo del sistema sobre la arquitectura base.

A esta altura del desarrollo muchos casos de usos no han sido implementados del todo, solo parcialmente o lo suficiente para probar algunas hipótesis y mitigar riesgos. Según se implemente más y más funcionalidad, se refinarán los requerimientos para asegurar que la funcionalidad correcta será entregada, buscando un balance entre calidad, alcance, tiempo y el refinar las características. Por eso, ésta fase es típicamente la que más tiempo consume.

6.5.1 Objetivos de la Fase

- Minimizar los costos de desarrollo por medio de la optimización de recursos, evitar el trabajo innecesario.
- Alcanzar cierto grado de paralelismo entre equipos. Este paralelismo puede acelerar el desarrollo de actividades significativas, pero también puede

incrementar la complejidad de la administración de recursos y la sincronización del flujo de trabajo.

- Desarrollar iterativa e incrementalmente un producto completo que esté listo para la transición a la comunidad de usuarios.

Para asegurar el progreso continuo y éxito de la fase, pueden tenerse en cuenta algunos aspectos como:

- Crear un equipo con una misión.
- Fijar metas claras y alcanzables para los desarrolladores.
- Hacer demostraciones y pruebas continuas del código, medir el progreso principalmente observando que el código es compilado y probado.
- Forzar la integración continua, ejecutar frecuentemente construcciones (de todo el programa o código) asegurando pruebas de integración de forma frecuente.
- Completar el análisis, diseño, desarrollo y prueba, de toda la funcionalidad requerida. Crear versiones útiles, alpha, beta u otras pruebas de liberación del producto.

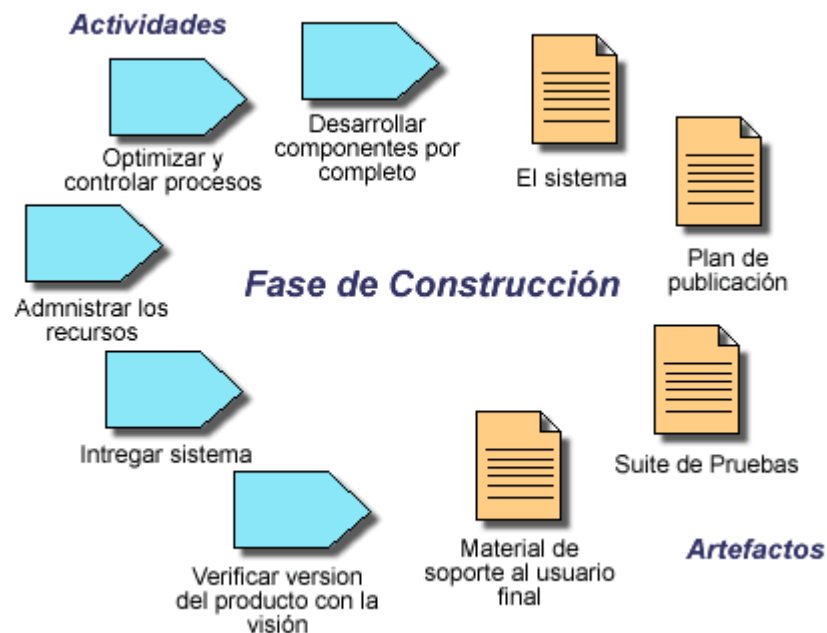


Figura 4.11: Actividades y Artefactos de la Fase de Construcción.

6.5.2 Iteraciones de la Fase

El número de iteraciones necesarias para la fase de construcción, varía de un proyecto a otro pero en la mayoría de los casos, tiene más iteraciones que otras fases (de tres o cuatro).

La necesidad de una iteración adicional en la fase de construcción, puede ser determinada al identificar cuáles componentes necesitan colaborar entre sí para proveer la funcionalidad de los casos de uso identificados. Estos componentes deben ser implementados y probados en cada iteración, hasta que todo el código pueda ser probado como una unidad, esto ofrece un mejor entendimiento del tiempo requerido para implementar los casos de uso, basándose en los recursos disponibles y el alcance del trabajo.

En cada iteración se necesita un plan de integración, que especifique cuáles capacidades se deberían probar en cada construcción y cuáles componentes necesitan integrarse para producir la funcionalidad requerida.

6.5.3 Hito: Capacidad Operacional

La fase de construcción termina con el hito de la capacidad operacional inicial, usado para determinar cuándo el producto está listo para ser publicado en una prueba beta del ambiente, que pueda responder preguntas como:

- ¿Es el producto publicado lo suficientemente estable para ser liberado a la comunidad de usuarios?
- ¿Están listas todas las personas involucradas en el proyecto para la transición a la comunidad de usuarios?
- ¿Son aceptables los gastos actuales de recursos vs. los gastos planeados?

Una vez obtenida una versión funcional de producto, se sigue a la fase que determinará, si la transición de dicha versión, es hacia otro ciclo de desarrollo o a los usuarios finales del producto.

6.6 Fase de Transición

Esta fase se enfoca en asegurar que el software esté listo para los usuarios finales. En la fase de transición, puede extenderse algunas iteraciones, incluyendo las pruebas del producto dentro de la preparación para su publicación y el hacer los ajustes menores basados en la retroalimentación de los usuarios. A este punto, la mayor parte de la estructura debería estar terminada y la retroalimentación de los usuarios debería enfocarse principalmente en la entonación, configuración, instalación y resultados de reutilización.

La fase de transición en el enfoque de RUP, difiere del desarrollo tradicional, principalmente porque se entra a ésta fase con una versión del sistema estable, integrada y probada.

6.6.1 Objetivos de la Fase de Transición.

Al final de la fase de transición los objetivos del ciclo de vida deben haber sido alcanzados y el proyecto debería estar en posición de ser cerrado. El fin del actual ciclo

de vida puede coincidir con el inicio del otro ciclo de vida, sobre el mismo producto, dirigiéndose a la próxima generación o versión del producto.

Los principales objetivos de esta fase incluyen:

- Una prueba beta, para validar el nuevo sistema contra las expectativas del usuario y la operación en paralelo con los sistemas heredados que el sistema reemplazará.
- Entrenar a los usuarios y a las personas encargadas de mantener el sistema, para alcanzar la adopción exitosa del mismo.
- Distribución del producto, que implica aspectos tales como, el empaquetamiento y producción comercial del software, fuerza de ventas, publicidad, distribución, actividades que ayudan a un lanzamiento exitoso del producto (ingeniería de publicación).
- Alcanzar un consenso con todas las personas involucradas en el proyecto, en que la base para la publicación está completa y es consistente con el criterio de evaluación de la visión.

En la Figura de 4.12 se presentan algunas de las actividades desempeñadas para alcanzar los objetivos de esta fase, junto a algunos de los artefactos trabajados durante la fase de transición. Es importante recordar que muchos artefactos persisten entre fases, siendo objeto de refinamientos, tal es el caso de la visión, la lista de riesgos, la suite de pruebas para el producto final, etc.

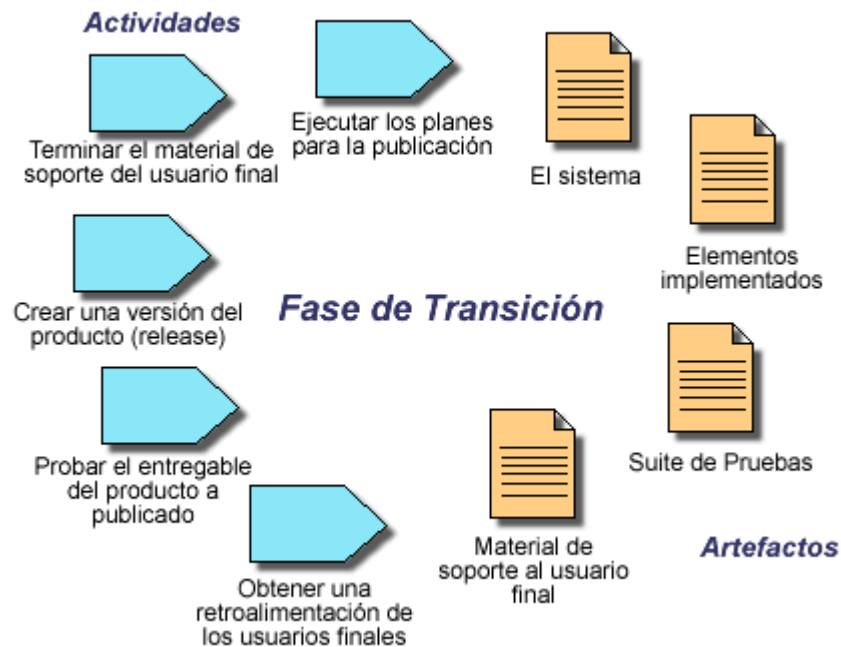


Figura 4.12: Actividades y Artefactos de la Fase de Transición

6.6.2 La Fase de Transición y las Iteraciones

En la fase de transición el número de iteraciones, puede variar de algo muy directo a algo extremadamente complejo, dependiendo del producto. Por ejemplo, una nueva versión de un producto de escritorio, ya existente puede ser muy simple requiriendo simplemente la corrección de algunos errores (bugs) menores, que se pueden realizar en una iteración. Sin embargo, se necesitan más iteraciones si se van a agregar características adicionales al producto y se va a integrar con otro sistema. Las actividades ejecutadas en las iteraciones de transición dependen de las metas que se quieran alcanzar. La mayoría de proyectos tienen una iteración en la fase de transición.

6.6.3 Transición y Ciclo de Desarrollo.

Un ciclo de desarrollo, es un paso a través de las cuatro fases: inicio, elaboración, construcción y transición. Al final de la fase de transición se ha completado un ciclo de desarrollo. Cada ciclo de desarrollo produce una generación del software. A menos que el proyecto sea cancelado, éste evolucionará dentro de la siguiente generación (secuencia de fases), pero con un énfasis diferente según las nuevas metas de proyecto requeridas. Estas sub-secuencias son llamadas ciclos de evolución.

Dos ciclos de desarrollo pueden solaparse, esto se da cuando la fase de inicio del siguiente ciclo de desarrollo, se inicia durante la fase de transición del actual ciclo de desarrollo. Se tiene la libertad de decidir si solapar o no dos ciclos de desarrollo, además de establecer el tamaño de dicho solapamiento. El tamaño del solapamiento, lo dará el momento de la fase de transición actual, en cual se decida comenzar el siguiente ciclo de desarrollo. Se debe tomar en cuenta que esto agrega una cantidad considerable de complejidad al proceso, lo que requiere un grado de madurez en la organización de procesos de desarrollos, con una avanzada configuración de administración procesos.

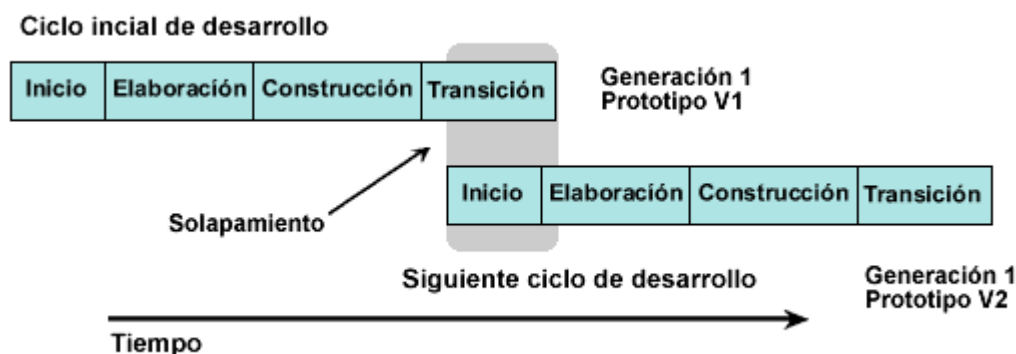


Figura 4.13 Solapamiento de Ciclos de Desarrollo.

6.6.4 Hito Versión del Producto.

El hito que marca el final de la fase de transición es el hito de la versión del producto (product release), donde se decide si los objetivos del proyecto fueron alcanzados y si es necesario comenzar otro ciclo de desarrollo. En éste punto, inicia un proceso de mantenimiento, operación y soporte. Esto puede involucrar el comienzo de un nuevo ciclo de desarrollo, para obtener una versión adicional de mantenimiento y resolver algunos problemas encontrados.

Llegado el final de la fase de transición, se deberían poder responder las siguientes preguntas:

- ¿Están los usuarios satisfechos?
- ¿Los recursos gastados actualmente vs. gastos planeados son aceptables? Si la respuesta es no, debería hacerse la pregunta siguiente:
- ¿Qué acciones pueden tomarse en un futuro para manejar este resultado?

7. ¿Quiénes deben usar RUP?

RUP es la metodología indicada para el desarrollo y publicación de proyectos de software críticos dentro de una organización. Esta metodología fue desarrollada pensando en dos grupos de usuarios:

- Desarrolladores de software, que trabajan como parte de un equipo de desarrollo de software.
- Personas que practican la ingeniería de procesos, específicamente gerentes e ingenieros de procesos de software.

Los desarrolladores de software pueden encontrar en RUP una guía, sobre qué es lo requerido de su rol (o de ellos como desarrolladores) en la definición de roles. Esta guía también proporciona el cómo, cada uno de estos roles colaboran entre si, en términos del trabajo detallado que es requerido para establecer el flujo de trabajo dentro de una iteración. Los desarrolladores pueden encontrar orientación, ayuda sobre definiciones, configuración e implementación de ingeniería de procesos.

8. ¿Cuándo usar RUP?

RUP puede ser utilizado desde el principio de un proyecto de software y puede continuar usándose en los subsecuentes ciclos de desarrollo del software después que la fase inicial ha finalizado. Sin embargo, la manera en la cual se use RUP, necesita ser trasformada para ajustarse a las necesidades. Existen algunas consideraciones que modifican el cuándo y cómo se utilizarán las diferentes partes de RUP, éstas son:

- El ciclo de vida del proyecto (número de iteración, longitud de cada fase, el largo del proyecto).
- Los objetivos de negocio del proyecto, visión, alcance y riesgos.
- El tamaño del esfuerzo para el desarrollo del software.

Aproximadamente 10.000 compañías están usando RUP. Estas compañías usan RUP en varios dominios de aplicación, de una manera equilibrada en proyectos grandes o pequeños. Esto muestra lo versátil de RUP y su amplia aplicabilidad. Algunos ejemplos de las industrias que usan RUP son empresa de: telecomunicación, transporte, servicios financieros, manufacturas, integradores de sistemas, entre otros.

Resumen

Ahora que ha finalizado esta unidad, usted será capaz de:

- Identificar los elementos básicos de RUP.
- Diferenciar entre el enfoque en cascada y el enfoque RUP.
- Comprender las características de la metodología RUP.
- Identificar las fases del ciclo de vida de RUP.
- Comprender qué se obtiene de cada fase.

Unidad 4: Examen de Autoevaluación

- 1) ¿Cuáles de las siguientes son características del enfoque RUP?
 - a) Dirigido por casos de uso.
 - b) Secuencial y iterativo.
 - c) Proceso iterativo e incremental.
 - d) Secuencias de actividades constantes.

- 2) ¿En la metodología RUP todas las fases tienen el mismo número de iteraciones?
 - a) Verdadero.
 - b) Falso.

- 3) ¿Cuáles de los siguientes son artefactos?
 - a) Un prototipo.
 - b) Diseñador de casos de uso.
 - c) Integrar los componentes.
 - d) Lista de riesgos.

- 4) El diseño, implementación y validación de la arquitectura base, son objetivos de la fase de:
 - a) Inicio.
 - b) Elaboración.
 - c) Construcción.
 - d) Transición.

- 5) Un ciclo de desarrollo concluye cuando se culmina:
 - a) Una fase.
 - b) Una iteración.
 - c) La fase de transición.
 - d) Todas las iteraciones de una fase.

- 6) ¿Cuáles de las siguientes ventajas de RUP?
 - a) Proporciona una definición común del proceso.
 - b) Un proceso de software configurable.
 - c) Brinda a un líder de proyecto un proceso de software por medio del cual administrar la planificación.
 - d) Un proceso de software accesible a todo el equipo de proyecto.

- 7) El hito de la fase de elaboración es:
- a) Hito de la capacidad operacional inicial.
 - b) Hito de la versión del producto.
 - c) Hito de los objetivos del ciclo de vida.
 - d) Hito del ciclo de vida de la arquitectura.
- 8) Las actividades pueden ser:
- a) Fraccionadas en pasos.
 - b) Agrupadas en disciplinas.
 - c) Ejecutadas como flujos de trabajos.
 - d) Asignadas a varios roles.
- 9) ¿La estructura dinámica de RUP está representada por?
- a) Ciclos.
 - b) Iteraciones.
 - c) Fases.
 - d) Disciplinas.
- 10) ¿Un ciclo de desarrollo no puede comenzar hasta concluir por completo el ciclo anterior?
- a) Verdadero.
 - b) Falso.

Respuestas de la Unidad 4: Examen de Autoevaluación

- 1) a y c
- 2) b
- 3) a y d
- 4) b
- 5) c
- 6) a, b, c y d
- 7) d
- 8) a, b y c
- 9) a, b y c
- 10) a